# Raku++ Internals

How a hand-written C++ implementation of Raku works — from source text to native code

The Raku++ project

# Contents

**Preface** **1**
Who this is for . . . . . 1
What this book is not . . . . . 2
How to read it . . . . . 2
Conventions . . . . . 3
A word about honesty . . . . . 3

**I Ground** **5**

**1 What Raku++ Is** **6**
1.1 The pipeline . . . . . 6
1.2 The four modes, in one paragraph each . . . . . 7
1.3 What the design is optimised for . . . . . 7
1.4 What it deliberately is not . . . . . 8
1.5 The ecosystem around the compiler . . . . . 9
1.6 A map of this book . . . . . 9

**2 The Shape of the Source** **11**
2.1 The numbers . . . . . 11
2.2 The library boundary . . . . . 13
2.3 What each file is for . . . . . 13
2.4 Reading conventions in the source . . . . . 16
2.5 Building it . . . . . 17
2.6 The test surface . . . . . 17

**3 Where It Sits in the Taxonomy** **18**
3.1 The lexer: separate, complete, run more than once . . . . . 19
3.2 The parser: recursive descent with a Pratt core . . . . . 20

3.3 The pass structure . . . 20
3.4 The part with no textbook box . . . 21
3.5 The regex engine, classified separately . . . 22
3.6 The back end: there is no IR . . . 23
3.7 What it is not . . . 24
3.8 Compared with three others . . . 24
3.9 Honest limitations . . . 25

**II The Front End** **27**

**4 The Lexer** **28**
4.1 The token . . . 28
4.2 `spaceBefore`: whitespace is significant . . . 29
4.3 Regex or division? . . . 29
4.4 Quote forms and heredocs . . . 30
4.5 Identifiers, sigils, twigils, Unicode . . . 30
4.6 Operators are a fixed, hand-ordered vocabulary . . . 31
4.7 Rule bodies are not tokenized . . . 32
4.8 What else the lexer carries out . . . 33
4.9 Errors . . . 33

**5 The Parser** **35**
5.1 Statement dispatch . . . 35
5.2 Statement modifiers . . . 36
5.3 Block or hash? . . . 36
5.4 Expressions: the Pratt core . . . 37
5.5 Declarations . . . 38
5.6 String interpolation is parsed, not concatenated . . . 39
5.7 Compile time: what the parser runs, and what it does not . . . 39
5.8 Errors . . . 40
5.9 Honest limitations . . . 41

**6 User-Defined Operators in a Single Pass** **42**
6.1 The tables . . . 42
6.2 Registration happens mid-parse . . . 43
6.3 How `5!` parses . . . 43
6.4 Precedence traits and the gaps of ten . . . 44
6.5 Custom brackets . . . 45
6.6 Operators are lexically scoped, and roll back . . . 45
6.7 Two escape hatches . . . 46

6.8 `use Foo` and imported operators . . . 46
6.9 The line this draws . . . 47

**7 The AST** **49**
7.1 A class hierarchy here, a fat struct there . . . 49
7.2 The nodes that carry the most . . . 50
7.3 Fields that are answers about the syntax . . . 51
7.4 Two nodes that only exist sometimes . . . 53
7.5 Borrowed pointers, and why the tree is immortal . . . 53
7.6 Seeing the tree . . . 54
7.7 The two emitters that must know every field . . . 54

**III The Value Model** **55**

**8 `Value`: the Fat Tagged Struct** **56**
8.1 Why not a union, a variant, or a class hierarchy . . . 58
8.2 What the fat struct buys . . . 58
8.3 Clever reuse of fields . . . 59
8.4 `ext`: the opaque slot . . . 60
8.5 The identity problem, and how `ext` solves it . . . 60
8.6 Ranges remember what they were written with . . . 61
8.7 The rendering hook . . . 62
8.8 What it costs . . . 62
8.9 The struct, version by version . . . 63
8.10 Honest limitations . . . 65

**9 Strings: `CowStr`** **67**
9.1 The problem . . . 67
9.2 What it is . . . 67
9.3 Does C++ have this already? It did, and removed it . . . 70
9.4 What it costs . . . 72
9.5 Rules for working with it . . . 73
9.6 Deliberately left on the table . . . 73
9.7 Checking that it is working . . . 74

**10 Interning and Comparison: `IStr` and `MName`** **75**
10.1 `MName`: the method name as the dispatcher sees it . . . 75
10.2 `IStr`: the storable counterpart . . . 77
10.3 Why they are separate types . . . 79

**11 Numbers** **80**
11.1 Layer 0: native integers, with overflow as a signal . . . . . 80
11.2 Layer 1: `BigInt` . . . . . 81
11.3 Layer 2: `Rat` . . . . . 83
11.4 Native integer containers . . . . . 84
11.5 Complex . . . . . 84
11.6 Where the arithmetic actually happens . . . . . 85
11.7 A bug worth remembering: numifying by throwing . . . . . 85

**12 Containers, Binding, and Copy Semantics** **87**
12.1 Scopes are hash maps with a parent pointer . . . . . 87
12.2 Where the sharing is broken . . . . . 89
12.3 Binding: `:=` and the `Proxy` . . . . . 90
12.4 The scalar-container metaphor . . . . . 91
12.5 Sigils are mostly a parse-time fact . . . . . 92
12.6 The list container . . . . . 92
12.7 Shaped arrays and typed containers . . . . . 94
12.8 `is rw`, and writing back to the caller . . . . . 94
12.9 Freeing cycles . . . . . 95
12.10 Honest limitations . . . . . 99

**IV The Interpreter** **101**

**13 The Tree Walk** **102**
13.1 The two entry points . . . . . 102
13.2 Execution registers . . . . . 103
13.3 Evaluating an expression . . . . . 103
13.4 Evaluating a statement . . . . . 104
13.5 Loop bodies . . . . . 106
13.6 Aliasing loops . . . . . 106
13.7 Sink context and `it does not matter what this returns` . . . . 107
13.8 Recursion, and the stack it needs . . . . . 107
13.9 What this costs, honestly . . . . . 107

**14 Calls and Parameter Binding** **109**
14.1 Building the argument list . . . . . 109
14.2 Activating the callee . . . . . 110
14.3 Binding parameters . . . . . 112
14.4 The `Callable`, and what it caches . . . . . 114
14.5 `is rw` write-back . . . . . 115

14.6 Lvalue-mode method calls . . . 115
14.7 What a call costs . . . 115

**15 Cooperative Control Flow** **117**
15.1 The insight . . . 117
15.2 The same trick, three more times . . . 118
15.3 The rest of the control exceptions . . . 119
15.4 `CATCH`, and where it lives . . . 120
15.5 Backtraces . . . 121
15.6 What was gained, and one bug it cost . . . 125

**16 Dispatch** **126**
16.1 Named calls . . . 126
16.2 Multiple dispatch . . . 127
16.3 Method dispatch: two worlds . . . 128
16.4 Re-dispatch . . . 130
16.5 Private methods, qualified calls, indirect calls . . . 130
16.6 `&builtin` as a value . . . 131
16.7 Where the time goes . . . 132
16.8 Honest limitations . . . 132

**17 The Object System** **134**
17.1 Registration and lookup . . . 135
17.2 Construction . . . 136
17.3 Attributes and accessors . . . 136
17.4 Roles . . . 137
17.5 Mixins: `but` and `does` . . . 138
17.6 `augment`, and the `supersede` that is not there . . . 139
17.7 Package stashes and `.WHO` . . . 140
17.8 Honest limitations . . . 140

**18 Laziness, Junctions, and the Wilder Values** **141**
18.1 Lazy and infinite sequences . . . 141
18.2 `gather` and `take`, without coroutines . . . 146
18.3 Junctions . . . 147
18.4 `Whatever` and `WhateverCode` . . . 148
18.5 Hyper operators . . . 148
18.6 Flip-flops . . . 149

**19 Node Specialization** **150**
19.1 What it does . . . 150

19.2 Why these shapes, and not constant folding . . . 151
19.3 What “cached” means here . . . 151
19.4 The guards . . . 153
19.5 Why it is not a new node kind . . . 154
19.6 Measured . . . 154
19.7 What went wrong on the way . . . 155
19.8 Codegen already had this . . . 155
19.9 Extending it . . . 156

**V The Regex and Grammar Engine 157**

**20 Compiling a Regex 158**
20.1 The pattern text arrives unparsed . . . 158
20.2 The node tree . . . 159
20.3 The byteset: a per-node character-class cache . . . 160
20.4 The parser . . . 160
20.5 Sigspace . . . 161
20.6 Ratchet . . . 161
20.7 Two whole-pattern optimisations . . . 162
20.8 Retired Perl 5 metacharacters . . . 162
20.9 Interpolation happens before compilation . . . 163
20.10 What the compiler produces . . . 163
20.11 `:P5` — the same engine wearing Perl 5 syntax . . . 164

**21 The Backtracking Matcher 168**
21.1 `FnRef`: a continuation that does not allocate . . . 169
21.2 The step budget . . . 169
21.3 `MState`: everything one match frame knows . . . 170
21.4 Two subrule paths . . . 171
21.5 Lookarounds . . . 171
21.6 The hooks: running Raku during a match . . . 172
21.7 When side effects fire . . . 173
21.8 Entry points . . . 173
21.9 Substitution . . . 174

**22 Grammars 175**
22.1 The grammar matcher . . . 176
22.2 A subrule call, in full . . . 176
22.3 `ParseNode`: the tree the matcher records . . . 178
22.4 Actions . . . 179

22.5 Parameterised rules . . . 179
22.6 Protoregexes . . . 180
22.7 Entry points and start rules . . . 181
22.8 Where grammars appear in this book again . . . 181

**23 Longest-Token Matching** **182**
23.1 Two rankers . . . 182
23.2 What ends a declarative prefix . . . 183
23.3 The gap-aware hybrid contract . . . 183
23.4 The automaton . . . 184
23.5 The cache, and an address-reuse bug . . . 185
23.6 Subrule expansion: three routes . . . 185
23.7 Proto dispatch . . . 186
23.8 The tie-break must be per-path . . . 187
23.9 Oracle notes . . . 187
23.10 Debugging and measured results . . . 188

**VI Unicode** **190**

**24 Graphemes, Normalization, Collation** **191**
24.1 Storage: UTF-8 bytes, grapheme semantics . . . 191
24.2 The fast answer: when a byte index is a grapheme index . . . 192
24.3 The slow answer: `GraphemeMap` . . . 192
24.4 The subsystems . . . 193
24.5 The tables are generated, and one generator is written in Raku . . . 194
24.6 Where Unicode shows up elsewhere . . . 195
24.7 Honest limitations . . . 195

**VII The Back Ends** **196**

**25 Four Ways to Run a Program** **197**
25.1 Mode 1 — interpret . . . 197
25.2 Mode 2 — `--bundle` . . . 198
25.3 Mode 3 — `--aot` . . . 198
25.4 Mode 4 — `--exe` . . . 199
25.5 Comparing them . . . 200
25.6 The fallback that makes `--exe` total . . . 201
25.7 What every mode shares . . . 201
25.8 Carrying modules inside a binary . . . 202

25.9 Mode 1 in a browser . . . . . 203

**26 The Code Generator** **204**
26.1 The mapping . . . . . 204
26.2 What is emitted . . . . . 206
26.3 Control flow . . . . . 206
26.4 Closures . . . . . 207
26.5 Classes and multis . . . . . 207
26.6 Signatures . . . . . 208
26.7 Sub-language pieces the generator hands back to the runtime . . 208
26.8 What it refuses . . . . . 209
26.9 The correctness constraint on resolving names at compile time . . 209
26.10 Inspecting the output . . . . . 210

**27 The `-O` Optimizer** **211**
27.1 What the generated code looks like without it . . . . . 212
27.2 Pass 1 — direct-arity calls . . . . . 212
27.3 Pass 2 — inline arithmetic . . . . . 213
27.4 Pass 3 — guarded native-int lanes . . . . . 214
27.5 Three defaults that are not `-O` . . . . . 215
27.6 Forwarding the C++ optimization level . . . . . 216
27.7 Measured . . . . . 216
27.8 The box stays; only its per-operation cost goes . . . . . 218
27.9 Correctness . . . . . 218
27.10 What is next . . . . . 218

**28 Dispatch Cost in Compiled Code** **220**
28.1 The four shapes . . . . . 220
28.2 What dispatch itself costs . . . . . 221
28.3 Cut 1 — cached builtin pointers . . . . . 222
28.4 Cut 2 — inline string comparisons . . . . . 222
28.5 Cut 3 — true named builtins, and the analysis that was wrong . . 223
28.6 The interpreter got the same treatment . . . . . 224
28.7 What is deliberately not done . . . . . 225
28.8 The general lesson . . . . . 225

**29 What a Compiled Binary Keeps** **226**
29.1 The problem is the linker's, not the compiler's . . . . . 226
29.2 The seam . . . . . 226
29.3 Five archives . . . . . 227
29.4 The stubs . . . . . 228

29.5 The levels . . . 229
29.6 The scan . . . 229
29.7 The triggers . . . 230
29.8 What it is worth . . . 231
29.9 The directives, and the one that proves it . . . 231
29.10 Where it declines . . . 232
29.11 The manifest . . . 232
29.12 The shape of the argument . . . 232

**30 AST Serialization and the Precompiled Parse 234**
30.1 The format . . . 234
30.2 Two independent switches . . . 235
30.3 What invalidates an entry . . . 236
30.4 Where it lives, and what it reports . . . 237
30.5 What is *not* cached . . . 237
30.6 The other consumer: embedded modules . . . 238
30.7 The AOT emitter, and what it adds . . . 238

**31 The JavaScript Back End 240**
31.1 The interface . . . 240
31.2 The same program, a third way . . . 241
31.3 The runtime that had to be written . . . 242
31.4 Three outputs, and a second tier . . . 243
31.5 Keeping two runtimes honest . . . 243
31.6 Where it diverges . . . 244
31.7 Speed . . . 244
31.8 Honest limitations . . . 245

**32 Raku in the Browser 246**
32.1 The whole boundary . . . 246
32.2 Input is one `rdbuf` swap . . . 247
32.3 Output, and the flush that is not optional . . . 247
32.4 The stack is chosen, not inherited . . . 248
32.5 Exceptions: `-fexceptions`, not `-fwasm-exceptions` . . . 248
32.6 Off the main thread . . . 249
32.7 How a line of output reaches the page . . . 250
32.8 Live mode: running as you type . . . 254
32.9 Embedded editors: one script tag . . . 256
32.10 What the host takes away . . . 257
32.11 Size and memory . . . 258
32.12 Why this is worth having . . . 258

**VIII Boundaries** **260**

**33 Modules** **261**
33.1 Parse time: only operators are looked at . . . 261
33.2 Run time: `loadModule` . . . 262
33.3 Where the source is found . . . 262
33.4 Failure is fatal, deliberately . . . 263
33.5 Questions people actually ask . . . 264
33.6 Divergences from Rakudo . . . 266
33.7 Debugging a module problem . . . 267

**34 The Installer and the Store** **268**
34.1 The layout . . . 268
34.2 Two writers, one reader contract . . . 269
34.3 The writer is the engine's . . . 270
34.4 The installer is a Raku program the binary carries . . . 271
34.5 Update means add . . . 275
34.6 Uninstall is garbage collection . . . 275
34.7 The checker . . . 276
34.8 What went wrong: damage travels between engines . . . 277
34.9 Interchange, stated precisely . . . 278

**35 `use nqp`: a Compatibility Subset** **280**
35.1 There is no NQP compiler here . . . 280
35.2 Zero cost when unused . . . 281
35.3 How the ops compile . . . 281
35.4 In code . . . 282
35.5 The argument buffers . . . 283
35.6 Native compilation . . . 284
35.7 What is covered . . . 284
35.8 Scope and non-goals . . . 285
35.9 Why this shape is the right one . . . 285

**36 NativeCall and the libffi Backend** **286**
36.1 libffi is loaded at run time, and its ABI is restated by hand . . . 286
36.2 What happens with no libffi . . . 288
36.3 How a call is made . . . 288
36.4 Variadics . . . 289
36.5 The type map . . . 290
36.6 Callbacks . . . 291
36.7 Your declaration is a promise nothing can check . . . 292

36.8 Tracing . . . 292
36.9 In compiled code . . . 293
36.10 Not supported . . . 293

**37 The Extension ABI** **294**
37.1 Extension or NativeCall? . . . 294
37.2 The one rule . . . 295
37.3 Hello, world . . . 295
37.4 Both halves of the linking bargain . . . 296
37.5 The value vocabulary . . . 297
37.6 Calling back into Raku . . . 299
37.7 Rooted handles, and the one way to leak . . . 300
37.8 Lifetime: handles belong to the call . . . 300
37.9 Errors . . . 300
37.10 Versioning . . . 301
37.11 Writing a module that also runs on Rakudo . . . 302
37.12 Packaging . . . 303
37.13 The naming convention . . . 304
37.14 Current limits . . . 304
37.15 The other direction . . . 304
37.16 A footnote on guessing . . . 306

**38 Concurrency** **307**
38.1 Stage 1: per-thread execution registers . . . 307
38.2 Stage 2: the symbol-table freeze . . . 309
38.3 Stage 3: the GIL . . . 310
38.4 Stage 3a: true parallelism, and then the flip . . . 311
38.5 Threads need big stacks . . . 312
38.6 Worker bookkeeping, and two real crashes . . . 312
38.7 The user-visible layer . . . 313
38.8 Signals, and a thing that must be turned off . . . 314
38.9 Testing it . . . 314
38.10 Honest limitations . . . 315

**IX Around the Compiler** **316**

**39 Tooling Built on the AST** **317**
39.1 `--lint`: static analysis that runs nothing . . . 317
39.2 The undeclared-variable gate: the same principle, with the program stopped . . . 318

39.3 `--highlight`: colouring with the compiler's own knowledge . . . 323
39.4 `--profile`: a routine-level wall-time profiler . . . 323
39.5 The REPL . . . 324
39.6 `--mcp`: the same session, driven by an agent . . . 325
39.7 `--jupyter`: the same session, in a notebook . . . 328
39.8 Pod and `--doc` . . . 329
39.9 `--dump-ast`, and the tool built on it . . . 330
39.10 The tools that are not in the binary . . . 330

**40 Measuring, and Proving** **332**
40.1 The benchmark policy . . . 332
40.2 The performance gate . . . 333
40.3 Three kernels, three resolutions . . . 334
40.4 The correctness gates . . . 334
40.5 Oracles . . . 335
40.6 Error messages are not behaviour . . . 336
40.7 What has and has not paid . . . 336
40.8 Four ways the measurement itself was wrong . . . 337
40.9 Where the remaining time is . . . 338
40.10 Honesty as a practice . . . 339

**X The Speed Campaign** **340**

**41 Constant Factors** **341**
41.1 The trigger, and the method . . . 341
41.2 The recurring kernels . . . 343
41.3 Batch one: the hash payload . . . 344
41.4 Batch two: the census and the cold block . . . 345
41.5 Batch three: pads, and the lever the falsifier exposed . . . 347
41.6 Batch four: the price of an assignment . . . 348
41.7 Batch five: the list container itself . . . 350
41.8 Where it landed . . . 352
41.9 The three decisions . . . 353

**42 Reading Other Engines** **354**
42.1 The method . . . 354
42.2 Nine engines, one line of inheritance each . . . 355
42.3 Already ours, before any study . . . 357
42.4 What the engines agree on . . . 357
42.5 The one item the shelf recommends and this engine does not take 358

42.6 Where it stops, for now . . . 359

**43 The Carry Chain** **360**
43.1 The trigger . . . 360
43.2 First pass: one limb at a time . . . 361
43.3 Second pass: the loop could not get out of its own way . . . 361
43.4 And then the copies, again . . . 363
43.5 The road not taken . . . 364
43.6 Where it landed . . . 364
43.7 What the two episodes share . . . 365

**44 Compiling While It Runs** **367**
44.1 What tiering up is . . . 367
44.2 The harness, and the whitelist that makes it sound . . . 368
44.3 The first back end: hand the loop to the machine's compiler . . . 369
44.4 The second back end: copy and patch . . . 370
44.5 The register file, and the one IR in this engine . . . 372
44.6 The four costs, and what each came to . . . 374
44.7 Hoisting, and the bug that priced it . . . 375
44.8 Measured . . . 376
44.9 How it is proved . . . 377
44.10 What went wrong on the way . . . 378
44.11 What it does not claim . . . 378
44.12 Where it goes . . . 379

**Appendices** **381**

**A The Tag Sets** **382**
A.1 `VT` — runtime value tags . . . 382
A.2 `NK` — AST node kinds . . . 383
A.3 `K` — regex node kinds . . . 384
A.4 `NqpOpc` — the `use nqp` subset . . . 384
A.5 Exception and control structs . . . 385

**B Flags and Environment Variables** **386**
B.1 Command-line flags that select a mode . . . 386
B.2 Inspection and tooling flags . . . 386
B.3 Flags that shape a build . . . 388
B.4 Flags that change how a run behaves . . . 388
B.5 Environment variables . . . 389

B.6 Build-time switches . . . . . 393
B.7 A note on flags that change behaviour . . . . . 393

**C Source Map** **394**
C.1 Where to look for what . . . . . 394
C.2 Rules that are easy to break by accident . . . . . 396

**D Glossary** **398**
If you have never built one before . . . . . 398

# Preface

This is a book about the inside of a compiler.

Raku++ is a hand-written implementation of the Raku programming language, written in C++17, with no third-party dependencies. It lexes, parses, interprets, and — in two of its five run modes — transpiles Raku to C++ and to JavaScript, and compiles it to a native binary. It carries its own regular-expression and grammar engine, its own Unicode subsystem built from the pinned UCD tables, its own arbitrary-precision arithmetic, a foreign-function interface that loads libffi at run time rather than linking it, a C ABI for native extension modules, and a concurrency runtime that runs interpreter compute on all cores by default, with a global interpreter lock kept as the opt-in escape hatch.

None of that is unusual for a language implementation. What is unusual is that all of it fits in about a hundred thousand hand-written lines that one person can read, and that almost every non-obvious decision in it was made against a measurement. This book is an attempt to write down both: the mechanisms, and the reasons.

## Who this is for

You should be comfortable reading C++ and comfortable reading Raku, but you do not need to be an expert in either, and you certainly do not need to have implemented a language before. Where a piece of compiler folklore is load-bearing — precedence climbing, Thompson construction, packrat memoisation, copy-on-write — it is explained where it is used rather than assumed, and Appendix D collects the whole vocabulary, general and local, in one alphabetical list.

Three kinds of reader were in mind:

- Someone who wants to **work on Raku++** and needs a map before they can usefully change anything.
- Someone building **their own interpreter or compiler**, who wants a worked example of a dynamic language implemented in a static one — including the parts that went wrong.
- Someone who writes **Raku** and wants to know what the machine underneath is actually doing when they write `given/when`, or a grammar, or `is native`.

## What this book is not

It is not a manual for the language, and not a manual for the `rakupp` command. Those exist: the guide in `docs/guide/` covers using Raku++, and `docs/guide/REFERENCE.md` is the exhaustive per-routine lookup sheet. Nor is it a specification of Raku; the reference implementation, Rakudo, and the Roast test suite hold that role, and this book is careful to say *where Raku++ diverges from them* rather than pretending it does not.

It is also not a tour of every function. The source is the normative reference, and it moves. What is written down here is the **shape** — the data structures that everything else is built on, the invariants they rely on, and the reasons a particular design was chosen over the obvious alternative.

## How to read it

The ten parts are ordered the way a program flows through the system: source text, then the tree, then the values, then execution, then the specialised engines, then the back ends, then the boundaries with the outside world. Read straight through and it is a narrative. Read a single part and it should still stand on its own; cross-references say where the rest of a thread lives.

If you are here for one thing in particular:

| You want | Start at |
|---|---|
| the overall map | Chapter 1, then Chapter 25 |
| what kind of compiler this is | Chapter 3 |
| how source becomes a tree | Part II |
| what a runtime value *is* | Chapter 8 |
| how a Raku call actually happens | Chapters 14 to 16 |
| regexes and grammars | Part V |

| You want | Start at |
|---|---|
| the native compiler | Part VII |
| Raku in a browser | Chapter 32 |
| installing modules, and the store zef shares | Chapter 34 |
| calling C, or being called from it | Chapters 36 and 37 |
| a loop that compiles itself while it runs | Chapter 44 |
| a term you have not met before | Appendix D, the glossary |

## Conventions

Source excerpts are tagged with the **file** they come from:

```
// src/Value.h
static Value integer(long long x) { Value v; v.t = VT::Int; v.i = x; return v; }
```

They are lightly trimmed for the page — an ellipsis, a dropped comment, an elided error string — but are otherwise verbatim. **File names, not line numbers, are the anchors.** The code moves; grep for the function or for the quoted line.

Raku code is shown as Raku:

```
my @primes = (2..*).grep(*.is-prime);
say @primes.head(5);          # (2 3 5 7 11)
```

Measured numbers carry their conditions. Unless a chapter says otherwise, they were taken on arm64 macOS with Apple clang, using the project's benchmark policy: interleaved A/B runs, a discarded warm-up, medians or minima of several repetitions, and a *control* kernel that the change under test cannot affect. A number without a control is an anecdote, and this book tries not to print anecdotes.

Sections headed **What went wrong** are not decoration. Several of the designs here are the second or third attempt, and the discarded attempts are usually more instructive than the surviving one.

## A word about honesty

Raku is a very large language, and Raku++ does not implement all of it. About ninety per cent of the declared Roast suite passes. The method resolution order is a depth-first walk rather than C3 linearisation. Multiple dispatch resolves ties by declaration order instead of raising an ambiguity error. Modules publish their whole environment to the global scope rather than only their exports. Macros are

not implemented at all, and slangs only where the lexer has a seam to hang a token on.

Every one of those is stated in the chapter where it belongs, under a heading that says so. A book about compiler internals that only described the parts that work would be a brochure.

# Part I

# Ground

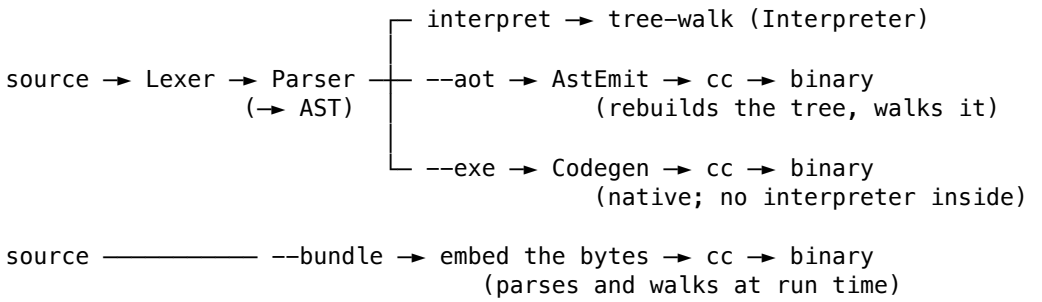
# Chapter 1

# What Raku++ Is

Raku++ is a Raku implementation written from scratch in C++17. It has no third-party dependencies: no parser generator, no regex library, no bignum library, no ICU, no libffi at link time, no tracing garbage collector. The binary it produces is one file, and the same source builds it on macOS, Linux, the BSDs, Windows, and — compiled to WebAssembly — inside a browser tab.

That constraint is not asceticism for its own sake. It has a specific consequence which shapes almost every chapter of this book: **when something is slow or wrong, the code responsible is in the tree**, and it can be changed. A good deal of what follows is the story of doing exactly that.

## 1.1 The pipeline




A single front end feeds five back ends. The front end is a hand-written character-level lexer and a recursive-descent parser with a precedence-climbing core for ex-

pressions; it produces a `Program`, which is a vector of statement nodes. Everything downstream consumes that one artifact.

Everything except the command-line driver is compiled into a static library, **`librakupp_rt.a`** — "the runtime". The `rakupp` executable links against it, and so does every binary the compiling modes produce. That is why a feature implemented once behaves identically whether a program is interpreted or natively compiled: there is one implementation of `Value` semantics, one set of built-ins, one method dispatcher, one regex engine, one Unicode subsystem.

## 1.2 The four modes, in one paragraph each

**Interpret** (the default) lexes, parses, and tree-walks. Startup is about two milliseconds; throughput pays the per-node dispatch cost. This is the mode the language is developed against, because it is the only one that can run every construct.

**`--bundle`** embeds the source bytes in a generated C++ stub and links it against the runtime. The result is standalone, but at run time it still lexes, parses and tree-walks — it is the interpreter in a box. The win is distribution, not speed.

**`--aot`** parses at build time and emits C++ that *rebuilds the identical AST* at startup, then interprets it. Parse errors surface at build time rather than run time; execution speed is the same as bundling, because it is the same tree walk. It emits one builder function per AST node, so the generated code grows with the program.

**`--exe`** parses at build time and transpiles the tree into C++ that implements the program directly: native control flow, native calls, with the runtime called only for value semantics. With `-O` it additionally runs its own optimizer before the C++ compiler sees anything. Anything it cannot transpile throws a `CodegenError`, which the driver catches and answers by bundling the whole program instead — so `--exe` never refuses a program.

Chapter 25 works one small program through all four.

## 1.3 What the design is optimised for

Three decisions dominate everything else, and each gets a full chapter later.

**One value type.** Every Raku value at runtime is the same C++ struct, `Value`, carrying a one-byte tag and a field for every kind of payload. Not a union, not a

`std::variant`, not a class hierarchy — a *fat struct*. Chapter 8 argues the case, which is stronger than it first looks, and prices the memory it costs.

**The C++ stack is the Raku stack.** There is no bytecode and no separate operand stack. `eval(Expr*)` returns a `Value`; `exec(Stmt*)` runs a statement. This makes the implementation small and makes native recursion depth the Raku recursion budget, which is why the entry point runs the program on a thread with a very large stack.

**Compile time runs almost nothing.** `BEGIN` becomes a tagged block that the interpreter schedules after the parse; `constant` is not folded; `use` loads no ordinary module during parsing. There are two parse-time side effects. The first is registering a user-declared operator in the parser's own tables, and Chapter 6 is about why that one is enough to support custom operators with real precedence in a single forward pass. The second is `use` of a *slang*: that module alone is run during the parse, so that the tokens it registers can be spliced into the lexer for the rest of the unit. Macros need a third thing neither provides — a user-built AST substituted for the call site — and are not supported.

## 1.4 What it deliberately is not

It is not the reference implementation. Rakudo is, and Raku++ is measured against Rakudo and against the Roast suite continuously rather than aspirationally. The current position is roughly ninety per cent of declared Roast assertions.

It has no tracing garbage collector, and no JIT unless one is asked for: the two tier-up back ends, `--jit` and `--cnp`, are off by default and are Chapter 44. Lifetime is reference counting, which frees a value the moment its last reference goes away. Beside it runs a backup cycle collector that looks only for groups of objects that refer to each other, which counting alone would leak; Chapter 12 describes it.

It implements compile-time metaprogramming only in part. `macro` and `quasi` are absent, for the structural reason given above. Swapping the grammar for a lexical scope — a slang — works at a fixed set of seams: the module is run and its tokens are called where the lexer would have started the productions it overrode, and a slang that reaches past them is refused by name. `RakuAST` turned out not to belong in that group: the tree a program asks for is a **view** built over the parse that has already happened, so nothing has to run mid-parse and nothing has to rewrite a grammar. It is implemented — the classes, `.AST`, `.DEPARSE`, `.EVAL`, `visit-children` and `.rakudoc`. What *is* supported is everything that can be done by adding an entry to a table during a single forward pass: all six custom-operator categories with preced-

ence and associativity traits, plus the whole runtime meta-object surface (`augment`, `.^add_method`, `does`/`but` mixins, `&routine.wrap`).

## 1.5 The ecosystem around the compiler

Several things in this book are not in the `rakupp` binary but are load-bearing for it, and appear where they are relevant:

| Thing | What it is |
|---|---|
| **Raku.js** | the same runtime compiled to WebAssembly — the interpreter in a browser |
| **the Roast harness** | `tools/run-roast.raku`, written in Raku, run by rakupp against the spec suite |
| **`perf-guard`** | `tools/perf-guard.raku`, the release gate that fails a regression instead of eyeballing one |
| **the showcase interpreters** | JavaScript, Perl 5, Python 3 and Lisp interpreters *written in Raku*, used as beyond-Roast tests |
| **`JSON::Native`** | the first native extension module, and the proof the extension ABI works |

The showcase programs deserve a note here because they recur. An interpreter for another language, written in Raku and run by Raku++, exercises grammars, recursion, string handling, closures and dispatch simultaneously and at a scale no unit test reaches. Each of them, when first written, produced a list of genuine bugs in Raku++; several fixes in later chapters were found that way.

## 1.6 A map of this book

| Part | Covers |
|---|---|
| I | this chapter, the layout of the source, and where the design sits in the compiler taxonomy |
| II | lexer, parser, user-defined operators, the AST |

| Part | Covers |
|---|---|
| III | `Value`, strings, interning, numbers, containers |
| IV | the tree walk, calls, control flow, dispatch, objects, laziness |
| V | the regex engine, the grammar engine, longest-token matching |
| VI | Unicode: graphemes, normalization, collation |
| VII | the five run modes, the two code generators, `-O`, what a binary keeps, serialization, the browser |
| VIII | modules, `use nqp`, NativeCall, the extension ABI, concurrency |
| IX | tooling built on the AST, and how any of this is proved |

# Chapter 2

# The Shape of the Source

Before the mechanisms, the map. This chapter is the one to come back to when a later chapter names a file and you want to know what else lives near it.

## 2.1 The numbers

`src/` holds about **274,000 lines** of C++. That figure is misleading on its own, because **93,000 of them are generated**: the Unicode tables — character names, properties, collation weights, normalization data, emitted from the pinned UCD and UCA 18.0 files in `tools/ucd/` — and the JavaScript runtime, which is written as JavaScript in `src/js-rt/` and baked into one C++ file so a binary carries the runtime it was built with. Nobody reads those, and nobody edits them where they land.

The hand-written implementation is about **182,000 lines**, and it is very unevenly distributed:

| File | Lines | What it is |
|---|---|---|
| `InterpreterCore.cpp` | 28,737 | the hot paths of the tree walk: `eval`, `exec`, operators, assignment, calls |
| `Parser.cpp` | 16,741 | statements, expressions, declarations, interpolation |

| File | Lines | What it is |
|---|---|---|
| `MethodCallPart2.cpp` | 10,222 | the method chain, continued |
| `Builtins.cpp` | 8,964 | the built-ins' helpers, `methodCall`, the head of the method chain |
| `Interpreter.cpp` | 7,559 | construction, the mainline run, the precompiled-module cache |
| `InterpreterModules.cpp` | 7,034 | module loading, `EVAL`, declarations |
| `MethodCallPart3.cpp` | 6,671 | the method chain, continued |
| `InterpreterBinding.cpp` | 5,642 | closures, signatures and binding, NativeCall |
| `Regex.cpp` | 5,428 | the regex and grammar engine |
| `Lexer.cpp` | 5,017 | tokenizer |
| `MethodCallTail.cpp` | 4,999 | the method chain, the end of it |
| `Codegen.cpp` | 4,652 | the `--exe` transpiler |
| `Interpreter.h` | 4,003 | the interpreter's own interface, plus the `rt*` helpers |
| `main.cpp` | 3,670 | the CLI, the compile drivers and the tooling subcommands |

Two shapes stand out and both are deliberate.

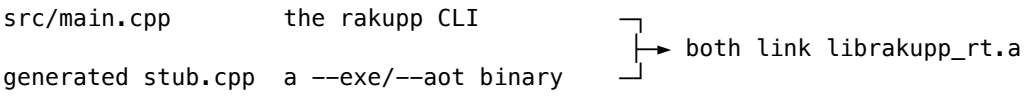
**The interpreter is one class in seven files.** The tree walk touches everything — scopes, calls, operators, assignment, control flow, module loading, the FFI marshaller, the concurrency runtime — and the pieces share the interpreter's private state rather than a clean interface, so the files are cut for compile time, not by concept. `InterpreterCore.cpp` holds every function the `perf-guard` kernels spend their time in, whatever it does, and the definition of `tctx_`, the per-thread execution state. The hot functions have to share a file: in separate files they no longer inline into each other, and a `thread_local` read from a file that does not define it goes through a call on every access — measured, that cost 14–35% on the kernels. The other six files (`Interpreter`, `...Modules`, `...Binding`, `...Calls`, `...Regex`, `...Oper-`

ators) hold the rest in source order, and InterpreterParts.h declares what they share. tools/source-helpers/ holds the scripts that made the cut, and the plan they followed; budget.raku there fails t/run.raku when a source file, a function or a widely included header grows past its recorded ceiling, so the next 59,000-line file is noticed long before it is one.

**The method dispatcher is split across six files for the same reason.** It used to be a single 9,138-line function, methodCallInner, and that stopped being compilable in a reasonable time. It is now six ordered *segments* — Builtins.cpp holds the head, then MethodCallPart1b, MethodCallPart1c, MethodCallPart2, MethodCallPart3, MethodCallTail — each returning std::optional<Value>, where nullopt means "not handled here, try the next segment". registerBuiltins, which fills the built-in routine table, is cut the same way: five pieces in the BuiltinsRegister*.cpp files, each calling the next, so the registrations still run in one order.

The critical property, stated in the source and worth repeating: **these are segments, not categories.** The chain is order-sensitive. Later arms deliberately catch what earlier ones decline. An arm belongs where its priority is, not where it reads nicely. MethodCallSegment.h gives MethodCallPart2, MethodCallPart3 and MethodCallTail one include prologue, so they cannot drift apart; MethodCallPart1b and 1c share BuiltinsParts.h with the head.

## 2.2 The library boundary




Everything except main.cpp and Repl.cpp compiles into librakupp_rt.a. The REPL lives in the executable rather than the library on purpose: nothing about an interactive session should be linked into the standalone binaries the compiling modes produce.

## 2.3 What each file is for

**Front end**

| File | Role |
|---|---|
| `Token.h` | the token struct: kind, text, position, `spaceBefore` |
| `Lexer.{h,cpp}` | source text to a flat `vector<Token>` |
| `Ast.h` | every node type, and the `NK` tag enum |
| `Parser.{h,cpp}` | tokens to a `Program`; the live user-operator tables |
| `Pod.{h,cpp}` | the `$=pod` DOM |

### Values

| File | Role |
|---|---|
| `Value.{h,cpp}` | the fat tagged struct, `CowStr`, `ClassInfo`, `Callable` |
| `IStr.h` | the interned-string field |
| `MethodName.h` | `MName`, the packed method name used by the dispatch chain |
| `BigInt.{h,cpp}` | arbitrary-precision integers, base 10^9 |
| `IntOps.h` | portable overflow-checked arithmetic and bit intrinsics |

### Execution

| File | Role |
|---|---|
| `Interpreter.h`, `Interpreter*.cpp` | the tree walk and nearly everything it reaches; `InterpreterCore.cpp` is its hot half |
| `InterpreterParts.h` | what the `Interpreter*.cpp` files share |
| `Builtins.cpp` | the built-ins' helpers and the method chain's head |
| `BuiltinsRegister*.cpp` | the built-in routine table, filled in five pieces |
| `BuiltinsSupply.cpp`, `BuiltinsNqp.cpp` | supplies and the `--exe` built-in natives; the `nqp::` ops |
| `BuiltinsParts.h` | what the `Builtins*.cpp` and `MethodCallPart1b/1c.cpp` files share |
| `MethodCall*.cpp` | the rest of the method chain |
| `BuiltinsShared.h` | helpers the split forced out of file scope |
| `Runtime.{h,cpp}` | the shared entry points, and the big-stack thread |

| File | Role |
|---|---|
| `IOSpec.cpp` | `IO::Spec::*` path algorithms |

### Engines

| File | Role |
|---|---|
| `Regex.{h,cpp}` | regex compilation, the matcher, `GrammarMatcher` |
| `LtmNfa.{h,cpp}` | the declarative-prefix NFA for longest-token matching |
| `Unicode.{h,cpp}` | normalization, grapheme segmentation, collation, properties |
| `unicode_*_gen.cpp` | the generated tables those read |
| `unicode_names.cpp` | character names — the single largest file in the tree |

### Back ends and tooling

| File | Role |
|---|---|
| `Codegen.{h,cpp}` | `--exe`: AST to C++ |
| `AstEmit.cpp` | `--aot`: C++ that rebuilds the AST |
| `AstSerial.{h,cpp}` | the binary AST format behind the precompiled parse |
| `AstDump.cpp` | `--dump-ast` |
| `SlimScan.{h,cpp}` | `--slim`: the feature scan over a parsed program |
| `FeatureGate.cpp` | the `X::Feature::NotBuilt` a cut feature throws |
| `ucd_seam.h` | the accessors the cuttable Unicode tables sit behind |
| `stubs/` | one throwing stand-in per cuttable feature |
| `Lint.{h,cpp}` | `--lint`, static analysis over the parsed tree |
| `Highlight.{h,cpp}` | `--highlight`, parse-aware syntax colouring |
| `Profiler.{h,cpp}` | `--profile`, the routine-level wall-time profiler |
| `Repl.{h,cpp}` | the interactive session |

### Boundaries

| File | Role |
|---|---|
| `Ffi.{h,cpp}` | the libffi backend: loader, ABI probe, type registry |
| `rakupp_ext.h` | the C ABI extension modules compile against |
| `ExtApi.cpp` | the host side of that ABI |
| `Platform.h` | the Windows/POSIX split, in one place |

## 2.4 Reading conventions in the source

Three habits recur, and knowing them saves a lot of confusion.

**Comments explain *why*, and often carry the measurement.** A comment in this tree is rarely a restatement of the code. It is much more often the reason the obvious version was rejected, sometimes with a number attached:

```
// src/Value.h — the CowStr rationale, trimmed
// Value is copied by value everywhere ... so holding a bare std::string
// meant a long string was memcpy'd on each of those. The cost is
// O(length) per OPERATION, which makes any pure-Raku tokenizer O(n^2):
// JSON::Fast spent 13.9 s on a 421 KB document that Rakudo parses in
// 50 ms, and the profile was all copying, not parsing.
```

When this book explains a decision, it is usually expanding a comment like that one.

**A "decided once" field is a fact about the syntax, not a cached result.** Several node types carry a small mutable field that starts at a sentinel and is written on first evaluation:

```
// src/Ast.h
template <typename T> struct DecidedOnce {
    std::atomic<T> v;
    operator T() const { return v.load(std::memory_order_relaxed); }
    DecidedOnce& operator=(T x) {
        v.store(x, std::memory_order_relaxed); return *this;
    }
};
```

The atomic is not for synchronisation. It is there because a node is shared between threads, every writer computes the same idempotent answer, and a plain field would make that a data race that ThreadSanitizer correctly reports. Relaxed atomics make it defined at plain-load cost on the architectures that matter. Chapter 19 is entirely about what these fields hold and, more importantly, what they must never hold.

**`rt*` functions are the compiled backend's vocabulary.** Anything named `rtAdd`, `rtIndexRef`, `rtAttrGet`, `rtCallB` is a runtime entry point that `Codegen` emits calls to. They are declared in `Interpreter.h` and are the contract between the transpiler and the runtime — which is why they are `inline` where the fast path matters. Chapters 26 to 28 are about them.

## 2.5 Building it

```
cmake -S . -B build
cmake --build build -j
```

`CMakeLists.txt` builds `librakupp_rt` from all of `src/` except `main.cpp`, then the `rakupp` executable. The glob is `CONFIGURE_DEPENDS`, but CMake caches it, so re-run the configure step after adding a source file.

The compiler matters more than usual here. Clang produces a binary between 1.2 and 2 times faster than GCC's on this codebase — the method dispatch chain and the tree walk are both inlining-sensitive in ways GCC handles less well — so Clang is what ships and GCC is kept as a portability gate. Link-time optimisation and `-mcpu=native` were both measured and both did nothing.

## 2.6 The test surface

| Where | What |
|---|---|
| Roast | the Raku specification suite, run by `tools/run-roast.raku` |
| `t/run.raku` | the local suite: examples and showcases, byte-compared to golden output |
| `t/regression/` | one file per fixed bug |
| `t/stress/` | concurrency and memory stress, also run under TSan and ASan |
| `tools/perf-guard.raku` | the performance gate, compared against a recorded baseline |
| the showcase interpreters | JavaScript, Perl, Python and Lisp, written in Raku |

The release checklist in `docs/dev/RELEASING.md` gates on all of them. Chapter 40 is about why the performance gate is there and what happens when it is skipped.

# Chapter 3

# Where It Sits in the Taxonomy

There is a question every compiler gets asked before any other: *what kind of compiler is it?* LL or LR, one-pass or multi-pass, interpreter or compiler, bytecode or native. The answer for Raku++ is that the question decomposes — the front end, the regex engine and the back end are classified on three different axes, and only when they are taken apart does the taxonomy say anything useful.

It is worth doing before the mechanisms, because most of the front end's limitations are not independent defects. They are what the box costs. A reader who knows the classification can predict the limitation list in Chapter 6 without being told it.

The one-line answer:

> A hand-written, single-pass, syntax-directed **recursive-descent parser with a Pratt (precedence-climbing) expression core and a dynamically extensible operator table**, feeding an **AST tree-walking interpreter**, with an optional **source-to-source back end** that emits C++.

Nothing in that sentence is generated. There is no grammar file, no parser generator, no state table, and no dependency — which is the same constraint Chapter 1 opened with, seen from the compiler-theory side.

*Single-pass* is the load-bearing word in that sentence, and it is the one with exceptions. The parse proper reads the token vector once; the lexer scans the source once before it, two seams throw the unread tail away and lex it again, and one of those runs the user's own code to do it. They are set out under "The pass structure" below, because a reader who takes the one-liner unqualified will predict the wrong limitation list — which is the opposite of what this chapter is for.

## 3.1 The lexer: separate, complete, run more than once

`Lexer::tokenize()` is a character-level scanner that runs to completion and returns a flat `std::vector<Token>`. Only then is a parser built over it:

```
// src/Parser.cpp
Lexer lx(src);
Parser p(lx.tokenize());
```

The lexer therefore never asks the parser anything. That is worth stating explicitly because the opposite arrangement is so common: C compilers famously need the parser's symbol table to decide whether an identifier is a type name, and the resulting parser-to-lexer feedback loop is known in the trade as the lexer hack. Raku++ has nothing of the sort; by the time the parser exists the lexer has finished.

The *other* direction is not empty, though, and this chapter used to say it was. A parser that reaches a `use` can discard the tokens it has not read yet and lex the remaining source again: `relexForQuoteWords` and `activateSlang` in `src/Parser.cpp` both truncate `toks_` at the cursor and splice on a freshly lexed tail. The lexer is still closed over its input and still answers no questions — it is simply run more than once. What forces that is always the same shape of event: a `use` teaches the *lexer* something, and the lexer is already past.

`tokenize()` also opens with a scan of its own. Before a single token exists it reads the source for `sub q`, `sub s` and `sub tr` declarations (`scanQuoteWordSubs`), because a routine of that name beats the quote form of the same name from its declaration onward, and the lexer has no scopes to ask. It is textual and deliberately narrow — a comment is skipped, quoted text is not code — but it is a pre-scan by any reasonable definition.

Raku's context-sensitivity is real, so the work has to happen somewhere; here it is resolved from the lexer's **own one-token history**. A bare `/` opens a regex in term position and divides in operator position, and `regexContext()` decides by inspecting the previous token alone — after most operators a term is expected, so a regex follows; after a value, a `)`, or a postfix `++`, division does. Chapter 4 gives the rule and the keyword set that patches the identifier case.

Two consequences follow from a lexer that is closed over its input before parsing begins, and both matter later:

**The operator vocabulary is static.** `lexOperator` matches against a hand-ordered table, first match wins, entries arranged longest-first by hand rather than by a

maximal-munch scanner. It tracks no user declarations. An operator spelling it does not know falls through to a single-character token — which is precisely how a user-declared operator reaches the parser at all.

**Rule bodies are not Raku tokens.** A `token`, `rule` or `regex` body is captured whole as one opaque token and re-lexed later by the regex engine (Chapter 20). The regex sub-language never enters the main token stream, which is why Part V can be read as a separate compiler.

## 3.2 The parser: recursive descent with a Pratt core

The parser is the classic hybrid:

| Construct | Technique |
|---|---|
| Statements, declarations, blocks | Recursive descent on the leading token |
| Expressions | Pratt, i.e. precedence climbing |
| Prefix and postfix operators | Interleaved around the primary term |

The split is not a matter of taste. Raku's operator table runs to some two dozen precedence levels, and pure recursive descent spends one mutually recursive function per level to express them — the textbook `expr → term → factor` ladder, with twenty-odd rungs. Precedence climbing collapses the ladder into a single loop driven by sixteen named binding powers, which is why `parseExpr` fits on a page (Chapter 5).

Placing it in the LL/LR hierarchy: it is **top-down, LL family, and neither LL(1) nor any fixed LL(k)**. The honest description is *recursive descent with bounded local backtracking*. The parser walks the token vector once, left to right, and rewinds at exactly four places in ten and a half thousand lines, each a short speculative probe that restores the position on failure. Because it memoizes nothing, it is not a packrat or PEG parser; because it builds no parse forest, it is not GLR or Earley. Ambiguity is resolved where it is met, not afterwards.

## 3.3 The pass structure

The staging is mostly as plain as the one-liner suggests. `BEGIN`, `constant`, `use` and `no` all become AST nodes that the interpreter acts on later; named subs are hoisted, but by the interpreter, which is the reason a sub needs no forward declaration while an

operator does. Nothing is memoized, nothing is revisited, and the AST that comes out is the only representation there will ever be.

Three things qualify that, and together they are the honest answer to *is it one-pass?*

**The lexer pre-scans.** `Lexer::tokenize()` reads the source for quote-word `sub` declarations before it tokenizes, as above. Nothing pre-scans it for *operators* — that is the pre-scan the limitation list further down is about, and the two are easy to conflate.

**Two seams re-lex the tail.** A `use` of a module that declares `sub tr`, or of a slang, rebuilds the token stream from the cursor on. The parse is still a single left-to-right walk; the tape under it was replaced mid-walk.

**A slang runs user code during the parse.** This is the one that changed. `rakuppActivateSlang` (`src/MethodCallPart3.cpp`) builds a scratch `Interpreter` with a compile-time `$*LANG`, *runs the slang module for real*, and keeps it alive alongside the parse; the grammar productions its roles override come back as seams, and the lexer then **runs the slang's own `token`** at the position it would itself have started a number, a value, an identifier, a sigilless variable, a pointy block or a routine declarator. `src/Slang.h` is the enumeration. Until September 2026 this chapter said flatly that no user code runs during the parse; for everything that is not a slang, it still does not.

One further pass sits between the parser and every back end, and it is not part of the parse at all: `DeclCheck` (Chapter 38) asks whether each variable the unit mentions is declared anywhere in its lexical chain, and refuses the program before it starts. It is a gate rather than an analysis — it computes nothing a later stage consumes, beyond the list of `no strict` names the native back end needs in order to emit no local for them.

## 3.4 The part with no textbook box

The central case is the **operator table, mutated while parsing**. Declaring `sub postfix:<!>` registers a new operator with its own precedence and associativity, taken from `is tighter`, `is looser` and `is equiv`, and tokens after that point parse differently from tokens before it.

That single fact puts the front end outside the LL/LR classification entirely. Both presuppose a grammar fixed before parsing starts; no static table can be built for a

grammar that is not fully known until the program has been read. The nearest term in the literature is an **adaptive**, or extensible, grammar.

In practice, top-down parsing with a live operator table is the only tractable way to implement one, which is why every language that lets users declare operators — Raku, Haskell, Prolog, Agda — has a top-down front end. The choice of recursive descent here was not made in spite of Raku's grammar. It was made because of it.

The extension mechanism is deliberately narrow, and Chapter 6 is about exactly how narrow. `use Foo` does not parse the module at compile time: the parser *text-scans* the module source for declarations of `infix:<...>` and its relatives, and registers those names so the importing file can parse them. That is lexical bookkeeping, not execution — the distinction Chapter 32 returns to.

One kind of module is the exception. If the source registers a slang — `use Slangify`, or a direct `define_slang` — bookkeeping cannot answer what it registered, so the module is run, in an interpreter of its own, to find out. That is a second adaptive mechanism sitting beside the operator table, and it is narrower than Rakudo's in a specific way. Rakudo mixes a role into the grammar and every production in it is fair game; here only a production with a seam can be overridden at all, and a slang that reaches past them — one that overrides a host production, or that changes only the actions — is refused by name rather than silently half-applied.

## 3.5 The regex engine, classified separately

Part V is a second compiler inside the first, and it belongs in a different box.

**The matcher is a backtracking recursive-descent walker** over a compiled node tree — the Perl and PCRE lineage, not the Thompson and RE2 lineage. It has to be. Raku regexes carry captures, embedded code blocks, assertions and recursive subrule calls, none of which a pure DFA can express, and a grammar is a class whose methods are regexes. Chapter 21 is the matcher; Chapter 22 is what a grammar adds on top of it.

**A Thompson NFA appears in exactly one role.** Longest-token matching has to answer "how far can each alternation branch's declarative prefix reach?", and the NFA answers it in one linear, execution-free scan of the input. It never matches on behalf of the engine — the real backtracking matcher then runs on the branches it ranked. It is also not yet the default: the shipped ranker probes each branch for its

greedy full-match end, and the NFA ranker is behind an environment variable, for reasons Chapter 23 sets out along with the phase plan for flipping it.

So the engine is a backtracking matcher that borrows one automaton for one question. Describing it as "an NFA engine" would be wrong in both directions.

## 3.6 The back end: there is no IR

Here the classification is unusually short, because a whole layer is missing. There is **no intermediate representation**: no bytecode, no three-address code, no SSA, no control-flow graph, no register allocation, no lowering pass. The AST is the sole representation the whole way through. That is what makes this an *AST interpreter* rather than a compiler in the Dragon-Book sense, for three of its four modes:

| Mode | What it is, taxonomically |
|---|---|
| default | Tree-walking interpreter |
| `--bundle` | Source in the binary, parsed at startup |
| `--aot` | AST rebuilt at startup, still tree-walked |
| `--exe` | Source-to-source compiler — a transpiler to C++ |

Only `--exe` is a compiler proper, and even there the optimizer is **AST-level pattern matching at emit time**: direct-arity calls, inlined `int64` arithmetic, guarded native-int expression lanes. Those are local, peephole-class transformations by any standard classification (Chapter 27). Everything classical — inlining, constant propagation, loop transforms, instruction scheduling, register allocation — is delegated to the host C++ compiler, which is the whole point of emitting C++ instead of machine code. Chapter 26 is about what that delegation buys and what it costs.

The interpreter's own speed work sits in the same place on the map. Node specialisation (Chapter 19) caches a decision on the syntactic shape of a node; it is a fast path on a tree walk, not a compilation step, and no amount of it turns the tree walk into something with an instruction stream.

**Since this chapter was written, one exception has landed, and the paragraph above has to be qualified.** Two opt-in backends compile hot loops *during* the run: `--jit`, through the same C++ emission `--exe` uses, and `--cnp`, by copying machine-code snippets out of the binary and patching them. The second lowers the loop to a flat list of register ops first — an intermediate representation, one loop wide, built and thrown away in about 15 µs, with no pass of any kind run over it. The rest of

the classification stands: the AST is still the sole representation of your *program*, and neither back end is reached unless it is asked for. Chapter 44 is both of them end to end.

## 3.7 What it is not

Stated plainly, since these are the usual guesses:

| Not | Why |
|---|---|
| LR, LALR, SLR | Bottom-up and table-driven; both are incompatible with an operator table that changes mid-parse |
| LL(1) | Several constructs need more than one token of lookahead |
| PEG or packrat | No memoization, no ordered-choice formalism |
| GLR or Earley | No parse forest; ambiguity is settled where it is met |
| Generated | The lexer and parser are hand-written C++ |
| Bytecode VM | No instruction set and no opcode dispatch loop |
| SSA-based | The only IR is `--cnp`'s one-loop op list, and nothing is in SSA form |
| A JIT by default | Nothing is compiled at run time unless `--jit` or `--cnp` is asked for; `--exe` compiles ahead of time |

## 3.8 Compared with three others

| | Front end | Grammar fixed? | Back end |
|---|---|---|---|
| **Raku++** | Recursive descent + Pratt, hand-written | No — live operator table, plus slang seams | AST tree-walk; `--exe` emits C++ |
| **Rakudo** | Self-hosted NQP grammars, code runs at parse time | No — slangs and macros | Bytecode, IR, JIT on MoarVM |
| **CPython** | Generated PEG parser | Yes | AST → bytecode → VM |

| | Front end | Grammar fixed? | Back end |
|---|---|---|---|
| **Clang** | Recursive descent, hand-written | Yes | LLVM IR, SSA, full pipeline |

Two rows of that table are worth a sentence each.

Raku++ shares its *front end* row with a production C++ compiler. Hand-written recursive descent is not a shortcut taken by small projects; it is what industrial compilers for languages with real syntax actually use, because generated parsers are hard to give good errors and impossible to give context-sensitivity.

And it shares its *grammar not fixed* column with Rakudo alone — which is a property of the Raku language rather than a choice either implementation made. Any Raku implementation has to solve it. The two solved it differently, and the difference explains most of what Chapter 6 says is missing here: Rakudo's front end *is* a Raku grammar, so running the program's own code during the parse is its ordinary case. Here it is the exception, bought at six named seams and two modes, and everything outside them parses with no user code running at all.

## 3.9 Honest limitations

Nearly every front-end limitation in this book is downstream of the classification above, and worth reading as a consequence rather than a bug list:

- **Operators must be declared before use.** The only way the parser learns of one is by reaching its declaration, and nothing pre-scans the file *for operators* — the lexer's own pre-scan looks for quote-word subs and nothing else. Subs are exempt only because the interpreter hoists them.
- **No macros, and slangs only where there is a seam.** A macro needs user code to run during the parse *and* to yield the AST that replaces the call; the first half now exists and the second does not. A slang works only at the productions `src/Slang.h` enumerates: the eight self-contained ones run their own tokens, Tuxic runs as a pair of parser modes, and one that reaches past both is refused by name.
- **Block versus hash stays a heuristic.** It follows Raku's documented rules, but a case a backtracking full grammar would settle by trying both can still be decided wrongly here.
- **No whole-program analysis.** `DeclCheck` does walk the tree before execution, but it answers one question and hands the back end nothing to optimize with;

with no IR, an `--exe` optimization still has to be expressible as a local rewrite during emission.

The other side of the ledger is the reason the design holds. One AST serves all five run modes with no lowering step between them, so a language feature is implemented once (Chapter 25). A parse error can point straight at a token, because there is no table-driven state to translate back into source. And there is exactly one implementation of the semantics — `librakupp_rt.a` — shared by the interpreter and by every binary it compiles.

# Part II

# The Front End

# Chapter 4

# The Lexer

Raku's grammar is too context-sensitive for a generated tokenizer, so the lexer is hand-written: a loop that skips whitespace and comments, dispatches on the current character to a handler, and appends one or a few tokens. The output is a flat `std::vector<Token>`.

What makes it interesting is not the loop. It is the set of decisions Raku forces the lexer to make that most languages never have to.

## 4.1 The token

```
// src/Token.h
enum class Tok {
    End, IntLit, NumLit, StrLit, VersionLit, StrInterp, RegexLit,
    SubstLit, QwList, Ident, Var, LParen, RParen, LBrace, RBrace,
    LBracket, RBracket, Semicolon, Comma, FatArrow, Op
};
struct Token {
    Tok kind = Tok::End;
    std::string text;    // identifier / operator spelling / raw literal
    std::string text2;   // s/// replacement; regex adverb flags
    long long ival; double nval;
    int line, col;
    bool spaceBefore = false;
    bool flag = false;   // S/// — the non-mutating substitution
};
```

Two absences are as informative as the contents.

**There is no keyword kind.** `if`, `sub`, `class`, `given` all arrive as `Tok::Ident`. The parser decides, from position, whether a given identifier is acting as a keyword. That is why Raku has no reserved-word list and why `my $if = 1` is legal.

**There is no operator identity.** Every operator arrives as `Tok::Op` carrying only its spelling. The lexer knows nothing about precedence, associativity, or which operators exist beyond a fixed vocabulary of spellings. All of that is the parser's problem, which is precisely what makes user-declared operators possible (Chapter 6).

## 4.2 `spaceBefore`: whitespace is significant

Raku is one of the few languages where a space changes the parse. `f()` is a call; `f ()` is `f` applied to a parenthesised list. `%h<k>` is a subscript; `%h <k>` is not. So every token records whether whitespace preceded it:

```
// src/Lexer.cpp — the tokenize loop
size_t before = pos_;
skipWhitespaceAndComments();
// an unspace isn't whitespace: `"xxxxxx"\.chars` keeps `.chars` tight-postfix
bool spaced = (pos_ > before && pos_ != unspaceEnd_) || before == 0;
// … lex the token …
t.spaceBefore = spaced;
```

The flag is true when skipping advanced the cursor — unless the cursor landed exactly after an *unspace*, a backslash followed by whitespace, which is written to be invisible and so does not count as space — and always for the very first token. The parser then leans on it constantly: to tell a postcircumfix from a list, a call from an application, a postfix operator from a fresh term.

## 4.3 Regex or division?

A bare `/` starts a regex in term position and divides in operator position. The lexer decides by looking at the *previous* token:

```
// src/Lexer.cpp — regexContext(), the Tok::Op case
case Tok::Op:
    return pv.text != "++" && pv.text != "--" &&
           pv.text != ">"  && pv.text != "<"  &&
           pv.text != ">>" && pv.text != "<<" &&
           pv.text != ">=" && pv.text != "<=";
```

After most operators a term is expected, so `/` opens a regex. After a value — a number, a variable, a closing bracket — or after a postfix `++`/`--`, division follows. After an identifier it is a regex only when the word is in a small set of keywords that take an expression (`if`, `while`, `say`, `grep`, `split`, and friends). `//` and `/=` are matched earlier so they lex as operators.

This is a heuristic in the honest sense: it implements Raku's actual rule for the cases the rule covers, and a sufficiently perverse program can still be surprised.

## 4.4 Quote forms and heredocs

`tryQuoteForm` handles the whole `q`/`qq`/`Q` family plus `m`, `rx`, `s`, `tr` and `qw`, with adverbs (`:w`, `:to`, `:!c`, ...) and **arbitrary delimiters** — any bracket pair, or any non-bracket character. Whether the result interpolates is decided here and encoded in the token kind: `qq` produces `Tok::StrInterp`, `q` produces `Tok::StrLit`. Adverbs that flip a `Q` into interpolating emit a feature sentinel the parser reads later.

Heredocs cannot be captured inline, because the body is on *later* lines. The lexer emits an empty-bodied token immediately and defers:

```
// src/Lexer.cpp — tryQuoteForm, on a :to adverb
heredocMarker_ = raw;                // the terminator, e.g. END
heredocInterp_ = (w == "qq");
out = make(heredocInterp_ ? Tok::StrInterp : Tok::StrLit, "");
```

The pending heredoc — marker, token index, interpolation flag — is queued. When the lexer reaches the newline that ends the current line, `processHeredocs` reads lines until the marker, dedents them, and **back-patches the token's text**. Because interpolation was already fixed by the token kind, the patched body needs no further decision.

Several heredocs can be opened on one line; the queue keeps them in order, each with its own interpolation-feature string.

## 4.5 Identifiers, sigils, twigils, Unicode

`lexIdentOrVar` reads a sigil (`$ @ % &`), an optional twigil (`*` dynamic, `.`/`!` attribute, `^` placeholder, `?` compile-time), then the name — which may contain `-` or `'`, may

be package-qualified with `::`, may be all digits (`$0`, `$^1`), and may be a Unicode identifier. The whole thing becomes one `Tok::Var`.

The rule about `-` and `'` inside a name is small and was a genuine bug, so it lives in the header where everyone who needs it can see it:

```
// src/Lexer.h
inline bool rakuIdentStart(char c) {
    return std::isalpha((unsigned char)c) || c == '_';
}
inline bool rakuIdentJoins(char sep, char next) {
    return (sep == '-' || sep == '\'') && rakuIdentStart(next);
}
```

A `-` continues a name only when a *letter* follows, never a digit. So `elems-1` is `elems - 1` and `$x-1` is `$x - 1`.

That rule has to be answered in two places: the lexer, and the string interpolation scanner in the parser. They disagreed. The interpolation scanner tested `isalnum`, glued the digit into the name, and handed `"$x-1"` to the expression parser — which re-split it correctly and then interpolated the *arithmetic result*. So `my $x = 5; say "$x-1"` printed `4` where Rakudo prints `5-1`. Six sites implemented the rule three different ways; the fix was to write it once, in a header, and delete the other five.

Superscripts are folded here too: a run like `x³` emits a synthetic tight `**` operator and an `IntLit 3`, so it parses as `x ** 3`.

## 4.6 Operators are a fixed, hand-ordered vocabulary

```
// src/Lexer.cpp — lexOperator
for (const char* op : ops) {
    std::string s(op);
    bool ok = true;
    for (size_t k = 0; k < s.size(); k++)
        if (peek(k) != s[k]) { ok = false; break; }
    if (ok) { /* consume s, return Tok::Op */ }
}
// unmatched: a single-character Op
```

This is not a longest-match scanner. It is a manually ordered table where longer entries are placed before their prefixes, and the first full match wins. Adding a built-

in operator means inserting it at the right position — a real maintenance hazard, and named as one.

The fall-through matters: anything unmatched becomes a **single-character `Tok::Op`**, so a novel user-defined operator arrives as *something* the parser can pick up.

For a while that was the whole answer, and it was not enough. A spelling like `sub infix:<%%%>` gets lexed a character at a time, and the built-in table swallows a prefix of it first — `%%%` came out as `%%` then `%`, so `5 %%% 2` divided by an empty hash. So the lexer's constructor now pre-scans the file for its own symbolic operator declarations and tries them, longest first, ahead of the table:

```
// src/Lexer.cpp — the constructor
for (const char* cat : {"infix:<", "prefix:<", "postfix:<"}) { … }
```

Only *this* file's declarations are visible that way; an imported symbolic operator still needs its module's own parse, which is a real limitation and one worth knowing before writing a module that exports one.

The vocabulary also covers Unicode aliases (≤ for `<=`), hyper metaoperators (`»op«`), and the set operators (`(elem)`, ∈). That part is static — but "the lexer tracks no user declarations" is no longer true of it as a whole.

## 4.7 Rule bodies are not tokenized

One construct is deliberately opaque. A `token`/`rule`/`regex NAME { … }` declaration is not lexed as Raku:

```
// src/Lexer.h
bool tryRuleDecl(std::vector<Token>& out, bool spaced);
```

`tryRuleDecl` captures the balanced-brace body **raw** and emits it as a single `Tok::RegexLit`. Regex syntax is a different sub-language — `<[a..z]>`, `**`, `%%`, `<?{ … }>` mean nothing to the Raku tokenizer — so keeping it out of the token stream avoids teaching the lexer two grammars. The regex engine re-lexes the captured text later, in Chapter 20.

The same applies to a bare `/…/` literal and the `s///` family: the pattern text travels in `Token::text` and the replacement in `Token::text2`, both unparsed.

## 4.8 What else the lexer carries out

Beyond tokens, `Lexer` produces three side channels the parser and the runtime need:

| Channel | Contents |
|---|---|
| `finishData()` | the text after `=finish`, for `$=finish` |
| `podData()` | rendered `=begin pod` content, for `--doc` |
| `declPod_` / `leadPod_` | `#=` and `#\|` declarator documentation, keyed by line |

Declarator pod is keyed by line, and the parser attaches it by where the lines fall among the tokens. A `#|` documents the next declaration below it, however far: blank lines and code that declares nothing leave it waiting, and the first declarator keyword or block after it takes it. `leadingPodAt(tok)` walks the tokens up from a declaration to that boundary, and remembers which declaration took each line, because a parameter has no keyword to stop the walk of the one after it. A `#=` documents the declaration beside it (the last one, when several share the line) or the one just above when it stands alone; that is `trailingPodAt(tok)`. The parser also records which lines a *parameter* has already claimed — otherwise the parameter documentation of a multi-line `sub MAIN` signature would be picked up a second time as the routine's own usage text.

## 4.9 Errors

A construct that runs off the end of the file gets a dedicated diagnostic:

```
// src/Lexer.h
[[noreturn]] void runawayQuote(const char* construct,
                               const char* finalDelim, int startLine) const;
[[noreturn]] void runawayTerm(const std::string& close,
                              const std::string& open, int startLine) const;
```

Two spellings, because Rakudo has two: the bare quote forms name the *construct*, the bracketed `q/rx/`comment forms name the *terminator*. Both quote the line the construct *started* on, which is the only coordinate still meaningful once the scan has eaten the rest of the file.

An `atEof` flag rides along on the resulting error. Only the REPL reads it, to tell "give me a continuation line" from "this is a syntax error" — which is how a half-typed

string literal at an interactive prompt asks for more input instead of failing (Chapter 39).

# Chapter 5

# The Parser

`Parser::parseProgram()` walks the token vector once, left to right, and produces a `Program`. It is recursive descent for statements and precedence climbing for expressions — the classic pairing, chosen because it is the one that survives contact with an irregular grammar.

It is a **single forward pass**, and on every path but one it executes nothing. The exception is a `use` that activates a slang, which runs that module and then re-lexes what is left of the unit (Chapter 3).

## 5.1 Statement dispatch

`parseStatementImpl` is a keyword ladder that only engages when the leading token is a `Tok::Ident`. It matches the identifier's *text* and delegates:

| Leading word | Goes to |
|---|---|
| `if` / `unless` | `parseIf` |
| `while` / `until` | `parseWhile` |
| `for` | `parseFor` |
| `loop` / `repeat` | the C-style and post-condition loops |
| `sub` / `multi` / `proto` / `method` / `submethod` | `parseSub` |
| `class` / `role` / `grammar` / `module` / `package` | `parseClass` |
| `my` / `our` / `state` / `has` / `constant` | strip the scope word, re-dispatch |
| `enum`, `subset` | `parseEnum`, `parseSubset` |
| `given` / `when` / `default` | the topicalisers |
| `use` / `no` / `need` | `UseStmt` |

| Leading word | Goes to |
|---|---|
| a phaser name | a `Block` tagged with the phaser |

Everything not caught is a bare expression statement:

```
// src/Parser.cpp — the parseStatementImpl fallthrough
auto es = std::make_unique<ExprStmt>();
es->e = parseExpression();
return applyModifiers(std::move(es));
```

Because keywords are matched by text and only in leading position, an `if` anywhere else is an ordinary identifier.

## 5.2 Statement modifiers

A trailing `if`, `unless`, `while`, `until`, `for`, `given` or `with` turns the statement into the corresponding control node and sets a `modifier` flag. `applyModifiers` recurses, so modifiers chain:

```
// src/Parser.cpp — applyModifiers, the `for` case, abridged
if (isIdent("for")) { advance();
    auto fs = std::make_unique<ForStmt>();
    fs->list = parseExpression();
    fs->modifier = true;      // no implicit block
    fs->body = wrapStmt(std::move(s));
    return applyModifiers(std::move(fs));
}
```

The flag is not cosmetic. `$x for @a` has no implicit block, so a `my` declaration inside `$x` is *not* scoped away when the loop ends — unlike `for @a { … }`. Every looping and topicalising node carries the same flag for the same reason.

## 5.3 Block or hash?

`{ … }` is Raku's classic ambiguity. The parser uses a small `looksHash` heuristic: it is a hash if the first token is a pair key followed immediately by `=>`, or a colon-pair (`:name`, `:$v`, `:!flag`), or a `%`-variable followed by `,` or `}`. The empty case is decided by **position**:

```
// src/Parser.cpp — a statement-leading '{' is a BLOCK unless looksHash
// src/Parser.cpp — parsePrimary, expression position:
if (a.kind == Tok::RBrace) isHash = true;    // {} in value context: a Hash
```

So a bare `{}` statement is an empty block, and `{}` where a value is expected is an empty hash. `{ $_ * 2 }` — first token a `$`-variable, no `=>` — is a block.

There is a second, subtler brace rule. A `}` that closed a **block** at the end of a line ends the statement, so an infix on the next line starts a new statement rather than continuing the expression:

```
($r, $g, $b) = (...).map: { ... }      # the statement ends here
%(r => $r, ...)                        # a new statement, not `} % (...)`
```

A subscript's `}` does not count, because `%h{'a'}` followed by a newline and `+ 3` really is a continuation. That is why the parser records the token *index* of the last block-closing brace, `lastBlockClose_`, rather than testing the token kind.

## 5.4 Expressions: the Pratt core

```
// src/Parser.cpp
ExprPtr Parser::parseExpr(int minbp) {
    ExprPtr lhs = parsePrefix();
    for (;;) {
        InfixInfo in = classifyInfix(cur());
        if (!in.valid || in.lbp < minbp) break;
        advance();
        int nextMin = in.rightAssoc ? in.lbp : in.lbp + 1;
        ExprPtr rhs = parseExpr(nextMin);
        // ... build a Binary / Assign / Range / Pair ...
    }
    return lhs;
}
```

Read a prefix term; then loop, consuming any infix whose left binding power is at least `minbp`, recursing for the right-hand side with a bound derived from the operator's precedence and associativity. For a left-associative operator the recursive bound is `lbp + 1`, which is what stops it re-associating.

Binding powers are a named enum, higher meaning tighter, and **spaced by ten**:

```
// src/Parser.cpp
enum {
```

```
    BP_OR = 10, BP_AND = 20, BP_ZIP = 25, BP_COMMA = 30, BP_ASSIGN = 40,
    BP_TERNARY = 50, BP_ORELSE = 60, BP_ANDTHEN = 70, BP_COMPARE = 80,
    BP_RANGE = 90, BP_CONCAT = 100, BP_REPLICATE = 110, BP_ADD = 120,
    BP_MUL = 130, BP_POW = 140, BP_PREFIX = 150
};
```

The gaps are the whole point, and the next chapter spends them.

Prefix operators (`- ! ~ + ? ++ --`) are handled in `parsePrefix`; postfix ones — `++`, `--`, method calls, subscripts, `.()` — in `parsePostfix`. The two interleave around the primary term, so `-$obj.foo[0]++` composes in the right order without a special case.

## 5.5 Declarations

The declaration parsers do the bulk of the work, because Raku declarations carry a great deal of information that only matters at run time. None of it is *checked* during parsing; it is recorded as data for the binder and the dispatcher.

**`parseSub`** reads the name (or an operator declaration, Chapter 6), the signature, a trait loop, an optional return type from `-->` / `of` / `returns`, and the body. It also handles `multi` and `proto`, `method`/`submethod`, alternate signatures `(a) | (b)` sharing one body, and the immediate-call form `sub f($n) {...}(1)`.

The trait loop is worth a note. Built-in traits (`is export`, `is native(...)`, `is rw`, the precedence traits) are consumed into dedicated fields; anything else is captured as a `SubTraitSpec` — a name and an unevaluated argument expression — and dispatched at run time to a user-defined `multi trait_mod:<is>`. That is how a module's `is json-name('x')` reaches its own handler without the parser knowing anything about it.

**`parseSignature`** builds a `std::vector<Param>`. One `Param` carries a remarkable number of flags, all of them from Chapter 7's struct: positional or named, optional or required, the slurpy kind (`*@` flattening, `**@` non-flattening, `+@` single-arg rule), a default expression, a type constraint, a `:D`/`:U` smiley, a `where` clause, a coercion type `Int(Str)`, `is rw` / `is copy` / `is raw`, and a nested sub-signature for destructuring parameters like `[$a, $b]`.

**`parseClass`** covers `class`, `role`, `grammar`, `module` and `package` in one function. It parses `is` and `does` parents, then a body of `has` attributes, `method`/`submethod`/`multi`

declarations, and — for grammars — `token`/`rule`/`regex` rules, whose pattern is the opaque `Tok::RegexLit` the lexer captured.

**`parseEnum`** and **`parseSubset`** leave their interesting parts as expressions: the enum's value list and the subset's `where` clause are evaluated by the interpreter, not the parser.

## 5.6 String interpolation is parsed, not concatenated

A `"...$x...{ code }..."` literal goes through `parseInterpString(raw)`, which builds an `InterpStr` node whose `parts` alternate literal `StrLit` chunks with parsed sub-expressions. It recognises `$x`, `@a[...]`, `%h{...}`, the capture variables `$/`, `$0`, `$<name>`, backslash escapes including `\x[...]` and `\c[NAME]`, and `{ EXPR }` blocks.

There is a commit rule for method chains that matches Raku's: `"$obj.foo"` interpolates the `.foo` only if a call or a subscript follows it; otherwise the `.foo` stays literal text.

Embedded fragments are parsed by a fresh lexer and parser, which **inherits the program's user-declared operators**:

```
// src/Parser.cpp — parseEmbeddedExpr
sub.userInfix_ = userInfix_;   sub.userInfixRight_ = userInfixRight_;
sub.userPrefix_ = userPrefix_; sub.userPostfix_ = userPostfix_;
sub.userCircumfix_ = userCircumfix_;
sub.userPostcircumfix_ = userPostcircumfix_;
```

so `"{ 5! }"` sees the program's own `postfix:<!>`.

## 5.7 Compile time: what the parser runs, and what it does not

This is a real divergence from Rakudo and it shapes what Raku++ can support.

- **Phasers** become `Block` statements tagged with a phaser name. `BEGIN` is not executed during parsing; the interpreter schedules it after the parse.
- **`constant`** is not folded. It becomes a declaring `VarExpr` that the interpreter binds.
- **`use` and `no`** become `UseStmt` nodes. No ordinary module is loaded, and even `no strict` is handled at run time rather than by toggling a parser mode. There are two exceptions, and both have to be. `use v6.X` sets `Parser::langRev_` as it

is parsed, because a language revision changes what the *rest of the file* means. `langRev_ >= 2` is what makes prefix `//` a defined-check rather than an empty regex, and what the module search consults for 6.e resolution rules. A pragma that decides how the following tokens parse cannot wait for run time. And `use` of a module that registers a *slang* does load and run it, in a scratch interpreter of its own, because there is no way to learn what a slang registered without running the code that registers it; the rest of the unit is then re-lexed through the tokens it handed back (Chapter 3).

- **Named subs are hoisted** — but by the *interpreter*, not the parser. `hoistSubs` pre-registers every named `SubDecl` in a scope before running its statements, which is why subs need not be declared before use.

Apart from a slang, the one parse-time side effect in the entire front end is registering a user-declared operator, which is lexical bookkeeping rather than execution. `use Foo` participates in exactly that much: `scanModuleOps` finds the module's source and *text-scans* it for operator declarations, so the rest of the importing file parses. It does not lex or parse the module (Chapter 32).

## 5.8 Errors

```
// src/Parser.h
struct ParseError : std::runtime_error {
    int line;
    std::string exType;    // X::Parameter::Twigil, … ("" = generic)
    std::vector<std::pair<std::string, std::string>> exAttrs;
    bool atEof = false;
};
```

A parse error carries a line, and optionally a *typed* diagnostic — an exception class name plus attributes — so that compile-time failures Roast checks by class can be reported as that class rather than as a generic message. The driver prints `===SORRY!===` and exits 1, matching Rakudo's compile-error shape. `atEof` is the REPL's continuation signal, as in the previous chapter.

`checkRedeclarations` runs after the parse over each scope's statement list and reports duplicate subs and types in the same scope — one of the few checks that happens before anything runs.

## 5.9 Honest limitations

- **Single pass, declared before use, for operators.** A custom operator used above its declaration will not parse. Subs are hoisted at run time; operator tables are populated textually.
- **A purely symbolic user infix may not be picked up at the use site.** The infix use-site branch keys on a `Tok::Ident`, so word-form infixes (`4 avg 6`) work; prefix, postfix and circumfix handle symbols fine.
- **The operator vocabulary in the lexer is hand-ordered**, so "longest match" is really "first match in a manually ordered table".
- **The block/hash rule is a heuristic.** It implements Raku's documented rules, but pathological cases a backtracking grammar would resolve can still surprise it.
- **No compile-time execution on the ordinary path**, which is why a `BEGIN` block cannot influence how later source parses, and why macros are out of scope. A slang is the exception, and it is a narrow one: `use` of a module that registers a slang runs that module in an interpreter of its own and re-lexes the rest of the unit through the tokens it registered (Chapter 3).

# Chapter 6

# User-Defined Operators in a Single Pass

Raku lets a program add operators to the language it is written in:

```
sub postfix:<!>($n) { [*] 1..$n }
say 5!;                                  # 120

sub infix:<avg>($a, $b) is tighter(&infix:<+>) { ($a + $b) / 2 }
say 4 avg 6 + 10;                        # 15 — (4 avg 6) + 10

sub circumfix:<⟦ ⟧>(*@x) { @x.sum }
say ⟦1, 2, 3⟧;                           # 6
```

That looks like it needs a parser that can rewrite its own grammar. It does not. It needs a parser that keeps a mutable table and reads the file exactly once.

## 6.1 The tables

```
// src/Parser.h — mutated during the parse, consulted at every operator site
std::map<std::string, int> userInfix_;    // name → left binding power
std::set<std::string> userInfixRight_;    // declared `is assoc<right>`
std::set<std::string> userPrefix_, userPostfix_;
std::map<std::string, std::string> userCircumfix_, userPostcircumfix_;
```

Five containers, all keyed by the operator's spelling. `userInfix_` maps to a binding power so a user infix can sit at any level; the bracket maps go from opening delimiter to closing delimiter.

## 6.2 Registration happens mid-parse

When `parseSub` reads a sub whose name is one of the operator categories followed by a colon, it pulls the operator's spelling out of the `<...>` and writes it into the tables **on the spot**:

```
// src/Parser.cpp — parseSub, operator declaration
if ((s->name == "infix" || s->name == "prefix" || s->name == "postfix" ||
     s->name == "circumfix" || s->name == "postcircumfix") && isOp(":")) {
    std::string cat = s->name; advance();               // ':'
    std::vector<std::string> w;
    if (isOp("<")) { advance(); w = readAngleWords(">"); }
    std::string opname = w.empty() ? "" : w[0];
    s->name = cat + ":<" + opname + ">";
    if (cat == "infix") { userInfix_[opname] = BP_ADD; declInfix = opname; }
    else if (cat == "prefix")  userPrefix_.insert(opname);
    else if (cat == "postfix") userPostfix_.insert(opname);
}
```

From that token onward, the operator is live. Everything *later* in the file sees it; everything earlier does not. That asymmetry is not a limitation invented here — it is Raku's own rule, and it falls out of the single pass for free.

The declaration is otherwise an ordinary sub. Its name becomes the string `"postfix:<!>"`, and subs live in the environment under a `&`-prefixed key, so at run time it is found exactly like any other routine.

## 6.3 How `5!` parses

Take `sub postfix:<!>($n) { [*] 1..$n }; say 5!;`.

The declaration runs the branch above and does `userPostfix_.insert("!")`. Later, parsing `5!`: `parseExpr` calls `parsePrefix`, which calls `parsePostfix(parsePrimary())`. `parsePrimary` returns the `5`. `parsePostfix` loops and reaches the user-postfix arm:

```
// src/Parser.cpp — parsePostfix
} else if ((cur().kind == Tok::Op ||
            (cur().kind == Tok::Ident && !cur().spaceBefore)) &&
```

```
            userPostfix_.count(cur().text)) {
    std::string opname = advance().text;
    auto call = std::make_unique<Call>();
    call->name = "postfix:<" + opname + ">";
    call->args.push_back(std::move(base));
    base = std::move(call);
}
```

`5!` becomes a `Call` node named `postfix:<!>` with one argument. The runtime never learns that an operator was involved; it dispatches a named call. Infixes work the same way through a branch at the top of the `parseExpr` loop, building a two-argument `Call` named `infix:<op>` and honouring the recorded binding power and associativity.

That is the whole trick, and it is worth stating plainly: **a user-defined operator is a parse-time spelling for an ordinary sub call.** Every mechanism that works on subs — closures, multi dispatch, `&infix:<avg>` as a value, passing one to `.reduce` — works on operators without further effort.

## 6.4 Precedence traits and the gaps of ten

A user infix registers at `BP_ADD` by default. A precedence trait resolves the referenced operator's level and slots the new one five away:

```
// src/Parser.cpp — parseSub, `is tighter/looser/equiv(&infix:<+>)`
int refBp = infixBpOf(refOp);
userInfix_[declInfix] = trait == "equiv"   ? refBp
                      : trait == "tighter" ? refBp + 5
                      :                      refBp - 5;
if (kind == "right") userInfixRight_.insert(declInfix);
```

This is what the by-ten spacing in the binding-power enum was reserved for. `is tighter(&infix:<+>)` yields 125: tighter than `+` at 120, looser than `*` at 130. `infixBpOf` resolves the referenced operator whether it is built in or itself user-declared, so traits chain.

The obvious question is what happens when someone declares eight operators each tighter than the last. The answer is that they collide at the same level after the gap is exhausted — the levels are integers with a fixed spacing, not a rational ordering. It has not come up in practice, and the alternative (a real partial order rebuilt on each declaration) has not been worth building.

## 6.5 Custom brackets

A `circumfix:<「 」>` or `postcircumfix` declaration registers an open-to-close mapping. `parsePrimary` consults `userCircumfix_` when it meets an unrecognised opening token, and `parsePostfix` consults `userPostcircumfix_` after a term. The token classifiers that decide "does this start a term?" and "does this start a list-operator argument?" consult the tables too:

```
// src/Parser.cpp — startsTermToken / startsListopArg, the Tok::Op case ends:
userPrefix_.count(t.text) || userCircumfix_.count(t.text);
```

so a custom bracket can open an argument to a list operator, not just a standalone term.

One guard is needed: `pcfxClose_` holds the active postcircumfix's closing bracket while its content is parsed, so a bracket pair spelled with the same character on both sides does not re-open itself inside its own contents.

## 6.6 Operators are lexically scoped, and roll back

A declaration inside a block must not leak out of it. If it did, a `sub postfix:<!!>` in some helper block would eat every later ternary's `!!`.

So every registration is logged, and `parseBlock` rewinds to its entry mark:

```
// src/Parser.h
struct OpUndo {
    char table; std::string name; bool existed; int oldBp; std::string oldClose;
};
std::vector<OpUndo> opUndo_;

void opRollback(size_t mark) {
    while (opUndo_.size() > mark) {
        OpUndo& u = opUndo_.back();
        switch (u.table) {
            case 'i': if (u.existed) userInfix_[u.name] = u.oldBp;
                      else userInfix_.erase(u.name); break;
            case 'p': if (!u.existed) userPrefix_.erase(u.name); break;
            // … 'P' postfix, 'r' right-assoc, 'c'/'C' the bracket maps …
        }
        opUndo_.pop_back();
    }
}
```

The undo record keeps the *previous* value, not just the fact of insertion, so a block that shadows an outer operator at a different precedence restores the outer one on exit rather than deleting it.

This is an undo log rather than a stack of scopes because registration is scattered across `parseSub` and the trait handling, and a single mark-and-rewind call at one place in `parseBlock` is much harder to get wrong than a matched push and pop at each registration site.

## 6.7 Two escape hatches

The single-pass rule has exactly two ways out *for operators*, and both are about another parse seeing the current one's tables. (`use` has an escape of its own — it can rebuild the token stream from the cursor onward — but that is the lexer's business rather than the operator table's, and Chapter 3 has it.)

**`EVAL`.** A snippet is parsed by a fresh `Parser`, which would know nothing about the enclosing program's operators. So the evaluator pre-seeds them:

```
// src/Parser.h
void declareUserOp(const std::string& kind, const std::string& name) {
    if (kind == "infix") userInfix_[name] = 120;
    else if (kind == "prefix")  userPrefix_.insert(name);
    else if (kind == "postfix") userPostfix_.insert(name);
}
```

which is why this works:

```
use MONKEY-SEE-NO-EVAL;
sub infix:<z>($a, $b) { $a * 10 + $b }
say EVAL("40 z 2");                    # 402
```

**String interpolation**, shown at the end of the previous chapter, copies the live tables wholesale into the sub-parser.

## 6.8 `use Foo` and imported operators

A module that declares an operator changes how the *importing file* parses. Nothing else about a module does. So `use Foo` triggers exactly one compile-time action, and it is not a parse:

```
// src/Parser.h
void scanModuleOps(const std::string& module);
void scanOpsIn(const std::string& src, const std::string& srcPath);
```

`scanModuleOps` resolves the module's *source file* through the same search path the loader uses and **text-scans** it — a substring search over raw bytes, no lexing, no parsing — for `sub`, `multi`, `proto` and `only` declarations of the five operator categories, registering what it finds.

Two consequences worth knowing:

- An operator spelled only in ASCII operator characters is **skipped**. A `multi infix:<*>(Color, Real)` is almost always extending a built-in, and registering it as a user operator would give it the default precedence and silently reshape every expression in the importing file.
- The scan can in principle be fooled by an operator declaration inside a string or a comment. Nothing has tripped on it yet.

Every source the scan read is recorded in `opScanned_` as a (path, content) pair. That is not for the parse; it is for the cache. A file's parse is only valid while the modules it scanned still declare what they declared — an imported module that gains or loses an operator changes how *this* file parses, without this file changing at all. Chapter 30 uses that list as a cache key.

## 6.9 The line this draws

Everything above is table manipulation: cheap, local, and reversible. That is why all six operator categories work, with precedence and associativity, without the parser running a line of the user's code to make them.

The features on the other side of the line — `macro`, `quasi`, slangs — need the parser to *execute user code mid-parse and then rewrite its own grammar with the result*. `macro` and `quasi` are still not implemented, and the reason is structural rather than a matter of effort: the second half of that sentence, substituting a user-built AST for the call site, has nowhere to happen in a parser with no compile-time evaluator.

Slangs crossed the line in September 2026, and it is worth being exact about how far. `use` of a module that registers one runs the module in a scratch interpreter with a compile-time `$*LANG`, collects the grammar productions its roles override, and re-lexes the remainder of the unit with the slang's own `token` running wherever the built-in lexer would have started one of those productions. That is user code

executing mid-parse, and the first half of the sentence above no longer describes this parser.

What it is still not is a grammar rewrite. Only the productions `src/Slang.h` enumerates — number, value, identifier, sigilless variable, pointy block, routine declarator, plus two parser modes — have a seam to hang a token on, and a slang that overrides a host production, or that changes only the actions, is refused by name rather than half-applied. Rakudo mixes a role into the grammar and every production in it is fair game; that difference is the whole of the gap that remains. Chapter 3 places the mechanism in the taxonomy, and `docs/dev/plans/SLANG-PLAN.md` is the full account.

`RakuAST` was on that list until the line was drawn properly. Asking for a program's syntax tree does not require running user code mid-parse; it requires being able to *describe* the parse that already happened. So it is a view over the existing tree, and it is implemented. `use L10N::DE;` predates the seams and never needed them: what that particular slang carries is a table of keyword spellings, and renaming a token is not rewriting a grammar.

There is a compensation. Because the language Raku++ accepts stays static enough to know in full at build time, the whole program can be compiled ahead of time — which is what the `--aot` and `--exe` modes exploit. The one remaining genuinely dynamic construct, a runtime-constructed grammar, is exactly what those modes fall back to bundling.

# Chapter 7

# The AST

The parser's output is a `Program`: a vector of statement pointers. Everything downstream — the interpreter, the two compiling back ends, the linter, the highlighter, the serialiser — consumes that one tree.

```
// src/Ast.h
struct Node { NK kind; int line; virtual ~Node() = default; };
struct Expr : Node { using Node::Node; };
struct Stmt : Node { using Node::Node; std::string label; };
using ExprPtr = std::unique_ptr<Expr>;
using StmtPtr = std::unique_ptr<Stmt>;
struct Program { std::vector<StmtPtr> stmts; };
```

## 7.1 A class hierarchy here, a fat struct there

The runtime's `Value` is a single struct with a tag and a field for every payload (Chapter 8). The AST is the opposite: a real C++ class hierarchy, one `struct` per node type, each with exactly the fields it needs. Both choices are right, for opposite reasons.

An AST node is **built once, lives for the whole run, and is visited by a `switch` on its tag**. Nodes are few, long-lived, and heterogeneous — some carry two pointers, `ClassDecl` carries seventeen fields including four vectors. A fat struct there would waste far more than it saved, and the virtual destructor costs nothing that matters when construction happens once.

A `Value`, by contrast, is created and destroyed millions of times per second and must be trivially copyable into vectors. Same program, opposite trade.

The `NK` enum tags every node so both the interpreter and the code generator can `switch` rather than dispatch virtually:

```
// src/Ast.h
enum class NK {
    IntLit, NumLit, StrLit, InterpStr, BoolLit, VarExpr, ListExpr,
    Assign, Binary, Unary, Call, MethodCall, Index, Ternary, Range,
    Pair, BlockExpr, ArrayLit, HashLit, NameTerm, RegexLit, SubstLit,
    ChainExpr, SymbolicRef, AllomorphLit, NqpOp,
    ExprStmt, VarDecl, SubDecl, IfStmt, WhileStmt, ForStmt, LoopStmt,
    Block, ReturnStmt, LastStmt, NextStmt, RedoStmt, UseStmt, EmptyStmt,
    GivenStmt, WhenStmt, RepeatStmt, Whatever, ClassDecl, SelfTerm,
    EnumDecl, NamedRegexDecl, SubsetDecl,
};
```

Forty-nine kinds for a language of Raku's size is small, and the reason is that most of Raku's surface syntax lowers into a handful of these. A hyper operator, a metaoperator, a junction constructor and a set operation are all `Binary` nodes with a different `op` string. A `given`/`when` chain is `GivenStmt` plus `WhenStmt`. Anything spelled as an operator, including every user-declared one, is a `Call`.

## 7.2 The nodes that carry the most

Three node types hold most of the language's declarative weight.

**`Param`** is not a `Node` at all — it is a plain struct, because signatures are lists rather than trees — and it is the densest thing in the header:

```
// src/Ast.h — Param, abridged
struct Param {
    std::string name;         // includes the sigil; empty for anonymous
    char sigil = '$';
    std::string type;         // constraint name; "" = unconstrained
    ExprPtr whereExpr;        // `where` clause
    ExprPtr litVal;           // literal parameter: MAIN('population')
    ExprPtr defaultVal;
    std::string namedKey;     // external name for :name($var)
    char slurpyKind = 0;      // 'f' = *@, 'n' = **@, '1' = +@
    bool named, slurpy, optional, required, invocant;
    int defConstraint;        // 0 none, 1 = :D, 2 = :U
```

```
    bool coerce;              // Int(Str)
    bool isRw, isCopy, isRaw;
    std::shared_ptr<std::vector<Param>> subSig;   // destructuring
    // …plus decided-once caches, below
};
```

None of that is validated at parse time. It is data for the binder (Chapter 14) and the multi-dispatcher (Chapter 16).

**`SubDecl`** carries the routine: name, parameters, alternate signatures sharing one body, the body, non-built-in traits kept as unevaluated expressions, the `multi`/`proto`/`method`/`submethod`/private flags, export and `our` scoping, the declared return type, declarator pod, and the four fields that describe an `is native` declaration.

**`ClassDecl`** covers `class`, `role`, `grammar`, `module` and `package` at once: parents and roles, attributes, methods, grammar rules, the `parameterized` flag for `role R[T]`, the `:ver`/`:auth`/`:api` adverbs both as strings *and* as expressions (because `module Zef:ver($?DISTRIBUTION.meta<version>)` is legal and must be evaluated when the declaration runs), the `is repr(…)` layout for NativeCall, and a statement body for the package forms.

## 7.3 Fields that are answers about the syntax

Several nodes carry a small mutable field that the parser leaves undecided and the first evaluation fills in:

```
// src/Ast.h — Binary
mutable DecidedOnce<signed char> simpleOp{-1};
mutable DecidedOnce<signed char> fastShape{-1};
mutable PublishedOnce<const void*> litVal{nullptr};
```

```
// src/Ast.h — Index
mutable DecidedOnce<signed char> fastShape{-1};
mutable DecidedOnce<long long> litIdx{0};
```

and similarly:

```
Block::hoistNeed          ForStmt::hasStateCache
NumLit::ratCache          Param::sigSimple, natSpec, typeKnown
Callable::arityShape, catchScan
```

**Two templates, and the difference between them is a real bug.** `DecidedOnce` is fine when the cached thing *is* the value: two threads compute the same `signed char`, either store wins, and a reader that sees either is correct. A *pointer* is different. The writer does two things — build the payload, then publish the pointer — and under relaxed ordering a reader may see the pointer without seeing the bytes it points at. On x86 the store buffer usually hides that; arm64 is weakly ordered and does not, and this project ships arm64 binaries. ThreadSanitizer reported it as a read of a `Value`'s type tag racing that `Value`'s constructor.

So the pointer-valued fields — `Binary::litVal`, `NumLit::ratCache`, `Callable::arityShape` and `catchScan` — are `PublishedOnce`: release on publish, acquire on read, and the publish is a compare-exchange, so a racing double-build has one winner and the loser frees its copy instead of leaking it. The template deliberately has the same drop-in shape as `DecidedOnce`, so a field can be upgraded without touching a use site; only the ordering changes.

One field in that list is now vestigial rather than decided: `StrLit::nfcDone` is permanently `true` and kept only so the precomp cache format is unchanged. Normalization used to happen lazily on first evaluation, in place — and the AST is shared by every thread, so it moved into the constructor.

What they hold is a **fact about the program text**, not a cached result. "Is this binary's left child a plain lexical and its right child a scalar literal?" is true before the program starts and stays true until it exits, because source code does not rewrite itself. Answering it costs several branches; answering it once per node instead of once per evaluation is the entire optimisation, and Chapter 19 measures what that is worth.

Two invariants keep them safe:

- **A value is never cached, only a classification** — except where the value is a literal, which is a constant by definition.
- **The sentinel means undecided**, so a fresh tree and a tree that has been running for an hour behave identically. That is what lets the serialiser skip these fields entirely (Chapter 30).

The `DecidedOnce<T>` wrapper is a relaxed atomic, for the reason given in Chapter 2: under parallel execution several threads may compute the same idempotent answer concurrently, and a plain field would make that a data race.

## 7.4 Two nodes that only exist sometimes

**`NqpOp`** implements the `use nqp` compatibility subset. It exists *only* when the parser saw `use nqp` and then met an `nqp::` call:

```
// src/Ast.h
enum class NqpOpc : uint16_t { Stmts, While, Until, IfNull, IseqI, AddI,
                               Substr, Chars, Concat, /* ~50 more */ };
struct NqpOp : Expr {
    NqpOpc op;
    std::vector<ExprPtr> args;
};
```

A program that never says `use nqp` builds no such node, so the interpreter's `NqpOp` arm is never reached and the implementation is dead code for that run. Chapter 35 is about why that zero-cost property is structural rather than an optimisation.

**`AllomorphLit`** exists because a numeric word inside a `<...>` list is simultaneously a number and its own source spelling: `<42>` is an `IntStr`, `<1/3>` a `RatStr`. The node keeps both the parsed numeric expression and the original text.

## 7.5 Borrowed pointers, and why the tree is immortal

A `Callable` — the runtime object behind every sub, block and method — does not own its code:

```
// src/Value.h — Callable, excerpt
const std::vector<Param>* params = nullptr;   // borrowed from the AST
const std::vector<StmtPtr>* body = nullptr;   // borrowed from the AST
std::shared_ptr<Env> closure;                 // owned
```

Only the environment is owned. The parameter list and the body are raw pointers into the tree. Calling a sub walks those AST statements directly; there is no copy, no bytecode, and no intermediate representation.

The consequence is that **the AST must outlive every value that refers to it**. For the main program that is automatic. For anything parsed later it is not, so the interpreter keeps them alive explicitly:

```
// src/Interpreter.h
std::vector<std::shared_ptr<Program>> keptPrograms_;   // EVAL'd ASTs
```

and `loadModule` pushes each module's `Program` onto the same vector (Chapter 33). Nothing is ever released from it. That is a deliberate, bounded leak: the number of distinct compilation units a process loads is small, and the alternative — refcounting subtrees — would cost on every call.

## 7.6 Seeing the tree

```
rakupp --dump-ast prog.raku
RAKUPP_DUMPTOKENS=1 rakupp prog.raku
```

`AstDump.cpp` prints the tree as an indented plain-text outline, one line per node with the fields that matter for that kind. It is the first thing to reach for when a parse produces something unexpected, and it is also how `tools/ast-opportunity.raku` counts syntactic patterns across a corpus — the tool that decided, with numbers, that constant folding was not worth building and node specialisation was (Chapter 19).

## 7.7 The two emitters that must know every field

Two back ends consume the tree structurally rather than semantically, and both break loudly if a field is added and forgotten.

**`AstEmit.cpp`** (`--aot`) emits C++ that rebuilds the identical tree. It throws `AstEmitError` on any construct it cannot reconstruct, and the driver answers by bundling the source instead — so a missed field degrades to a slower binary rather than a wrong one.

**`AstSerial.cpp`** (the precompiled parse) writes and reads a binary encoding. Its defence is stronger, and worth copying: **both directions are driven by one visitor per node type**, so a field cannot be written but not read. That is the failure mode which makes a format like this quietly wrong rather than loudly broken. A version constant in the header is bumped whenever the shape changes, and a mismatched entry is ignored rather than reinterpreted.

# Part III

# The Value Model

# Chapter 8

# `Value`: the Fat Tagged Struct

Raku is dynamically typed. A variable holds an `Int` now and a `Hash` later, routines dispatch on the runtime types of their arguments, lists can be infinite, and an object can grow a role while the program runs. Raku++ is written in statically-typed C++17 with no tracing garbage collector. One data structure carries the entire distance between those two facts.

```
// src/Value.h — as it stands
enum class VT : uint8_t { Nil, Any, Bool, Int, Num, Str, Array, Hash, Code,
                          Range, Pair, Type, Whatever, Object, Rat, Regex,
                          Match, Complex };
enum class PK : uint8_t { None, List, Hash, Code, PairV, Obj, Match };

struct Value {                      // 128 bytes
    long long i = 0;                // Int; a Type's definiteness; a Seq's position
    double    n = 0;                // Num; the real part of a Complex
    CowStr    s;                    // Str; also the Type name and the Pair key
    IStr      hashKind;             // "" Hash, else Set/Bag/Mix/Buf… — interned
    VT   t = VT::Any;               // the discriminator
    bool b, isList, itemized, objKeyed, readonly, namedArg,
         natSigned, natFloat;       // packed together so they cost one word
    PK   pk_ = PK::None;            // what the payload slot holds
    int  natBits = 0;               // native width uint8/int16/…; 0 = not native
    std::shared_ptr<void>     p_;   // THE payload slot: a list, a hash, a
                                    // Callable, a Pair's value, an object — or a
                                    // MatchData carrying three of them at once
    std::shared_ptr<ValueExt> x_;   // the cold block: Rat parts, Complex imag,
                                    // Range ends, BigInt, shape, ofType…
```

```
                                    // null on almost every value
    IStr enumName, enumType;        // enum key and enum type — interned
};
```

Two `shared_ptr`s, one interned tag, a packed word of flags. That is not what this chapter was written against, and the original is worth seeing, because every argument below is easier to follow when the fields are all in the open:

```
// src/Value.h — the 392-byte original, for the arguments that follow
struct Value {
    VT t = VT::Any;              // the discriminator
    bool b; long long i; double n, im;    // Bool / Int / Num / Complex imag
    CowStr s;                    // Str; also the Type name, the Pair key
    IStr hashKind, enumName, enumType;    // interned secondary tags
    bool isList, itemized, readonly, namedArg, fatRat, objKeyed;
    std::shared_ptr<ValueList> arr;                    // Array / List / Seq
    std::shared_ptr<std::map<std::string, Value>> hash;
    std::shared_ptr<Callable> code;
    std::shared_ptr<Value> pairVal, pairKey;
    std::shared_ptr<ObjectData> obj;
    std::shared_ptr<void> ext;   // opaque: Promise, Channel, lazy-seq state
    std::shared_ptr<BigInt> big, ratN, ratD;
    std::shared_ptr<std::vector<long long>> shape;
    long long rFrom, rTo; bool rExFrom, rExTo, rNum;   // Range
    std::string ofType; int natBits; bool natSigned, natFloat;
};
```

Eighteen tags, eleven `shared_ptr`s, and a pile of flags. It looks indefensible. It is not, and the case for it is the most important argument in this book.

**Read the field names below as the original's, because they still work.** The three campaigns that took 392 to 128 (Chapter 41) moved fields; they did not rename anything a reader or a caller uses. `arr`, `hash`, `code`, `pairVal` and `obj` became one kind-tagged slot, and `big`, `ratN`, `ratD`, `pairKey`, `ext`, `shape`, `ofType`, `im` and the `Range` ends went behind a lazily-allocated cold block — but every one of them is still spelled `v.arr()`, `v.ratN()`, `v.rFrom()`, and returns what it always did. Roughly 3,900 call sites depended on that, which is why the accessors were introduced *before* the fields moved. The struct got smaller; the model did not change.

The one place the change is visible is the `Match`, and it is visible because the design below forced it: a `Match` genuinely needs `arr`, `hash` and `pairVal` at once, so it is the single licensed co-occupant of the payload slot, riding in a `MatchData` body that holds all three. The next section is why that mattered.

## 8.1 Why not a union, a variant, or a class hierarchy

The instinct is: a value is one of *N* types, therefore a sum type. All three standard spellings of that instinct are wrong here, each for a different reason.

**A `Value` is frequently several fields at once.** A union models mutual exclusivity — one active member, selected by the tag — and that is simply not what a Raku value is:

- a **`Match`** sets the subject string `s`, the span `rFrom`/`rTo`, the positional captures in `arr`, *and* the named captures in `hash`, all live together;
- a **`Pair`** is the key in `s` plus `pairVal`;
- an **enum value** is the ordinal in `i` plus `enumName` and `enumType`;
- a **`Rat`** is `ratN` plus `ratD` plus the `fatRat` flag;
- and the cross-cutting flags — `isList`, `itemized`, `readonly`, `namedArg`, `ofType`, `natBits` — are set *regardless of the tag*. A `readonly` `Int` and an `itemized` `Array` are both ordinary.

So `VT t` does not mean "which one field is valid". It means "how to read a bag of fields, several of them set". That is a struct.

**A raw union of these members is undefined behaviour.** They are non-trivial C++ types — a string, eleven `shared_ptr`s. Making it legal means hand-written placement-`new`, a tag-switched copy constructor, move constructor, both assignment operators and destructor. At that point you have rebuilt `std::variant`, which re-imposes the one-active-member model *and* forces `std::get`/`std::visit` on the roughly sixteen thousand lines of interpreter and built-ins that today just read `v.i`, `v.s`, `v.arr`.

**The memory a union would save is smaller than it looks.** The overlappable members are mostly `shared_ptr`, which is null and therefore allocation-free when unused, and by the first point many of them cannot be overlapped at all.

**A class hierarchy** would put a virtual call on every field access and a heap allocation on every integer. For a tree-walker whose values are overwhelmingly transient scalars, that is the worst of the three.

## 8.2 What the fat struct buys

**No virtual dispatch and no allocation for scalars.** An `Int` is a `Value` with `t == VT::Int` and `i` set. Building one is a stack operation:

```
// src/Value.h
static Value integer(long long x) { Value v; v.t = VT::Int; v.i = x; return v; }
```

**Uniform copy and move.** Passing a `Value` by value, returning it from `eval`, storing it in a vector — all use the compiler-generated operations. The `shared_ptr` members make copies of arrays, hashes and objects a refcount bump, and give them **shared identity**, which is exactly what Raku's container semantics need. Mutating an object attribute through a copied `self` handle works for this reason and no other.

**Coercion is a method, not a cast.**

```
// src/Value.h
bool truthy() const;      long long toInt() const;   double toNum() const;
std::string toStr() const;  std::string gist() const;
std::string typeName() const;
```

`~$x` calls `toStr`, `+$x` calls `toNum`, boolean context calls `truthy`. There is no C++ inheritance making `Int` "be a" `Cool`; these six functions encode the numeric and string towers directly, each a `switch` on `t` in `Value.cpp`.

## 8.3 Clever reuse of fields

Several Raku types have no `VT` of their own. They are an existing tag plus a tag-like marker, and knowing which is which explains a lot of otherwise puzzling code.

**A `Junction` is an `Array` tagged by `enumName`.** `any(1,2,3)` is a `VT::Array` whose elements are the eigenstates, with `enumName` set to one of `any`, `all`, `one`, `none`:

```
// src/Value.cpp — typeName(), the VT::Array case
if (enumName == "any" || enumName == "all" ||
    enumName == "one" || enumName == "none") return "Junction";
```

**Set, Bag and Mix are Hashes tagged by `hashKind`.** So are `Proxy`, `Failure`, `Date`, `DateTime`, `Scalar` and several more; `hashKind` is a secondary type tag drawn from a closed vocabulary.

**An allomorph is a number that is also its own string.** `<42>` is `VT::Int` with `s` holding `"42"` and `hashKind` naming `IntStr`:

```
// src/Value.h
bool isAllomorph() const {
    return (t == VT::Int || t == VT::Rat || t == VT::Num || t == VT::Complex) &&
           (hashKind == "IntStr" || hashKind == "RatStr" ||
```

```
            hashKind == "NumStr" || hashKind == "ComplexStr");
}
```

**An `Int` grows a bignum only when it must.**

```
// src/Value.h
static Value bigint(const BigInt& b) {
    Value v; v.t = VT::Int;
    if (b.fitsLL()) v.i = b.toLL();
    else v.big = std::make_shared<BigInt>(b);
    return v;
}
```

**A `FatRat` is a `Rat` with a flag.** Same storage, but the flag carries the type identity — contagious through arithmetic — and exempts the value from the denominator spill described in Chapter 11.

**A native integer container is a `Value` with a width.** `my uint8 $b` sets `natBits = 8`, `natSigned = false`, and every assignment masks to that width.

## 8.4 `ext`: the opaque slot

One member is deliberately untyped:

```
std::shared_ptr<void> ext;
```

It carries whatever state a value needs that has no business in the struct: a `PromiseState`, a `Channel`, a `TapHandle`, a `LazySeqState`, a `CueState`, or the endpoint objects of a `Range`. Each consumer knows what it parked there and static-casts it back.

That is not as loose as it sounds, because the tag plus `hashKind` always determine which kind of state can be present. But it is the one place in the value model where the type system has been switched off on purpose, and it is worth being aware of when adding a case.

## 8.5 The identity problem, and how `ext` solves it

Some Raku types are *reference* types: two of them are the same one only when they are the same object. `Buf`, `Instant` and `Duration` are three. Here they are plain tagged scalars — a `Buf` is a `Str` with `hashKind = "Buf"`, an `Instant` a `Num` — and with no address to compare, `===` fell through to comparing the *rendering* and declared two independently built buffers identical.

That is not academic. It made `@!outstanding-writes .= grep({ $_ !=== $p })` unemptiable for Promises, and it was worse for `Buf`, which is mutable: dropping one buffer from a list by `!=== $buf` threw away every buffer that happened to hold the same bytes.

```
// src/Value.h
inline bool identityScalar(const Value& v) {
    return (v.t == VT::Str && v.hashKind == "Buf") ||
           (v.t == VT::Num && (v.hashKind == "Instant" ||
                               v.hashKind == "Duration"));
}
inline Value& identify(Value& v) { v.ext = std::make_shared<char>(); return v; }
```

Each freshly built one stamps a unique token into `ext`. A plain `Value` copy carries the token along — which is precisely what "the same object" means for these types — and `===` compares it. `Blob` stays out: it is immutable and compares by value in Rakudo too.

The same trick answers a smaller question. `$*INIT-INSTANT` is one `Instant` read many times, and `$*INIT-INSTANT === $*INIT-INSTANT` must be `True`, so both reader sites go through one function that hands back the same stored token.

## 8.6 Ranges remember what they were written with

A `Range` iterates over integers in `rFrom`/`rTo`, or over doubles in `n`/`im` when it is fractional. But `1/2 .. 1/3` must keep its `Rat`s and `True .. False` its `Bool`s for `.min`, `.max`, `.bounds` and rendering.

```
// src/Value.h
struct RangeEnds { Value from, to; };
inline void setRangeEnds(Value& r, const Value& from, const Value& to) {
    auto keep = [](const Value& v) {
        return v.t == VT::Rat || v.t == VT::Num || v.t == VT::Bool ||
               v.t == VT::Nil || v.t == VT::Any || v.t == VT::Type;
    };
    auto renders = [&](const Value& v) { return keep(v) || v.t == VT::Int; };
    if (!keep(from) && !keep(to)) return;
    if (!renders(from) || !renders(to)) return;
    attachRangeEnds(r, from, to);
}
```

The endpoints are parked in `ext`, and only when they would actually render differently. `..` numifies a `Str` endpoint (`"2"` becomes `2`) and a list endpoint (its element

count), so those must *not* be carried; and an Int renders identically either way, so carrying it would only cost an allocation on the very hot 1..n path.

That last clause is the pattern to notice. Almost every field in Value is guarded by a test that keeps the common case free.

## 8.7 The rendering hook

Value.h cannot call into Builtins.cpp, but a Range's endpoints must render with .raku, which lives there. The solution is a function pointer:

```
// src/Value.h
using RakuReprFn = std::string (*)(const Value&);
extern RakuReprFn g_rakuRepr;
```

A raw pointer is zero-initialised before any dynamic initialisation runs, so installing it from another translation unit is order-safe — unlike a std::function, which would have its own construction order. There are a handful of these hooks in the runtime (g_forceLazy, g_endlessLazy), each solving the same layering problem the same way. Not every hook is one: g_objListItems and g_deproxy are std::functions, which is the alternative this paragraph argues against — they are installed early enough that it has not mattered, which is the sort of thing that is true until it is not.

## 8.8 What it costs

| | bytes |
|---|---|
| long long | 8 |
| std::string (libc++) | 24 |
| CowStr | 40 |
| **Value** | **392**, on the build these numbers were taken from |

Every Value carries every field, live or not — or, since the cold block, every field it might plausibly need with the rest one pointer away. A ValueList of integers is about fifty times the size of a vector<int64_t> on the build these numbers come from. That is the price of branch-free field access and trivial copyability, and it is a real price: the profile of a method-call-heavy loop puts 31% of the time in heap allocation and 11% in Value copy and destruction.

The number moves, which is why this chapter shows two listings. It has been 392, 376, 344, 208 and 128 bytes, and the next section walks the whole lineage — including the step that made it smaller and slower, and had to be reverted.

The table above prices the 392-byte original because that is the version the argument was made against, and because the ratio it produces — fifty times a `vector<int64_t>` — is the number that made the case for shrinking. At 128 bytes the same ratio is sixteen. The design did not change; the tax did.

That instability is also the single most important input to the extension ABI in Chapter 37, which is why an extension module never sees this struct at all.

## 8.9 The struct, version by version

`Value` is the most-rewritten data structure in the project, and every rewrite was measured. The lineage is worth having in one place, because the individual steps are scattered across four later chapters and because two of them are lessons rather than wins.

| | bytes | what changed |
|---|---:|---|
| original | 392 | four `std::string`s, eleven `shared_ptr`s, all inline |
| ordinary work | 376 | field-level tidying, no design change |
| *attempted* | *360* | **flags packed to remove padding — reverted** |
| batch 1 | 344 | `hashKind`, `enumName`, `enumType` interned (Chapter 10) |
| batch 2 | 208 | the cold block: rarely-used fields behind one lazy pointer |
| batch 4 | 128 | five payload pointers become one kind-tagged slot |
| design A | *56* | proposed: intrusive refcount, kind-specific bodies, packed tags |
| design C | *32* | proposed: the string leaves the struct too |

**The reverted step is the most instructive.** In July 2026 the flags — `fatRat`, the three `Range` booleans, `natBits` and its two companions — sat between 8- and 16-byte members and forced 23 bytes of pure padding. Grouping them was free: no behaviour change, no risk, 376 down to 360. It measured **2.5% slower**, consistently, over six alternating rounds.

The explanation is that the hot field offsets did not move — `t`, `b`, `i`, `n`, `im`, `s`, `hashKind` stayed exactly where they were — while `arr` and every pointer after it shifted by eight bytes. Smaller is not automatically faster when the thing you shrink is padding a cache line was absorbing anyway, and a *free* change can still cost. The size campaign that eventually succeeded did not repack fields; it moved them out of the struct.

**Batch 1 replaced types, not layout.** Three of the four `std::strings` were secondary type tags drawn from a closed vocabulary — container kinds, type names — so they became interned 8-byte handles (Chapter 10). Fifteen non-trivial members became twelve, `sizeof` went 392 to 344, and a JSON parse got 4 to 6 per cent faster at every document size while a twelve-thousand-entry hash build went from 302 to 265 nanoseconds per entry. The win is not the bytes; it is that a copy stopped running three string constructors.

**Batch 2 asked what a typical value actually carries.** A census instrumented every `Value` destruction and counted which pointers were live: of thirty million destructions, **25.6 million carried none** of `big`, `ratN`, `ratD`, `pairKey`, `ext`, `shape` or the `Range` fields. So those moved behind one lazily-allocated, copy-on-write block, and an `Int` stopped paying for a `Rat` it does not have. Reads never allocate; writes clone a shared block first.

**Batch 4 collapsed the payload.** The same census said `arr`, `hash`, `code`, `pairVal` and `obj` are mutually exclusive on every live value — with exactly one exception, the `Match`, which needs three at once — so five pointers became one `shared_ptr<void>` plus a one-byte kind tag, and the `Match` got a combined body. That is where the `MatchData` mentioned at the top of this chapter comes from: it exists because the union argument this chapter opens with is *true*, and the one place it is true had to be given somewhere to live.

Batch 4 also produced the campaign's sharpest reminder that a census bounds what typical programs do and not what the language allows. A container declared `is default(9)` carries its element default *alongside* its payload — a live co-occurrence the thirty-million-destruction census never sampled, because it is rare per value and

unremarkable per program. Three regression files segfaulted within minutes of the change. The default now lives in the cold block, where rare-per-value belongs.

**What the shrinking bought that nobody planned.** Two consequences arrived from outside the campaign's own goals. The first is in Chapter 42: the cost of parking a `Value` in long-lived addressable storage, which had made a partially lowered bytecode IR structurally impossible, is a function of how much struct there is to construct and destroy — 11.2 nanoseconds per node at 376 bytes, zero at 128. The second is in Chapter 12: at 128 bytes with two `shared_ptr`s and no self-referential member, a `Value` is *trivially relocatable*, which is what lets a list of them grow by `memcpy`. Neither was a design goal. Both follow from the same reduction.

**What is left.** Fifty-six bytes is reachable without touching the string: an intrusive refcount instead of `shared_ptr`, kind-specific bodies instead of the generic cold block, the tag word packed, and `i`/`n` unioned. Thirty-two means the string leaves the struct, and the probe says the two ways of doing that differ by seven times in opposite directions — a heap body is 1.6× slower than today, an inline buffer 3.7 to 4.4× faster, and the second costs a fifteen-hundred-site type change. Sixteen is not a layout change at all: it needs values to stop being refcounted and the container to stop living inside the value, which is a rewrite of assignment and binding rather than a batch.

## 8.10 Honest limitations

- **Memory per value**, as above. It is the accepted cost of the design, not an oversight, but it is the thing to attack first if the design is ever revisited.
- **Nothing in a `Value` may point at itself.** This one has no compiler enforcement and no test, and breaking it is silent. `ValueList` relocates its buffer with a `memcpy` when the standard library allows it (Chapter 12), which is only sound because moving a `Value`'s bytes to a new address and not destroying the source is equivalent to move-constructing and destroying: true of the scalars, of `IStr`'s interned pointer, and of the `shared_ptr` slots. The near-miss is already in the struct — libstdc++'s short `std::string` stores a pointer to its own inline buffer, which is why the relocation path is decided by a run-time probe rather than assumed. Any future field with an interior pointer, a self-registering handle, or an intrusive list node breaks that path, and it will break it by producing wrong data rather than by failing to compile.
- **`ext` is untyped.** Nothing but convention stops two subsystems from parking incompatible state in it for the same tag.

- **Reference semantics are emulated.** The identity-token trick covers the three scalar types that needed it; a fourth would need the same treatment added by hand.
- **The tag set is closed.** Adding a `VT` means auditing every `switch` over it in the interpreter, the code generator, the serialiser and the dumper — which is exactly why new Raku types are added as an existing tag plus a marker instead.

# Chapter 9

# Strings: `CowStr`

`Value` holds its `Str` payload in a `CowStr`, not a `std::string`. This chapter is about why that class had to be written rather than reached for, and it is the clearest example in the project of a design forced entirely by measurement.

## 9.1 The problem

A `Value` is copied by value everywhere: every argument pass, every operand evaluation, every list element, every return from `eval`. With a `std::string` member, a long string was `memcpy`'d on every one of those.

That is O(length) **per operation**, which makes any tokenizer written in Raku quadratic. `JSON::Fast` needed 13.9 seconds on a 421 KB document that Rakudo parses in 51 milliseconds, and the profile was copying, not parsing.

Measured, 20,000 operations on a 200 KB string:

| | `std::string` | `CowStr` |
|---|---:|---:|
| pass the string to a sub, no work done | 142 ms | 24 ms |
| `nqp::ordat` on a global | 79 ms | 11 ms |

## 9.2 What it is

Two arms, exactly one live:

```
// src/Value.h
class CowStr {
    std::string s_;                         // short: inline
    std::shared_ptr<const StrBody> p_;      // long: shared, immutable
    static constexpr size_t kPromote = 23;

    void take(std::string x) {
        if (x.size() >= kPromote) {
            p_ = std::make_shared<const StrBody>(std::move(x)); s_.clear();
        } else { s_ = std::move(x); p_.reset(); }
    }
public:
    const std::string& str() const { return p_ ? p_->text : s_; }
    std::string& mut() { if (p_) { s_ = p_->text; p_.reset(); } return s_; }
};
```

Short strings stay inline, where `std::string`'s own small-buffer optimisation already makes a copy free and sharing would cost an allocation to save nothing. Long strings are promoted once into a shared immutable body, after which copying is a refcount bump.

Three decisions inside that deserve their own paragraphs.

### 9.2.1 Promotion is eager

At construction, not lazily on first copy. Raku++ runs work in parallel, and a lazy promotion would have to mutate the source from a `const` copy constructor — two threads copying the same `Value` would race on it.

Eager promotion means **a `const CowStr` is never written to at all**, which is what makes the type safe to share without a lock. That single property is what lets everything else here be lock-free.

### 9.2.2 The threshold is 23, and it used to be 64

The relevant numbers are the small-buffer capacities, measured:

| standard library | `sizeof(std::string)` | SSO capacity |
|---|---|---|
| libc++ (clang) | 24 | 22 |
| libstdc++ (g++ 14, 16) | 32 | 15 |

The original threshold was 64, chosen to sit above every mainstream capacity. That left a band in which a copy was a couple of words and no allocation happened either way, and it promoted only where copying was clearly the thing being paid for. It was not a tuned number; it was a deliberately safe one.

It was also, by construction, wrong at one end. Between libc++'s 22 and the threshold's 64 sat a band where the inline arm had already stopped being free: a 30-byte string is a heap `std::string` that mallocs **on every copy**, while a 70-byte one is a shared body that copies by refcount. A 33-byte string cost more to pass around than a 68-byte one, for no reason but the constant.

So the threshold moved to 23 — libc++'s boundary, the point where the inline representation stops being free — and the policy inverted with it: promote wherever `std::string` would allocate anyway, rather than wherever sharing is obviously worth it. On libstdc++, whose capacity is 15, the new value leaves a 16..22 band that is now promoted where it need not be; that is the cost of having one constant rather than a per-library one, and it is small.

The honest ledger for the change, measured as instruction counts at best of five runs: mid-band strings copied twice **+19.8%**, built and read once **+4.9%**, `texts-plit` **+3.0%**, and one measured counter-case at **−5.6%**. Grammar parsing, string comparison and string appending are all flat, because their strings are not in the band. The change is kept because the shapes it helps are commoner than the shape it hurts, not because the measurement is emphatic — and the counter-case is the next section.

### 9.2.3 The counter-case: a mid-band string used as a hash key

A promoted string costs **two** allocations, not one: `make_shared` fuses the control block with `StrBody`, but `StrBody::text` heap-allocates its own buffer. An unpromoted mid-band string costs the single `std::string` malloc. Copies are free afterwards, so promotion pays from roughly the second copy onward.

A hash key never gets there. It is built, hashed, and copied *out* into the hash's own key storage, and the `Value` is then dropped — one construction, one read, no copy that a shared body could make cheap. Two hundred thousand mid-band keys measure 5.6% more instructions after the threshold change, and that is the whole shape of the trade written in one program.

The fix is not the threshold. It is `StrBody` storing its text inline rather than as a `std::string` member, which would collapse promotion to a single allocation and

remove the loss entirely — the same change the 32-byte Value design needs, for the same reason.

### 9.2.4 The body caches string properties

This is the part no general-purpose string type can do, and the real reason CowStr exists rather than a third-party copy-on-write string.

```
// src/Value.h
struct StrBody {
    std::string text;
    mutable std::atomic<signed char> allAscii{-1};
    mutable std::atomic<signed char> crFree{-1};
    mutable std::atomic<long long>   nGraphemes{-1};
};
```

Raku string positions are **grapheme** positions, and Raku++ stores UTF-8 bytes. So every indexing operation must first establish whether a byte index is also a grapheme index. Establishing that by scanning is O(position) per character examined — which is the *other* half of the quadratic, independent of copying.

Because the promoted body is immutable, the answer can be computed once and cached on it. Three flags suffice:

- allAscii: every byte below 0x80, so a byte index is a codepoint index;
- crFree: no carriage return which, together with allAscii, means a byte index is a **grapheme** index — CR LF is the one ASCII sequence that clusters under UAX #29;
- nGraphemes: the answer to .chars.

They are reached through cowAllAscii, cowByteIsGraphemeIndex and cowGraphemeCount in BuiltinsShared.h.

The caches need no lock. Two threads may each compute one, but the text is immutable, so they compute the same answer and the store is idempotent. They live on the body, so they exist only for promoted strings — which is exactly the case that needs them; short strings are cheap to rescan.

## 9.3 Does C++ have this already? It did, and removed it

**libstdc++'s pre-C++11 basic_string was refcounted copy-on-write.** C++11 made that non-conforming. The string requirements added contiguous storage,

O(1) non-`const` `operator[]` and `data()`, and iterator-invalidation rules under which one copy's mutation must not disturb another copy's references. Copy-on-write cannot satisfy all of them at once: non-`const` element access would have to detach, and detaching is O(n). GCC 5's dual ABI (`_GLIBCXX_USE_CXX11_ABI`, `std::__cxx11::basic_string`) exists to carry that break. libc++ was never copy-on-write.

C++11's replacement answer was **move semantics**, which solves "stop copying temporaries" and not this problem. Every copy in the list at the top of this chapter is a genuine copy, not a move: an argument passed to a function that keeps it, an element stored into a list.

**None of the reasons copy-on-write was banned apply to `CowStr`,** because it is not trying to be a `std::string`. Its `operator[]`, `data()`, `begin()` and `substr` are `const`-only; the mutation door for text is `mut()`, which detaches explicitly and hands back a real `std::string&`. What C++11 outlawed was copy-on-write *hiding behind* an interface that promises O(1) non-`const` element access. `CowStr` promises no such thing.

### 9.3.1 The alternatives, and why each is not it

| | why not |
|---|---|
| `std::string_view` | non-owning; `Value` owns its string |
| small-buffer optimisation | already the short arm — that is what `s_` relies on |
| `shared_ptr<const std::string>` | already the long arm; `CowStr` is mostly the *policy* for choosing between the two |
| `std::atomic<std::shared_ptr<T>>` | only needed if the pointer were rebound concurrently, which eager promotion guarantees it never is |

Outside the standard the pattern is thoroughly proven and lives attached to frameworks: `folly::fbstring` is the closest relative — a drop-in `basic_string` with three tiers, the top one copy-on-write with an atomic refcount; Qt has done "implicit sharing" across its whole container library for about twenty-five years; `absl::Cord` is a tree of refcounted immutable chunks, which wins when *concatenating* large strings rather than copying them; Rust's `Cow<'_, str>` makes the same idea explicit in the type system. LLVM went the other way entirely, with `StringRef` and `Twine` and no copy-on-write anywhere.

**Why not adopt one?** Folly and Qt are large dependencies for a subset of what is needed, and this project vendors nothing it can write in a page. But the deciding reason is the property cache: `fbstring`, `QString` and `Cord` would all give the copy elision and none of the `allAscii`/`crFree`/`nGraphemes` caching, which is the half of the fix that removed the *scanning* quadratic. Bolting that onto a third-party string means a side table keyed on the buffer address, with its own lifetime and thread-safety problem — strictly worse than owning forty lines.

The threshold difference makes the same argument from the other end. `fbstring` promotes at 255 because copy avoidance is all it buys. `CowStr` promotes at 23 because promotion also buys the cache — an order of magnitude earlier, for a type that gets something out of the body besides sharing.

## 9.4 What it costs

| | bytes |
|---|---|
| `std::string` (libc++) | 24 |
| `std::shared_ptr` | 16 |
| **`CowStr`** | **40** |
| `StrBody` | 56 |

Both arms are stored side by side even though only one is ever live, so `CowStr` is the sum rather than the maximum. That was a 16-byte, 4% growth of `Value` when this was written and `Value` was 392 bytes; `Value` is 128 now, so the same 16 bytes are an eighth of it. Paid on every `Value` everywhere. The performance gate passed and Roast came out marginally up, so it is bought and paid for — but it is a real tax and should be named as one. On libstdc++ the same layout is 48 bytes, because `sizeof(std::string)` is 32 there.

A promoted string also costs **two** allocations: `make_shared` fuses the control block with `StrBody`, but `StrBody::text` heap-allocates its own buffer for anything past the small-buffer capacity.

`StrBody` grew to 56 bytes for a reason worth knowing, because it is the other half of the string-scan story. Beside the cheap scanning flags it carries two lazily built byte-offset tables — `cpIndex`, the byte offset of codepoint *i*, and `gIndex`, the byte offset of grapheme *g*, each with an end sentinel. They are what makes a positional operation on a non-ASCII string O(1) instead of a rescan from the start, which is the non-ASCII half of the quadratic the ASCII fast path had already closed. They are

installed by compare-and-swap rather than by the idempotent store the flags use: two threads may both build one, and the loser deletes its own copy.

## 9.5 Rules for working with it

The type forwards about 1,100 read sites unchanged, which is the point. The places where it does not behave like a `std::string` are worth knowing.

- **`mut()` is the write door for text, and it detaches.** After `mut()` the string is inline again and stays inline until it is next *assigned*, which is where promotion happens. A long string mutated in a loop is therefore unpromoted for the whole loop; if that ever shows up in a profile, build into a local `std::string` and assign once at the end.
- **References from `str()` do not survive an assignment.** The same rule as `std::string`, but easier to forget through the implicit conversion.
- **Both converting constructors are `explicit` on purpose.** With an implicit `std::string` to `CowStr` conversion alongside the `CowStr` to `const std::string&` one, every `cond ? value.s : someString` became ambiguous in both directions.
- **Comparisons and + are spelled out by hand.** Two real reasons: no conversion is considered for a built-in operator when neither operand is a class type with a candidate, so `CowStr == CowStr` needs its own overload; and `std::string`'s comparisons are templates, and template deduction never considers a user-defined conversion, so `value.s == "Int"` needs a real candidate too. A macro generates the twenty-four resulting overloads.
- **Do not add a non-`const` `operator[]`, `begin()` or `data()`.** That is exactly the interface that made copy-on-write non-conforming for `std::string`. Reach for `mut()`.
- **Do not cache `body()` across an assignment.** It is the raw `StrBody*`, and the `shared_ptr` that owns it can be replaced.

## 9.6 Deliberately left on the table

Both are real, both unmeasured, and neither should be done on faith — the project's rule is that a performance change is an interleaved A/B against `perf-guard` under the same conditions.

- **A union instead of two side-by-side members.** `fbstring` is a union and stays at `sizeof(std::string)`; the same here returns 16 bytes per `CowStr`. It costs

the very readable `p_ ? p_->text : s_` and needs hand-written copy, move and destroy. Worth it only if `Value`'s size shows up in a measurement — and one attempt to blame `Value`'s size has already been measured and rejected.

- **One allocation instead of two** for a promoted string, by storing the refcount and the caches inline ahead of the characters. Only matters if the promotion *rate* is high.

## 9.7 Checking that it is working

The failure mode is silent: a change that puts a hot string back on the copying path costs time and nothing else. Two cheap checks.

**Measure scaling, not speed.** Time the operation over inputs of $n$, $2n$, $4n$, $8n$. Doubling per doubling is fine; quadrupling is the bug this class exists to prevent. That is the test which found `findnotcclass` after four other sites had already been fixed.

**Confirm promotion is actually happening.** A string that never crosses 23 bytes is never promoted and caches nothing — which is correct, but means a benchmark built from short strings proves nothing about either mechanism. The bar is low enough now that most real text clears it; it was 64, and a good deal of plausible-looking test data used to sit under it.

# Chapter 10

# Interning and Comparison: `IStr` and `MName`

Two small headers, ninety lines each, both solving the same shape of problem: **a string that is compared far more often than it is created**. They arrived from different directions and ended up with almost the same interface, which is itself the point.

## 10.1 `MName`: the method name as the dispatcher sees it

Method dispatch on a built-in value is a long ladder of name comparisons in `Built-ins.cpp` and the five `MethodCallPart*.cpp` files that continue it — roughly 1,640 of them. So the comparison itself is hot.

The path to this fix is worth following, because two plausible diagnoses were ruled out by measurement before the real one was found.

A profile of `for ^3000000 { "ab".chars }` put **60% of the time in string comparison**: `_platform_strlen` 19%, an out-of-line `std::operator==(const string&, const char*)` 19%, `DYLD-STUB$$strlen` 14%, `memcmp` 8%.

That looks like the ladder's length. Earlier measurement had seemed to exonerate it: `.chars` sits 177 comparisons into the file and `.uc` 812, yet they cost the same. **Both facts are true.** Position *in the file* is not the number of comparisons *executed*, because the arms are guarded by invocant type — a `Str` invocant only ever runs the `Str` arms, and `.chars`, `.uc` and `.uniname` are all in that same group.

The real problem was that `strlen` was being called **at run time on a string literal**. Clang normally folds that away, and in a small function it does — a 1,700-branch toy reproduction compiles to zero `strlen` calls. But `methodCallInner` was about 8,900 lines with 1,640 comparisons, far past the optimizer's inlining budget: `std::operator==` stayed out of line, and out of line it cannot see the literal, so it measured it with `strlen` on every call.

The fix does not depend on the optimizer's mood. Wrap the name in a type whose `operator==` takes the literal **by reference to array**, so the length is part of the template argument:

```
// src/MethodName.h
struct MName {
    const std::string& s;
    std::size_t n;        // cached length: the first test on every comparison
    std::uint64_t pre;    // first 8 bytes packed, so short names compare as one

    explicit MName(const std::string& v)
        : s(v), n(v.size()), pre(pack(v.data(), v.size())) {}

    template <std::size_t N>
    RAKUPP_ALWAYS_INLINE bool operator==(const char (&lit)[N]) const {
        if (n != N - 1) return false;
        if (N - 1 <= 8) return pre == pack(lit, N - 1);
        return std::memcmp(s.data(), lit, N - 1) == 0;
    }
};
```

One `const MName m{mName};` at the top of the function, and every `m == "chars"` site becomes a size check — which rejects nearly every candidate outright — followed by an inlined comparison. Four compile errors resulted, all of them stream or concatenation overloads, which is why the struct also forwards `+`, `<<` and the handful of `std::string` members the chain uses.

Two details earn their keep.

**`always_inline` is not optional.** Left to itself the optimizer emits `MName::operator==<N>` out of line in this function too, and an out-of-line call costs more than the comparison it performs.

**The first eight bytes are packed into a `uint64_t` at construction**, so any name of eight characters or fewer — which is most of them — compares as a single integer against a literal packed at compile time, with no `memcmp` at all.

Measured, one million iterations:

| | before | wrapper | + always_inline & packing |
|---|---|---|---|
| `'ab'.chars` | 1.63 s | 1.19 s | **1.17 s** |
| `'ab'.uc` | 1.78 s | 1.15 s | **0.95 s** |
| `'ab'.uniname` | 1.61 s | 1.08 s | **0.71 s** |

`.chars` barely moves because it is reached early. `.uniname` is deep in the `Str` arm and gains the most.

### 10.1.1 Where that leaves dispatch

Name comparison went from 60% of the profile to **8.5%**, which retires it as a target. A perfect-hash dispatch table would only be chasing that 8.5%, and it is not a drop-in: the ladder is not a pure dispatch on the name. Arms are guarded by invocant type and argument shape, and later generic arms deliberately catch what earlier specific ones decline. Turning 1,640 of those into table entries means giving each its own guard *and* preserving the fall-through order between them — a large, risky rewrite for a single-digit percentage.

The 42% now sitting in allocation and `Value` churn is the real remaining target, and it is a different problem (Chapter 16).

## 10.2 `IStr`: the storable counterpart

`Value` carried four `std::string` members — `hashKind`, `enumName`, `enumType`, `ofType` — that are **empty on almost every value**. An empty `std::string` is not free: it is 24 bytes, and it is constructed, copied and destroyed on every one of the roughly two million `Value` copies a 278 KB JSON parse makes.

Measured on that parse:

- `Value`'s implicit constructor, destructor and copy were **12.8% of self time**, the top line of the profile;
- `std::string == const char*` — which calls `strlen`, then `memcmp` — plus its helpers was about **10% more**, and `hashKind` is compared against a literal on paths that run per parameter per call.

The strings that ever reach these fields are a small closed set in practice: container kinds and type names. So intern them.

```
// src/IStr.h
struct IStr {
    struct Entry { std::string s; std::size_t n; std::uint64_t pre; };
    const Entry* e = nullptr;      // null IS the empty string

    static const Entry* intern(const char* p, std::size_t k) {
        if (!k) return nullptr;
        static std::deque<Entry> storage;   // stable addresses
        static std::unordered_map<std::string, const Entry*> index;
        static std::shared_mutex mu;
        std::string key(p, k);
        { std::shared_lock<std::shared_mutex> rd(mu);
          auto it = index.find(key);
          if (it != index.end()) return it->second; }
        std::unique_lock<std::shared_mutex> wr(mu);
        // ... re-check, then append and index ...
    }

    // interned, so identity IS equality
    bool operator==(const IStr& o) const { return e == o.e; }
};
```

Eight bytes, trivially copyable, trivially destructible. A default-constructed `IStr` costs a zeroed pointer where a `std::string` cost a constructor call. Comparing two `IStrs` is a pointer comparison, because interning makes identity equality. Comparing against a literal uses the same packed-prefix trick as `MName`.

**Interning takes a lock, but only on assignment from text.** Copies and comparisons — the operations `Value` does millions of times — never reach it. Readers share the lock; only a miss serialises.

The operator surface is deliberately the same shape as `MName`'s, so the roughly 1,300 existing `.hashKind == "Buf"` and `.ofType.empty()` sites compiled unchanged.

### 10.2.1 The intern table is never freed

That is deliberate. Entries are type names and kind tags, bounded by the program's own vocabulary, and a stable address is what makes the handle comparable by pointer.

It also imposes a rule, and there is one field that breaks it. `ofType` is **not** interned, and must stay that way until one site moves:

```
// src/Value.h
// `IO::Path`'s `:CWD` rides in `ofType` because a path value has no other
// use for it, so this field can hold a runtime DIRECTORY NAME rather than
// a type name. The intern table is append-only by design, so a program
// walking many directories would add an entry per directory and never
// release it.
std::string ofType;
```

This is the interesting failure mode of interning, and it is worth stating as a general rule: **intern a closed vocabulary, never an open one.** A field that can hold arbitrary runtime data — a path, a user string, a key — turns an append-only table into an unbounded leak. Everything else in `Value` is drawn from the program's own vocabulary of type names and is safe.

## 10.3 Why they are separate types

`MName` borrows a `const std::string&` and caches its length and prefix. It is a **view**, built at the top of a function and discarded at the bottom; it cannot be stored.

`IStr` **owns** a handle into a global table. It can live in a struct, be copied freely, and outlive the text it was built from.

They could in principle be unified — `IStr` could serve both roles — but a method name is compared once per call and never stored, so paying the intern lock for it would be strictly worse. Two types, two lifetimes, one idea.

# Chapter 11

# Numbers

Raku's numeric tower is exact where it can be. `1/3` is a rational, not a float. `2**200` is an integer, not an overflow. `0.1 + 0.2 == 0.3` is `True`, because a decimal literal is a `Rat`. Implementing that on a machine with 64-bit registers means three layers, each with a boundary the previous one falls through.

## 11.1 Layer 0: native integers, with overflow as a signal

Most integers in most programs fit in a `long long`, so that is the representation: `Value` with `t == VT::Int` and the value in `i`. Promotion to bignum happens exactly when an operation overflows, which means overflow must be *detected* rather than avoided.

```
// src/IntOps.h
inline bool add_ovf(long long a, long long b, long long* r) {
#if defined(__GNUC__) || defined(__clang__)
    return __builtin_add_overflow(a, b, r);
#else
    unsigned long long u = (unsigned long long)a + (unsigned long long)b;
    long long s = (long long)u; *r = s;
    return ((a ^ s) & (b ^ s)) < 0;   // both operands' sign differs from result
#endif
}
```

`IntOps.h` exists because MSVC has neither `__builtin_*_overflow` nor `__int128`. It provides `add_ovf`, `sub_ovf`, `mul_ovf`, `ctzll`, `clzll` and a `RAKUPP_HAS_INT128` flag, mapping each onto intrinsics where they exist and a hand-written fallback where

they do not. The multiply fallback for MSVC on ARM64 computes the wrapped product and then verifies it by division, which is slow and correct; the x64 path uses `_mul128` and checks that the high half is the low half's sign extension.

This header is what makes the arithmetic fast paths in Chapters 27 and 28 possible: `rtAdd` can inline the native case precisely because the overflow test is one instruction on the compilers that matter.

## 11.2 Layer 1: `BigInt`

```
// src/BigInt.h
struct BigInt {
    int sign = 0;                  // -1, 0, +1
    std::vector<uint32_t> mag;     // little-endian limbs, base 1e9
    static const uint32_t BASE = 1000000000u;
};
```

Base 10^9 rather than a power of two. That choice trades some arithmetic density for the thing a language runtime does constantly: **decimal conversion is free**. `toString` walks the limbs and prints nine digits each; `fromString` parses in nine-digit chunks from the right. A binary base would make printing a repeated division by 10, which for a language where every integer is eventually stringified is the wrong trade.

Multiplication is schoolbook, O(n·m). There is no Karatsuba and no FFT: the sizes that appear in practice are small, and the workloads where they are not are dominated by other costs.

Division is base-10^9 long division — Knuth's algorithm D. The per-limb quotient digit is **estimated** from the leading limbs and then corrected, which is what normalising the divisor so its top limb is at least `BASE/2` buys: it bounds the estimate's error to 2, where without it a small leading limb can make the estimate wrong by a factor of `BASE`.

It did not always work that way, and the reason it changed is a good example of a wart that was only theoretical until it was not. The quotient digit used to be found by **binary search** over `[0, BASE)`, at about thirty `BigInt` multiplications per limb. That made `gcd` — Euclid over `divmod`, which every `Rat` construction calls — take 865 ms on a 1,437-digit over 812-digit pair where Rakudo takes 3. And it was not a corner case: `Math::NumberTheory`'s `FatRat` digit expansions reduce `Rats` with ~600-digit parts on every step, so that file simply never finished.

### 11.2.1 The 64-bit fast path

Almost every bignum operation in real Raku code is on values that are not, in fact, big. They arrive as `BigInt` because a `Rat` stores its numerator and denominator as `BigInts` unconditionally — and *every* `Rat` construction calls `gcd` and then two `divmods`.

Measured on values that fit in 64 bits, `divmod` cost 2.1 microseconds and `gcd` — which is Euclid over `divmod` — cost 15.8 microseconds. So every decimal literal and every `p/q` in a program cost roughly 10 microseconds to build.

```
// src/BigInt.cpp — divmod, the fast path
// One hardware divide instead of the base-1e9 long division below, whose
// per-limb BINARY SEARCH costs ~30 BigInt multiplications.
if (a.fitsU64() && b.fitsU64()) { /* two 64-bit divides, rebuild */ }
```

```
// src/BigInt.cpp — gcd
// Euclid entirely in registers when both fit — the general loop below
// builds two BigInts per step and calls divmod, and Value::rat() calls
// this on EVERY Rat it constructs.
```

The guard is `fitsU64()`, which is a three-limb comparison against the base-10^9 digits of 2^64 − 1:

```
// src/BigInt.h
bool fitsU64() const {
    if (mag.size() > 3) return false;
    if (mag.size() < 3) return true;
    if (mag[2] != 18u)        return mag[2] < 18u;
    if (mag[1] != 446744073u) return mag[1] < 446744073u;
    return mag[0] <= 709551615u;
}
```

`Rat` construction became about nineteen times faster.

### 11.2.2 Two conversions to 64 bits, deliberately different

```
// src/BigInt.h
long long toLL() const;                   // SATURATES on overflow
unsigned long long toU64Wrap() const;     // WRAPS, two's complement
```

`toLL` saturates because its callers are indices, codepoints and range bounds, where a silent wrap produces garbage rather than a wrong-but-defined answer. `toU64Wrap` wraps because its caller is a native `uint64/int64` container, which is *defined* to keep

the low bits — and saturating there turned every 64-bit digest word into `0x7FFF_FFFF_FFFF_FFFF`.

Two functions with the same signature and opposite policies is a smell in general. Here it is the correct answer, and the comment in the header says so at length precisely because the next person will want to merge them.

## 11.3 Layer 2: `Rat`

A `Rat` is a normalised pair of `BigInt`s:

```
// src/Value.h
static Value rat(BigInt n, BigInt d) {
    if (d.sign == 0) return ratZ(std::move(n), std::move(d));
    Value v; v.t = VT::Rat;
    if (d.sign < 0) { n = -n; d = -d; }          // sign lives in the numerator
    BigInt g = BigInt::gcd(n, d);
    if (!g.isZero()) { /* divide both by g */ }
    v.ratN = std::make_shared<BigInt>(n);
    v.ratD = std::make_shared<BigInt>(d);
    return v;
}
```

Normalisation is eager: the denominator is positive and the pair is reduced at construction. That is what makes == on `Rat`s a component-wise comparison rather than a cross-multiplication, and it is why `gcd`'s speed matters.

A decimal literal is a `Rat`, built by the parser as a numerator and denominator pair (`NumLit::ratNum`, `ratDen`, with decimal-string fields for the cases that overflow `long long`). The node caches the constructed `BigInt` parts, because a literal in a hot loop would otherwise re-allocate and re-reduce on every evaluation.

### 11.3.1 The spill, and the type that is exempt

Raku caps a plain `Rat`'s denominator at 64 bits. Arithmetic that would produce a larger one degrades to `Num`:

```
// src/InterpreterCore.cpp — applyArith, the Rat result
bool fat = (l.t == VT::Rat && l.fatRat) || (r.t == VT::Rat && r.fatRat);
Value v = Value::rat(std::move(n), std::move(d)); v.fatRat = fat;
if (!fat && v.ratD && !v.ratD->fitsU64()) return Value::number(v.toNum());
return v;
```

`FatRat` is the same storage with a flag, and the flag does two things: it carries the type identity, and it exempts the value from the spill so a `FatRat` stays an arbitrary-precision rational forever. The flag is **contagious** — one `FatRat` operand makes the result a `FatRat` — which is the semantics Raku specifies and also the only way a chain of operations can stay exact.

`Rat.new` is subtly different from the `/` operator: a zero denominator must be *preserved* rather than treated as an error, normalised to ±1/0 with 0/0 kept as 0/0. That is `ratZ`, and taking `Str` or `Num` of such a value throws.

## 11.4 Native integer containers

`my int8 $x` is not a different type at runtime; it is a `Value` carrying a width:

```
// src/Value.h
int natBits = 0;       // 0 = not native
bool natSigned = false;
bool natFloat = false; // num32: truncate to float32 on assignment
```

Every assignment masks to the declared width, so `my uint8 $b = 300` stores 44. The width is derived from the declared type name once and cached, because deriving it involved a `substr` and therefore an allocation per typed parameter per call:

```
// src/Value.h
static int natWidthOfType(const std::string& ofType, bool& sign);
```

```
// src/Ast.h — Param
mutable DecidedOnce<int> natSpec{-1};   // bits<<1 | signed, decided once
```

Typed arrays and hashes use the same table through `ofType`, so `my uint32 @a` stores masked elements.

## 11.5 Complex

`VT::Complex` reuses `n` for the real part and `im` for the imaginary one — the only tag that uses `im` at all, apart from a fractional `Range`'s upper bound. Coercing a `Complex` back to a real checks the imaginary part against `$*TOLERANCE`, which defaults to 1e-15 and is looked up dynamically first, then lexically.

The language revision matters here: under `use v6.e.PREVIEW`, `sqrt(-1)` is a `Complex` rather than `NaN` — and so is `log(-1)`, and `log10`/`log2` of a negative, all of which

come through one `rtLogReal` so the branch is written once. `Interpreter::langRev_` carries the revision *of the code currently running*: each compilation unit records the revision it was compiled under, every routine is stamped with its unit's, and the call path switches to the callee's for the duration of the call. That is what lets a module compiled under 6.e keep `Complex` results when a 6.d program calls into it, without the 6.d program's own `sqrt` changing meaning.

## 11.6 Where the arithmetic actually happens

One function, `applyArith(op, l, r)`, implements every binary operator for every combination of numeric types, plus string operators, set operators, junction autothreading and `Whatever` currying. It is shared by the interpreter and the compiled backend, which is why the two agree byte for byte.

Its structure is a dispatch chain, and the order is performance-critical. Integer-integer and number-number cases are tested first, with a character switch on the operator rather than a chain of string comparisons. String comparisons were originally *late* in that chain, and paid about 110 nanoseconds walking past mixin delegation, negated operators, set operations, junction autothreading, `Whatever` currying and the `Version` branch before reaching the actual work — which is why `~`, `eq` and friends got their own fast path at the top too (Chapter 28).

## 11.7 A bug worth remembering: numifying by throwing

`Value::toNum()` numified a `Str` with `std::stod`, which **throws `std::invalid_argument`** when the string does not begin with a number. The result was caught and turned into `0.0` — the right answer, arrived at by raising, unwinding and catching a C++ exception.

Speculatively numifying a string that turns out not to be one is completely routine in an interpreter; the invocant of every `"ab".method` call reached that code. So the language's slowest control-transfer mechanism sat on its hottest path, at roughly 7 microseconds per method call.

`std::strtod` reports the same failure by leaving its end pointer at the start of the input, and costs nothing:

```
// src/Value.cpp
static double strToNumOr0(const std::string& s) {
    if (s.empty()) return 0.0;
```

```
    const char* p = s.c_str(); char* end = nullptr;
    errno = 0;
    double d = cnum::strtod(p, &end);
    if (end == p) return 0.0;   // nothing numeric at the front — stod would throw
    return d;
}
```

Measured over 300,000 iterations:

| | before | after | Rakudo |
|---|---|---|---|
| `'ab'.chars` | 2.23 s | **0.73 s** | 0.34 s |
| `'ab'.uniname` | 2.51 s | **0.47 s** | 0.36 s |

Roast was unchanged across the switch, which is the point: `strtod` and a caught `stod` agree on every input that reaches this path.

The general lesson is not about `stod`. It is that a C++ exception used as a *value-returning* mechanism — rather than for an error that genuinely aborts something — is a performance bug waiting for a profiler. The same realisation drives the cooperative control flow in Chapter 15.

# Chapter 12

# Containers, Binding, and Copy Semantics

A `Value` copy shares its payload. Raku sometimes wants that and sometimes wants the opposite, and the rules for which are the subtlest part of the value model. This chapter is about where the sharing is broken on purpose, how the things Raku calls *containers* are modelled without a container object, and, at the end, how shared payloads that refer to each other are freed.

## 12.1 Scopes are hash maps with a parent pointer

```
// src/Interpreter.h
struct Env {
    std::unordered_map<std::string, Value> vars;   // "$x", "@a", "%h", "&sub"
    std::shared_ptr<Env> parent;
    bool routineFrame = false;   // $/ scopes here
    bool loopFrame = false;      // a loop's `state` frame
    std::unique_ptr<EnvExtras> ex;

    Value* find(const std::string& name) {
        auto it = vars.find(name);
        if (it != vars.end()) return &it->second;
        return parent ? parent->find(name) : nullptr;
    }
    Value& define(const std::string& name, Value v) {
        return vars[name] = std::move(v);
```

```
    }
};
```

The **full sigilled name is the key**. Subs live in the same map under a `&`-prefixed key, so `&foo` and `$foo` occupy different namespaces without a second table. Lexical scoping *is* the parent walk in `find`; there is no separate symbol table anywhere in the interpreter.

Reading a variable returns a **copy** of the slot. Writing goes through `lvalue(Expr*)`, which hands back a `Value*` pointing straight into the owning `Env`'s map:

```
// read  $x
if (Value* p = tctx_.cur->find(ve->name)) return *p;      // a COPY

// my $x — make the slot here, hand back its address
tctx_.cur->define(ve->name, std::move(init));
return &tctx_.cur->vars[ve->name];

// $x = 5
*lv = rhs;
```

The three declarators differ only in *which* `Env` holds the slot:

| Declarator | Storage |
|---|---|
| `my` | the current lexical `Env` |
| `our` | the package env, ultimately the global one; also republished under a qualified name when the package block closes |
| `state` | a per-`Callable` `stateEnv`, created exactly once and shared across calls |

`state` is the only one that needs care under concurrency, and it gets a `std::once_flag` on the `Callable` so the persistent environment is created exactly once even if two threads call the routine simultaneously.

### 12.1.1 `EnvExtras`: the eight rarely-used maps

An `Env` is constructed for every routine call **and every block**. It also needs somewhere to keep `is rw` write-through links, `temp`/`let` restorations, `is default` values, and the set of names declared `is dynamic` — eight containers which are empty in the overwhelming majority of scopes.

Constructing and destroying eight empty maps per block is pure overhead for the ordinary case, so they live behind one lazily-allocated pointer:

```
// src/Interpreter.h
EnvExtras& x() { if (!ex) ex = std::make_unique<EnvExtras>(); return *ex; }
const EnvExtras& xr() const {
    static const EnvExtras kEmpty;
    return ex ? *ex : kEmpty;
}
```

Writers call `x()` and materialise it; readers call `xr()` and get a shared empty instance, so a lookup never allocates. This is a small pattern but a recurring one: **make the common case cost nothing, and pay only where the feature is actually used.**

## 12.2 Where the sharing is broken

Copying a `Value` bumps refcounts on the `shared_ptr` members, so a raw copy of an `Array` shares its backing `ValueList`. Raku wants:

```
my @a = 1, 2, 3;
my @b = @a;      # a COPY
@b[0] = 99;
say @a;          # (1 2 3) — unchanged
```

So the interpreter deliberately breaks the sharing at = assignment, and **only for the `@` and `%` sigils**:

```
// src/Interpreter.cpp — coerceArray
if (v.t == VT::Array) {
    if (v.itemized) { … }            // an itemized array is ONE element
    if (v.ext) return v;             // a lazy seq stays lazy
    Value r = Value::array(*v.arr);  // *v.arr copies the element list
    r.isList = false; return r;
}
```

`*v.arr` dereferences the pointer and copies the underlying `ValueList`, so `@b` gets its own storage. That copy is the most visible O(n) event in the language, and it is the reason `ValueList` is a container this project owns rather than a `std::vector` — see *The list container* below. Hashes copy the same way. Scalar assignment is a plain struct overwrite, so `my $x = @a` stores an `Array` value that *does* still share `@a`'s buffer — because an item container is a reference to one thing.

Two consequences to keep straight:

- **The copy is one level deep**, matching Rakudo. `my @b = @a` copies the top-level buffer, but a nested itemized array inside is copied as a `Value` struct, so its `arr` pointer is still shared.
- **A lazy sequence is not copied.** The `if (v.ext) return v` line is what keeps `my @a = 1, 2, 4 ... *` from trying to drain an infinite list.

The compiled backend has its own mirror of this rule, `rtArrayVal`, with the same fresh-buffer semantics, so interpreted and compiled code agree.

## 12.3 Binding: `:=` and the `Proxy`

`=` copies a value into an existing container. `:=` rebinds the container itself: after `$y := $x`, the two names *are* the same container and a write to either is seen by both.

But `Env` stores `Values` by value in a map, so two map slots cannot literally be the same storage. The alias is faked with a `Proxy`:

```
// src/InterpreterCore.cpp — $y := $x, scalar case
Value proxy = Value::makeHash(); proxy.hashKind = "Proxy";
(*proxy.hash)["FETCH"] = fetch;    // a builtin Code closing over the owning Env
(*proxy.hash)["STORE"] = store;
*lvalue(target) = proxy;
```

`$y`'s slot holds a `Hash` tagged `Proxy` whose two closures read and write `$x`'s slot in the environment that owns it. Reading a variable notices the tag and calls `FETCH` instead of returning the hash:

```
// src/InterpreterCore.cpp — reading a variable
Value* p = tctx_.cur->find(ve->name);
if (p) {
    if (p->t == VT::Hash && p->hashKind == "Proxy" && p->hash) {
        auto it = p->hash->find("FETCH");
        if (it != p->hash->end())
            return callCallable(it->second, { *p });
    }
    return *p;
}
```

Binding chains dereference one extra level so `$z := $y := $x` works.

This costs a call on every read of a bound variable, which is why the check is placed inside the *slow* half of the variable-read path and why the fast paths in Chapter 19 decline any value with a non-empty `hashKind`.

**Array binding is much cheaper**, because sharing a buffer is exactly what a `shared_ptr` copy already does:

```
// src/InterpreterCore.cpp — @a := @b
if (a->op == ":=" && rhs.t == VT::Array) {
    Value b = rhs; b.itemized = false; *lv = b;
}
```

The cleared flag is `itemized`, not `isList`, and the difference is two bugs rather than a detail. Clearing `isList` would make a bound `List` become an `Array`, which is wrong — `my @a := @b` where `@b` is a `List` must stay a `List`, and `URI::Path` is where that surfaced. What has to be cleared is the *itemization*: `$(1,2,3)` is one scalar item that does not flatten, and binding it to an `@` variable that then flattens it is how `DBDish::Pg` overflowed its stack.

`Value b = rhs` copies the struct and shares `rhs.arr`. This is the deliberate opposite of `@a = @b` above. `constant` reuses the binding path — a constant is `:=` in disguise.

The user-visible `Proxy` type — `Proxy.new(:FETCH{...}, :STORE{...})` — is the same mechanism exposed, so a program can build one explicitly.

## 12.4 The scalar-container metaphor

Raku's `$` variable is really a *Scalar container* holding a value, with introspectable machinery: `.VAR`, `is default`, type constraints. Raku++ models that without a separate container object, using flags on the `Value` plus per-`Env` side tables.

**`.VAR`** builds a `Hash` tagged `"Scalar"` reporting the variable's name, value and default.

**`is default(v)` and typed defaults** live in `EnvExtras::varDefault`, a per-scope map. `my Int $x` stores `(Int)` as both the initial value and the reset default. Assigning `Nil` walks the chain to find it:

```
// src/InterpreterCore.cpp — $x = Nil restores the container's default
for (Env* en = tctx_.cur.get(); en; en = en->parent.get()) {
    auto di = en->varDefault.find(nm);
    if (di != en->varDefault.end()) { dv = di->second; break; }
    if (en->vars.count(nm)) break;   // owner scope, no declared default
}
*lv = dv;                             // else Any
```

**Type constraints** on a scalar are enforced at assignment for the core nominal types, throwing `X::TypeCheck::Assignment` on a mismatch.

**`readonly`** is a flag on the `Value` itself, set when binding a plain `$` parameter. Mutating operations such as `s///` check it and die.

**`temp` and `let`** push restoration closures onto the scope's extras. `tempRestores` run whenever the scope leaves; `letRestores` run only on an *unsuccessful* exit, which is the distinction the language draws.

## 12.5 Sigils are mostly a parse-time fact

The runtime dispatches on the `VT` tag, not the sigil. The sigil — the first character of the name — is consulted exactly twice: to pick the empty container shape at declaration (`@` gives an empty `Array`, `%` an empty `Hash`, `$` an `Any`), and to choose the assignment coercion (`coerceArray`, `coerceHash`, or a scalar overwrite).

After that, behaviour follows the tag and two flags:

- **`isList`** marks a `VT::Array` that is a `List` or `Seq` rather than an `Array`. It renders with parentheses instead of brackets and flattens in list context. Same storage, different behaviour.
- **`itemized`**, set by `$(...)` or `$[...]`, marks an array that counts as *one* element in list context rather than flattening.

That pair carries a surprising amount of Raku's list semantics, and most of the one-argument-rule subtleties in the built-ins reduce to testing them.

## 12.6 The list container

`ValueList` is the type behind every Array — `Value::arr` is a `shared_ptr<ValueList>` — and it is also the argument list of every call. Until 2026-09 it was a `typedef` for `std::vector<Value>`. It is now `RVec<Value>`, a container this project owns (src/ValueVec.h), implementing the `std::vector` subset the tree uses with `std::vector`'s semantics: raw-pointer iterators, growth invalidating everything, `push_back(v[0])` still legal.

It differs in two places, both of them measured.

**It grows in one pass.** `std::vector` reallocating means filling a second buffer element by element and then walking the old buffer *again* to destroy each source. `RVec` constructs the new element and destroys the old one in a single walk. Over a hundred megabytes of `Value`, one pass instead of two is the larger half of this change: `arrayops` retires 30% fewer instructions, `listbuild` 26%, `sortnums` 19%.

**Where the standard library permits it, that pass is a `memcpy`.** A `Value` is *trivially relocatable* — moving its bytes to a new address and not destroying the source is equivalent to move-constructing and then destroying, which is exactly what a reallocation does. Nothing in it points at itself: `i/n` are scalars, `IStr` is an interned pointer, the payload and cold-block slots are `shared_ptr` pairs.

Nothing except one member. `CowStr`'s inline `std::string` is relocatable on libc++ and MSVC, which compute a short string's data address from `this`, and is **not** on libstdc++, which stores `_M_p` aimed at its own inline buffer — a bitwise copy there reads the original object's storage and dangles the moment it is reused. `bitwiseRelocOk()` asks the library at run time rather than trusting a macro, relocating a short string and checking where the copy's characters come from, and a library that fails the probe keeps correct behaviour and loses only the speedup.

That probe is worth a warning to anyone editing `Value`. Its first version asked whether a short string's `data()` lay *inside* the string object — which is true everywhere, because that is what the small-buffer optimisation is — so it answered "not relocatable" on every platform, the `memcpy` path never ran for a whole day of measurement, and nothing failed, because the fallback is correct. tools/reloc-probe.cpp prints which path is live, and exists precisely because a correctness-preserving fallback can hide a dead fast path indefinitely.

### 12.6.1 Small blocks come off a free list

A `ValueList` is not only a list. It is the argument list of every interpreted call, and for a one- or two-element list the heap block *is* the cost: 32 ns against a call that takes about 265. `RVec` keeps a thread-local free list of blocks for capacities 1 through 4, so allocation is a pop and release is a push. The one-argument shape drops to 9.5 ns; `fib` retires 8.8% fewer instructions, a two-argument call loop 13%.

The lists are kept **per exact capacity**, not per rounded-up size class, and that is the whole design decision. One four-element block for every small request is simpler and marginally faster on call-shaped work — but a `ValueList` is also every Array's payload, and a program holding a million one-element arrays then holds a million four-element blocks: 560–663 MB against 292–296, paying 22% of its cycles in the extra memory traffic. The two users of the type want opposite things from a block-sizing policy, and per-capacity lists are what give both of them what they want.

This is Perl 5's arena-and-free-list discipline (`sv.c`) applied to the one allocation two separate investigations had already named as the thing to remove — see Chapter 41.

## 12.7 Shaped arrays and typed containers

```
// src/Value.h
std::shared_ptr<std::vector<long long>> shape;  // my @a[2;3]
std::string ofType;                              // Array[Int], my Int @a
```

A shaped array is a fixed row-major structure with its dimensions recorded; an unshaped one leaves the pointer null. `makeShapedContainer` builds one, optionally filling it from a flat list.

`ofType` carries the element type for a typed container, comma-joined when there is more than one parameter (`Hash[Int,Str]` becomes `"Int,Str"`). It also drives native-width masking for `my uint8 @a`, through the same `natWidthOfType` table as scalars.

## 12.8 `is rw`, and writing back to the caller

Because arguments are passed as `Value` copies, a mutated `is rw` parameter has to be written back into the caller's variable. The call site passes the argument *expressions* alongside the values, and the binder records the link:

```
// src/Interpreter.h — EnvExtras
std::map<std::string, std::pair<Expr*, std::shared_ptr<Env>>> rwLinks;
std::map<std::string, Value> rwSynced;
std::map<std::string, Value*> rwDirect;
std::set<std::string> rwDead;
```

There are two mechanisms because there are two situations. `rwLinks` holds the caller's argument expression and environment, so an assignment to the parameter can re-resolve the lvalue and push the value through *immediately* — the caller sees the change mid-call, which is what Raku specifies. `rwSynced` records what was last pushed, so the copy-out backstop at return can skip parameters that are already up to date; without it a late copy-out would re-apply stale values over the callee's own later edits.

`rwDirect` is the simpler case: a hyper-operator element call already has the caller's slot as a pointer, with no expression to re-evaluate. `rwDead` marks a raw or `rw` parameter bound to a literal, so assigning it dies with `X::Assignment::RO` rather than silently writing into a temporary.

This works for **direct** calls, where the caller supplied the argument expressions. Multi-dispatched and indirect calls can lose that fidelity — a known limitation, documented rather than hidden.

## 12.9 Freeing cycles

Every payload this chapter has described is reference-counted: an array, a hash, an object, a Pair's value, a container cell. The count lives in the body (`src/Ref.h`), and the release that takes it to zero frees the body on the spot. That is what gives Raku++ its small floor and its pause-free run time, and it has the classic hole: two bodies that refer to each other keep each other's count at one after the program has let go of both. Issue #151 was the plainest form of it, a loop that made a pair of objects pointing at each other and dropped them. Rakudo held that loop at 143 MB; Raku++ grew by 235 MB every 20,000 pairs and freed none of them.

The fix is a **backup cycle collector** (`src/Cycle.h`, `src/Cycle.cpp`). Reference counting stays the primary mechanism and keeps everything it was already good at. The collector only looks for groups that counting cannot free, and frees them by clearing their contents, after which ordinary counting does the rest.

### 12.9.1 Why the end of a scope cannot decide

The obvious first idea is to free an object when the variable that named it goes out of scope. It is wrong whenever the object escaped: pushed onto an outer array, captured by a closure, stored in another object, returned. Proving that nothing escaped is escape analysis through `new`, `BUILD`, `is rw` accessors and every method call, and in Raku the honest answer is "might escape" almost everywhere.

What the end of a scope *can* supply is a trigger. A group becomes garbage at the moment its last outside reference goes away, so every release that leaves a container alive is a moment it may have done so. The container is then a **candidate**.

### 12.9.2 Trial deletion

The algorithm is the synchronous form of Bacon and Rajan's cycle collection (ECOOP 2001), done without touching the live counts:

1. From the candidates, walk the references that bodies hold.
2. For each body reached, count the references to it from other bodies the walk reached.

3. A body whose real count is higher is held from outside the walk. It is live, and so is everything it reaches.
4. Whatever is left is referenced only from inside the group: garbage.
5. Run the garbage objects' `DESTROY`s while every link is still intact, then count again, because a `DESTROY` may have stored `self` somewhere.
6. Clear the garbage bodies' contents. Their counts fall to zero and the normal release path frees them.

The safety argument is one sentence. A body is cleared only when every reference its count records comes from a body in the garbage set. A reference the walk cannot see (a C++ temporary, a native callback's capture, another thread's copy) leaves the count higher than the walk explains, so the body looks held and nothing is freed. A walk cut short by a budget leaves edges uncounted, with the same effect. The single error that could free something live is counting an edge that holds no reference, so `forEachEdge`, the function that lists a body's references, reports only handles that each own exactly one count. When a count comes out *below* what the walk explains, that is proof of such an error, and the collection is abandoned.

Two holders need care. A hash entry handed out as a live Pair keeps the entries alive through a shared generation, not through a count, so a hash with outstanding aliases is treated as held from outside. And the `DESTROY` registry holds one count on every object whose class declares a `DESTROY`; a flag bit on the body tells the walk to account for it.

### 12.9.3 Candidates that hold nothing

The first design held a counted reference to every candidate, and that is the design to avoid: it delays every free until the next collection, and a handle closed by its destructor would close later than it does today. The candidate list holds **raw pointers**. A flag bit in the body's `pflags_` word marks it as buffered, and the bits above carry its index in the list. When a buffered body's count reaches zero, the release path removes it (`cycleForget`) before freeing it. Being a candidate never keeps anything alive, so a collection can wait for 1,024 candidates without changing what a program observes.

### 12.9.4 Which releases make a candidate

The plan was to note candidates at scope exits only. Measured against eight ways a program drops a cycle (reassigning an outer variable, replacing an array element, `:delete`, `@a = ()`, a sunk `N.new.link`, `.shift`, overwriting an attribute, the result list

of a `.map`), scope exits caught one. Rakudo frees all eight. The drops that matter happen everywhere a `Value` is released, and the only place that sees all of them is the release itself.

Hooking every release costs on the hottest path in the engine. A comment in `Ref.h` records that inlining the destroy call there once cost 2.5% from code size alone. The answer folds the decision into the count:

```
// src/Ref.h
void release() noexcept {
    if (p_) {
        std::uint32_t old = p_->refs_.fetch_sub(1, std::memory_order_acq_rel);
        if ((std::int32_t)old <= 1) payloadReleaseSlow(p_, old);
    }
}
```

An object's body is created with the top bit of its count set (`kRefTracked`). Its count therefore reads negative, so every release of it leaves the inline path, exactly as the last release of any body does, and the inline test is still one compare. The out-of-line half frees on a last release and otherwise notes a tracked body as a candidate. Every reader of a count masks the bit. A buffered object has the bit cleared while it waits, so its later releases stay inline, and gets it back if a collection finds it alive.

Arrays, hashes, Pairs and cells are not tracked: they are released far more often than objects, and the cycles they form on their own are rarer. They are noted where a scope exit drops them. When the drop frees one instead, its own contents lose a reference too, so the note looks into the dying body, a bounded number of levels down.

### 12.9.5 Scopes are nodes too

A routine that declares `my sub helper`, or keeps a block in a variable, makes a cycle every time it runs: the frame holds the block, and the block holds the frame as its closure. Before the collector, every call of such a routine leaked its whole frame; Rakudo freed 20,000 of 20,000 and Raku++ none. Frames are `Env`s reached through `std::shared_ptr`, not payload bodies, so the walk treats an `Env` as a node of its own. Its count is the pointer's `use_count`, and an edge to it is a handle that shares its control block (an aliasing or non-owning handle is not counted). The walk does not enter a program's, a module's, a package's or an `EVAL`'s scope: the process holds those for good, and their slots are most of what a program has.

Breaking such a cycle clears the routine, not the scope. Resetting a garbage `Callable`'s `closure` and `state` scope releases the frame, and the frame's own count takes it from there. A frame is not cleared in place: interpreter code holds raw pointers into frames' slots, and a frame that dies by its count dies the way it always has.

The trigger here is a frame that a closure keeps alive when its routine returns or its loop iteration ends: its variables become candidates. A kept frame can hold any amount of memory, so these candidates weigh 64 toward the 1,024 that start a collection. Counted as one each, 300 calls of a routine keeping a block beside a 1 MB string peaked at 294 MB; weighted, at 18 MB.

### 12.9.6 When it runs

A collection reads counts while it walks, and parallel mode has threads changing them. It therefore runs only at a **quiet point**: on the main thread, with no worker live, at the end of a scope (a pooled frame returned, a loop iteration's scope cleared, a flat loop body's iteration) or at the top of a compiled loop iteration. While workers run, candidates wait, and the threshold doubles each time a collection is put off. Candidates may be noted on any thread; the list sits behind a spinlock that costs an exchange and a store when uncontended.

`DESTROY` runs from the collection, at those same points. That is no new licence: the registry sweep already runs `DESTROY` when constructing an object pushes the registry past its threshold. A collection never starts while an exception unwinds.

### 12.9.7 Bounding the cost

The danger is a hot routine whose local variable names the root of a large structure that is cyclic but live, such as a tree with parent links. Every return would walk the tree to find it held. Four bounds keep that in check:

- **A node budget per candidate.** A walk that would add more than 1,024 nodes stops adding, and pins the nodes it has not looked inside. A pinned node counts as held from outside.
- **A read budget per walk.** The node cap alone was not enough: a walk read every element of a node before checking the cap. Running `t/scaling` with a collection at every scope exit made two of its shapes quadratic, because a growing attribute array was read whole each time. A walk now stops after 16,384 values, and the node it stopped inside is pinned.

- **Shared work within a collection.** Candidates are walked one at a time, but a node an earlier walk reached is not walked again. A collection that finds nothing doubles the threshold for the next, up to 2 ^ 20; one that finds garbage resets it.
- **No node for nothing.** An empty array or hash is not given a node at all, since it holds nothing and so is on no cycle. Together with an open-addressing index in place of `std::unordered_map` and one reused queue, that took the strategy's worst measured case (200,000 children each pointing back at a live parent) from +8.4% to +2.9% instructions.

One more saving is static. A class whose attributes can only hold values (a native, `Int`, `Str`, `Num`, `Rat`, `Bool`, ...), counting its parents' and roles' attributes, cannot be on a cycle, and its objects are not tracked. The verdict is cached on the `ClassInfo` the first time one of its objects is released, a point every way of making an object passes through. It is a heuristic, and it errs only toward a leak: a value type can be subclassed (`class A is Int { has Int $.next }`), and a cycle made only of such objects has no tracked member.

### 12.9.8 How it was checked

`t/leak/` holds one file per shape of cyclic garbage and per shape of live cycle that must survive, each counting `DESTROY` calls. It passes under Rakudo as well. Every file also runs with `RAKUPP_CYCLE_POISON=1`: instead of clearing a collected object, the collector gives it a class whose every method dies with `CYCLE-COLLECTED[...]` in the message. `DESTROY` still runs, so every count still holds, and any later use of a collected object fails loudly. `RAKUPP_CYCLE_TRIGGER=1` makes a collection run at nearly every scope exit, which matters because at the normal threshold most Roast files are too short to collect at all. Roast under that setting, with and without poison, passed 220,845 of 220,845 tests, and the threaded paths ran clean under ThreadSanitizer.

On the benchmark kernels the collector costs under 0.6% on twenty of twenty-two. The object-construction kernel pays 1.3–1.5%: each of its 200,000 objects goes on the candidate list once and comes off when it is freed.

## 12.10 Honest limitations

- **Some cycles still leak.** A cycle larger than one walk's budget, and one that runs through a promise, a channel, a lazy generator or a native callback, are

not freed. A program that always has a worker thread running collects only at the moments it has none.

- **`:=` on a scalar costs a call per read.** The `Proxy` is correct and general, and it is not free.
- **`is rw` write-back is expression-based.** Where the argument expression is not available or not re-resolvable, the write-back does not happen.
- **The one-level copy rule** matches Rakudo, but it surprises people regularly, and no amount of documentation seems to fix that.

# Part IV

# The Interpreter

# Chapter 13

# The Tree Walk

Everything so far has been about turning text into a tree. This part is about what happens when something runs that tree, and the first thing worth saying about the runner is what it does not contain.

There is no bytecode. `Interpreter::eval(Expr*)` returns a `Value`; `Interpreter::exec(Stmt*)` runs a statement and returns its value. Both are recursive, both switch on the node's `NK` tag, and the C++ call stack *is* the Raku call stack.

That decision buys a small implementation and costs throughput, and every optimisation in Chapters 19, 27 and 28 exists because of it.

## 13.1 The two entry points

```
// src/Interpreter.h
Value eval(Expr* e);
Value exec(Stmt* s, bool sink = false);
Value execBlock(Block* b, std::shared_ptr<Env> scope, bool sink = false);
```

`exec` takes a **sink** flag. When a statement's value is discarded — a loop body, a statement in the middle of a block — the flag says so, and an assignment can skip materialising its (possibly very large) result. It is a small thing that turns the naive `$s ~= ...` in a loop from quadratic into linear, in combination with the in-place append described in Chapter 27.

`execBlock` runs a statement list in a given scope, handling the phaser schedule around it.

## 13.2 Execution registers

The state that belongs to one thread of Raku execution is gathered into a struct rather than scattered across interpreter members:

```
// src/Interpreter.h — ExecContext, abridged
struct ExecContext {
    std::shared_ptr<Env> cur;           // current lexical scope
    std::vector<Env*> dynStack;         // the dynamic ($*foo) caller chain
    int callDepth = 0;
    Env* curStateEnv = nullptr;
    std::vector<std::shared_ptr<ValueList>> gatherStack;
    std::vector<ValueList*> supplyStack;
    std::vector<Value*> makeTargets;
    std::string pkgPrefix;

    bool returning = false;  Value returnV;      // cooperative return
    uint64_t frameTop = 0, curRoutineFrame = 0;
    int loopCtl = 0;         uint64_t curLoopFrame = kNoFrame;
    int givenCtl = 0;        Value givenV;  uint64_t curGivenFrame = kNoFrame;

    struct CallSite { int line; const Value* code; };
    std::vector<CallSite> callFrames;           // for callframe(N), backtraces
    int wantLvalue = 0;  Value* lvalueOut = nullptr;
};

static thread_local ExecContext tctx_;
```

It is `static thread_local`, so each real worker thread owns its own set. That is the foundation of the concurrency model in Chapter 38 — with per-thread registers, running Raku on a second thread does not need a register swap at every handover.

Two chains, not one. **`cur`** is the lexical chain: a callee's parent is the scope it was *written* in, reached through its `Callable::closure`. **`dynStack`** is the dynamic chain: the caller's scope, pushed on entry, walked only when resolving a `$*foo`. Keeping them separate is what makes lexical and dynamic scoping both correct and both cheap.

## 13.3 Evaluating an expression

`eval` is a switch. The interesting arms are the ones that are *not* a straightforward recursion.

**`VarExpr`** is the hottest node in most programs. Its fast path is a `find` and a copy:

```
// src/InterpreterCore.cpp — eval(VarExpr), the plain-lexical fast path
if (Value* p = tctx_.cur->find(ve->name))
    if (!(p->t == VT::Hash && p->hashKind == "Proxy"))
        return *p;
```

with the `Proxy` check placed so that the common case is one comparison. Names with a twigil — `$*dyn`, `$?FILE`, `$^a`, `$/` — never reach this path; each has its own lookup rule, and the fast paths in Chapter 19 exclude them by testing the second character of the name.

**`Binary`** dispatches through `applyArith`, but first consults two decided-once fields on the node: `simpleOp`, which records whether this operator needs special handling at all, and `fastShape`, which records the syntactic shape of the operands. Chapter 19 is entirely about those. A third, `specialArm`, names which special-cased arm handles the operator, so `~~` or `|` goes straight there instead of past a hundred tests for other spellings; `Unary` has the same thing as `evalPath`.

**`Index`** reads a container element, and has a matching `fastShape`.

**`Call`** and **`MethodCall`** are Chapters 14 and 16.

**`InterpStr`** evaluates each part and concatenates. The parts were already parsed into sub-expressions by the front end, so there is no string scanning here at all.

**`NqpOp`** exists only under `use nqp` (Chapter 35).

## 13.4 Evaluating a statement

`exec` is the same shape, with the control-flow statements doing the real work. Two mechanisms recur through all of them.

### 13.4.1 Hoisting

Before a statement list runs, two things are pre-registered.

**`hoistSubs`** walks the list and defines every named `SubDecl` in the scope, so a sub is callable across its whole enclosing scope regardless of textual order. This is why subs need no forward declaration while *operators* do (Chapter 6): sub visibility is a runtime fact, operator visibility a parse-time one.

**`hoistExprDecls`** pre-declares `my` variables that are buried inside expressions — in a ternary branch, in an `nqp::` argument — because Raku scopes them to the block, not to the expression.

Both are guarded by a decided-once flag so the walk happens once per block rather than once per entry:

```
// src/Interpreter.h
void hoistExprDecls(const std::vector<StmtPtr>& stmts, Env* env,
                    DecidedOnce<signed char>* cache = nullptr);
```

```
// src/Ast.h — Block
DecidedOnce<signed char> hoistNeed{-1};  // -1 undecided, 0 no, 1 yes
```

For a block with nothing to hoist — the overwhelming majority — the cost after the first entry is reading one byte.

### 13.4.2 Phasers

`execBlock` runs the phaser schedule around the statement list:

| Phaser | When |
|---|---|
| `ENTER`, `FIRST` | at block entry, in source order |
| `LEAVE` | at block exit, reverse order, always |
| `KEEP` | at block exit, only on a successful exit |
| `UNDO` | at block exit, only on an unsuccessful one |
| `NEXT` | at the end of each loop iteration |
| `LAST` | once after a loop's final iteration |
| `CATCH`, `CONTROL` | as handlers around the block |

```
// src/Interpreter.h
void runEnterPhasers(const std::vector<StmtPtr>& stmts);
void runLeavePhasers(const std::vector<StmtPtr>& stmts, bool ok = true);
void runNextPhasers(const std::vector<StmtPtr>& stmts,
                    std::shared_ptr<Env>& scope);
```

The program-level phasers — `BEGIN`, `CHECK`, `INIT`, `END` — are scheduled by `run()` rather than by a block: `BEGIN` in source order after the parse, `CHECK` reversed, `INIT` immediately before the mainline, `END` at process exit. This is where Chapter 5's compile-time decision becomes visible: a `BEGIN` block runs *after* the whole file has been parsed, so it cannot influence how later source is read.

## 13.5 Loop bodies

Every loop form funnels through one function:

```
// src/Interpreter.h
bool runLoopBody(Block* b, std::shared_ptr<Env> scope,
                 const std::string& label = "",
                 bool isFirst = true, bool isLast = true,
                 ValueList* collect = nullptr,
                 const std::function<void()>& rebind = nullptr);
```

It runs one iteration and handles `redo`, `next` and `last` — returning `false` when the loop should stop — plus the `FIRST`/`NEXT`/`LAST` phasers. The `collect` parameter is how a loop used in **value context** works: `my @x = do for @y { ... }` appends each iteration's value rather than discarding it. The `rebind` callback re-binds the loop variable for a `redo`.

Concentrating this in one function is what makes `for`, `while`, `until`, `loop` and `repeat` agree about control flow, which they historically did not.

## 13.6 Aliasing loops

`for @a { $_ = ... }` must write **into** `@a`, so the topic has to alias the element rather than copy it. That requires knowing what the loop source *expression* is, not just its value, so the interpreter has a small family of functions that peer at the source expression:

```
// src/Interpreter.h
std::shared_ptr<std::map<std::string, Value>> valuesAliasSource(Expr*);
std::shared_ptr<ValueList> derefArrayAlias(Expr*);
bool scalarListAlias(Expr*, std::vector<Value*>& slots);
Value* topicAliasSlot(Expr* topic, bool skip);
Expr* peelGrepFilter(Expr* listExpr, Expr*& pred);
```

The last one is the sort of detail that only shows up against a real test suite: `for %h.values.grep(...) { $_ = ... }` must *still* alias, because Rakudo's `grep` is `is raw`. So the loop peels a trailing `.grep(PRED)` off the source expression, aliases the underlying container, and applies the predicate as a filter during iteration.

## 13.7 Sink context and `it does not matter what this returns`

Statement values matter in Raku — the last statement of a block is its value — so `exec` returns one. But most statements' values are discarded, and building them can be expensive.

The `sink` flag threads that knowledge down. Its most valuable use is assignment: in sink context, `evalAssign` does not need to produce the assigned value as a result, so `@big = @other` does not build a second copy to throw away. Combined with `rtCatAssign`'s in-place append (Chapter 27), this is what makes string building in a loop linear.

## 13.8 Recursion, and the stack it needs

Because the C++ stack is the Raku stack, recursion depth is bounded by the thread's stack size. The default for a non-main thread on macOS is 512 KB, which a recursive Raku sub overflows within about a hundred frames.

So every entry point runs the program on a thread with a very large stack:

```
// src/Runtime.h
int rakuppRunBigStack(const std::string& src, std::vector<std::string> args,
                      const std::string& fileName, const std::string& exePath,
                      const std::vector<std::string>& libPaths = {});
int rakuppMainOnBigStack(int (*body)(void*), void* ctx);
```

and worker threads get the same treatment through `BigStackThread`, which reserves 256 MiB of *virtual* address space — committed only as used (Chapter 38). A compiled `--exe` binary calls `rakuppMainOnBigStack` for its own main body, so the recursion budget is the same in every mode.

The interpreter also keeps a `callDepth` register and raises a clean Raku error when it gets too deep, so a runaway recursion produces a diagnostic rather than a segmentation fault.

## 13.9 What this costs, honestly

Re-dispatching every AST node on every execution is inherently slower than bytecode or a JIT — but by how much is a question with an answer, and the answer is smaller than the sentence suggests. Counted directly: the dispatch `switch` costs **0.32 ns**, and a node visit costs **46 to 85 ns**. The re-dispatch is under one per cent

of what visiting a node costs. Chapter 42 has the measurement and what follows from it.

Where the rest goes is the profile of a method-heavy loop, after the fixes in Chapters 10 and 11:

| | share |
|---|---|
| heap allocate and free | 31% |
| `Value` copy and destroy | 11% |
| method-name comparison | 8.5% |
| the dispatch function's own body | 6.1% |

Note what is *not* there: no interpreter loop overhead line, no instruction decode. The tree walk itself is not the problem; **allocation and value churn are**, and they would still be there in a bytecode VM built on the same value model.

That is why the optimisation work in this book goes after allocation — the per-call `ValueList`, the per-operation `Value` box, the per-copy string — and not after the dispatch mechanism. The one place the dispatch mechanism itself was worth attacking is Chapter 19, and even there the win came from removing copies rather than from removing branches.

All three of those allocations have since been attacked, and the sentence above should now be read in the past tense: the per-copy string by `CowStr` (Chapter 9), the per-operation `Value` by the native lanes (Chapter 27), and the per-call `ValueList` twice — removed outright in compiled code by the optimiser's first pass, and made nearly free in interpreted code by the block free list of Chapter 12. What is left in the 31% is the object and payload allocations that a program genuinely asks for.

# Chapter 14

# Calls and Parameter Binding

Calling something is the most frequent non-trivial act in a Raku program, and in a tree-walker it is also one of the most expensive: the floor for a trivial call was around forty-six nanoseconds when this chapter was first measured, and only about a quarter of that was dispatch. Where the other three quarters went is what shaped the optimizer of Chapter 27 — and, later, what got most of the same win back inside the interpreter. Both are easier to see once the mechanism is laid out.

A call happens in three phases: build the argument list, activate the callee, bind the parameters. Each has a fast path, and each fast path exists because something measurable was in the way.

## 14.1 Building the argument list

`evalArgs` evaluates the argument expressions into one flat `ValueList`:

```
// src/InterpreterCore.cpp — evalArgs, abridged
} else if (a->kind == NK::Unary && ((Unary*)a.get())->op == "|") {
    Value v = eval(...);                                  // a Slip: |@a / |%h
    if (v.t == VT::Array || v.t == VT::Range)
        for (auto& x : v.flatten()) args.push_back(x);
    else if (v.t == VT::Hash && v.hash)
        for (auto& kv : *v.hash) {                        // |%h → named args
            Value p = Value::pair(kv.first, kv.second);
            p.namedArg = true; args.push_back(p);
        }
} else {
```

```
    Value v = eval(a.get());
    if (v.t == VT::Pair && a->kind == NK::Pair &&
        !((PairExpr*)a.get())->quotedKey && ident)
        v.namedArg = true;
    args.push_back(std::move(v));
}
```

Two things to notice.

**An argument list is just a `ValueList`.** Named arguments are ordinary `Pair` values carrying a `namedArg` flag; the positional/named split happens later, at bind time. That is why spreading (`|@a`), slurping (`*@rest`) and flattening are all list operations rather than a separate protocol.

**And that list is the most frequent one in the tree**, which is why it stopped being a `std::vector`. One is built, filled, passed and destroyed on every interpreted call, and for the one- or two-element shape almost the whole of its cost was the heap block: 32 nanoseconds against a call that takes about 265. `ValueList` now takes small blocks off a thread-local free list — allocation is a pop, release is a push — and that shape costs 9.5 ns. See *The list container* in Chapter 12 for the container itself, including the reason the free list is kept per exact capacity rather than rounding every small request up to one size class: an argument list and a million-element array want opposite things from that policy, and they are the same type.

**Only a *syntactic* pair is a named argument.** `f(k => 1)` and `f(:k(1))` pass a named argument; `f($pair)` and `f(3 => 4)` pass a positional one, even though all four produce a `Pair`. The test is on the *expression* kind, and it also excludes a quoted key — `f('a' => 1)` is positional, per Raku's rule. Getting this wrong makes every module that passes pairs around behave subtly differently, so the check is deliberately narrow.

## 14.2 Activating the callee

```
// src/Interpreter.h
Value callCallable(const Value& codeVal, ValueList args,
                   const std::vector<ExprPtr>* rwArgs = nullptr,
                   bool ownFrame = false, bool arityCheck = false);
Value callCallableRaw(const Value& codeVal, ValueList args,
                      const std::vector<ExprPtr>* rwArgs,
                      bool ownFrame = false, bool arityCheck = false);
```

`callCallable` is a thin **wrapper layer**: if the routine has been `&r.wrap({...})`'d, it runs the wrapper stack, each level able to `callsame` into the next; otherwise it passes straight through:

```
// src/InterpreterCore.cpp — callCallable
if (codeVal.code && !codeVal.code->wrappers.empty()) { /* run the stack */ }
return callCallableRaw(codeVal, std::move(args), rwArgs);
```

`callCallableRaw` handles the special callables first — a native FFI sub, a `Format`, junction autothreading, a multi-dispatcher, a builtin — and then activates a user routine:

```
PooledFrame frame;                                    // a per-call frame, reused
auto& env = frame.env;
c.state.env->parent = c.closure ? c.closure : global_; // once
env->parent = c.state.env;        // frame → state env → closure → … → global
tctx_.dynStack.push_back(caller_scope);               // the OTHER chain
```

The frame is *pooled*, not freshly allocated, and the reason is one of the sharper measurements in this book. A call frame is a `shared_ptr<Env>`: a control block, an `Env`, and the hash buckets inside it, allocated and destroyed per call. A thread-local pool of up to thirty-two hands one back instead, clearing the map but keeping its bucket array — the same trick the pad vector plays with its capacity. A frame that anything captured (a closure, an `rwLink`, a state chain) fails a `use_count` test on release and is simply dropped rather than reused, which is what makes the reuse safe.

For a while only `callCallableRaw` had it, and `invokeMethod` did not. That asymmetry alone was the difference between a sub call at about twice Rakudo's cost and a method call at 5.8 times it.

Two facts are established here and everything else depends on them.

**The callee's parent is its lexical closure, not its caller.** Free variables resolve through the scope the routine was *written* in. The caller's scope goes on `dynStack`, used only for `$*foo`.

**Each activation bumps `frameTop`**, and a routine — as opposed to a bare block — records its own frame number as `curRoutineFrame`. Those two counters drive the cooperative control flow in the next chapter.

### 14.2.1 The call registers

A handful of one-shot parameters are passed to the next activation without widening every signature:

```
// src/Interpreter.h
static thread_local Value* topicWriteback_;
static thread_local Value* builtinTopicWB_;
static thread_local bool noAutothread_;
static thread_local int loopPhaserCtl_;
static thread_local const std::vector<Value*>* pendingRwSlots_;
```

Each is set immediately before a call and consumed at the top of `callCallableRaw`. `topicWriteback_` is how `@a.grep({ $_++; True })` writes into `@a`'s element: the driver points it at the element, and the mutated implicit `$_` is copied back after the call.

They are `static thread_local` and the comment in the header explains why in one sentence: as plain members they were written by *every* call on *every* thread, and they were ThreadSanitizer's top report — 2,761 lines on a program with no sharing in it at all. The set-then-consume window is contiguous within one thread, so thread-local is not a workaround, it is exactly their semantics.

## 14.3 Binding parameters

`bindParams` maps the argument list onto the signature. There is a fast path and a general path.

```
// src/InterpreterCore.cpp — bindParams
if (simple) {                       // all plain positional $ params, no nameds
    for (size_t i = 0; i < params.size(); i++) {
        Value v = i < args.size() ? args[i]
                                  : typedDefault(params[i].type, '$');
        v.readonly = true;
        env->define(params[i].name, std::move(v));
    }
    return;
}
for (auto& a : args)                // general: split named from positional
    if (isNamedArg(a)) named[a.s] = a.pairVal ? *a.pairVal : Value::any();
    else positional.push_back(a);
```

Whether a signature qualifies for the fast path is a static property of the signature, so it is decided once and stored on the *first* parameter:

```
// src/Ast.h — Param
mutable DecidedOnce<signed char> sigSimple{-1};   // only element [0] is read
```

The general path covers everything else:

- **positional parameters with defaults**, evaluated in the parameter scope so `sub f($g, $a = $g/2)` can see earlier parameters;
- **readonly, `is rw`, `is copy`, `is raw`** — a plain `$` parameter is marked readonly unless it is one of those or the invocant:

  ```
  if (p.sigil == '$' && !p.isRw && !p.isCopy && !p.invocant)
      v.readonly = true;
  ```

- **slurpies**: `*@rest` flattens, `**@rest` does not, `+@rest` applies the single-argument rule, `*%named` collects the rest;
- **named parameters**, including the alias forms `:a(:$b)` and nested `:x(:y(:z($a)))`, where every layer's key answers;
- **sub-signature destructuring**, `sub f([$a, $b])`, recursing through `Param::subSig`;
- the implicit **`$_`** topic, and placeholder parameters `$^a`, `$^b`, bound in sorted order.

### 14.3.1 What is *not* checked here

**Type constraints, `where` clauses and `:D`/`:U` smileys are not enforced for an ordinary, non-multi call.** Single dispatch is largely duck-typed at the bind boundary. Those checks live in `scoreCandidate`, which is multi dispatch's business (Chapter 16).

There is one exception, `typeCheckBind`, used when a lone candidate is being bound and a mismatch should raise `X::TypeCheck::Binding`. It caches its verdict on the parameter — but **only once true**:

```
// src/Ast.h — Param
mutable DecidedOnce<signed char> typeKnown{0};
```

The asymmetry is important and the header says why: a type name that is unresolvable now can become resolvable later, when a class further down the file is declared

or a module is loaded. Caching the *positive* answer is safe; caching the negative one is a bug that would only show up in a program whose declarations are ordered a particular way.

## 14.4 The `Callable`, and what it caches

```
// src/Value.h — Callable, abridged
std::string pkg, name;
const std::vector<Param>* params;      // borrowed from the AST
const std::vector<StmtPtr>* body;      // borrowed from the AST
std::shared_ptr<Env> closure;
StateSlot state;                       // { shared_ptr<Env> env; once_flag init; }
BuiltinFn builtin;                     // set ⇒ this is a builtin
ValueList candidates;                  // multi-dispatch candidates
ValueList wrappers;                    // .wrap stack, outermost last
PublishedOnce<signed char> hoistNeed{-1};
PublishedOnce<signed char> arityShape{-1};
int arityMaxPos = 0, arityReqPos = 0; bool arityUnbounded = false;
PublishedOnce<signed char> catchScan{-1};
Stmt* catchBlkCache = nullptr;
```

The `state` env and its `once_flag` are wrapped in a `StateSlot` for a reason worth knowing: `std::once_flag` is not copyable, so a bare pair of members would make `Callable` uncopyable. The holder's copy constructor is empty, which also gets the semantics right — a cloned routine starts with fresh `state` slots rather than sharing the origin's, as it does in Rakudo.

The four published-once fields are all static properties of the routine's AST that `callCallableRaw` used to recompute **on every call**: whether anything needs hoisting, whether an arity pre-check applies and what its bounds are, and whether the body contains an inline `CATCH` block. Each is cheap once and worthless repeated — a parse that calls a routine 73,603 times paid for them 73,603 times.

`state.env` is created exactly once, under that `once_flag`, and chained between the per-call frame and the closure. That is the whole implementation of `state`: the variable lives in a scope that is created once per routine rather than once per call.

## 14.5 is rw write-back

Because arguments are Value copies, a mutated is rw parameter must be written back. The call site passes the argument *expressions* alongside the values, and on a normal return each such parameter's final value is written back by re-resolving its expression:

```
// src/InterpreterCore.cpp — copyOutRw
if ((p.isRw || p.sigil == '\\') && pi < rwArgs->size())
    if (Value* lv = lvalue((*rwArgs)[pi].get()))
        *lv = env->vars[p.name];
```

setupRwLinks additionally arranges for an assignment *inside* the callee to push through immediately, so the caller sees the change mid-call. The bookkeeping that keeps those two mechanisms from fighting is described in Chapter 12.

The whole thing is guarded by a sticky flag, anyRwLinks_, so a program that never uses is rw never runs the per-assignment hook at all.

## 14.6 Lvalue-mode method calls

$obj[$i] = $v on a class whose AT-POS is return-rw @!arr[$i] must write the *real* element, not a returned copy. That needs a channel from the subscript site into the routine's return-rw:

```
// src/Interpreter.h — ExecContext
int wantLvalue = 0;      // 0 off, else the callFrames depth being served
Value* lvalueOut = nullptr;
```

The subscript-lvalue path sets wantLvalue to the current frame depth plus one before invoking AT-POS; a return-rw executing at exactly that depth fills lvalueOut with the address of its operand. The pointer survives the frame because its target lives in the object's shared containers.

Matching on depth rather than on a plain flag is what keeps an inner call from accidentally answering an outer call's request.

## 14.7 What a call costs

Measured against the runtime library, 2 million iterations, minimum of six:

| Shape | ns/call |
|---|---|
| direct C++ call, user-sub shape | 46.3 |
| cached `BuiltinFn*` call | 47.4 |
| by-name lookup: hash, `unordered_map::find`, `std::function` | 55.0 |

The lookup tax is real but modest at 8 to 9 nanoseconds. The important number is the **floor**: about 46 nanoseconds for a *trivial* call, nearly all of it the `ValueList` — at the time, a heap-allocating `std::vector` built per call.

Dispatch was a quarter of the overhead. The argument list was the rest. That finding shaped two different pieces of work, years apart in the book's ordering and months apart in fact.

The first is the optimiser of Chapter 27: its opening pass gives fixed-arity subs direct `Value` parameters and removes the list entirely, which buys more than any lookup cache can. That only ever helped compiled code.

The second is the free list above, which attacked the same allocation from inside the interpreter and took most of it: the one-argument shape from 32.35 to 9.48 ns, `fib` 8.8% fewer instructions, a two-argument call loop 13%. The lesson is not that the optimiser was unnecessary — it removes the list rather than making it cheap, and it is still the larger win where it applies. It is that a cost identified once can be worth attacking twice, from different sides, and that "the allocation is the cost" survived being true for long enough to be acted on in two entirely different places.

# Chapter 15

# Cooperative Control Flow

The natural way to implement `return` in a tree-walker is to throw a C++ exception and catch it at the routine boundary. It is correct, it is short, and it is what Raku++ did first.

It is also slow. On macOS a C++ throw walks the dynamic loader's unwind information under a lock, costing tens of microseconds. In the WebAssembly build, where C++ exceptions run through JavaScript trampolines, it is very much worse. And `return` is not rare — it is on the exit path of a large fraction of all routines.

## 15.1 The insight

A `return` only needs to unwind C++ frames if there is a **callable boundary** between it and its routine. If the `return` executes directly inside its routine's own statements, or inside a native `for` loop in the interpreter, then no C++ frame between them belongs to another Raku routine — and "unwinding" is just a matter of *stopping the statement loop*.

Distinguishing those two cases needs a way to ask "has a callable been entered since my routine started?", and the answer is two counters.

```
// src/Interpreter.h — ExecContext
bool returning = false;  Value returnV;
uint64_t frameTop = 0;         // incremented per callCallableRaw activation
uint64_t curRoutineFrame = 0;  // frameTop at the enclosing ROUTINE entry
```

Every activation bumps `frameTop`. A routine records the value at its own entry. So the test is an equality:

```
// src/InterpreterCore.cpp — return
if (tctx_.curRoutineFrame != 0 && tctx_.frameTop == tctx_.curRoutineFrame) {
    tctx_.returning = true; tctx_.returnV = std::move(v);
    return Value::any();                        // set a flag, do not throw
}
throw ReturnEx{v};                              // a boundary was crossed
```

The statement loops check the flag after each statement and bail out, and `callCallableRaw` consumes it at the routine boundary:

```
// src/InterpreterCore.cpp — after each statement in the activation loop
if (tctx_.returning) {
    if (isRoutine) { tctx_.returning = false; last = std::move(tctx_.returnV); }
    break;                // a bare block just propagates it to its routine
}
```

A bare block does *not* consume the flag — it breaks out and lets it propagate — because a `return` inside `if { … }` returns from the enclosing routine, not from the block.

## 15.2 The same trick, three more times

`next`, `last` and `redo` use an identical mechanism with their own register and frame counter:

```
// src/Interpreter.h — ExecContext
static constexpr uint64_t kNoFrame = ~uint64_t(0);  // "no frame is armed"
int loopCtl = 0;                      // 0 none, 1 next, 2 last, 3 redo
uint64_t curLoopFrame = kNoFrame;     // frameTop of the innermost native loop
```

```
// src/InterpreterCore.cpp — LastStmt
if (t.empty() && tctx_.frameTop == tctx_.curLoopFrame) {
    tctx_.loopCtl = 2; return Value::any();
}
throw LastEx{t};              // labelled, or across a frame: unwind
```

That sentinel is worth a paragraph, because the obvious spelling was wrong in a way that hid for a long time. `curLoopFrame` was initialised to `0` and the guard read `curLoopFrame != 0 && frameTop == curLoopFrame` — but `0` is also the **mainline's own frame number**. So every loop and every `given` written at the top level of a program failed that guard, fell out of the cooperative path, and threw: about eighty

microseconds per iteration on macOS, in exactly the programs a beginner writes. One sentinel that cannot collide with a real frame number fixed it, and the guard collapsed to a single comparison.

`when`, `default` and `succeed` got the same treatment, and for a specific measured reason. `given $v { when Int {...} when Str {...} ... }` executed per row of a data set means a throw per row:

```
// src/Interpreter.h — ExecContext
int givenCtl = 0;             // 1 = a when matched; break the given
Value givenV;
uint64_t curGivenFrame = 0;
```

In each case the rule is the same: **same frame, set a flag; different frame or a label, throw.** Labelled control always throws, because a labelled `last` targets a specific enclosing loop that may be several frames up, and walking to it is exactly what unwinding is for.

## 15.3 The rest of the control exceptions

Everything that genuinely has to cross frames is still an exception, and they are all tiny structs:

```
// src/Interpreter.h
struct ReturnEx { Value v; };
struct ExitEx { int code = 0; };
struct LastEx { std::string label; };
struct NextEx { std::string label; };
struct RedoEx { std::string label; };
struct DoneEx {};                    // exits a whenever/supply/react body
struct BreakGivenEx { Value v; bool hasVal = false; };
struct LeaveEx { Value v; bool hasVal = false; };
struct ResumeEx {};                  // .resume inside a CATCH
struct StopGatherEx {};              // a lazy gather has enough
struct ProceedEx {};                 // leave a when, keep matching later ones
struct RakuError { Value payload; std::string message; };
struct WorkerAbortEx {};             // unwind a background worker at shutdown
```

`RakuError` is the user-visible one: every Raku exception is a `RakuError` carrying a payload `Value` — normally an exception object — and a message. `WorkerAbortEx` is deliberately *not* a `RakuError`, so a user's `CATCH` cannot accidentally swallow a shutdown (Chapter 38).

## 15.4 `CATCH`, and where it lives

A `CATCH` block is a `Block` statement with `isCatch` set, sitting inside the block it guards. Finding it means scanning the body — so the answer is cached on the `Callable`:

```
// src/Value.h — Callable
DecidedOnce<signed char> catchScan{-1};   // 1 = the body holds an inline CATCH
Stmt* catchBlkCache = nullptr;            // …and which one
```

When a `RakuError` reaches a block that has one, the handler runs with `$_` and `$!` bound to an exception instance. `exceptionFor` guarantees that instance is always *defined*, whatever was actually thrown, because a `when X::Foo` inside the handler needs something to smartmatch against.

`.resume` throws `ResumeEx`, caught at the throw point. Because a bare `ResumeEx` with nothing to absorb it would reach `std::terminate`, the interpreter tracks how many `CATCH` handlers are live and turns a `.resume` outside any of them into a catchable Raku error:

```
// src/Interpreter.h
int catchDepth_ = 0;
```

Typed exceptions are built rather than hard-coded:

```
// src/Interpreter.h
[[noreturn]] void throwTyped(const std::string& type,
        std::vector<std::pair<std::string, std::string>> attrs,
        const std::string& message);
Value makeTypedEx(const std::string& type,
        std::vector<std::pair<std::string, Value>> attrs,
        const std::string& message);
```

so `X::TypeCheck::Binding` arrives with its `got` and `expected` attributes populated and can be matched on class rather than on message text. That distinction matters: the project's rule is that error *message* wording is only copied from Rakudo where Roast or the documentation actually asserts it — the behaviour is what must match, not the prose.

### 15.4.1 An error that reaches its CATCH without a throw

A `die` is a throw too, and Roast measures what that costs. `S04-exception-handlers/catch.t` dies half a million times inside two blocks:

```
for ^500000 {
    CATCH { default { } }
    { CATCH { }; die "foo" }
}
```

Each iteration raised four C++ exceptions: the `die`; two more to get it out of the inner block, whose `CATCH` matched nothing; and a `BreakGivenEx` for the `default` that took it. The file ran for 37 seconds, against Rakudo's 2.5.

The `when` or `default` in a handler now matches the way one in a `given` does: the handler runs with `curGivenFrame` set to its own frame, and reads `givenCtl` after each of its statements. The rest follows the pattern this chapter started with. A block with a `CATCH` runs its statements through `execStmtHanding`, which lets an error arrive as a value wherever no other C++ frame stands between it and the handler:

```
// src/Interpreter.h
struct HandedError { RakuError err; std::exception_ptr raised; };
```

A statement that is only a call of the built-in `die` has its error made by `dieError`, the same function the built-in throws from, and never thrown. A bare block nested in such a block runs with a `handOff` slot: an error that leaves it is put there, after the inner block's own `CATCH`, `LEAVE`s and `let`s have had their turn, and the enclosing statement loop takes it as if it had caught it. Anything else, a `die` inside a called routine for instance, still raises, and the loop catches it as before. When no handler within reach takes the error, the outermost block that has no caller to hand it to raises it once. `raised` keeps the original C++ exception when there was one, so the object that propagates is the object that was thrown.

The loop above now runs without a throw, and the file takes 0.4 seconds. Only `execBlock`'s statement loop works this way. A routine body, and therefore a `try` block, runs its statements in `callCallableRaw`, where a `die` still raises.

## 15.5 Backtraces

```
// src/Interpreter.h — ExecContext
struct CallSite { int line; const Value* code; };
std::vector<CallSite> callFrames;
```

One entry per live routine activation: the line the *call* was written on, and the routine itself. It is pushed next to `dynStack`, which every call already pays for, so it costs a vector append.

That is what `callframe(N)` walks — `Log::Async` stamps every message with `callframe(1)` — and what a program reads through `Backtrace.new`, which answers the chain at the point of call; an `Int` argument drops that many innermost frames, so a routine can report its caller's position rather than its own.

The routine's declaring file comes from `Callable::declFile`, recorded when the routine was declared, because by the time a backtrace is taken the module that declared it is long gone from the current scope. Frames with no declaring routine — the mainline, a bare block — take the file whose *top level* is running, `curDeclFile_`. That is the program ordinarily, but a module load switches it while the module's body runs, and so does `EVALFILE`, which is how a backtrace taken inside an `EVALFILE`d file names that file instead of the program that read it.

### 15.5.1 Where the chain has to be captured

For a long time all of that machinery existed and almost nothing used it. An uncaught error printed its message and stopped: no file, no line, no chain. `$!.backtrace` after a `try` answered a single frame on the line of the question rather than the line of the throw.

The reason is the interesting part, and it is the same C++ unwinding this chapter has been about. `callFrames` is maintained by a scope guard:

```
struct CFGuard { ExecContext& t; Interpreter* self; int line;
    ~CFGuard() { if (!t.callFrames.empty()) t.callFrames.pop_back(); self->curLine_ = line; }
};
```

When a `RakuError` is thrown, the C++ runtime unwinds through every frame between the throw and the handler, running exactly those destructors on the way. By the time any `catch` block runs, the stack it would want to describe has already been dismantled, frame by frame. **A backtrace cannot be taken where the exception is caught. It has to be taken where it is thrown.**

There are two places that could do it. One is the unwind itself: have `CFGuard` notice `std::uncaught_exceptions() > 0` and append its own frame to a thread-local record on the way out. That is cheaper per throw, since it records only the frames actually unwound — but it puts a thread-local read on *every routine return* in the interpreter, and it needs a discipline at every `catch` site to stop frames from a swallowed exception leaking into the next trace.

The other is the throw. `RakuError` is the one type every Raku-level error is raised as, so giving it a constructor makes every one of the roughly 320 throw sites in the tree capture without any of them being edited:

```
RakuError::RakuError(Value p, std::string m)
    : payload(std::move(p)), message(std::move(m)) {
    bt = btCaptureNow();
}
```

This is the version that shipped. The cost model decided it: a constructor costs one reference-count bump per live frame *on the path that throws*, and exactly nothing on the path that does not. The unwind-based version is faster in the rare case and slower in the common one, which is the wrong way round.

The record is one shared allocation:

```
struct BtFrame  { std::shared_ptr<Callable> code; int line = 0; };
struct BtRecord { std::vector<BtFrame> frames; std::string originFile; };
```

`code` is an *owning* pointer, not the borrowed `const Value*` the live stack holds, precisely because the record has to outlive the unwind that destroys what the live stack points at.

### 15.5.2 Handing it to the program

`exceptionFor` is the single funnel through which a caught `RakuError` becomes a Raku exception object — `try`, `CATCH`, the mainline handler and `.resume` all go through it — so that is where the record is attached, as an opaque handle rather than a materialised list. A `try` that never asks for the backtrace pays one pointer store; the `BacktraceFrame` objects are built the first time a program actually asks, and cached back onto the exception.

An existing record is never overwritten. A `rethrow`, a `.resume`, or a `die $caught` keeps the position the exception was *first* raised from, which is the only position that answers "where is the bug".

### 15.5.3 Errors that happen in one place and are noticed in another

Two constructs break the assumption that an error has a single position, and both are common enough that reporting the wrong one is worse than reporting none.

A **Failure** is created by `fail`, or by any failed coercion, and then lies dormant until something uses the value. The throw happens at the *use*. Reported naively, every

such error blames a line that merely read a variable. So a Failure records its own creation:

```
Value rakuppNewFailure() {
    Value f = Value::makeHash(); f.hashKind = "Failure";
    f.extM() = btCaptureNow();
    return f;
}
```

and `failureDetonate` puts the creation chain in front and labels the throw `Actually thrown at:`, which is Rakudo's spelling. That helper replaced 26 hand-written `Value::makeHash(); hashKind = "Failure"` pairs scattered across five files — the kind of duplication that guarantees a later change reaches only some of them.

Failures are the one part of this feature that is not free, because ordinary code makes them in bulk. The measured cost is about 9% of making a Failure, of which roughly half is the walk and half the allocations. The first version cost 12.6%; the difference is where the record lives. Storing it as a `__bt` entry in the Failure's hash cost a map node and a second `Value`; a Failure's own `ext` handle was sitting unused, so the record went there instead.

A **`start` block** dies on a worker thread, and some later `await` on some other thread is where a program hears about it. `PromiseState` carries the worker's record across the boundary alongside the cause, and `await` reports the worker's frames first, then labels its own line `Awaited at:`.

### 15.5.4 One renderer

Everything that prints a chain goes through one function, so the uncaught printer, `.gist`, `Backtrace.Str`, `warn`, the JSON handler and the REPL cannot drift apart on what an error looks like. The style struct is what separates them: the terminal gets a source-line excerpt under the origin frame, the exception type, folded runs of identical frames and colour; `.gist` and `Backtrace.Str` get none of it, because those are strings that programs print and compare rather than terminal output.

There is one deliberate departure from Rakudo's frame text. Rakudo prints `in method new`; Raku++ prints `in method Foo::new`. In a program where six classes each declare a `new`, the bare form does not say which one ran. The package goes inside the name part, ahead of the `at`, so anything parsing these lines by splitting on `at` is unaffected.

## 15.6 What was gained, and one bug it cost

The cooperative path removes a C++ throw from the exit of most routines and the body of most loops and `given` blocks. On macOS, where a throw is expensive, and in WebAssembly, where it is much worse, that is the difference between a tolerable and an intolerable tree-walker.

The mechanism has one hazard, and it bit: **the counters must be maintained consistently across every activation kind.** Methods go through `invokeMethod` rather than `callCallableRaw`, and for a period that path did not establish the routine frame boundary the same way. The symptom was specific and baffling — a method lost an early `return` when the `return` was inside a loop, but only in some call contexts. The cooperative flag was set, and the frame that should have consumed it did not recognise itself as a routine boundary, so the return value evaporated.

The fix was to establish the boundary in `invokeMethod` exactly as `callCallableRaw` does, and the regression test is a method with a `return` inside a loop, called every way the language allows.

The general lesson is worth stating: **an optimisation that replaces a language-level mechanism with a hand-maintained invariant is only as correct as the least careful place that maintains it.** C++ unwinding could not be got wrong this way, because the compiler maintained it. Frame counters can.

# Chapter 16

# Dispatch

Raku has more dispatch than most languages: named subs, multiple dispatch on argument types, method dispatch on an invocant, private methods, meta-object calls, and a re-dispatch protocol (`callsame`, `nextsame`, `callwith`, `nextwith`, `samewith`) that lets a routine hand control to the next candidate.

Raku++ implements them as **three distinct mechanisms** that meet at the edges, and knowing which one is in play explains most of the surprises.

## 16.1 Named calls

`evalCall` resolves every named call the same way:

```
// src/InterpreterCore.cpp — evalCall, abridged
if (Value* f = tctx_.cur->find("&" + c->name))
    return callCallable(*f, std::move(args), &c->args, false, …);
…
auto it = builtins_.find(c->name);        // built-in fallback
```

The environment first, walking the parent chain; the builtin table second. That order is the whole rule, and it has three consequences worth spelling out.

**A lexical `my sub` shadows a builtin.** That is Raku's semantics and it comes for free.

**A module's exported sub also shadows a builtin**, because the loader copies it into the global environment, which is the last link of the chain (Chapter 33).

**A compiled program must reproduce this order**, which it cannot do by resolving names against the builtin table at compile time. That is a real bug that happened — `--exe` printed the built-in `val`'s answer where the interpreter called a module's exported `val` — and Chapter 28 describes the fix.

The argument *expressions* are passed alongside the values (`&c->args`) so that `is rw` write-back can re-resolve them.

## 16.2 Multiple dispatch

A `multi sub` or `multi method` is one `Callable` with `isMultiDispatcher` set and a `candidates` vector; each `multi` declaration pushes its `Code` onto it. A `proto` declares the group without being a candidate.

At call time, `scoreCandidate` scores every candidate against the actual arguments, returning `-1` for "does not apply" or a non-negative **specificity**:

```
// src/InterpreterCore.cpp — scoreCandidate, per positional parameter
if (subsets_.count(p->type)) {
    if (!subsetMatches(p->type, pos[i])) return -1;
    score += 2;
} else if (!typeMatchesArg(pos[i], p->type)) return -1;
if (p->defConstraint == 1 && !isDefined(pos[i])) return -1;   // :D
if (p->defConstraint == 2 &&  isDefined(pos[i])) return -1;   // :U
if (p->defConstraint) score++;
if (!p->type.empty() && p->type != "Any" && p->type != "Mu") {
    score++;                                      // constrained beats not
    if (p->type == pos[i].typeName()) score++;  // exact beats supertype
}
```

Arity gates run first — too few required arguments, or too many for a non-slurpy signature, is an immediate `-1`. Literal parameters (`multi fact(0)`) and sub-signature destructuring are treated as very specific. A required named parameter that was not supplied is `-1`.

The winner is the highest score:

```
const Value* best = nullptr; int bestScore = -1;
for (auto& cand : c.candidates) {
    int s = scoreCandidate(cand, args);
    if (s > bestScore) { bestScore = s; best = &cand; }   // strict >
}
if (!best || bestScore < 0) throw /* X::Multi::NoMatch */;
```

**Two honest notes.** The comparison is a strict `>`, so on equal scores the *first-declared* candidate wins and there is no `X::Multi::Ambiguous` diagnostic. And a genuine no-match does throw `X::Multi::NoMatch`, with the candidate list in the message.

### 16.2.1 Subsets

A `subset` is a refinement type that participates in dispatch:

```
// src/Interpreter.h
struct SubsetInfo { std::string base; const Expr* where = nullptr; };
std::unordered_map<std::string, SubsetInfo> subsets_;
bool subsetMatches(const std::string& name, const Value& v, int depth = 0);
```

`subsetMatches` checks the base type and then evaluates the `where` expression with the value as the topic. The `depth` parameter is a recursion guard, because a subset can be defined in terms of another subset and nothing prevents a cycle.

A subset match scores `+2`, above a plain nominal match, which is what makes `multi f(Even $n)` beat `multi f(Int $n)`.

## 16.3 Method dispatch: two worlds

An `Int`, a `Str` or an `Array` is a native `Value` — it has no `ClassInfo` and no method table. A user class instance is a `VT::Object` with both. So there are two dispatch worlds.

**User objects** go through `ClassInfo::findMethod`, which *is* the method resolution order:

```
// src/Value.h
Value* findMethod(const std::string& m, ClassInfo** owner) {
    auto it = methods.find(m);
    if (it != methods.end()) { if (owner) *owner = this; return &it->second; }
    if (parent) { if (Value* r = parent->findMethod(m, owner)) return r; }
    for (auto& p : extraParents) if (p)
        { if (Value* r = p->findMethod(m, owner)) return r; }
    if (owner) *owner = nullptr;
    return nullptr;
}
```

Own methods, then the first parent recursively, then each extra parent. It is a depth-first walk, **not C3 linearisation**, and for the diamond hierarchies where the two differ it can pick a different method than Rakudo. The `owner` out-parameter reports

which class the method was found in, so re-dispatch can resume from that class's parent.

**Built-in values** go through `methodCall`, a very long cascade of branches keyed on the method name and, inside each branch, on the invocant's tag:

```
if (m == "uc")       return Value::str(mapCase(inv.toStr(), true, 0));
if (m == "chars")    return Value::integer(graphemeCount(inv.toStr()));
if (m == "is-prime") { /* Miller–Rabin on inv.toInt() */ }
// … some 1,640 more …
```

The C++ `if`-ladder *is* the built-in method set. That is the pragmatic counterpart to the fat `Value`: since every native value is the same struct, its methods are one dispatch function rather than per-type classes.

The ladder is split across four files as ordered segments (Chapter 2), each returning `std::optional<Value>`. The name is wrapped in an `MName` at the top so the comparisons are integer compares (Chapter 10).

### 16.3.1 What runs before the ladder

Two things are consulted first, and both are bridges between the two worlds.

**Junction autothreading.** If the invocant is a junction, a small allow-list of methods (`Bool`, `gist`, `new`, …) act on the whole junction; everything else autothreads, returning a junction of the per-eigenstate results.

**`augment` on a built-in type.** Methods a program adds to a built-in with `augment class Int { … }` are parked in a map of maps and consulted before the native branches, so they can override built-ins:

```
// src/Builtins.cpp — before the native branches
if (!builtinExt_.empty() && inv.t != VT::Object) {
    std::string tn = inv.t == VT::Type ? inv.s : inv.typeName();
    if (Value* f = lookup(tn))
        return invokeMethod(*f, inv, std::move(args), rwArgs);
    for (const std::string& anc : typeAncestry(tn))
        if (anc != tn) if (Value* f = lookup(anc))
            return invokeMethod(*f, inv, …);
}
```

The ancestry walk is what makes `augment class Cool { … }` reach an `Int` and a `Str`. The guard on `builtinExt_.empty()` is why a program that never augments anything pays one branch.

This is also why two of the inline built-in fast paths in Chapter 28 are `builtinExt_`-guarded: an inlined `abs` must stop inlining the moment a program augments `Int`.

## 16.4 Re-dispatch

`callsame`, `nextsame`, `callwith`, `nextwith` and `samewith` need to know what the *next* candidate is, and that differs by context: the next-less-specific multi candidate, the same method on the owning class's parent, or a built-in that a user method shadowed.

```
// src/Interpreter.h
struct RedispatchCtx {
    std::function<Value(ValueList)> next;      // callsame / nextsame / …with
    std::function<Value(ValueList)> restart;   // samewith
    ValueList sameArgs;
    bool lastcall = false; bool fromChain = false;
};
static thread_local std::vector<RedispatchCtx> redispatchStack_;
```

Each dispatch site that can be re-entered pushes a context knowing how to invoke the next thing, and the original arguments for the `*same` variants. `invokeMethodChain` is the method version: it invokes a method found from a given class and pushes a context that resumes from that class's parent, recursively.

One field guards a subtle case:

```
// src/Interpreter.h — ExecContext
size_t redispatchFloor = 0;   // frames below this are another routine's
```

Without it, a `callsame` inside a routine could reach a re-dispatch context belonging to a *caller*, and dispatch into something unrelated.

The stack is `static thread_local` for the reason given in Chapter 14: as a plain member it was written by every dispatching call on every thread.

## 16.5 Private methods, qualified calls, indirect calls

The `MethodCall` node carries the variants as flags:

| Written | Flag | Meaning |
|---|---|---|
| `$o.m` | — | ordinary dispatch |

| Written | Flag | Meaning |
|---|---|---|
| `$o.?m` | `maybe` | return `Nil` rather than dying when absent |
| `$o!m` | `bang` | private method, reachable only through `self!m` |
| `$o.Class::m` | `methodQual` | dispatch to a specific class's method, past overrides |
| `$o."$name"()` | `methodExpr` | the name is computed at run time |
| `$o.=m` | `mutate` | call and assign back to the invocant |
| `$o>>.m` | `hyper` | apply to each element |
| `$o.^m` | `meta` | a meta-object call |

The flags combine: `self!"$name"()` is `bang` *and* `methodExpr`, and the name is not known until the expression has been evaluated — so the check that a private call is lexically inside its class runs after that, not before the invocant. The private branch used to take the interpolation's raw source text as the name and dispatch to a method spelled `!$name` (issue #43).

`.^` calls go to the meta-object. Most are answered directly — `.^name`, `.^methods`, `.^attributes`, `.^parents`, `.^roles` — but `.HOW` must return a *persistent* object, because `T.HOW does SomeRole` mixins have to stick. So `ClassInfo` carries its meta-object rather than building one per call:

```
// src/Value.h — ClassInfo
Value howObj;   // the persistent .HOW metaobject
```

## 16.6 `&builtin` as a value

`&say` must be a `Callable` value, and `&dir.wrap({...})` must mutate *the* `Callable` the call path consults — that is exactly how `File::Find`'s test suite mocks `dir`. So a reference to a builtin is memoised:

```
// src/Interpreter.h
std::map<std::string, Value> builtinRefs_;
```

populated lazily, and empty for programs that never take a builtin reference — which keeps the check on the call path free.

## 16.7 Where the time goes

After the two fixes in Chapters 10 and 11, a method call on a built-in costs roughly three times what Rakudo charges, down from about twenty. The profile of `for ^6000000 { "ab".uc }`:

| | share |
|---|---|
| heap allocate and free | 31% |
| `Value` copy and destroy | 11% |
| method-name comparison | 8.5% |
| the dispatch function's own body | 6.1% |

The 42% in allocation and value churn was the remaining target when this profile was taken, and it had a known shape: both `methodCall` and `methodCallInner` take their invocant and argument list **by value**, so every call copies a `Value` and allocates a `ValueList`.

Two of the three costs in that sentence have since shrunk without the by-value signatures changing at all. A `Value` carried eleven `shared_ptr` members then and carries two now (Chapter 41), so the copy is a fraction of what it was; and the `ValueList` for a short argument list no longer reaches the allocator, because small blocks come off a free list (Chapter 12). What is left is the copy itself, which is smaller, on a path that is otherwise unchanged.

Switching both to `const&` is mechanically small: eighteen compile errors, each either "make a local copy here" or "let this helper take `const&`". It has not been done because it trades away the accidental safety that copying provides against a callee mutating the container its own invocant lives in. That wants an aliasing audit of 178 call sites, not a quick pass — and the case for spending that audit is weaker now than when it was written, because the cost it would recover has been reduced twice by changes that needed no audit at all.

## 16.8 Honest limitations

- **The method resolution order is depth-first, not C3.** Diamond hierarchies can resolve differently from Rakudo.
- **No `X::Multi::Ambiguous`.** Equal-specificity candidates resolve silently to the first declared.

- **Role conflicts are diagnosed**, so this is no longer a limitation: composing two roles that define the same method raises `X::Role::Unresolved::Method` unless the class resolves it, as in Rakudo. Only the order the roles are listed in inside the message differs.
- **The ladder's order is load-bearing and undocumented in the code beyond a warning.** An arm moved for readability is a behaviour change.

# Chapter 17

# The Object System

Raku's object surface is one of the largest in the language: classes with inheritance, roles, runtime mixins, a meta-object protocol, `augment` and package stashes. A reader arriving from C++ or the JVM would expect a subsystem to match. There is not one — the whole of it is two structs and the operations that read them, and most of this chapter follows from how little those structs hold.

A user class is a `ClassInfo`. An instance is an `ObjectData` that points at one. There is no per-object method table, no vtable, and no per-class code generation.

```
// src/Value.h — abridged
struct ClassInfo {
    std::string name;
    std::shared_ptr<ClassInfo> parent;
    std::string nativeParent;                          // `is Str`, `is Cool`
    std::vector<std::shared_ptr<ClassInfo>> extraParents;
    std::vector<ClassAttr> attrs;
    std::map<std::string, Value> methods;              // Code values
    std::map<std::string, std::string> rules;          // grammar rules
    std::vector<std::string> ruleOrder;
    bool isGrammar = false, isRole = false;
    std::string repr;                                  // is repr('CStruct')
    std::string ver, auth, api;
    std::set<std::string> requiredMethods, doneRoles;
    Value howObj;                                      // persistent .HOW
    std::shared_ptr<Env> declEnv;                      // for attr defaults
    ClassDecl* decl = nullptr;                         // for role parameters
    std::vector<std::pair<std::string, Value>> roleParamBindings;
```

```
};

struct ObjectData {
    std::shared_ptr<ClassInfo> cls;
    std::map<std::string, Value> attrs;
    Value boxed; bool hasBoxed = false;                    // for `5 but Role`
};
```

Everything a class *is* lives in that first struct, including the grammar rules if it happens to be a grammar — which is how a grammar is just a class whose methods can be regexes (Chapter 22).

## 17.1 Registration and lookup

`ClassInfos` are registered in one flat map at declaration:

```
// src/Interpreter.h
std::unordered_map<std::string, std::shared_ptr<ClassInfo>> classes_;
std::unordered_map<std::string, std::string> classAliases_;
```

Flat, not per-compilation-unit. Two modules declaring the same unqualified class name collide — a real divergence from Rakudo, listed in Chapter 33.

The alias map softens the consequences of flatness. Registering `URI::Path` also aliases its tail, `Path`, unless a real class already claims that name. And a qualified name that is a *partial* path resolves by unique suffix match:

```
// src/Interpreter.h — resolveClassAlias
if (n.find("::") != std::string::npos) {
    const std::string suffix = "::" + n;
    const std::string* hit = nullptr;
    for (auto& kv : classes_) {
        if (...kv.first does not end with suffix...) continue;
        if (hit) return n;          // ambiguous — leave it alone
        hit = &kv.first;
    }
    if (hit) return classAliases_.emplace(n, *hit).first->second;
}
```

so `Globber::Match` inside `unit class IO::Glob` finds `IO::Glob::Globber::Match`. Only qualified names take that road — a bare `Match` must not silently find a nested one — and an ambiguous suffix is left alone rather than guessed at. The successful answer is memoised into the alias map.

## 17.2 Construction

The default `new` and `bless` share one path. It walks the class chain **parent-first**, gives each attribute its default, folds named arguments into the attribute map, and then runs `BUILD` and `TWEAK`:

```
// src/MethodCallPart2.cpp — default construction, abridged
for (auto it = chain.rbegin(); it != chain.rend(); ++it)      // parent-first
    for (auto& at : (*it)->attrs) {
        Value dv = at.hasDefVal ? at.defVal
                 : at.def       ? eval(at.def)
                 : at.sigil == '@' ? Value::array()
                 : at.sigil == '%' ? Value::makeHash() : Value::any();
        od->attrs[at.name] = dv;
    }
for (auto& arg : args)
    if (arg.t == VT::Pair) od->attrs[arg.s] = *arg.pairVal;
if (Value* build = ci->findMethod("BUILD")) invokeMethod(*build, self, args);
if (Value* tweak = ci->findMethod("TWEAK")) invokeMethod(*tweak, self, args);
```

Parent-first matters: a subclass's default for an inherited attribute must win, and it does because it is written last.

Attribute defaults are *expressions*, evaluated at construction in the scope the class was declared in — which is what `ClassInfo::declEnv` is for. The compiled backend precomputes them where it can, into `ClassAttr::defVal`, and falls back to the expression otherwise.

`is required` throws when construction supplies no value, carrying the reason string from `is required("it is a good idea")` into the message. `is built` makes a *private* attribute settable by name at construction anyway.

## 17.3 Attributes and accessors

`$!x` is a direct read or write of the attribute map. `$.x` is a public accessor: method lookup runs first, and on a miss `findAttr` supplies the attribute. The compiled backend uses the same map through two helpers:

```
// src/InterpreterBinding.cpp
Value rtAttrGet(const Value& self, const std::string& name) {
    if (self.t == VT::Object && self.obj) {
        auto it = self.obj->attrs.find(name);
        if (it != self.obj->attrs.end()) return it->second;
```

```
    }
    return Value::any();                    // absent attribute → (Any)
}
Value& rtAttrRef(Value& self, const std::string& name) {
    return self.obj->attrs[name];           // autovivifies
}
```

`ClassAttr` carries more than storage: the declared type and its `:D`/`:U` smiley, `is rw`, `is required`, `is built`, a container trait (`has %.a is Set`), a `handles` list for delegation, an object-keyed flag, and **user traits**.

That last one is how third-party traits reach their own code. `has $.id is json-name('licenseId')` records `{"json-name", Str}` on the attribute, surfaced on the `Attribute` meta-object, which is exactly what `JSON::Unmarshal`'s role checks and accessors look for. The parser does not know what `json-name` means and does not need to.

Assigning through an accessor is guarded: `$obj.attr = v` on a private or read-only attribute throws, while `$obj!attr` and a plain method stay writable. The declared type is enforced too, which needs the attribute's type at the assignment site — hence:

```
// src/Interpreter.h — ExecContext
std::string lastLvalueAttrType;
```

recorded by the method-call lvalue arm so a role-typed `has C $.x is rw` rejects `42`.

## 17.4 Roles

A role is a `ClassInfo` with `isRole` set, a set of `requiredMethods`, and a set of `doneRoles`. Composition copies the role's methods and attributes into the class and records membership.

Required methods are checked **at class declaration**, and the check is more generous than a lookup, because there are four honest ways to satisfy a requirement:

```
// src/InterpreterModules.cpp — for each name a role requires
ok = hasImpl(ci, rq, nullptr)      // a non-stub implementation anywhere in the type
  || classOwn.count(rq);           // …or the class's own stub: a deliberate promise
if (!ok && attrCovers(ci, rq)) ok = true;   // …or a public attribute's accessor,
                                            //    or an attribute `handles` delegation
if (!ok)
    throw RakuError{Value::typeObj("X::Comp::AdHoc"),
```

```
        "Method '" + rq + "' must be implemented by " + clsName +
        " because it is required by roles: " + rl + "."};
```

A *stubbed multi* is checked per candidate signature rather than by name, so a class that implements one of two required signatures is still incomplete.

`.does` and ~~ consult `doesRole`, which is true for the role itself, for directly or transitively composed roles, and for roles done by parents:

```
// src/Value.h
bool doesRole(const std::string& rn) const {
    if (isRole && name == rn) return true;
    if (doneRoles.count(rn)) return true;
    if (parent && parent->doesRole(rn)) return true;
    for (auto& p : extraParents) if (p && p->doesRole(rn)) return true;
    return false;
}
```

Attribute composition deduplicates by the address of the *declaring* attribute node, recorded in `ClassAttr::declId`. So a diamond composition — two roles both doing a third — contributes the shared attribute once rather than twice.

**Parameterised roles** carry their parameters as a `Param` list on the AST declaration, and a composition binds arguments to them:

```
// src/Value.h — ClassInfo
std::vector<std::pair<std::string, Value>> roleParamBindings;
```

The bindings are injected into the scope of the composing class's methods, so the role's body sees `%phase-defaults` in `does Cro::Policy::Timeout[%h]`. `makeRolePun` handles the anonymous case, `R[Int].new`, by building a punned class on the spot.

Two roles defining the same method is a **conflict**, not a race: composition raises `X::Role::Unresolved::Method` — "Method 'r' must be resolved by class E because it exists in multiple roles (R1, R2)" — unless the class declares the name itself. Rakudo says the same sentence with the roles in the other order.

## 17.5 Mixins: `but` and `does`

`5 but Role` and `%h does R` add a role to a value at run time.

For a value that is already an object, the role is composed into a fresh anonymous subclass. For a **non-object base** there is no `ObjectData` to extend, so the value is *boxed*:

```
// src/InterpreterOperators.cpp — mixinValue
obj = std::make_shared<ObjectData>();
obj->boxed = base;          // the original 5 is kept here
obj->hasBoxed = true;
// … build an anonymous subclass composing the role, wrap in a VT::Object …
```

Dispatch on the result checks the role's methods first; anything not found — and not an identity method — is delegated back to the box:

```
// src/Builtins.cpp — methodCall
if (inv.t == VT::Object && inv.obj && inv.obj->hasBoxed && inv.obj->cls &&
    !inv.obj->cls->findMethod(m) && !inv.obj->cls->findAttr(m)) {
    static const std::set<std::string> keepOnObj =
        {"does","HOW","WHAT","WHICH","defined","DEFINITE"};
    if (!keepOnObj.count(m))
        return methodCall(inv.obj->boxed, m, args, rwArgs);
}
```

So `(5 but Role).succ` runs `Int.succ` on the `5`, while the role's methods and `.does`/`.WHAT` see the mixed object. The `keepOnObj` set is the list of questions that must be answered *about the mixin* rather than about the box.

`$x but Pair` mixes an attribute rather than a role, using the same machinery with a synthesised anonymous role — which is what `anonMixinSeq_` names.

## 17.6 `augment`, and the `supersede` that is not there

`augment class Foo { … }` on a user class merges methods into the existing `ClassInfo`. On a **built-in type** there is no `ClassInfo`, so the methods go into a side table consulted before the native dispatch ladder:

```
// src/Interpreter.h
std::unordered_map<std::string,
                   std::unordered_map<std::string, Value>> builtinExt_;
```

keyed by type name then method name, and searched along the native ancestry so augmenting `Cool` reaches `Int` and `Str`. Both forms need `use MONKEY-TYPING`, as they do in Rakudo — without it the parser refuses `augment` outright.

`supersede` is a different matter: it is **not implemented in any form**. The word is in the syntax highlighter's keyword list, which is the only place it appears, so `supersede class S { ... }` reaches the ordinary declaration path and dies with `X::Redeclaration`. Replacing a method on a live class is `.^add_method`.

Chapter 16 covers the dispatch side; Chapter 28 covers why two compiled fast paths have to check whether this map is empty.

`.^add_method($name, $code)` writes into `ClassInfo::methods` directly, which is the whole of runtime method injection.

## 17.7 Package stashes and `.WHO`

```
// src/Interpreter.h
std::map<std::string,
         std::shared_ptr<std::map<std::string, Value>>> pkgStashes_;
```

One shared map per package **name**, and the sharing is the point. `.WHO` used to build a fresh empty `Hash` on each call, so `EXPORTHOW.WHO.<grammar> = SomeHOW` wrote into a temporary and died with "Target is not assignable". Reading a stash also re-syncs the package's qualified globals into it, so `our`-scoped symbols appear.

A `module` or `package` has no `ClassInfo` at all, so its `:ver`/`:auth`/`:api` adverbs live in a separate small map, `pkgMeta_`, which is what `.^ver` answers for a package.

## 17.8 Honest limitations

- **Depth-first method resolution**, as in Chapter 16.
- **Role conflicts are silent**, last writer wins.
- **The class registry is flat**, so unqualified class names from different modules collide. The alias machinery reduces the pain but does not remove the cause.
- **`ClassInfo::decl` is a raw `ClassDecl*`** into the AST, kept alive by the same "the tree is immortal" rule as `Callable::body` (Chapter 7).

# Chapter 18

# Laziness, Junctions, and the Wilder Values

Three parts of Raku resist a straightforward tree-walker: infinite lists, values that are several values at once, and `gather/take`, which wants a coroutine. None of the three gets a new `VT`. All three are an existing tag plus some state.

## 18.1 Lazy and infinite sequences

An infinite list like `1, 2, 4 ... *` or `(1..Inf).map(*²)` obviously cannot be a materialised `ValueList`. Raku++ attaches a generator to an otherwise ordinary array value.

```
// src/Interpreter.h
struct LazySeqState {
    std::function<bool(ValueList&)> appendNext;  // one more; false = exhausted
    bool infinite = false;   // truly unbounded: elems/pop/[*-1] must die
};
```

The already-computed **prefix** lives in the `Value`'s `arr`; the generator lives in the opaque `ext` slot — the same slot a `Promise` or a `Channel` would use. `appendNext` appends exactly one element and reports whether more exist.

### 18.1.1 Building one

`seqOp` implements the `...` operator. It splits the left side into a seed list — a trailing `Code` seed becomes the *generator* — and classifies the right endpoint. A bounded endpoint (`1 ... 10`) is computed eagerly in a loop, capped at a million. An infinite endpoint builds a `LazySeqState`:

```
// src/Interpreter.cpp — seqOp
if (infinite) {
    auto st = std::make_shared<LazySeqState>();
    st->infinite = true;
    st->appendNext = [...](ValueList& cache) -> bool {
        /* feed the last `arity` cached elements to the generator, or step
           the detected progression; push the next value */
    };
    out.ext = st;
}
```

When there is no explicit generator, the operator *detects the progression* from the seed: a constant arithmetic difference, a constant geometric ratio, or — for strings — repeated `succ`/`pred`. That is Raku's rule, and it is why `1, 3, 5 ... 99` and `'a', 'b' ... 'z'` both work with no closure written.

`...` is also **list-associative**: `1 ... 5 ... 1` and `'A'...'Z', 'a'...'z'` are one operator over a list of lists, where each group's first element closes the previous segment. That is `seqOpGroups`, shared with the compiled backend.

A bare `1..Inf` assigned to an array builds a simpler counting generator, in `coerceArray`:

```
auto st = std::make_shared<LazySeqState>(); st->infinite = true;
auto next = std::make_shared<long long>(start);
st->appendNext = [next](ValueList& cache) -> bool {
    cache.push_back(Value::integer((*next)++)); return true;
};
a.ext = st;
```

### 18.1.2 Forcing elements

Consumers grow the prefix on demand:

```
// src/InterpreterBinding.cpp
void Interpreter::materializeLazy(const Value& v, size_t n) {
    auto st = std::static_pointer_cast<LazySeqState>(v.ext);
```

```
    while (v.arr->size() < n && v.arr->size() < CAP)
        if (!st->appendNext(*v.arr)) break;
}
```

So `@lazy[5]` materialises six elements and then indexes; `.head(3)` materialises three; `.first(&pred)` pulls one at a time until the predicate matches.

Operations that need the *end* of an infinite list cannot complete, and say so:

```
// src/MethodCallTail.cpp
if (m == "elems" || m == "end" || m == "pop" || m == "tail" ||
    m == "reverse" || m == "sort" || m == "sum" || m == "min" ||
    m == "max" || m == "join" || m == "Str" || m == "gist")
    throw RakuError{Value::typeObj("X::Cannot::Lazy"),
                    "Cannot " + m + " a lazy list onto an Array"};
```

A *finite* lazy value — a `gather` that outgrew its probe — instead forces full materialisation, which is the right answer for it.

### 18.1.3 Lazy `.map` and `.grep`

`.map` over a lazy source builds a **new** `LazySeqState` that pulls from the source on demand, which is what makes `(1..Inf).grep(*.is-prime).head(5)` terminate:

```
// src/MethodCallTail.cpp — .map over a lazy source
st->appendNext = [self, src, fn](ValueList& cache) -> bool {
    size_t si = cache.size();
    self->materializeLazy(src, si + 1);          // pull one more
    if (si >= src.arr->size()) return false;
    cache.push_back(self->callCallable(fn, { (*src.arr)[si] }));
    return true;
};
```

`.grep` loops pulling source elements until its predicate matches; `.skip` builds a shifted view.

### 18.1.4 The cap

Every materialisation path shares a hard ceiling of **one million elements**. It is a runaway safety net, not a semantic limit, and it means “infinite” is bounded in practice. A consumer that genuinely needs the millionth element of a generated sequence will get it; one that needs the ten-millionth will not.

`Value::flatten()` has a tighter one for an endless `Range`: ten thousand elements. The next section is about what that ceiling cost, and what replaced it.

### 18.1.5 Answering without the elements

A ceiling turns a non-terminating operation into a terminating one, which is what it is for. It also turns a question the implementation cannot answer into one it answers *wrongly*:

```
$ rakupp -e 'say [+] 1..Inf'      # before
50005000
```

That is the sum of `1..10000` — the prefix, folded and handed back as the total, with nothing in the output to mark it partial. A reduce is written to consume its list element by element, and an endless list has no end to consume to, so the fold has no *value*. But its partial results usually have a **limit**, and that limit follows from the range's endpoints without touching a single element:

| | | |
|---|---|---|
| `[+] 1..Inf` | `Inf` | the partial sums 1, 3, 6, 10, ... are unbounded |
| `[*] 1..Inf` | `Inf` | so are the partial products |
| `[*] ^Inf` | `0` | zero is absorbing: every partial from there on is 0 |
| `[*] -0.5..Inf` | `-Inf` | one negative element, and the walk misses zero |
| `[-] 1..Inf` | `-Inf` | the tail it subtracts runs away |
| `[min] 1..Inf` | `1` | the partial minima settle on the low bound at once |
| `[max] 1..Inf` | `Inf` | the partial maxima chase the high one |
| `[~] 1..Inf` | `X::Cannot::Lazy` | the strings grow without approaching a string |

The rule the table follows: **answer with the limit of the partial folds when the bounds fix it; otherwise say it cannot be done; never fold a prefix and present it as the whole.** `~` has no limit to name in the string domain, and a user-supplied operator has no known one at all, so both refuse.

The sign rule for `*` is the one case that needs care. A `Range` is bounded below, so it has finitely many negative elements, and their count settles the sign of an otherwise unbounded product — three of them in `-2.5..Inf`, hence `-Inf`. Zero overrides all

of it, but only when the walk actually lands on zero: `-3..Inf` does, `-2.5..Inf` steps straight from `-0.5` to `0.5` and does not.

The check lives in one function, `endlessReduce`, and every entry point calls it *before* flattening — the `[op]` metaoperator, `prefix:<[op]>(...)`, an `&prefix:<[op]>` reference, `.reduce`, and `rtReduce`, the one the native code generator emits, so a compiled binary agrees with the interpreter. `.sum` and the `sum` builtin answer from the bounds too, rather than from a truncated `toList`.

An endless **lazy** list is a different matter: its elements do not follow from any bounds, so there is nothing to compute a limit from and every operator refuses. A *merely* lazy one — finite, but holding only the prefix something has pulled so far — must not be folded from its cache either, or `[+] @lazy` answers for whatever happened to be materialised. It is drained first, and refused only if it will not drain:

```
// src/InterpreterOperators.cpp — endlessReduce, the lazy arm
auto st = std::static_pointer_cast<LazySeqState>(v.ext);
const size_t CAP = 1000000;                       // materializeLazy's ceiling
if (st->appendNext)
    while (v.arr->size() < CAP && st->appendNext(*v.arr)) {}
if (v.arr->size() < CAP && !isEndlessLazy(v)) return false;  // drained: fold it
throw RakuError{Value::typeObj("X::Cannot::Lazy"), "Cannot reduce a lazy list"};
```

Whether it will drain cannot be decided when the view is built. A `.map` over an endless source is endless, always — it inherits the flag. A `.grep` over one may still end, because the predicate can stop it:

```
(^Inf).grep({ last if $_ > 5; True }).eager.join   # 012345
```

That is Roast's own test, so a grep view is *not* born endless. Instead it learns: if `appendNext` ever finds the source has stopped growing because the source is endless and hit its ceiling, the view marks itself endless from that point on, and the second `isEndlessLazy` check above — the one after the drain — sees it.

### 18.1.6 One sentinel, two meanings

A `Range` keeps its endpoints in two `long longs`, and an endless one parks `±LLONG_MAX` in them. So does a range whose endpoint is merely too large for a `long long`, which made `1..10**100` indistinguishable from `1..Inf` and gave it the prefix treatment as well. The written endpoint settles it — a big-`Int` bound in `Value::big`, a real infinity or a `Whatever` in `RangeEnds` — and a range of integers is summed by Gauss rather than walked:

```
Value count = applyArith("+", applyArith("-", hi, lo), Value::integer(1));
return applyArith("div", applyArith("*", count, applyArith("+", lo, hi)),
                  Value::integer(2));
```

`count × (lo + hi) / 2`, in the exact integer tower, so `(1..10**100).sum` is a 200-digit answer and `(1..10).sum` is still `55`. Roast's `S03-operators/range-int.t` asserts both.

Two paths still walk the prefix, deliberately. `[\+] 1..Inf`, the triangular form, produces a *list* rather than a fold, and stops at ten thousand partial sums instead of building a lazy sequence of them. And `.flat` over a list containing an endless range materialises the prefix without marking the result endless, so a reduce downstream of it never learns that anything ran away.

## 18.2 `gather` and `take`, without coroutines

`gather { … take … }` should suspend the block at each `take` and resume it when another element is demanded. That is a coroutine, and a tree-walker built on the C++ stack does not have one.

The strategy is **probe and double**. The collector and a per-gather element cap live on the execution context:

```
// src/Interpreter.h — ExecContext
std::vector<std::shared_ptr<ValueList>> gatherStack;
std::vector<size_t> gatherLimits;   // 0 = unlimited
```

The block is first run under a small cap of 64. If it finishes within the cap it was finite, and the result is returned eagerly. If the cap was *hit*, the result becomes a `LazySeqState` that grows by **re-running the block** with a larger cap:

```
// src/Interpreter.cpp
st->appendNext = [this, runGather](ValueList& out) -> bool {
    ValueList grown;
    bool more = runGather(out.size() + std::max<size_t>(64, out.size()), grown);
    for (size_t i = out.size(); i < grown.size(); i++) out.push_back(grown[i]);
    return more;
};
```

Doubling keeps the re-run cost amortised linear. A `take` that pushes past the current cap unwinds the block with an empty marker exception:

```
// src/InterpreterCalls.cpp — take
auto& coll = *tctx_.gatherStack.back();
for (auto& x : a) coll.push_back(x);
if (lim && coll.size() >= lim) throw StopGatherEx{};
```

So an infinite `gather` runs its block only far enough to satisfy each demand, stops, and re-enters later for more.

The honest cost of this design: **the block runs more than once, from the start.** A `gather` whose block has side effects — printing, or mutating a counter — will repeat them. That is a genuine divergence from a coroutine implementation, and the reason the initial probe cap is generous enough that most real `gathers` never re-run at all.

## 18.3 Junctions

A junction has no `VT` of its own. `any(1, 2, 3)` is a `VT::Array` whose elements are the eigenstates, tagged by `enumName`:

```
// src/Value.cpp — typeName(), the VT::Array case
if (enumName == "any" || enumName == "all" ||
    enumName == "one" || enumName == "none") return "Junction";
```

They are built by the `any/all/one/none` routines, the matching methods, and the `|`, `&`, `^` infix operators.

**Autothreading** — distributing an operation over the eigenstates and recombining — happens at each place a value is consumed, and there are four such places.

**Operators.** A comparison *collapses* to a single `Bool` according to the junction type; any other operator produces a **new** junction of the per-eigenstate results:

```
// src/InterpreterCore.cpp — applyArith
Value out = Value::array(); out.enumName = j.enumName;
for (auto& e : *j.arr)
    out.arr->push_back(applyArith(op, jleft ? e : l, jleft ? r : e));
return out;                        // any(1,2) + 10  ==>  any(11, 12)
```

**Method calls**, with a small allow-list that acts on the whole junction. **Callable invocation**, in `callCallableRaw`. **Smartmatch**, in `~~`.

That is why `if 3 == any(1, 2, 3)` works: the `==` sees a junction on the right, threads the comparison across the eigenstates, and collapses `any` to `True`.

One escape hatch exists, because `Junction.THREAD` must pass each eigenstate — junctions included — through whole:

```
// src/Interpreter.h
static thread_local bool noAutothread_;   // one-shot, consumed by the next call
```

## 18.4 `Whatever` and `WhateverCode`

`*` is `VT::Whatever`. An expression containing one *curries* into a `WhateverCode` — a `Callable` with a flag and an arity:

```
// src/Value.h — Callable
bool isWhateverCode = false;
long long whateverArity = 0;   // `* + *` consumes 2
```

so `* + 1` becomes a one-argument closure and `* + *` a two-argument one. Composition works because currying an expression that already contains a `WhateverCode` produces another one.

Deciding whether to curry is a syntactic question, answered by walking the expression for a literal `*`:

```
// src/Interpreter.h
static bool exprHasWhateverLit(const Expr* e);
```

In *subscript* position `*` means something else entirely — `@a[*-1]`, `@a[*]` — and is handled by a dedicated path, `idxW`, rather than by currying.

## 18.5 Hyper operators

»op« and friends apply an operator element-wise, with rules about which side may be extended. All spellings — `>>op<<`, »op«, `>>[&op]<<` — funnel into one core:

```
// src/Interpreter.h
Value hyperCore(Value& l, Value& r, bool strictL, bool strictR,
                const std::function<Value(const Value&, const Value&,
                                          Value*, Value*)>& apply,
                Value* lroot = nullptr, Value* rroot = nullptr,
                bool wantSlots = false);
Value hyperUnary(const std::string& op, Value v);         // -«(...)
Value hyperPostfixApply(const std::string& op, Value v);  // @a»++
```

The `strictL/strictR` flags are the dwimmy-versus-strict distinction Raku draws between `<<` and `>>` on each side; the slot parameters exist because `@a»++` must write *through* to the elements, which is the same write-back problem as an `is rw` parameter and uses the same one-shot register.

## 18.6 Flip-flops

`ff` and `fff` need per-*site* state — the same operator at two places in a program has two independent latches:

```
// src/Interpreter.h
struct FlipFlop { bool on = false; long long seq = 0; };
std::unordered_map<const void*, FlipFlop> ffState_;
```

keyed on the AST node's address, which is stable for the life of the program. The `seq` counter is there because the result while the latch is on is the *count of elements since it fired*, not a plain `Bool`.

This is the one place in the interpreter where a map keyed on a node pointer is the right answer rather than a hazard. It works here because AST nodes are never freed. Chapter 23 describes a case where the same idea failed for exactly the opposite reason — freed addresses being recycled.

# Chapter 19

# Node Specialization

This chapter is a single optimisation, described in full, because it is the clearest worked example in the project of the method the whole thing runs on: count the opportunity before building anything, remove work rather than adding cleverness, and keep a control in the measurement.

## 19.1 What it does

`evalBinary` and `evalIndex` recognise a handful of syntactic **shapes**, record that verdict on the node, and take a short path.

| shape | example |
|---|---|
| `$var OP literal` | `$n < 2`, `$n - 1` |
| `literal OP $var` | `2 * $n` |
| `$var OP $var` | `$a + $b` |
| `@arr[$var]` / `@arr[literal]` | `@a[$i]`, `@a[0]` |

Three costs disappear on those paths.

**The variable's copy.** `eval()` on a `VarExpr` returns the value *by value* — hundreds of bytes, several strings, eleven refcount increments. The fast path takes a `Value*` out of the environment and hands it to `applyArith`, which already took `const&`.

**The literal's reconstruction.** The `1` in `$n - 1` was rebuilt into a fresh `Value` on every visit. It is now built once and kept on the node.

**The probes that cannot apply.** Every binary operation ran two string comparisons for `DateTime/Duration` handling, plus the hyper and `Z/X` checks. None of them can match two plain scalars.

For `@arr[$i]` the win is the same in kind: the general path opens by copying the whole container into a local `Value` in order to read one element out of it.

## 19.2 Why these shapes, and not constant folding

The idea started as "let `-O` fold the tree before walking it". That was measured first, with `tools/ast-opportunity.raku`, which reads `rakupp --dump-ast` and counts the patterns a tree optimizer would rewrite. Across 48 real files — the showcase interpreters, the examples, the tools — 51,353 AST nodes:

| pattern | sites | per 1k nodes |
|---|---|---|
| constant folding (both operands literal) | 37 | 0.7 |
| **binary with ONE literal operand** | **1,282** | **25.0** |
| `$x = $x + ...` self-update | 24 | 0.5 |
| constant condition | 0 | 0.0 |

Classical constant folding has essentially nothing to fold in real Raku: 0.07% of nodes, and none of it in a loop. Dead branches: none at all. What *is* plentiful is the operand shapes above, and they sit in loop conditions and index arithmetic, where they run millions of times.

That table is the whole reason this is a specialisation pass and not a folding pass. It took two minutes to produce, and it prevented building the wrong thing. **Re-run the tool before assuming any other classical optimisation is worth building**; the static count is an upper bound on the opportunity.

## 19.3 What "cached" means here

"Cache" is a misleading word for this, so be precise. In most contexts caching means remembering a computed result so you do not recompute it. **Nothing about the result is remembered.** What is remembered is a fact about the source code.

The entire mechanism is two extra fields on each `Binary` node — one byte and one pointer. No table, no map, no key, nothing global:

```
// src/Ast.h — Binary
mutable DecidedOnce<signed char> fastShape{-1};   // which shape this node is
mutable DecidedOnce<const void*> litVal{nullptr}; // the literal, shapes 1 and 2
```

Walk one node, `$n - 1`, through its life.

**After parsing**, the fields say nothing:

```
op        = "-"
lhs      -> VarExpr("$n")
rhs      -> IntLit(1)
fastShape = -1            <- nobody has looked at this node yet
litVal    = nullptr
```

**On the first evaluation** the interpreter asks a question *about the syntax*: is the left child a plain lexical, and the right child a scalar literal? Both yes, so it writes the answer down:

```
fastShape = 1             <- "variable, operator, literal"
litVal   -> Value(1)      <- the 1 from the source text
```

**On every evaluation after that**, including the millionth, the node reads `fastShape == 1` and goes straight to: look up `$n`, check its type, apply the operator. It never asks the question again.

So the two fields hold two different kinds of thing.

- **`fastShape` is a classification of the syntax** — "this expression is written as variable-operator-literal". That is a fact about the *text of the program*. It is true before the program starts and stays true until it exits, because source code does not rewrite itself.
- **`litVal` is the literal's value.** This is cached in the ordinary sense, but a literal is a constant by definition: if the source says `1`, it is 1 forever.

**`$n` appears in neither field** — not its value, not its address, not the scope it was found in. Every evaluation does a fresh lookup by name through the current environment and a fresh type check on whatever comes back. Which is why all of this behaves normally:

```
my $v = 1;
for ^4 { say $v + 1; $v = $v ~~ Int ?? "10" !! 1 }   # 2, 11, 2, 11
```

One AST node, four evaluations, a variable that changes both value and type underneath it, and the right answer each time. The visits where `$v` holds a `Str` still take

the fast path, because `Str` is in the guard; had it become a `DateTime`, that visit alone would have fallen through to the general path.

Why store anything at all, then? Because the classification is not free — it is several branches: check both child node kinds, check the variable name's second character, check the literal is not a bignum or a `Rat`. Asking that once per *node* instead of once per *evaluation* is the whole optimisation. A `fib(29)` run evaluates `$n - 1` about 1.6 million times and answers the question once.

It is less a cache than a **sticky note on the node**, written the first time anyone reads it. The precedent sits directly above it in the same struct: `simpleOp` does exactly the same thing for "is this a plain operator, or one of the special-cased ones?", also decided once, also from the syntax alone. `specialArm` beside it goes one step further for the special-cased ones: which arm of `evalBinary` answers this spelling. Without it `~~` passed 82 string tests, and `|` 118, on every evaluation before reaching its own code.

## 19.4 The guards

The fast path is declined, and the untouched general path runs, unless:

- **the name is a plain lexical** — second character a letter or _, which excludes every twigilled and special name (`$*dyn`, `$?FILE`, `$^a`, `$/`), each of which has its own lookup rules;
- **the value is `Int`, `Num`, `Str` or `Bool` with an empty `hashKind`**, which excludes `Proxy`, junctions, `DateTime`, `Duration`, `Failure`, allomorphs and undefined values;
- **for a subscript**: no adverb, not multidimensional, not a slice, a plain materialised `Array` (no `ext`, so no lazy sequence), and a **non-negative** in-range integer index.

`Rat` is deliberately left out even though it would probably qualify; so is anything with a shared payload, so that a cached literal can never be mutated through.

Falling through costs nothing incorrect. The only things the fast path evaluates are variable lookups and cached literals, neither of which has side effects, so nothing is ever evaluated twice.

## 19.5 Why it is not a new node kind

The obvious implementation is a new `NK::` per shape — a superinstruction — rewritten into the tree by a pass. This is deliberately not that.

A new node kind has to be taught to the parser, `AstDump`, `AstEmit` (which serialises the tree for `--aot`), `Codegen`, `Lint`, and every `switch` over `NK`; any of those missing a case is a silent wrong answer. Caching a verdict *inside* the existing node needs none of them, degrades to the old behaviour by construction, and leaves the AST dump and both compiling back ends seeing exactly the tree they saw before.

It also needs no `-O` flag. The transformation cannot change semantics — same `applyArith`, same operands — so it is always on, decided lazily per node.

## 19.6 Measured

Alternating A/B, medians of seven, both binaries interleaved so that machine drift hits each equally:

| kernel | base | specialised | delta |
|---|---:|---:|---:|
| vars — `$a OP $b` | 840.8 ms | 686.5 | **−18.3%** |
| lits — `$a OP 1` | 728.9 | 600.0 | **−17.7%** |
| fib | 478.6 | 422.4 | **−11.7%** |
| asg | 395.1 | 349.5 | **−11.5%** |
| index — `@a[$i]` | 195.3 | 178.4 | **−8.7%** |
| hash | 42.6 | 40.0 | −6.2% |
| loopsum | 208.6 | 209.6 | +0.5% |
| **ctl — method dispatch only** | 327.9 | 327.0 | **−0.3%** |

**The control is the point of the table.** It is a loop of pure method dispatch, containing nothing the specialisation can touch, and it does not move. Without it the other rows would only be evidence that the machine was quieter the second time.

Roast came out identical: 196,590 assertions, 631 files fully passing, with not one file moving in either direction.

## 19.7 What went wrong on the way

Three mistakes, all invisible to reasoning and obvious to measurement. They are the reason this chapter exists.

**The first version made the control 5.7% *slower*.** It bound the right operand through a `std::optional` so the literal could be used by reference — which cost the *general* path its copy elision, since `Value r = eval(...)` constructs in place where `emplace` move-constructs. The lesson is structural: **add an early exit, never restructure the shared code underneath it.** The final version leaves everything after the fast path byte-identical.

**The literal cache was a reassignable `shared_ptr`.** Under parallel execution a second thread rebuilding it could free the object while this thread held a raw pointer into it. It is now allocated once and never freed: the node outlives every evaluation, so there is nothing to reclaim before exit, and a racing double-build leaks one `Value` instead of dangling.

**A negative subscript nearly changed behaviour.** The first `evalIndex` version wrapped negative indices from the end, copying logic from a branch further down that plain arrays never reach. `my $i = -1; @a[$i]` must throw `X::OutOfRange`. Rakudo could not have caught this — it rejects the spelling at compile time — so it was found by diffing against the **previous rakupp binary**, which is now the habit: for a change that is supposed to be semantically invisible, the baseline binary is the oracle, not another implementation.

## 19.8 Codegen already had this

Worth knowing before anyone ports it: `--exe` does not need it. The code generator keeps variables in C++ locals, so there is no lookup and no copy, and it calls `applyArith` directly rather than going through `evalBinary`, so the temporal and hyper probes never existed there. With `-O` it goes one level deeper still, into a raw `int64` lane that never materialises a `Value` for the arithmetic at all (Chapter 27).

What this change really did was close part of the gap between the interpreter and what the code generator had been doing all along — which is why the win was large.

Two residual costs do remain in **non-`-O`** codegen, and they were measured rather than guessed. Taking the generated C++ for a 3M-iteration literal-heavy loop,

hand-editing one temporary at a time, compiling each variant with identical flags and interleaving the runs (medians of 7, outputs verified identical):

| variant | time | delta |
|---|---|---|
| as generated | 640.5 ms | — |
| op name hoisted to a `static const std::string` | 636.1 | −0.7% |
| **literal hoisted to a `static const Value`** | **598.1** | **−6.6%** |
| both | 602.0 | −6.0% |

So the op-string temporary is free — it is a one-character SSO string, and hoisting it is within noise. The whole win is `Value::integer(1LL)` being rebuilt every iteration: an object constructed and destroyed three million times to hold the number 1. (`Value` was 376 bytes when that was measured and is 128 now, so the ratio has moved; the shape of the finding has not.)

That is the same finding as on the interpreter side in a different costume, and it suggests the same fix: emit each distinct literal once at file scope and reference it. Not implemented — it only affects `--exe` *without* `-O`, since with `-O` the int lane never builds the `Value` at all, and it needs its own pass through the `--exe` golden tests and `run-optbench`.

## 19.9 Extending it

The same treatment fits other shapes, in rough order of expected value:

- `Assign` where the target and the left operand are the same variable (`$x = $x + 1`) — write through the slot instead of building and storing;
- `%h{$k}` and `%h<lit>`, the associative twin of the array path;
- `Unary` on a plain lexical (`-$x`, `!$x`, `++$i`);
- method calls on a plain lexical invocant, which is where the *other* half of the profile lives.

Whatever is added, the two rules that made this one safe still apply: **guard on the runtime value every time, and leave the general path underneath untouched.**

# Part V

# The Regex and Grammar Engine

# Chapter 20

# Compiling a Regex

Raku's regex sub-language is not a bolt-on. It has its own quoting rules, its own whitespace conventions, named rules, code assertions, and grammars built out of it. Raku++ implements it as a **second, small recursive-descent compiler** inside the runtime, producing a node tree that a backtracking matcher walks.

The whole thing is one class in one file:

```
// src/Regex.h
class Regex {
public:
    explicit Regex(const std::string& pattern, const std::string& flags = "");
    bool ok() const;
    bool search(const std::string& subject, long startPos, RxMatch& out) const;
    bool matchAt(const std::string& subject, long pos, RxMatch& out,
                 const SubResolver& r,
                 const std::set<std::string>* lexNames = nullptr) const;
};
```

## 20.1 The pattern text arrives unparsed

The Raku lexer does not tokenize regex syntax (Chapter 4). A `/.../` literal, an `rx//`, an `s///` and a `token NAME { ... }` body all travel through the token stream as raw text. The regex compiler re-lexes that text with its own scanner.

That separation is the reason two grammars can coexist without either contaminating the other, and it is why the constructor takes a `std::string` rather than a token range.

Adverbs arrive as a **flags string** alongside the pattern: `i` for ignorecase, `s` for sigspace, `r` for ratchet, `m` for ignoremark, `5` for Perl 5 pattern syntax (the last section of this chapter). A `token` compiles with `r`; a `rule` with `sr`.

## 20.2 The node tree

```
// src/Regex.h
enum class K {
    Lit, Any, Class, Seq, Alt, Conj, Rep, Group,
    AnchorStart, AnchorEnd, WBLeft, WBRight, Nop,
    Subrule, Look, Code, VarMatch, CapStart, CapEnd, CondRef
};
```

Twenty kinds — the last one exists only for Perl 5 patterns, and this chapter ends with it. The interesting information is in the fields rather than the tags:

```
// src/Regex.h — Node, abridged
struct Node {
    K k;
    std::string lit;                                // Lit
    std::vector<std::unique_ptr<Node>> kids;
    std::vector<std::pair<unsigned char, unsigned char>> ranges;   // Class
    std::vector<std::pair<uint32_t, uint32_t>> cpRanges;
    std::vector<std::string> clusterMembers;        // multi-codepoint graphemes
    std::string classFlags, negClassFlags, uprop;
    bool negate = false, icase = false, imark = false, multiline = false;
    mutable uint32_t byteset[8];                    // built on first use
    mutable std::atomic<bool> bytesetReady{false};
    long min = 0, max = -1; bool greedy = true, possessive = false;  // Rep
    std::unique_ptr<Node> sep; bool sepTrail = false;    // X+ % Y
    std::string repCode;                            // ** { ... }
    bool runOnly = false, ltmStop = false;          // Code
    bool firstMatch = false, classCombo = false;    // Alt
    int capIndex = -1; std::string capName; bool listCap = false;   // Group
    std::string ruleName, ruleArgs, ruleAlias;      // Subrule
    bool aliasDotted = false, ruleCapture = true;
    mutable const GrammarRuleMeta* metaCache = nullptr;
    bool behind = false;                            // Look
```

```
    mutable long lookMin = 0, lookMax = -1;
    mutable std::unique_ptr<LtmNfa> ltmNfa;         // Alt
};
```

Three fields deserve attention now; the rest appear where they are used.

## 20.3 The byteset: a per-node character-class cache

A character class has to answer "does this byte match?" for every position it is tested at, taking into account ranges, named classes, negation and case-folding. Computing that from the class's own data each time is a loop.

So each `Class` node caches the answer for all 256 bytes as a bitmap, built on first use:

```
mutable uint32_t byteset[8];                    // 256 bits
mutable std::atomic<bool> bytesetReady{false};
```

The flag is an *acquire/release* atomic rather than a relaxed one, and the comment says why: it gates a 32-byte array, so a relaxed store would not guarantee that a reader which sees the flag also sees the bytes. This is the one place in the codebase where the weaker ordering used everywhere else would be wrong, and it is worth noticing as a general point — `DecidedOnce` is safe because it publishes a *single word*.

Codepoints above 255 do not fit a byteset, so they live in `cpRanges` and are tested separately. Multi-codepoint graphemes — a class member written `\c[A, COMBINING ACUTE ACCENT]` — live in `clusterMembers` and are compared as whole clusters.

## 20.4 The parser

Five mutually recursive functions, one per precedence level:

```
// src/Regex.h
NodePtr parseAlt();     // a | b   and   a || b
NodePtr parseConj();    // a & b
NodePtr parseSeq();     // concatenation
NodePtr parseQuant();   // * + ? ** N..M, with % separators
NodePtr parseAtom();    // everything else
```

`parseAtom` is the large one. It handles literals and quoted literals, `.`, the escape classes `\d \w \s` and their negations, `<[...]>` and `<-[...]>` character classes with ranges, named classes and Unicode properties `<:Nd>`, groups `[ ]` (non-capturing)

and `( )` (capturing), named captures `$<x>=[...]`, subrule calls `<name>` with their aliasing and capture variants, lookarounds, code assertions `<?{...}>` and `<!{...}>`, side-effect blocks `{...}` and `:my` declarations, `$var` interpolation, the capture-span markers `<(` and `)>`, anchors, and word boundaries.

Two parse-time pieces of state make the adverbs scoped rather than global:

```
// src/Regex.h
bool curIcase_ = false;   // :i / :!i, scoped to the enclosing group
bool curImark_ = false;   // :m / :ignoremark, likewise
int assertDepth_ = 0;     // >0 inside an assertion, so parseSeq stops at `>`
```

so an inline `:i` inside a group applies to that group's nodes and no further — which is why `icase` and `imark` are per-*node* flags rather than per-regex ones.

## 20.5 Sigspace

Under `:s`, or in a `rule`, whitespace in the pattern is significant and means "match optional whitespace here". That is implemented structurally rather than by a matcher flag:

```
// src/Regex.h
static NodePtr wsWrap(NodePtr inner);   // Seq(inner, <.ws>)
```

Each atom is wrapped in a sequence with a non-capturing `<ws>` subrule after it. The consequence is that sigspace needs no support anywhere in the matcher: the `<ws>` calls are ordinary subrule nodes.

## 20.6 Ratchet

`token` and `rule` are *ratcheting*: their quantifiers are possessive and their matches commit — no backtracking into a token once it has matched.

```
// src/Regex.h
bool ratchet_ = false;
```

The flag makes every `Rep` node possessive at compile time. It is also what makes the packrat memo in Chapter 22 sound: a rule that does not backtrack has exactly one match at a given position, so caching it is safe.

## 20.7 Two whole-pattern optimisations

**A single-character rule is inlined at its call sites.** A grammar full of `token space { <[\ \t]> }` would otherwise pay a full subrule call per character:

```
// src/Regex.h
bool rootIsSingleChar() const;
long trySingleChar(const std::string& s, long pos) const;   // pos+1, or -1
```

The call site tests the property once, caches the answer on the rule's metadata, and thereafter tests the character directly.

**Lookbehind width is computed once.** A naive lookbehind scans every earlier start position, which is O(pos) per attempt. Instead the inner pattern's possible width is derived and the scan window bounded:

```
// src/Regex.h
std::pair<long, long> nodeWidth(const Node* n, MState& st) const;
```

```
// src/Regex.cpp — the Look case
if (!n->lookWidthReady) {
    auto w = nodeWidth(child, st);
    n->lookMin = w.first; n->lookMax = w.second; n->lookWidthReady = true;
}
long hi = pos - n->lookMin;
long lo = n->lookMax < 0 ? 0 : pos - n->lookMax;
```

O(width) instead of O(position).

## 20.8 Retired Perl 5 metacharacters

Raku deliberately reuses some Perl 5 regex syntax for other things, and Roast checks that the old spellings produce a specific error rather than silently meaning something else:

```
// src/Regex.h
struct ObsoleteEscape { std::string seq; };
const std::string& obsolete() const;   // non-empty: a retired P5 metachar
```

The constructor records the offending sequence — `\A`, `\z`, `\G`, `\p`, `\Q`, `\1` — so the caller can raise `X::Obsolete` instead of reporting a failed match. Distinguishing "this pattern is wrong" from "this pattern did not match" is worth the extra field.

Retired, that is, in *Raku* syntax. Every one of those spellings is alive and meaningful under the `:P5` adverb, where the same constructor parses the same text with a different front-end — the last section of this chapter.

## 20.9 Interpolation happens before compilation

`/$var/` and `/@words/` are resolved in the pattern *text*, before the regex is compiled, by three interpreter functions:

```
// src/Interpreter.h
std::string interpRegexPattern(const std::string& in);  // $var atoms
std::string spliceRegexVars(const std::string& pat);    // regex-valued vars
std::string rxInterpArrays(const std::string& pat);     // /@arr/ → alternation
```

An array interpolates as a **longest-first literal alternation**, which is the semantics Raku specifies and which the sorting has to implement explicitly. A regex-valued variable is spliced in as source text at construction, so `rx/ $x /` where `$x` is a `Regex` composes.

A `$var` that must be read *at match time* — because the variable changes during the match — is a different node, `K::VarMatch`, evaluated through the hooks in the next chapter.

## 20.10 What the compiler produces

The result of construction is a root node plus a little bookkeeping:

```
// src/Regex.h
int ncaps_;                                        // positional capture count
std::set<int> listCaps_;                           // indices under a quantifier
std::shared_ptr<std::set<std::string>> listNames_; // named keys under one
std::shared_ptr<std::set<std::string>> hashNames_; // %<name>= keys
```

The two “list” sets exist because Raku’s capture arity is decided by the *pattern*, not by the match. A capture under a repetition quantifier is an array **even when it matched once or not at all**. That fact is a property of the compiled pattern, so it is computed once by `collectListNames` walking each quantified atom, and then shared — as a `shared_ptr` to a frozen set — into every match result the pattern produces. Sharing rather than copying matters because a grammar parse produces one of these per rule invocation.

## 20.11 `:P5` — the same engine wearing Perl 5 syntax

Raku specifies an escape hatch: `m:P5/.../` (long form `:Perl5`, also on `s///` and `rx//`) promises that the pattern is Perl 5 syntax — `( )` captures, `[ ]` character classes, `\1` backreferences, `(?i)` inline modifiers, `|` as leftmost-first alternation. Roast holds it to an 11-file, 918-assertion corpus (`S05-modifier/Perl_0.t` through `Perl_10.t`) generated from perl's own `re_tests` suite.

The implementation is a **second front-end, not a second engine**. Seven parser functions produce the same `Node` tree everything else in this chapter produces, and the matcher of Chapter 21 runs it unchanged:

```
// src/Regex.h
bool p5_ = false;
bool p5Multi_ = false;   // (?m): ^/$ anchor at line boundaries
bool p5DotAll_ = false;  // (?s): `.` also matches \n
bool p5Ext_ = false;     // (?x): whitespace and # comments insignificant
NodePtr p5Alt();  NodePtr p5Seq();   NodePtr p5Quant(NodePtr atom);
NodePtr p5Atom(); NodePtr p5Group(); NodePtr p5Escape();
NodePtr p5Class();
```

The structure is a port of the Perl-regex parser inside `showcase/perl/perl.raku` — the Perl 5 interpreter written *in* Raku — widened to the full `re_tests` surface: lookaround, named groups, backreferences, conditionals, inline modifier groups. The four `(?imsx)` state booleans mirror `curIcase_`: saved on entering a group, restored on leaving it, so a mid-group `(?i)` scopes exactly as Perl scopes it.

### 20.11.1 How the adverb reaches the constructor

Two roads lead here, and they meet in the constructor. On the `m//` road the Raku lexer bakes leading adverbs into the literal's text — `m:P5:i/ab/` travels as the string `":P5 :i ab"` — and unknown adverbs used to fall through `skipWs()` untouched, becoming pattern *text* that could never match: `:P5` was silently `Nil` on every input. Now the constructor pre-scans a leading adverb run without committing, and commits only when `P5` or `Perl5` is among it — because in Perl 5 syntax a bare `:` is a literal, so the ordinary scoped-adverb machinery must never see the pattern. On the `s///` road, `substSelect` parses the adverb run itself (it used to *throw* on `:P5`) and passes the fact down as the flag character `5`.

Either way the constructor ends at the same line:

```
// src/Regex.cpp — the constructor, P5 branch
if (p5_) {
    root_ = p5Alt();
    if (!eof()) throw P5BadPattern{}; // e.g. an unmatched `)`
}
```

with parse errors collapsing to `ok_ = false` exactly like Raku-syntax ones.

### 20.11.2 Most of Perl desugars to nodes that already existed

The engine's node inventory turned out to be almost sufficient. Some of the mappings are one-to-one — `(?:…)` is a `Group` with no index, `(?=…)` and `(?<!…)` are `Look` with the right two booleans, `\1` is the in-flight backreference `VarMatch` that Raku uses for `$0` inside a pattern. The rest are small structural rewrites:

| Perl 5 | becomes |
|---|---|
| `\b` | `Alt(WBLeft, WBRight)` — either edge of a word |
| `\B` | that `Alt` inside a negated `Look` |
| `.` (no `/s`) | a negated `Class` holding `\n` |
| `a\|b` | `Alt` with `firstMatch` — Perl `\|` is Raku `\|\|`, never LTM |
| `[a\D]` | a `classCombo` `Alt`: the positive members, then `¬d` |
| `[^a\D]` | De Morgan: `Conj(¬a, d)` — all at one position, last consumes |
| `(?#…)` | consumed before quantifier detection, so `a(?#x){3}` repeats `a` |
| `(?{…})` | nothing — a constant no-op |
| `(??{"…"})` | a constant string sub-compiles; the root node is stolen |

The conditional `(?(N)yes|no)` on a *group number* is the one construct with no structural equivalent — whether group N participated is match-time state — and it is why `K::CondRef` exists: one node, the group number in `min`, the two branches as kids, and a four-line matcher case that picks a kid by looking at `st.caps`. The conditional on an *assertion*, `(?(?=…)yes|no)`, needs no new node at all. The parser reads the condition **twice** — once straight, once with its negation flipped, because `Node` trees have no clone — and emits a first-match `Alt` whose branches each lead with their guard:

```
(?(COND)yes|no)   →   Alt||( Seq(COND, yes), Seq(¬COND, no) )
```

### 20.11.3 Three places the engine had to learn something

Perl's anchors are almost rakupp's anchors. `$` matching at the end *or just before a final newline* is what this engine's `$` always did, so `\Z` came free. `\z` — the absolute end, trailing newline not welcome — did not exist, and `(?m)^` is the engine's `^^` minus one position:

```
// src/Regex.h — Node
bool absEnd = false;  // AnchorEnd: `\z` — not before a final newline
bool p5Line = false;  // AnchorStart, (?m) `^`: after \n, NOT at the end
```

The second flag encodes a genuine difference: `"a\nb\n" =~ /b\s^/m` must fail in Perl, because Perl's multiline `^` does not match in the void after a final newline, while the engine's `^^` does.

The third lesson is in `Rep`. The matcher refuses zero-width quantifier iterations — the standard guard against `(x*)*` looping forever. Perl instead admits **one** empty iteration and then stops, and `re_tests` 1327 checks the observable consequence: in `2(]*)?$\1` the group matches empty, *participates*, and the backreference then matches empty too. The relaxation sits in the `Rep` case, gated so Raku patterns keep the strict guard:

```
auto more = [&](long np) {
    if (np != p) return self(self, count + 1, np);
    return p5_ && count + 1 >= mn && finish(np, count + 1);
};
```

### 20.11.4 Where the semantics are Rakudo's, not Perl's

Three decisions were made by the test corpus rather than by `perlre`, and the source says so at each site. Named groups do not take positional indices — in Perl, `(?<meow>...)` is also `$1`; in Rakudo's `:P5` (and per `Perl_10.t`) it is `$<meow>` only, and `$/[0]` is the first *unnamed* group. A backreference to a group that never matched **fails** — modern Perl agrees, but it was the corpus (`(a)|\1` against `"x"` must not match) that settled it after the implementation initially guessed the old matches-empty behaviour. And interpolation follows Perl rather than Raku: a `$var` in a `:P5` pattern splices its value as **raw regex source** (`interpP5Pattern`), not as the quotemeta'd literal of `interpRegexPattern` — `my $r = '\d+'` really compiles

the \d+. Rakudo fudges its own interpolation tests as not-yet-implemented; rakupp passes them.

The payoff of the front-end-only shape is that everything downstream is shared: rx:P5 values flow through .match, .split, .comb and .subst untouched, :g and :nth and substitution plumbing never learn that the pattern was foreign, and the corpus exercises the one backtracking matcher — 918 assertions of perl's own regression suite running against the same matchNode that grammars use.

# Chapter 21

# The Backtracking Matcher

Chapter 20 ended with a node tree and no way to run it. Running it is where regex engines usually acquire their bulk: backtracking state, capture bookkeeping, the machinery to undo a branch that failed halfway through. This one acquires almost none of it, because the backtracking is the C++ stack unwinding and everything else fits in a single frame struct.

The matcher is continuation-passing. One function walks the node tree:

```
// src/Regex.h
bool matchNode(const Node* n, MState& st, long pos, const FnRef& k) const;
```

"Match node `n` at position `pos`; if it succeeds, call `k` with the position after it; return whether the whole thing ultimately succeeded."

That signature is the entire design. Backtracking is the C++ stack unwinding when a continuation returns `false`, and every construct that has alternatives simply tries them in order:

```
// src/Regex.cpp
case K::Seq: {
    // match kids[0], and in its continuation match the rest
}
case K::Alt: {
    // try each branch in ranked order; the first whose continuation
    // succeeds wins
}
case K::Rep: {
    // greedy: try one more repetition first, then k(pos)
```

```
    // frugal: try k(pos) first, then one more repetition
}
```

There is no explicit backtracking stack, no thread list, no bytecode. The continuation *is* "the rest of the pattern", and the C++ frame that holds it *is* the choice point.

## 21.1 `FnRef`: a continuation that does not allocate

A `std::function` for the continuation would heap-allocate on every node visit, which for a matcher is fatal. Continuations here live only for the duration of the match call chain — they are never stored — so the callable can be borrowed from the caller's stack:

```
// src/Regex.h
struct FnRef {
    void* ctx;
    bool (*fn)(void*, long);
    template <class F, class = std::enable_if_t<
                  !std::is_same_v<std::decay_t<F>, FnRef>>>
    FnRef(F&& f)
      : ctx((void*)&f),
        fn([](void* c, long v) {
              return (*(std::remove_reference_t<F>*)c)(v); }) {}
    bool operator()(long v) const { return fn(ctx, v); }
};
```

Two words, no heap, one indirect call. The safety argument is the lifetime rule stated in the comment: continuations are never stored, so the borrowed lambda always outlives the call. Break that rule — store an `FnRef` anywhere — and the result is a dangling pointer.

## 21.2 The step budget

Continuation-passing plus backtracking means a pathological pattern can recurse without bound. `/[a*]* b/` against a long non-matching string is the classic.

```
// src/Regex.cpp — the top of matchNode
if (++st.steps > 8000000) throw StepLimitExceeded{};
```

Eight million steps is far beyond any real match and trips in well under a second on the pathological cases. The exception is caught at each match entry point and reported as a failure to match:

```
// src/Regex.h
struct StepLimitExceeded {};
```

The budget is carried *across* the start positions of an unanchored search, so a quadratic scan cannot escape it by restarting.

This is a blunt instrument and named as one: it prevents a hang, it does not prevent a pathological pattern from being slow, and a legitimate match that genuinely needs more than eight million steps would be reported as a non-match. No such match has appeared.

## 21.3 `MState`: everything one match frame knows

```
// src/Regex.h — MState, abridged
struct MState {
    const std::string& s;
    std::vector<std::pair<long, long>> caps;                // $0, $1, ...
    std::map<std::string, std::pair<long, long>> named;     // $<x>
    std::map<std::string, std::vector<ParseNode>> children;  // subrule trees
    const SubResolver* resolver = nullptr;   // plain-regex subrule path
    class GrammarMatcher* grammar = nullptr; // grammar path
    const std::set<std::string>* lexNames = nullptr;  // my regex NAME shadows
    std::map<int, std::vector<std::pair<long, long>>> capReps;
    long startPos = 0;
    long capFrom = -1, capTo = -1;          // the <( ... )> span
    const GrammarHooks* hooks = nullptr;    // interpreter callbacks
    const std::string* curSym = nullptr;    // the proto candidate's :sym<...>
    long firstCode = -1;                    // where the LTM prefix ends
    long litPrefix = -1;                    // leading literal run
    long steps = 0;
};
```

Captures are **byte spans**, not strings. Nothing is copied out of the subject until a `Match` object is built, which keeps backtracking cheap: undoing a capture is restoring a pair of integers.

`capFrom` and `capTo` implement `<( ... )>`, which narrows what the *overall match* reports while matching continues outside it.

## 21.4 Two subrule paths

A `<name>` call inside a pattern resolves one of two ways, and the difference is fundamental.

**The grammar path** threads the continuation *through* the callee:

```
// src/Regex.cpp — case K::Subrule
if (st.grammar) {
    if (!n->metaCache) n->metaCache = &st.grammar->nameMeta(n->ruleName);
    return st.grammar->matchSubMeta(*n->metaCache, n->ruleName, n->ruleArgs,
                                    ..., st, pos, k, ...);
}
```

Because `k` crosses the call, `<a> <b>` can backtrack **into** `<a>` when `<b>` fails. That is what Raku's grammars require and what most regex engines with "subroutine calls" do not provide.

**The plain-regex path** uses a `SubResolver`, which is atomic: it matches the named rule at a position and answers with a span. No continuation crosses, so no backtracking into the callee.

Between them sit the built-in rules — `<alpha>`, `<digit>`, `<ident>`, `<ws>` and the rest — resolved directly by `builtinRuleMatch`, with one carve-out:

```
// src/Regex.cpp
if (!(st.lexNames && st.lexNames->count(n->ruleName))) { ... }
```

A lexical `my regex ident { ... }` shadows the built-in of that name. The check is a set lookup on a set that is usually null.

The `metaCache` field is a per-node cache of the name resolution, so the hot path never re-resolves a rule name through a string-keyed map. It is safe because compiled regexes live in the matcher's own cache, so the node and the matcher share a lifetime.

## 21.5 Lookarounds

A zero-width assertion matches its inner pattern in an **isolated capture state**, so captures inside a lookahead do not leak:

```
// src/Regex.cpp — case K::Look
MState sub{st.s, std::vector<std::pair<long,long>>(ncaps_, {-1,-1}),
           {}, {}, st.resolver, st.grammar};
```

```
sub.hooks = st.hooks; sub.startPos = pos;
m = matchNode(child, sub, pos, [](long) { return true; });
…
return (m != n->negate) ? k(pos) : false;
```

The hooks are propagated into the sub-state, so an embedded code block or `$var` inside a lookahead still evaluates against the interpreter's live scope. The final line is the whole of positive-versus-negative: `negate` flips the sense, and either way the position does not advance.

## 21.6 The hooks: running Raku during a match

A Raku regex can contain executable code — `{ … }` side effects, `<?{ … }>` assertions, `:my` declarations, `$var` atoms, and `** { … }` quantifier bounds. The matcher cannot evaluate Raku, so it calls back:

```
// src/Regex.h — GrammarHooks, abridged
std::function<bool(const std::string&)> hasAction;
std::function<void(const ParseNode&)> onRule;
std::function<bool(const std::string&, long, long,
                   const NamedMap&, const ParamMap&)> assertPass;   // <?{…}>
std::function<void(…)> run;        // :my / {…}
std::function<void(…)> runCaps;    // …with positional captures, so $0 works
std::function<std::string(…)> str;                  // a $var atom
std::function<std::pair<long,long>(…)> range;       // ** { … }
std::function<std::shared_ptr<void>()> saveState;
std::function<void(std::shared_ptr<void>)> restoreState;
std::function<bool(const std::string&, std::string&, std::string&)> namedRule;
```

Every hook is optional. A plain `Regex` with none of them set treats code constructs leniently rather than failing, which is what lets the same engine serve a simple `~~ /…/` and a full grammar parse.

The current named and positional captures are passed *into* the hook, so the block's `$/` can offer `$<x>` and `$0` — the assertion in `/ (\d+) <?{ $0 > 10 }> /` needs them, and they exist only inside the matcher.

`saveState` and `restoreState` exist for one specific purpose: the longest-token ranker in Chapter 23 measures branch lengths by *probing*, and a probe must not leave the interpreter's `:my` variables and deferred `makes` behind. They snapshot and roll back that state around a measurement pass.

## 21.7 When side effects fire

This is the subtlest semantic decision in the engine, and it differs between the two paths.

**On the grammar path, blocks fire eagerly.** A completed subrule fires its action method immediately, and a later backtrack does **not** unfire it. That sounds wrong, and it is what Rakudo does: `HTTP::Header` sets header fields from the actions of a parse whose `TOP` ultimately fails on a missing trailing newline, and expects the fields to be there. A later `<?{ ... }>` assertion also needs to see what an earlier `{ ... }` set.

**On the plain-regex path, bare blocks are queued.** The matcher is continuation-passing and backtracking, so a block sitting on a branch that is later abandoned would otherwise fire anyway — and fire again on every retry. So the plain entry points queue them, unwind the queue when a branch fails, and run them in order only once the overall match is accepted.

The gating flag is why: only the plain-regex entry points turn queuing on and drain it. The grammar path wants eager.

`hasAction` exists so that rules with no action method cost nothing. Assembling a `ParseNode` for a completed rule is not free, so the matcher asks first whether anyone is listening.

## 21.8 Entry points

```
// src/Regex.h
bool search(const std::string& subject, long startPos, RxMatch& out) const;
bool matchAt(const std::string& subject, long pos, RxMatch& out,
             const SubResolver& r, const std::set<std::string>* lexNames) const;
std::vector<RxMatch> searchExhaustive(const std::string& subject,
             const SubResolver& r, const std::set<std::string>* lexNames) const;
```

`search` is the unanchored scan: try each start position in turn. `matchAt` is anchored, used by grammar subrule calls. `searchExhaustive` implements `:exhaustive` — every match at every start position and every length — which is the only mode that does not stop at the first success.

The result is a `RxMatch`: spans, capture arrays, named spans, the child `ParseNode` trees for subrules, and the shared frozen sets that record which captures are list-valued. The interpreter turns that into a Raku `Match` object, which is a `VT::Match`

`Value` carrying the subject in `s`, the span in `rFrom`/`rTo`, positional captures in `arr` and named ones in `hash` — the canonical example of a `Value` with four fields live at once (Chapter 8).

## 21.9 Substitution

`s///`, `.subst` and `tr///` share one entry point on the interpreter, because the occurrence-selection adverbs are the complicated part:

```
// src/Interpreter.h
std::string substSelect(const std::string& subj, const std::string& pat,
                        Value* replArg, ValueList& args, long& nsub,
                        bool literal = false,
                        const std::string* tmplRepl = nullptr,
                        Value* matchResult = nullptr);
Value substApply(Value* target, const std::string& pattern,
                 const std::string& repl, bool nonMut);
```

`substSelect` handles `:g`, `:x`, `:nth`, `:p`, `:c` and the same-family adverbs (`:samecase`, `:samespace`, `:samemark`), which adjust the replacement to match the case, spacing or marks of what it replaced.

`substApply` is the single call the compiled backend emits, so a compiled `s///` and an interpreted one run the same code — including the `readonly` check that makes `s///` on a bound parameter die.

# Chapter 22

# Grammars

A grammar is the largest thing Raku's regex sub-language is asked to do: a whole parser, written declaratively, with a parse tree and a set of actions falling out of it. Four working language interpreters in this project's showcase — for JavaScript, Perl 5, Python 3 and Lisp — are built that way, so what follows is load-bearing rather than a demonstration feature.

A Raku grammar is a class whose methods can be regexes. That is not a simplification for the book — it is how it is implemented. `grammar G { ... }` produces a `ClassInfo` with `isGrammar` set, its rules in a string map beside its methods:

```
// src/Value.h — ClassInfo, the grammar fields
std::map<std::string, std::string> rules;      // name → pattern text
std::vector<std::string> ruleOrder;            // DECLARATION order
std::map<std::string, std::string> ruleKind;   // token / rule / regex
std::map<std::string, std::vector<std::string>> ruleParams;
bool isGrammar = false;
```

Inheritance works because `findRule` walks the parent chain exactly as `findMethod` does, so `grammar Mine is Base` overrides individual rules.

The pattern text is stored **unparsed**, as the lexer captured it (Chapter 4). Compilation happens lazily, per rule, on first use.

## 22.1 The grammar matcher

```
// src/Regex.h
class GrammarMatcher {
public:
    struct Rule { std::string pattern, kind; std::vector<std::string> params; };
    std::map<std::string, Rule> rules;
    std::map<std::string, std::vector<std::string>> protos;
    GrammarHooks hooks;

    bool parse(const std::string& input, const std::string& top, bool subparse,
               ParseNode& out, long& endOut);
    bool matchSub(const std::string& name, const std::string& args,
                  const std::string& capKey, Regex::MState& st, long pos,
                  const FnRef& k);
private:
    std::unordered_map<uint64_t, MemoEntry> memo_;
    std::unordered_map<std::string, NameMeta> nameMeta_;
    std::map<std::string, std::unique_ptr<Regex>> cache_;
    std::vector<std::map<std::string, std::string>> scope_;
};
```

A `GrammarMatcher` is built **per parse**. That single fact removes a whole category of problems: every cache in it — compiled rules, name metadata, the packrat memo, the longest-token automata — dies with the parse, so none of them needs invalidation logic.

`Interpreter::grammarParse` builds one, fills `rules` and `protos` from the `ClassInfo`, wires the hooks to the interpreter, and runs it.

## 22.2 A subrule call, in full

`<name>` inside a rule reaches `matchSubMeta`, which is where most of the engine's cleverness lives. In order:

1. **Resolve the name** — or rather, read the already-resolved metadata, which the calling node cached (Chapter 21).
2. **Check the memo**, if this rule is ratcheting.
3. **Compile the rule body**, if this is its first use, and cache it.
4. **Bind parameters**, if the call was `<r($x, 'lit')>`.
5. **Match**, threading the caller's continuation through.

6. **Record a `ParseNode`** in the parent's capture frame.
7. **Fire the action method**, if the grammar's action object has one.

### 22.2.1 The per-name metadata

```
// src/Regex.h
struct GrammarRuleMeta {
    bool ratchet = false; int id = 0;
    Regex* singleChar = nullptr;   // body is one char matcher: inline it
    const void* rule = nullptr;
    Regex* noArg = nullptr;        // pre-compiled parameterless body
    const std::vector<std::string>* proto = nullptr;
    bool isWs = false;
    bool scoped = false;           // body declares `:my`
    bool dynDep = false;           // ...or reads a dynamic var: do not memoise
    std::string builtinClass;      // unknown name → built-in class flags
    mutable std::shared_ptr<LtmNfa> protoNfa;
    mutable bool protoNfaTried = false;
};
```

Computed once per name and reached through a pointer cached on the calling AST node, so the hot path never touches a string-keyed map.

Two flags on it encode a real correctness rule. `scoped` means the rule's body declares `:my` variables, so the interpreter's scope must be saved and restored around the call. `dynDep` means the body declares `:my` **or reads a dynamic variable** — and therefore its match can depend on caller state that is not part of the memo key, so **it must not be memoised**.

That second one is the kind of condition that is obvious in hindsight and invisible in advance. A memo keyed on (rule, position) is only sound if the rule is a function of (rule, position).

### 22.2.2 The packrat memo

A ratcheting rule does not backtrack, so its first complete match at a position *is* the match. Caching it collapses the exponential re-descent that recursive longest-token probing would otherwise cause:

```
// src/Regex.h
struct MemoEntry {
    bool matched = false;
```

```
    long end = 0;
    long declEnd = 0;      // where the declarative prefix ended
    long litPrefix = 0;    // leading literal run, for the LTM tie-break
    long capFrom = -1, capTo = -1;
    std::vector<std::pair<long, long>> caps;
    std::map<std::string, std::pair<long, long>> named;
    std::shared_ptr<const ChildMap> kids;      // frozen: replays share
    std::shared_ptr<const std::set<std::string>> listNames;
    std::shared_ptr<const std::set<int>> listCaps;
    std::shared_ptr<const std::map<int,
        std::vector<std::pair<long,long>>>> capReps;
};
std::unordered_map<uint64_t, MemoEntry> memo_;
```

The key is a 64-bit integer built from the rule id, the position and the bound parameters — not a string, because a string key would allocate on every subrule call.

The child map is stored **frozen behind a `shared_ptr`**, which is the detail that makes the memo pay. Replaying a memoised match is then a refcount bump rather than a subtree copy, and a deeply nested grammar replays large subtrees constantly.

## 22.3 `ParseNode`: the tree the matcher records

```
// src/Regex.h
struct ParseNode {
    std::string name;
    std::string actualRule;    // for a proto: the winning name:sym<...>
    long from = 0, to = 0;
    std::vector<std::pair<long, long>> caps;
    std::map<std::string, std::pair<long, long>> named;
    std::shared_ptr<const ChildMap> kids;     // null = leaf
    std::shared_ptr<const std::set<std::string>> listNames;
    std::shared_ptr<const std::set<int>> listCaps;
    std::shared_ptr<const std::map<int,
        std::vector<std::pair<long,long>>>> capReps;
};
using ChildMap = std::map<std::string, std::vector<ParseNode>>;
```

The child map is a map to a **vector**, so repeated captures of the same name collate into a list — which is Raku's rule for `<pair>*`.

Except that the rule is stronger than "collate when repeated". A capture under a repetition quantifier is list-valued **even when it matched once or not at all**, so the arity has to come from the *pattern* rather than from the match. That is what `listNames`, `listCaps` and `capReps` carry, shared from the compiled `Regex` rather than rebuilt per node.

The interpreter walks the finished tree and builds `Match` values from it, with `$<name>` reading the child map and `$0` reading the positional spans.

## 22.4 Actions

An action object is an ordinary Raku object whose methods are named after the grammar's rules. When a rule completes, its method runs and can call `make` to attach a value to the match.

The engine fires actions **during** the match, through the hooks:

```
// src/Regex.h — GrammarHooks
std::function<bool(const std::string&)> hasAction;
std::function<void(const ParseNode&)> onRule;
```

`hasAction` gates the node assembly, so a rule with no corresponding method costs nothing. `onRule` fires the method with the completed node.

Three properties of that design, all deliberate:

- **A later backtrack does not unfire an action.** This matches Rakudo; the reasoning is in Chapter 21.
- **A memo replay reuses the first firing** rather than firing again, so packrat caching does not multiply side effects.
- **`make` writes into a target on the execution context** — `ExecContext::makeTargets` — which is how the value attaches to the right match rather than to a global.

## 22.5 Parameterised rules

`<r($x, 'lit')>` binds arguments to the rule's declared parameter names:

```
// src/Regex.h
std::vector<std::map<std::string, std::string>> scope_;
const std::map<std::string, std::string>& currentParams() const;
Regex* compiled(const std::string& name, const std::string& argstr,
```

```
                std::map<std::string, std::string>& boundOut);
std::string interpParams(const std::string& pat,
                         const std::map<std::string, std::string>& sc) const;
```

Bindings are strings, and they are interpolated into the pattern *text* before compilation. So `token line($indent) { $indent \N* }` compiles a different `Regex` for each distinct indent value — and the compiled-rule cache is keyed on name plus argument values for exactly that reason.

That design has a clear cost (a compilation per distinct argument tuple) and a clear benefit: parameterised rules need no support at all in the matcher. It is also what makes an off-side-rule parser — a Python tokenizer written in a Raku grammar — possible, and writing one is how the interpreter's `:my` dynamic-variable restore bug was found.

`currentParams` merges outer dynamic-variable parameters into the current frame's, so a nested rule can see an enclosing rule's parameter.

## 22.6 Protoregexes

```
proto token statement {*}
token statement:sym<if>    { <sym> \s+ <condition> <block> }
token statement:sym<while> { <sym> \s+ <condition> <block> }
```

A `proto` is a dispatch group. Its candidates are collected by name:

```
// src/Regex.h
std::map<std::string, std::vector<std::string>> protos;
```

and `<sym>` inside a candidate matches that candidate's own `:sym<...>` literal — which is why `MState` carries a `curSym` pointer.

Candidate selection is **longest-token matching**, not declaration order, and that is the next chapter.

`ParseNode::actualRule` records which candidate won, so `$<statement>.made` can tell an `if` from a `while`.

## 22.7 Entry points and start rules

```
bool parse(const std::string& input, const std::string& top, bool subparse,
           ParseNode& out, long& endOut);
```

`.parse` requires the whole input to be consumed; `.subparse` does not. `:rule<name>` picks a start rule other than `TOP`. Both go through the same function with a flag, so there is one implementation of the anchoring rules.

## 22.8 Where grammars appear in this book again

Grammars were for a long time the one construct the native code generator refused, so a program containing one was compiled by bundling rather than transpiling. That is no longer so: `src/Codegen.cpp` emits the rule table, and `--exe` on a grammar reports `Compiled (native)` (Chapter 26). They are also the heaviest exercise in the test suite: the showcase interpreters for JavaScript, Perl 5, Python 3 and Lisp are grammar-driven, and each of them, when first written, produced a list of genuine bugs in this engine — the `:my` restore, a subrule-alias mis-parse, `|` versus `||` ranking, and an optional-block state corruption among them.

# Chapter 23

# Longest-Token Matching

Raku's `|` alternation does not try its branches left to right. Each branch is ranked by how far its **declarative prefix** can reach into the input — the leading part of the pattern made of literals, character classes and quantifiers, up to the first procedural construct. The branch with the longest reachable prefix is committed first; ties break by more-literal-first, then by declaration order. `||` is the opt-out: sequential first match, no ranking.

```
say ("abcd" ~~ / [ ab { } cd ] | abc /).Str;   # abc
```

The first branch *could* match four characters, but its declarative prefix ends at the code block, at length 2. `abc`'s prefix is 3, so `abc` wins.

That annotation is also the shortest proof of which ranker is running. Ask for the old one and the answer changes:

```
$ rakupp        -e 'say ("abcd" ~~ / [ ab { } cd ] | abc /).Str'   abc
$ RAKUPP_LTM=0  rakupp -e '…the same program…'                      abcd
```

Two properties of that example matter and pull in opposite directions. The ranking is by **prefix** length, not by greedy match length. And ranking must run **no user code** — the `{ }` above must not fire for a branch that loses.

## 23.1 Two rankers

**The probe** (`RAKUPP_LTM=0`, and still the fallback) runs each branch once for its greedy full-match end, snapshotting and rolling back side effects through the `saveState`/`restoreState` hooks, and commits branches in longest-end-first order.

It is cheap and it is wrong twice: it ranks by the wrong length, and it descends into user-code paths that true longest-token matching never visits. That is why it is no longer what runs.

**The NFA** (the default since v3.0.0) builds a Thompson automaton per alternation, lazily, from the compiled node tree, and answers "how far could each branch's declarative prefix reach?" in one linear scan of the input. No code execution, no backtracking. The commit phase then runs the real engine on the ranked branches, so captures, assertions and side effects behave exactly as before.

An automaton cannot execute code. That is not a limitation here — it is precisely what makes it the structurally correct ranking oracle.

## 23.2 What ends a declarative prefix

Two very different reasons a prefix stops, and the distinction is the whole correctness argument.

**Spec enders** — the prefix genuinely ends here, and ranking at this point is *correct*: a bare `{...}` code block, a back-reference (`$0`, `$<name>`), a `||` tail (the first branch's prefix continues, per the specification), runtime quantifier bounds `** {$n}`, and `&` conjunction.

**Model gaps** — the *builder* could not model the construct, so ranking might unfairly demote this branch: an unexpandable subrule, Unicode-property and grapheme-cluster classes, `:m` on a character *class*, non-ASCII `:i` folding, lookarounds, and a blown build bound (200 depth or 4,000 states).

Zero-width assertions are **transparent**: anchors, word boundaries, `:my` declarations, and the code assertions `<?{...}>` / `<!{...}>` are epsilon transitions for ranking purposes. Roast checks this — `a <?{ 1 }> .+ | aa` against `"aaa"` matches `aaa`, so the assertion does not end the prefix.

Over-permissiveness is safe: the commit engine enforces assertions for real, and a branch whose assertion fails simply fails at commit and hands over to the next ranked branch.

## 23.3 The gap-aware hybrid contract

The NFA must never be *less* correct than the probe it replaced. So:

> The NFA decides an alternation only when **every** branch's prefix ended for a spec reason. If any branch hit a model gap, that alternation falls back to the probe.

```
// src/LtmNfa.h
bool anyModelGap() const { return anyGap_; }
```

Each expansion route closed converts more alternations from probe fallback to NFA ranking. A branch whose prefix cannot reach any accept state on the given input is not a candidate at all — the commit never tries it, matching Rakudo's own pruning.

## 23.4 The automaton

```
// src/LtmNfa.h
struct Pred {
    char kind = 'A';   // 'L' literal cp, 'C' class node, 'A' any,
                       // 'S' whitespace (<ws>), 'F' builtin flag class
    const void* node = nullptr;
    uint32_t lit = 0;
    bool icase = false;
};
struct State {
    std::vector<std::pair<int,int>> eps;
    std::vector<std::pair<int,int>> edges;
    int acceptBranch = -1;
    int litDepth = 0;
};
```

Transitions are codepoint predicates that **reuse the `Class` node's own match data** — byte ranges, codepoint ranges, negation, folding. There is no second Unicode implementation, which is both less code and structurally safer: the ranker cannot disagree with the matcher about what a class contains.

The `Pred` borrows the `Regex`'s `Node`, relying on the regex outliving its cached automaton — the same lifetime the byteset cache already assumes.

Every branch gets its **own** entry epsilon-edge from state 0. Sharing entry states once let a sibling branch's `a*` loop re-anchor other branches, which produced ranks that were not merely wrong but incoherent.

## 23.5 The cache, and an address-reuse bug

The automaton is cached **on the `Alt` node**:

```
// src/Regex.h — Node
mutable std::unique_ptr<LtmNfa> ltmNfa;
```

The first implementation used a side map keyed on `Node*`, which is the obvious thing and was wrong. A process compiles many regexes, freed node addresses get recycled by the allocator, and the map handed out an automaton built for a completely different pattern. The symptom, found by the phase-1 harness on a Roast alternation file, was empty or nonsense ranks.

That is the exact opposite of the flip-flop state map in Chapter 18, which *is* keyed on a node address and *is* correct — because AST nodes are never freed while regex nodes are. The rule to extract: **a pointer is only a valid key when its target's lifetime is at least the map's.**

For expanded subrules there is a second staleness axis. Named regexes register last-wins, so a re-declared token changes what the same compiled node resolves to. The stored pattern text doubles as a staleness stamp:

```
// src/LtmNfa.h
bool stillValid(const GrammarHooks* hooks) const;
std::map<std::string, std::string> expandStamps_;   // name → pattern text used
```

## 23.6 Subrule expansion: three routes

A <name> inside a branch resolves through an expansion context:

```
// src/LtmNfa.h
struct LtmExpand {
    const GrammarHooks* hooks = nullptr;
    std::function<int(const std::string& name, const void*& regexOut,
                      char& flagOut)> grammar;
};
```

**Lexical.** Interpreter-registered `my token/rule/regex` bodies, handed over as text plus the match path's exact flag spelling (`rule` becomes `sr`, `token` becomes `r`). The automaton compiles and **owns** the callee, inlining its prefix recursively. Recursion is treated as a spec prefix end.

**Grammar.** Already-compiled rule bodies from the grammar's own table, so nothing is recompiled and the automaton borrows the `Regex`'s nodes. The resolver answers with a small integer: refuse, here is a compiled body, this is `<ws>`, or this is a single-character built-in class. Protos, parameterised rules and dynamically-dependent bodies refuse, which becomes a model gap and therefore a probe fallback.

**`<sym>`** inlines as the current candidate's `:sym<...>` literal, for proto dispatch.

`<ws>` is modelled as `\s*` for ranking. That is deliberately approximate — the commit engine enforces the real `<!ww>` word-boundary gate — and it is ranking-grade rather than semantics-grade, which is all the ranker needs.

### 23.6.1 The composed character class

A class written `<[\-+.] +uri_alpha +digit>` is parsed into a synthesised first-match `Alt`. Semantically it is a **one-character union**, so the builder must union its members:

```
// src/Regex.h — Node
bool classCombo = false;   // a SYNTHESIZED Alt for a composed char class
```

Without the tag those nodes were modelled like a user `||`, which takes only the first member. That under-matched the prefix and pruned whole branches — the symptom was a URI grammar failing on its `scheme` rule.

## 23.7 Proto dispatch

Protoregex candidates are ranked by the same machinery:

```
// src/LtmNfa.h
static std::unique_ptr<LtmNfa> buildForBranches(
    const std::vector<const void*>& regexes,
    const std::vector<std::string>& syms, const LtmExpand* ctx);
```

One union automaton over all candidates' bodies, one branch per candidate, with `<sym>` inlined per branch. It is cached on the `GrammarRuleMeta`, and since the matcher is per-parse, that cache dies with the parse and needs no staleness management at all. Any model gap in any candidate discards the automaton and the probe dispatch stands.

## 23.8 The tie-break must be per-path

Equal prefix length breaks by "more literal wins", and that count must be computed **per path during the scan**, not stored per state.

The builder originally kept a static literal depth on each state, max-merged at join states. In:

```
token TOP { <foo> | <bar> }
token foo { \w\w }
token bar { aa | <foo> }
```

on input `"bb"`, `bar`'s dead `aa` path — two literals — leaked its count onto the join state that `bar`'s live `<foo>` path — zero literals — also reaches. So `bar` stole the tie from the earlier-declared `foo`.

`rank()` now carries, per live state, the length of the leading literal run along the path that reached it, frozen at the first non-literal edge; accepts record the *path's* value, not the state's.

```
// src/LtmNfa.h
struct Ranked { int branch; long prefixEnd; long litPrefix; };
std::vector<Ranked> rank(const std::string& s, long pos) const;
```

sorted by prefix end descending, then literal prefix descending, then declaration order — the specification's tie-break chain.

## 23.9 Oracle notes

Three behaviours confirmed against Rakudo while building this, each of which would otherwise have been guessed wrong:

- **`multi token` protos do not rank.** Rakudo ranks proto candidates by declarative prefix only when they are declared `token t:sym<x>`; `multi token t:sym<x>` candidates dispatch in declaration order, and so does a direct `.subparse(:rule<proto>)`. Both rankers here rank `multi token` candidates too — a pre-existing divergence, deferred rather than hidden.
- **`||` continues through its first branch.** `a || b`'s declarative prefix is `a`'s prefix: not empty, and not both.
- **Code assertions are transparent** to ranking, as above.

## 23.10 Debugging and measured results

| Variable | Effect |
|---|---|
| `RAKUPP_LTM=0` | fall back to the legacy probe ranker everywhere |
| `RAKUPP_LTM_DEBUG=1` | rank both ways and print disagreements |
| `RAKUPP_LTM_RANKDUMP=1` | print each NFA-decided alternation's ranking |

`RANKDUMP` needs no other flag, which makes it the one-command answer to "which ranker am I getting?" — if it prints, the automaton decided.

`RAKUPP_LTM_DEBUG` was the phase-1 harness: before anything consulted the automaton's answer, the probe path computed both rankings and printed every disagreement with its pattern context, for classification against Rakudo. Its block still sits in that fall-through, *after* the NFA branch, so on today's default it only fires for alternations that fall back. Pair it with `RAKUPP_LTM=0` to get the both-ways comparison back.

Same binary, same machine, full Roast, measured while the flip was being decided (v3.0.0; the engine's standing has moved a long way since, so read these as the comparison they were, not as today's count):

| ranker | assertions | `longest-alternative.t` | `proto-token-ltm.t` |
|---|---|---|---|
| probe (`RAKUPP_LTM=0`) | 197,116 | 45/62 | 10/10 |
| NFA (now the default) | 197,117 | 47/62 | 10/10 |

The NFA's failure set on the alternation file is a strict subset of the probe's, which is what made the flip safe. A grammar benchmark — twenty compiles of a JSON grammar corpus — runs in 28 to 34 milliseconds in both settings, so automaton construction is fully amortised by the node cache. The twenty-eight grammar showcase runs are byte-identical across settings.

The default changed in v3.0.0 and `RAKUPP_LTM=0` is kept for one release after it, which is the general policy for a switch that changes behaviour rather than speed: an escape hatch and a bisection tool, then the probe path retires.

Two details that live in the resolver rather than the automaton: `namedRule` and `ltm-Resolve` are what a protoregex dispatch goes through, and a lexically scoped `<ws>` is

resolved at the call rather than the declaration, so a grammar that redefines it gets its own.

# Part VI

# Unicode

# Chapter 24

# Graphemes, Normalization, Collation

Raku specifies strings at the level of **graphemes** — what a reader would call a character — not codepoints and certainly not bytes. `"e\c[COMBINING ACUTE ACCENT]".chars` is 1. Sorting respects the Unicode Collation Algorithm. `"\c[LATIN SMALL LETTER E WITH ACUTE]".uniname` answers a real name from the character database.

None of that is optional, and Raku++ implements all of it from the pinned Unicode 18.0 data with no external library.

## 24.1 Storage: UTF-8 bytes, grapheme semantics

A `Str` holds UTF-8 bytes in its `CowStr`. Raku's *indices* are grapheme indices. Those two facts have to be reconciled on every string operation, and how that reconciliation is made cheap is the most consequential engineering decision in this part of the system.

Strings are normalised to **NFC** on the way in — Raku's NFG storage model — so a literal is normalised once, in place, on first evaluation:

```
// src/Ast.h — StrLit
bool nfcDone = false;   // NFC-normalized in place on first eval
```

## 24.2 The fast answer: when a byte index is a grapheme index

Most strings in most programs are ASCII. For those, byte index, codepoint index and grapheme index are all the same number, and every conversion is a no-op — if you can establish it cheaply.

```
// src/BuiltinsShared.h
size_t asciiRun(const std::string& s, size_t limit);
bool allAscii(const std::string& s);
bool byteIsGraphemeIndex(const std::string& s);
bool cowAllAscii(const CowStr& s);
bool cowByteIsGraphemeIndex(const CowStr& s);
long long cowGraphemeCount(const CowStr& s);
```

The `cow*` forms are the memoised ones. They read the flags cached on the promoted string body (Chapter 9), so the answer is computed once per string rather than once per character examined.

The condition is narrow and exact: **all bytes below 0x80, and no carriage return.** CR LF is the one ASCII sequence that forms a single grapheme cluster under UAX #29, so an ASCII string containing a CR can still cluster.

Getting this wrong is not a correctness bug — it is a *complexity* bug. The scanning operations call these once per character examined, so the difference between memoised and not is the difference between a linear tokenizer and a quadratic one.

## 24.3 The slow answer: `GraphemeMap`

When a string can genuinely cluster, the translation between codepoint indices and grapheme indices is built explicitly:

```
// src/Unicode.h
class GraphemeMap {
public:
    explicit GraphemeMap(const std::vector<uint32_t>& cps);
    bool trivial() const { return starts_.empty(); }
    size_t count() const;                  // graphemes
    size_t cpAt(size_t g) const;           // codepoint index where g starts
    size_t graphemeAt(size_t cp) const;    // grapheme containing codepoint cp
private:
    size_t ncps_;
```

```
    std::vector<size_t> starts_;           // empty when 1 grapheme == 1 cp
};
```

The class exists because `substr`, `index` and `flip` each used to re-derive the translation — or, worse, skip it entirely and index codepoints, which is silently wrong for any text carrying a combining mark.

The common case costs one linear scan and no allocation: a string with no codepoint above U+02FF and no CR cannot cluster, so `starts_` stays empty and `trivial()` is true. Only a string that can actually cluster pays the full UAX #29 walk.

## 24.4 The subsystems

```
// src/Unicode.h — the interface, abridged
std::vector<uint32_t> uniNormalize(const std::vector<uint32_t>&, int mode);
size_t uniGraphemeCount(const std::vector<uint32_t>& cps);
std::vector<size_t> uniGraphemeStarts(const std::vector<uint32_t>& cps);
size_t uniClusterEndUtf8(const std::string& s, size_t pos, size_t len);
int uniCollate(const std::vector<uint32_t>& a, const std::vector<uint32_t>& b);
int32_t uniCharByName(const std::string& name);
std::string uniNameOf(uint32_t cp);
bool uniNumValue(uint32_t cp, long long& num, long long& den);
std::string uniGeneralCategory(uint32_t cp);
std::string uniScript(uint32_t cp);
bool uniMatchesProp(uint32_t cp, const std::string& prop);
uint32_t uniSimpleUpper(uint32_t cp);
std::vector<uint32_t> uniCaseMap(uint32_t cp, int kind); // 0lo 1up 2ti 3fold
std::string uniBlockOf(uint32_t cp);
int uniBinaryProp(uint32_t cp, const std::string& prop);
```

**Normalization** implements all four forms — NFD, NFC, NFKD, NFKC — over codepoint sequences: canonical decomposition, canonical ordering by combining class, and canonical composition.

**Grapheme segmentation** is UAX #29 extended grapheme clusters, which is more than “base plus combining marks”: it covers Hangul syllable composition, regional indicator pairs (flag emoji), emoji modifier sequences and zero-width joiner sequences, and the CR LF rule.

**Collation** is the Unicode Collation Algorithm against the default table, producing a three-way comparison. That is what `coll` and `.collate` use, and it is why sorting Raku strings is not `strcmp`.

**Names** go both ways: name to codepoint for `\c[NAME]`, codepoint to name for `.uniname`. This is the single largest file in the tree at 41,174 lines, and it is a binary search plus a hash lookup at run time.

**Properties** cover general category, script, block, numeric value, bidi class, mirroring, and the binary and enumerated properties. `uniMatchesProp` is what backs `<:Nd>` and `<:L>` inside a regex character class — which is why a regex class with a Unicode property in it is a `Class` node with a `uprop` string rather than a byteset (Chapter 20).

**Case mapping** has two tiers: simple 1:1 mappings from `UnicodeData`, and full 1:N mappings from `SpecialCasing` and `CaseFolding`. The second is not optional: uppercasing the German sharp s produces two characters, and Turkish dotted and dotless i are language-sensitive.

## 24.5 The tables are generated, and one generator is written in Raku

```
tools/ucd/                  pinned UCD + UCA 18.0 data files
tools/gen_unicode_*.py      table generators
tools/gen-unicode.raku      …and the ones written in Raku
src/unicode_*_gen.cpp       the output: 79,400 lines
```

The tables are checked in rather than generated at build time, which keeps the build free of a data dependency and makes the Unicode version a reviewable part of the source tree.

Two of the generators — the names and one of the property tables — are written in **Raku and run by rakupp itself**. That is not a stunt. It is a load-bearing test: generating a 41,000-line table exercises string handling, hashing, sorting and file I/O at a scale no unit test reaches, and a bug in any of them shows up as a table that does not compile or does not match.

It is also the project's standing rule for tooling: build the ecosystem tools in Raku, run them with rakupp, and let the compiler eat its own output.

## 24.6 Where Unicode shows up elsewhere

**In the lexer.** Identifiers may contain Unicode letters and combining marks, so `consumeIdentChars` decodes codepoints rather than reading bytes. Superscript digit runs fold into a ** operator.

**In the regex engine.** A character class carries three parallel representations for three ranges of complexity: a 256-bit byteset for bytes, codepoint ranges for anything above 255, and a list of whole cluster strings for multi-codepoint grapheme members. `:i` folds through `uniCaseMap`; `:m` (ignoremark) compares NFD first-starter base codepoints and consumes the rest of the cluster.

**In the longest-token automaton.** Its transition predicates reuse the regex `Class` node's data rather than reimplementing any of this — the single most important structural decision in Chapter 23, because two Unicode implementations that disagreed would produce rankings that could not be debugged.

**In collation-sensitive built-ins.** `cmp`, `leg`, `unicmp`, `coll`, `.sort` and the ordering operators each pick their comparison from this layer.

## 24.7 Honest limitations

- **Language-sensitive casing is not exposed.** The full case mappings are implemented, but there is no locale parameter, so the Turkish `i` rules are not reachable.
- **Collation is the default table**, with no tailoring and no locale.
- **Normalization is by codepoint sequence**, so a caller that has bytes pays a decode and an encode around it.
- **The Unicode version is pinned in the source tree**, so updating it is a regeneration and a review rather than a dependency bump — deliberate, but it does mean the tables age.

# Part VII

# The Back Ends

# Chapter 25

# Four Ways to Run a Program

The same binary will run the same Raku source four different ways, and the choice is not academic: one mode is for editing and re-running, one for handing someone a program that needs no interpreter installed, one for cutting start-up time, one for throughput. They share the entire front end and diverge completely after it.

One front end, five back ends. This chapter takes one small program through the four that produce a runnable artefact from C++ — interpret, `--bundle`, `--aot` and `--exe` — and shows exactly what each produces. The fifth, `--target=js`, does not go through C++ at all and has its own chapter; it is the one case where the shared-runtime principle everything here rests on had to be given up.

```
# demo.raku
sub square($n) { $n * $n }
my $total = 0;
for 1..5 { $total += square($_) }
say $total;
```

The CLI in `main.cpp` picks a mode from the flags and calls one of four drivers.

## 25.1 Mode 1 — interpret

```
demo.raku ─► Lexer ─► Parser ─► Program (AST)
          ─► Interpreter.run(Program)  ─► walk the tree, node by node
```

`main` reads the source and calls `rakuppRunBigStack`, which lexes, parses, and hands the `Program` to `Interpreter::run`. Evaluation is the recursive `eval`/`exec` of Chapter 13: the `for` loop iterates and re-executes its body block, `square($_)` looks the sub

up in the environment chain and calls it, `$total += ...` re-dispatches through `applyArith`.

Nothing is cached or compiled — the same AST nodes are re-interpreted on every iteration. Startup is about two milliseconds. This is the only mode that runs every construct in the language, so it is the one the language is developed against.

## 25.2 Mode 2 — `--bundle`

```
demo.raku ─► (embedded verbatim as bytes) ─► generated stub.cpp
          ─► cc stub.cpp librakupp_rt.a ─► ./demo
```

The driver does **not** parse the program at build time. It emits a small stub that embeds the source as a byte array and calls the runtime:

```
static const unsigned char SRC[] = { 115,117,98,32,... };
int main(int argc, char** argv) {
    std::string src(reinterpret_cast<const char*>(SRC), SRC_LEN);
    /* … collect argv … */
    return rakupp::rakuppRunBigStack(src, args, "demo.raku", exe);
}
```

then compiles it and links against the runtime. The result is standalone — no `rakupp` needed on the target machine — but at run time it still lexes, parses and tree-walks. It *is* the interpreter in a box. The win is distribution and a roughly ten-millisecond start; run time equals interpreting.

## 25.3 Mode 3 — `--aot`

```
demo.raku ─► Lexer ─► Parser ─► Program        ← parsed at BUILD time
          ─► serializeAst(Program) ─► bytes ─► C++ that carries them
          ─► cc gen.cpp librakupp_rt.a ─► ./demo
```

This is genuine ahead-of-time work: the program is **parsed at build time**, so parse errors are reported then rather than at run time, and the tree travels as bytes:

```
// Generated by `rakupp --aot`. The Raku program was parsed ahead of time;
// its AST travels below as bytes and is rebuilt at startup, so there is no
// lexing or parsing at run time.
static const unsigned char kAst[] = { 82,75,65,83,17, /* … */ };

int main(int argc, char** argv) {
    Program prog;
```

```
    deserializeAst(std::string(reinterpret_cast<const char*>(kAst), sizeof kAst), prog);
    return rakupp::rakuppRunProgramBigStack(prog, args, "demo.raku", exe, "");
}
```

`RKAS` and a format version — 17 at the time of writing — then the tree. The generated `main` rebuilds the identical `Program` and hands it to `rakuppRunProgram`, which interprets it with **no lexing or parsing at run time**. It is the serializer of Chapter 30, with a C++ wrapper around it: one format, used by the precomp cache and by this mode alike.

It was not always so, and the reason it changed is the better half of the story. `AstEmit` used to emit one small builder function per AST node, listing each node type's fields by hand — and that was quietly **lossy**. A field nobody remembered to list simply did not survive, so `my Int(Str) $n = "42"` compiled into a binary that threw a type error, because `VarExpr::declCoerce` was not among the fields it emitted. `Node::line`, `Stmt::label`, most of `Param` and nearly all of `SubDecl` went the same way, which is how a compiled program's diagnostics lost their line numbers. The serializer cannot have that bug: its writer and its reader are one visitor per node type instantiated twice, so a field cannot be saved but not restored, because both directions are the same source line.

Because the interpreter runs the embedded tree, `--aot` handles the **whole language, grammars included**. If `AstEmit` ever meets a node it cannot rebuild, it throws `AstEmitError` and the driver falls back to mode 2.

The trade-offs moved with the mechanism, and mostly in `--aot`'s favour. The generated file for the four-line `demo.raku` above is 2,109 bytes — *smaller* than the 4,182 `--exe` produces for the same program, where the old per-node emitter would have run to tens of thousands of lines. Building it takes 0.66 s against `--bundle`'s 0.48, not the ten times slower it used to be. And the edge is no longer only parse errors: a 2,001-line program starts in 4.5 ms under `--aot` against 8.2 under `--bundle`, because the bundled binary still lexes and parses its own source on every run and this one does not.

## 25.4 Mode 4 — --exe

```
demo.raku ─► Lexer ─► Parser ─► Program
          ─► Codegen.transpileToCpp(Program) ─► C++ source
          ─► cc gen.cpp librakupp_rt.a ─► ./demo   (no interpreter inside)
```

`Codegen` walks the tree and emits C++ that **implements the program directly** — native control flow, native calls — calling the runtime only for `Value` operations. For `demo.raku`, abridged:

```
#include "Interpreter.h"
using namespace rakupp;
static Interpreter RT;
static Value v_stotal = Value::any();     // top-level `my $total`
static const BuiltinFn* __bfp0 = nullptr; // say — resolved once at startup

static Value u_square(ValueList __a) {
    Value v_sn = rtPos(__a, 0);
    return applyArith("*", v_sn, v_sn);
}
static void __rakupp_register() { __bfp0 = RT.builtinPtr("say"); }

int main(int argc, char** argv) {
    __rakupp_register();
    try {
        v_stotal = Value::integer(0LL);
        {                                          // for 1..5 — a real C++ loop
            long long __lo = Value::integer(1LL).toInt();
            long long __hi = Value::integer(5LL).toInt();
            for (long long __i = __lo; __i <= __hi; __i++) {
                Value v__t0 = Value::integer(__i);          // $_
                try { v_stotal = applyArith("+", v_stotal,
                                            u_square(ValueList{v__t0})); }
                catch (const NextEx&) { continue; }
                catch (const LastEx&) { break; }
            }
        }
        rtCallB(RT, __bfp0, "say", ValueList{v_stotal});
    } catch (const RakuError& e) { std::cerr << e.message << "\n"; return 1; }
    return 0;
}
```

The loop is a native `for`, the sub call is a direct C++ call, and only value semantics dip into the runtime. This is the only mode whose runtime performance differs, and the only one the optimizer touches.

## 25.5 Comparing them

| | parse when | runs how | whole language | speed |
|---|---|---|---|---|
| interpret | run time | tree walk | yes | baseline |
| `--bundle` | run time | tree walk | yes | baseline |
| `--aot` | build time | tree walk | yes | baseline |
| `--exe` | build time | native C++ | falls back | several times faster on hot loops |

The important row is the last column of the third row: **three of the four modes run at the same speed**, because three of them are the same tree walk. Bundling and AOT buy distribution and build-time error reporting, not throughput.

## 25.6 The fallback that makes --exe total

`Codegen` throws on any construct it cannot transpile:

```
// src/Codegen.h
struct CodegenError { std::string msg; };
```

mainly grammars, symbolic references (`::($name)`, which name a variable that a native local does not have at run time), and a handful of NativeCall shapes that need copy-back the generated call site cannot express. The driver catches it and **transparently bundles the whole program instead**.

So `--exe` never refuses a program. It native-compiles what it can and bundles the rest, always producing a correct binary. The user is told which happened.

## 25.7 What every mode shares

**One runtime.** Everything except the CLI is in `librakupp_rt.a` — plus four feature archives a compiled binary may leave out, which is Chapter 29. A feature implemented once behaves identically in all four modes. `Value` is the shared currency, `callBuiltin`/`callCallable` the shared calling convention, `rtIndexRef`/`rtAttrRef`/`rtArrayVal` the shared container operations, and `callNative` the shared FFI.

**One big stack.** Every entry point runs the program body on a thread with a large reserved stack, including a compiled binary's `main`, so the recursion budget matches across modes.

**One compile-time gate.** Before any mode does its own work, the parsed unit is asked whether every variable in it is declared, and a program that fails is refused rather than started — so a typo on line 90 is not discovered after 89 lines of output, and `--exe` reports it itself instead of emitting C++ that names an identifier it never declared. `--aot` and `--bundle` inherit it at build time; a `--bundle` binary, which parses its embedded source at run time, is checked again then, since that is the interpreter path. The pass, and why it is careful to the point of standing down, is in Chapter 39.

**Modules are always interpreted.** `Codegen` emits a call to `Interpreter::rtUse`, a thin mirror of the interpreter's `use` handling, which calls the same `loadModule`. Only the *main program* is compiled; a compiled binary still loads and interprets its modules (Chapter 33) — though it can carry their serialised ASTs inside itself, which is the next section.

## 25.8 Carrying modules inside a binary

A standalone binary that needs its modules' source files on the target machine is not very standalone. So the compiling modes resolve the `use` graph at build time and embed each module's serialised AST:

```
// src/Interpreter.h
struct BundledModule { std::string name, blob, finish, src; };
std::vector<BundledModule> collectModuleGraph(
    const Program& prog, const std::vector<std::string>& searchPath,
    std::set<std::string>* exportsOut = nullptr);
void rakuppRegisterModule(const std::string& name, const char* blob,
                          size_t blobLen, const std::string& finish);
```

Registration happens once at startup, and `loadModule` consults the embedded table before it looks at the disk.

A module that cannot be found, parsed or serialised is **simply left out** rather than failing the build — the binary falls back to loading it from disk, which is also what has to happen for anything only a running program can name (`require ::($x)`, a computed `use lib`).

`--bundle` needs one extra thing, and it is a subtle one. It parses the main program at *run* time, and that parse has to scan each `used` module for the operators it declares (Chapter 6) — otherwise a program using an imported operator stops parsing once the module tree is gone. So bundled binaries also carry the module **sources**:

```
// src/Parser.h
void rakuppRegisterModuleSource(const std::string& name,
                                const char* src, size_t len);
const std::string* rakuppEmbeddedModuleSource(const std::string& name);
```

`--exe` and `--aot` parse at build time and never need it.

## 25.9 Mode 1 in a browser

The same runtime compiled to **WebAssembly** is Raku.js: the interpreter running in a browser tab, with the same `Value` semantics and no server. It is mode 1 with a different host — but the host removes the filesystem, the threads and the dynamic loader, which changes enough to be worth its own chapter. That is Chapter 32, at the end of this part.

# Chapter 26

# The Code Generator

Compiling a dynamic language to C++ invites one particular mistake: writing a second implementation of the language inside the code generator, which then has to be kept in agreement with the first one for ever. Raku++ declines that trade, and declining it is why the generator is under three thousand lines against an interpreter of twenty-three thousand.

`--exe` transpiles the AST into a self-contained C++ program. The interface is one function:

```
// src/Codegen.h
std::string transpileToCpp(Program& prog, bool optimize = false,
                           const std::string& srcPath = "",
                           const std::set<std::string>& moduleExports = {},
                           const std::string& srcText = "");
struct CodegenError { std::string msg; };
```

The design principle is stated in one sentence and everything follows from it: **the generator never reimplements the runtime.** It emits C++ that calls the same functions the interpreter calls. That is why the two produce byte-identical output, and why a feature implemented once works in both.

## 26.1 The mapping

| Raku | Generated C++ |
|---|---|
| `$a + $b`, `$a eqv $b` | `applyArith("op", a, b)`, or an inline `rt*` helper |
| `say`, any named builtin | `rtCallB(RT, __bfpN, "say", ...)` |
| `.map`, `.sort`, any method | `RT.methodCall(inv, "map", ...)` |
| a block, `-> $x {...}`, `* + 1` | `Value::closure([=](ValueList& a){ ... })` |
| `@a[i]` / `%h{k}` read, write | `rtIndexGet(...)` / `rtIndexRef(...)` |
| `[+] ...` | `rtReduce("+", ...)` |
| `class` | register a `ClassInfo` at startup; methods become closures |
| `$!x` | `rtAttrGet` / `rtAttrRef` |
| `multi` | one C++ function per candidate, plus a `rtTypeMatch` dispatcher |
| `enum` | global `Value::enumVal` constants |
| `gather/take` | push a collector onto the execution context |
| phasers, `CATCH` | reordered emission, a C++ `try` and a `when` chain |
| `use Foo` | `RT.rtUse("Foo")` — the module is still interpreted |

The `rt*` family in `Interpreter.h` is the generator's vocabulary — about sixty functions, each the runtime's implementation of one construct:

```
// src/Interpreter.h — a sample
Value  rtIndexGet(const Value& base, const Value& key, bool isHash);
Value& rtIndexRef(Value& base, const Value& key, bool isHash);
Value  rtArrayVal(const Value& v);       // list-assignment semantics
Value  rtSlipVal(const Value& v);        // |x as a list element
Value  rtHashLit(const ValueList& items);
Value  rtNamedPair(const std::string& k, Value v);
Value  rtReduce(const std::string& op, const Value& list);
Value  rtTypedDefault(const char* type, char sigil);
ValueList rtMainArgs(const std::vector<std::string>& argv);
```

Several of them exist *only* because the interpreter had the same logic inline and the two would otherwise have drifted. `rtArrayVal` is the compiled twin of `coerceArray`, with the same fresh-buffer rule (Chapter 12), and it was factored out precisely so there is one definition of what `@a = expr` means.

## 26.2 What is emitted

A generated file has five sections.

**A prologue** including `Interpreter.h` and defining one static `Interpreter RT` — the runtime object that owns the builtin table, the class registry and the method dispatcher.

**Static globals** for top-level variables. `my $total` becomes `static Value v_stotal;`, with the sigil encoded into the name (`$` becomes `s`) so that `$x` and `@x` do not collide.

**Functions**, one per user sub. Their parameters arrive as a `ValueList`, and positionals are pulled out by index:

```
static Value u_square(ValueList __a) {
    Value v_sn = rtPos(__a, 0);
    return applyArith("*", v_sn, v_sn);
}
```

**A registration function**, `__rakupp_register`, run before `main`'s body. It resolves cached builtin pointers, registers `ClassInfos` for the program's classes, installs named regexes, and registers embedded modules.

**`main`**, which sets up `@*ARGS`, runs the registration, and executes the mainline inside a `try` with the phaser and exit handling around it.

## 26.3 Control flow

Raku control flow becomes C++ control flow wherever it can.

A `for` over a literal range becomes a native counting loop with the topic rebuilt per iteration, wrapped in a `try` that catches the control exceptions:

```
for (long long __i = __lo; __i <= __hi; __i++) {
    Value v__t0 = Value::integer(__i);
    try { /* body */ }
    catch (const NextEx&) { continue; }
    catch (const LastEx&) { break; }
}
```

Note that compiled code catches the *exception* forms of `next` and `last` rather than reading the cooperative registers of Chapter 15. It can afford to: the cooperative path exists to avoid a throw in the tree-walker's hot loop, and a compiled loop's body is already native.

`if`, `while`, `until` and the ternary become their C++ equivalents. Conditions that are comparisons use the `bool`-returning helpers (`rtLtB`, `rtEqSB`) rather than building a `Bool Value` and reading it back — a default, not an `-O` feature.

## 26.4 Closures

A block becomes a C++ lambda wrapped as a `Value`:

```
// src/Value.h
static Value closure(std::function<Value(ValueList&)> fn) {
    Value v; v.t = VT::Code; v.code = std::make_shared<Callable>();
    v.code->builtin = [fn](Interpreter&, ValueList& a) { return fn(a); };
    return v;
}
```

This is where the dual nature of `Callable` pays off. A `Callable` is *either* an AST body plus a closure environment, *or* a C++ function. The interpreter builds the first kind; the code generator builds the second. Everything downstream — `.map`, `.sort`, `callsame`, `&f` as a value, passing a block to a user routine — treats them identically, because it only ever calls through `callCallable`.

The capture is by value (`[=]`), so a compiled closure captures copies of the `Values` it uses. That is the same observable behaviour as the interpreter's environment capture for reads; for a closure that *mutates* a captured variable the code generator has to emit a shared slot instead.

## 26.5 Classes and multis

A `class` becomes registration code: a `ClassInfo` built at startup, its attributes filled in with precomputed defaults where possible (`ClassAttr::defVal`), and its methods installed as closures. There is no C++ class, no vtable, and no inheritance in the generated code — the runtime's object model does all of it, exactly as it does when interpreting.

A `multi` becomes one C++ function per candidate plus a dispatcher that scores with `rtTypeMatch`. That is a simplification of the interpreter's `scoreCandidate`, which is the honest reason a few multi shapes are on the fallback list.

## 26.6 Signatures

The generator has to bind arguments without the interpreter's `bindParams`, so there is a small family of binding helpers:

```
// src/Interpreter.h
Value  rtPos(const ValueList& a, size_t idx);
bool   rtHasPos(const ValueList& a, size_t idx);
Value  rtNamed(const ValueList& a, const std::string& key);
bool   rtHasNamed(const ValueList& a, const std::string& key);
Value  rtSlurpyPos(const ValueList& a, size_t from);
Value  rtSlurpyNamed(const ValueList& a);
size_t rtPosCount(const ValueList& a, size_t from = 0);
Value& rtPosRef(ValueList& a, size_t i);   // `is rw`: a ref into the list
```

A signature is compiled into a sequence of these. Defaults become `if (!rtHasPos(...)) ...`; a slurpy becomes one call; `is rw` binds a reference into the caller-visible argument list.

## 26.7 Sub-language pieces the generator hands back to the runtime

Some constructs are emitted as a single runtime call rather than transpiled, because transpiling them would mean duplicating an engine:

- **regexes and substitutions** — `RT.regexMatch`, `RT.substApply`;
- **`gather`/`take`** — a collector pushed on the execution context;
- **`use`** — `RT.rtUse`;
- **the `...` sequence operator** — `RT.seqOp` and `RT.seqOpGroups`;
- **hyper operators** — `rtHyperMethod` and the hyper core;
- **indirect method calls** — `rtIndirectMethod`: the generated site evaluates the name expression, and the runtime either calls the `Code` it produced (with the invocant first) or dispatches the name it stringifies to, through the one method dispatcher;
- **`nqp::` ops** — `rtNqpOp`, the same function the interpreter calls (Chapter 35).

Each of those is a place where "call the runtime" is not a compromise but the correct answer: there is one implementation, so there is one behaviour.

## 26.8 What it refuses

`CodegenError` is thrown for anything the generator cannot express, and the driver answers by bundling the whole program. The main cases:

- **grammars**, which need the full engine plus the interpreter hooks;
- `no strict` in a unit whose declaration check could not finish (below);
- a few **NativeCall shapes** — `is native(&lib-sub)`, a library name computed by an expression, `is rw` out-parameters, and buffer or `CArray` parameters that need copy-back;
- anything involving a construct the generator has simply not learned.

The failure is total-per-program rather than per-construct: one refused construct bundles the entire file. That is a deliberate simplification — a hybrid binary that interpreted some routines and compiled others would need the two halves to share an environment, which is a much larger design.

## 26.9 The correctness constraint on resolving names at compile time

The generator resolves a named call at compile time, deciding whether it is a user sub or a built-in. **A `used` module can take that name away.**

```
# lib/OnlySub.rakumod
unit module OnlySub;
sub val() is export { 'from-module' }
```

The interpreter's `evalCall` looks in the environment *before* the builtin table (Chapter 16), so the exported sub wins. Compiled code never did that lookup and printed the built-in `val`'s answer instead — a silent divergence.

So the generator is given the module graph's exported names, and for those it emits a call that looks in the environment first:

```
// src/Interpreter.h
Value callEnvFirst(const std::string& name, ValueList args);
```

Everything else keeps the cached builtin pointer, so the cost lands only on names a module actually claims. The carve-out is the interpreter's own: a **non**-exported module sub of a built-in's name stays module-private, so the importer still reaches the built-in while the module's own code sees its own.

That fix is a good example of the general hazard in compiling a dynamic language: **any decision made at compile time is a promise about the run-time environment,** and every such promise needs a reason to be true.

### 26.9.1 The other promise: every variable has a declaration

A variable compiles to a C++ local, which is only possible because Raku declares its variables — and `no strict` is precisely the pragma that says it need not. The generator emitted a reference to a local for the name anyway, so *any* program using the pragma failed in the C++ compiler:

```
nostrict.rakupp.gen.cpp:35:9: error: use of undeclared identifier 'v_szz'
```

Not a `CodegenError`, so not a fallback either — `--exe` simply could not build these programs. The reason to believe the promise now comes from the pass that already answers this exact question, `DeclCheck` (Chapter 39): `findLaxVars` returns the names a lexically lax region auto-vivifies, and those compile to `RT.laxVarRef("$x")` — a slot in the live environment — instead of a local.

What makes the set safe to act on is the pass's *source-text backstop*: a name survives into it only if the unit spells a declaration for it nowhere, and a name the generator never declares cannot collide with one it does. When the check could not finish (`EVAL` and friends stand it down) the set may be short, so the generator throws `CodegenError` and the program is bundled — the same answer as any other construct it cannot promise anything about.

## 26.10 Inspecting the output

```
rakupp --cpp prog.raku          # print the generated C++
rakupp --cpp -O prog.raku       # ...with the optimizer on
rakupp --exe -o prog prog.raku  # compile it
```

`--cpp` is the debugging tool for everything in this chapter and the next. When compiled and interpreted output differ, the first step is to read the emitted C++ for the construct that differs; it is ordinary C++, and the divergence is usually visible.

# Chapter 27

# The –O Optimizer

An optimizer for a language this dynamic has a narrower job than the name suggests. It is not looking for cleverness; it is looking for generality the program never asked for and is being charged for anyway. Chapter 14 measured a call and found the cost sitting in the boxed argument list rather than in dispatch, and the three passes here follow from that one finding.

The finding has since been acted on twice. This chapter is the first way: compile the list out of existence where the signature allows it. The second came later and from inside the interpreter — make the allocation itself nearly free, with a free list of small blocks (Chapter 12) — which narrowed the gap these passes close without changing what they do. Removing a cost is still worth more than cheapening it, and the passes below still measure what they always did against the `-O`-off baseline; the honest note is that the baseline moved.

`--exe` and its inspection twin `--cpp` accept `-O`. Everything under it is **semantics-preserving**: it changes how the compiled program computes, never what it computes. It is opt-in and off by default.

```
rakupp --exe    prog.raku -o prog     # no optimizer
rakupp --exe -O prog.raku -o prog     # optimizer on
rakupp --cpp -O prog.raku             # print the optimized C++
```

## 27.1 What the generated code looks like without it

By default the transpiler is faithful but generic: every value is a boxed `Value`, operators in value position go through `applyArith`, and every user-sub call packs its arguments into a `ValueList` — a heap allocation per call.

That last clause was true without qualification when these passes were written, and it is the sentence they were written against. It is now half true: the short lists a call actually builds come off a free list rather than the allocator (Chapter 12), so what remains is a list built and torn down per call without touching `malloc`. The passes below still remove it outright, which is still worth more.

```
// sub fib($n) { $n < 2 ?? $n !! fib($n-1) + fib($n-2) }
static Value u_fib(ValueList __a) {
    Value v_sn = rtPos(__a, 0);
    return rtLtB(v_sn, Value::integer(2LL))       // condition: always inline
         ? v_sn
         : applyArith("+",
             u_fib(ValueList{applyArith("-", v_sn, Value::integer(1LL))}),
             u_fib(ValueList{applyArith("-", v_sn, Value::integer(2LL))}));
}
```

For a hot recursive function that is two costs on every one of about 1.6 million calls: a `ValueList` allocation, and value-position `applyArith` calls that dispatch on the operator *string* before touching the operands.

## 27.2 Pass 1 — direct-arity calls

A sub whose signature is entirely **plain required positional scalars** — no named, slurpy, optional, defaulted or destructured parameters — gets a direct-`Value` overload, plus a boxed adapter so any unusual call site still resolves. C++ overload resolution picks the right one.

```
static Value u_fib(Value v_sn) { ... }                              // fast
static Value u_fib(ValueList __a) { return u_fib(rtPos(__a, 0)); }  // adapter
```

A call site takes the fast overload when it passes exactly the right number of plain positional arguments — no `:name(...)` pairs, no `|@slurp`. Multi subs, indirect calls through `&fib`, and method calls are untouched.

The same idea covers **37 named builtins**, each of which has a real C++ function (`rtBAbs`, `rtBUc`, `rtBSin`, ...) rather than a `std::function` in a map. Chapter 28 tells that story, because the reason it works is not the one that was first assumed.

## 27.3 Pass 2 — inline arithmetic

For the common operators the generator emits inline helpers instead of `applyArith`:

| ops | helper | fast path |
|---|---|---|
| `+ - *` | `rtAdd` / `rtSub` / `rtMul` | native `int64`, overflow to bignum |
| `**` | `rtPow` | integer power by squaring |
| `< <= > >= == !=` | `rtLt` ... | `int64` compare |
| `% %% div` | `rtMod` / `rtDivides` / `rtDiv` | floored `int64` |
| `~` | `rtConcat` | direct concat when both are `Str` |
| `eq ne lt gt le ge` | `rtEqS` ... | byte-wise when both are **plain** `Str` |

```
// src/Interpreter.h
inline Value rtAdd(const Value& l, const Value& r) {
    long long z;
    if (rtBothInt(l, r) && !rakupp::add_ovf(l.i, r.i, &z))
        return Value::integer(z);
    return applyArith("+", l, r);     // Rats, bignums, mixed types, coercions
}
```

`rtBothInt` tests that both operands are `VT::Int` and neither has grown a bignum. Overflow promotes to bignum exactly as `applyArith` does.

The string comparisons matter more than the integer ones, because they sat *late* in `applyArith`'s dispatch chain — about 118 nanoseconds against 10 for a direct call. The guard is that both operands are **plain** `Str`: no `Version`/`IO`/`Buf` tag and no enum identity. A tagged value falls back to the full chain, which preserves `Version` part-comparison, enum stringification, junction autothreading and `Whatever` currying.

With both passes plus pass 3's condition lane, `fib` becomes:

```
static Value u_fib(Value v_sn) {
    return ([&]() -> bool {                    // condition lane
        do { if (!(rtIntBox(v_sn))) break;     //   guard the box
```

```
            return (v_sn.i < 2LL); } while (0);   //   compare as raw int64
        return rtLtB(v_sn, Value::integer(2LL));   //   guard failed
    }())
         ? v_sn
         : rtAdd(u_fib(rtSub(v_sn, Value::integer(1LL))),
                 u_fib(rtSub(v_sn, Value::integer(2LL))));
}
```

No heap allocation, no string dispatch. Under a real C++ optimizer that collapses to tight native-integer recursion.

## 27.4 Pass 3 — guarded native-int lanes

Passes 1 and 2 still build a boxed `Value` for every intermediate result: `$sum += $_ * 2 - 1` constructs four of them per evaluation. Pass 3 removes them.

A straight-line integer expression whose leaves are **int literals and plain scalar variables** is computed in raw `int64`. Each leaf is tag-guarded at run time, each operation is overflow-checked, and the result is stored **into the target's existing box** with no `Value` construction at all:

```
// $sum += $_ * 2 - 1     (inside a native range loop)
{ bool __lok = false; do {
    if (!(rtIntBox(v__t0) && rtIntSlot(v_ssum))) break;
    long long __t1; if (rakupp::mul_ovf(v__t0.i, 2LL, &__t1)) break;
    long long __t2; if (rakupp::sub_ovf(__t1, 1LL, &__t2)) break;
    long long __t3; if (rakupp::add_ovf(v_ssum.i, __t2, &__t3)) break;
    v_ssum.i = __t3; __lok = true;
} while (0);
if (!__lok) { v_ssum = rtAdd(v_ssum,
                  rtSub(rtMul(v__t0, Value::integer(2LL)),
                        Value::integer(1LL))); } }
```

Any guard failure, overflow, or domain failure falls through to the untouched boxed emission. That is sound because **lane leaves are pure** — literals and variable reads — so re-evaluating them on the slow path has no side effects.

The lane applies to statement-position assignment to a plain scalar, statement- position `++`/`--`, and conditions. The store additionally requires `rtIntSlot`: not an enum-typed box, whose stringification is its name rather than its number.

Operations covered: `+ - *` overflow-checked, unary minus, floored `%` mirroring `rtMod` bit for bit, `%%`, and the six comparisons. Everything else — `Num`s, strings, `Rat`s, bignums, array elements, method calls — fails the lane at compile time or its guards at run time.

## 27.5 Three defaults that are not –O

Three changes look like optimisations and are shipped as defaults in every mode, because each fixes a **correctness wart** rather than adding speed.

**In-place `~=`.** `$s ~= …` naively rebuilds the whole string each step, so *n* appends do quadratic work. The interpreter and Rakudo both build strings in linear time, so this was a wart:

```
// src/Interpreter.h
inline void rtCatAssign(Value& l, const Value& r) {
    if (l.t == VT::Str && r.t == VT::Str) { l.s += r.s; return; }
    l = applyArith("~", l, r);
}
```

The interpreter pairs it with sink context, so a loop body's assignment does not copy its growing result either.

**`.sort($key)` extracts the key once per element.** `.sort` takes either a 2-ary comparator or a 1-ary key extractor, and the difference is asymptotic: a comparator is *supposed* to run per comparison, but a key extractor runs **once per element**. Raku++ used to call the 1-ary block inside the comparator:

```
// after — a Schwartzian transform: n calls, then compare the keys
ValueList keys(items.size());
for (size_t i = 0; i < items.size(); i++)
    keys[i] = callCallable(blk, {items[i]});
std::stable_sort(order.begin(), order.end(), [&](size_t x, size_t y) {
    return valueCmp(keys[x], keys[y]) < 0;
});
```

Sorting codepoints by the length of their Unicode name:

| N | before | after | Rakudo |
|---|---|---|---|
| 8 K | 5.70 s | 0.29 s | 0.34 s |
| 64 K | 18.09 s | 0.68 s | 0.63 s |
| 128 K | still running at 95 s | 1.33 s | 1.09 s |

The ratio grows with log $n$, which is the shape the change predicts.

**Never numify a string by throwing**, which is Chapter 11's `stod` story and was found while profiling the comparator case above.

## 27.6 Forwarding the C++ optimization level

--exe compiles the generated C++ at -O2 by default; a level on the flag is passed through:

| flag | codegen passes | C++ compile |
|---|---|---|
| *(none)* | off | -O2 |
| -O | on | -O2 |
| -O3 / -Os / -O0 | on | that level |

**The integer passes only pay off *with* C++ inlining.** At -O0 the `rt*` helpers become real calls with no fast path, so -O0 is *slower* than the default. It is for inspecting the generated C++, not for speed.

-O is a speed switch and not a size lever: the binary is dominated by the statically linked runtime, mostly the Unicode tables, not by the program's own code. -Os shrinks nothing that matters and costs 20 to 50% of the lane speed-up. Use -O for speed and `strip` if size matters.

## 27.7 Measured

--exe, best of six after a discarded warm-up, startup-inclusive:

| Benchmark | --exe | --exe -O | speed-up | what -O reached |
|---|---|---|---|---|
| fib | 166.5 ms | **47.0 ms** | 3.5× | direct-arity calls, condition lane |
| loopsum | 27.1 ms | **8.6 ms** | 3.2× | the += lane |
| streq | 46.5 ms | **17.5 ms** | 2.7× | int lanes atop inline `eq` |

| Benchmark | `--exe` | `--exe -O` | speed-up | what `-O` reached |
|---|---|---|---|---|
| arrayops | 102.0 ms | **77.8 ms** | 1.3× | map/grep body boxing |
| regex | 61.9 ms | 60.3 ms | 1.0× | the regex engine dominates |
| hash | 15.3 ms | 14.9 ms | 1.0× | hash slots, not scalars |
| sortnums | 49.9 ms | 48.6 ms | 1.0× | `.sort` dominates |
| bigint | 29.2 ms | 29.2 ms | 1.0× | `BigInt` multiply |
| strcat | 3.9 ms | 4.8 ms | 0.8× | ~= already linear by default |

The pattern is clear and worth stating: **where the time is inside a runtime method — the regex engine, bignum multiply, `.sort` — `-O` cannot reach it.** What it removes is boxing and allocation in the program's own code.

A dedicated showcase suite in `tools/optbench/`, each program written to lean on one pass, is compiled twice and checked byte-identical against the interpreter and Rakudo before being timed:

| benchmark | `--exe` | `--exe -O` | speed-up | rakudo |
|---|---|---|---|---|
| sieve | 1029.3 ms | **25.4 ms** | **40.6**× | 994.5 ms |
| powmod | 531.5 ms | **50.6 ms** | 10.5× | 716.8 ms |
| intsum | 283.1 ms | **35.9 ms** | 7.9× | 624.0 ms |
| fibcalls | 701.3 ms | **190.9 ms** | 3.7× | 1353.3 ms |
| stringbuild | 22.3 ms | 21.9 ms | 1.0× | 204.7 ms |

`sieve`'s whole inner loop — `while $d * $d <= $n`, `if $n %% $d`, `$d++` — runs as raw `int64` under pass 3, taking it from a tie with Rakudo at plain `--exe` to far ahead. `stringbuild` is flat because in-place ~= is default in both.

## 27.8 The box stays; only its per-operation cost goes

A common question: does a native-compiled `my int $x` become a bare C++ `long long`? **No.** The generated code keeps one uniform representation at every optimization level. `my int $s` compiles to `Value v_ss`.

- **Default ––exe:** `$s = $s + $i` is `rtAdd`, which constructs a fresh `Value` for the result.
- **–O pass 3:** the arithmetic runs as raw `int64` in registers and the result is written into the existing box's `.i` slot. No `Value` is constructed per operation — but the *storage* is unchanged, and loops still copy full `Values`.

Why keep the box? Because the transpile is uniform: any expression must be able to flow anywhere — into a runtime function, an array element, `say()`, an untyped assignment — and Raku's `my int` is readable in `Any` context while a non-native `Int` can hold `Nil` or promote to bignum. Emitting a bare `long long` local is only sound once an analysis proves the variable never escapes into a `Value` context, which is a genuine escape-analysis pass and the natural successor to pass 3.

## 27.9 Correctness

`–O` is validated to produce output byte-for-byte identical to the interpreter:

- every benchmark program matches with `–O` on;
- every deterministic example compiles with `––exe –O` and matches its golden output;
- the arithmetic fallbacks are checked directly — int64 overflow to bignum, `Int/Num` mixes, string coercion, `<=>` (left on the general path), and `+=` at the int64 boundary;
- the lane fallbacks likewise: `+=/++/*=` crossing int64, floored `%` with negative operands, `%= 0`, and comparisons whose intermediates overflow.

The design leans on the fallback. Anything the fast path does not recognise routes to the same runtime code the non-`–O` build uses, which is why "identical output" is a checkable claim rather than a hope.

## 27.10 What is next

- **value-position lanes** — laneable integer expressions inside larger expressions (call arguments, list elements) still box;

- **`Num` lanes** — the same trick for `double`, with tag guards and no overflow checks;
- **array-element lanes** — `@a[$i]` inside the lane, with a bounds and tag guard;
- **native int locals** — the big one, and a real escape-analysis pass: prove a typed local never escapes into a `Value` context and never leaves `Int`, then emit a raw `long long` with no box at all, eliminating both the storage and the per-iteration copies.

# Chapter 28

# Dispatch Cost in Compiled Code

A compiled Raku++ program reaches code four different ways, and they are not equally fast. This chapter measures each, describes the three cuts made to them, and — because it is the more useful part — records the analysis that turned out to be wrong.

Method: micro-benchmarks against the built runtime, 2 million iterations per shape, seven repetitions, first discarded, minimum reported.

## 28.1 The four shapes

For `sub square($n) { $n * $n }` plus `say square(5)`, the generated C++ contains all four:

| Raku | Generated C++ | Dispatch |
|---|---|---|
| `square(5)` | `u_square(ValueList{...})` | direct C++ call; inlinable |
| `say ...` | `rtCallB(RT, __bfp0, "say", ...)` | cached pointer |
| `$n * $n` | `applyArith("*", ...)`, or `rtMul` under `-O` | string-dispatch chain, or inline |
| `$x.method` | `RT.methodCall(inv, "m", ...)` | name-keyed `if`-ladder |

## 28.2 What dispatch itself costs

Trivial function body, identical `ValueList` construction in every variant, so the differences are pure dispatch:

| Shape | ns/call | |
|---|---|---|
| direct C++ call | 46.3 | the floor |
| cached `BuiltinFn*` call | 47.4 | **+1.1 — dispatch eliminated** |
| by-name: hash, `find`, `std::function` | 55.0 | +8.7 vs direct |
| `RT.callBuiltin("chr", ...)` end to end | 66.0 | the pre-change path |

Two readings. The by-name lookup tax is real but modest, and caching the pointer recovers essentially all of it.

**The more important reading is that the floor itself is high.** About 46 nanoseconds for a *trivial* call, nearly all of it the `ValueList` — at the time, a heap-allocating `std::vector` built per call. Dispatch was a quarter of the overhead; the argument list was the rest.

That floor has since moved twice: `ValueList` stopped being a `std::vector`, and its small blocks stopped reaching the allocator (Chapter 12), which took the one-argument shape from 32 nanoseconds to 9.5. The passes below still pay — they remove the list rather than making it cheap, and removing is worth more than cheapening — but the gap they close is narrower than this table implies.

Operators are a different story, because `applyArith` has integer and number fast paths at the top of its chain:

| Op shape | ns/call |
|---|---|
| `applyArith("+", int, int)` — early fast path | 7.8 |
| `rtAdd(int, int)` — the `-O` lane | 5.6 |
| `applyArith("~", str, str)` — **late in the chain** | **118.1** |
| `rtConcat(str, str)` — direct | 9.6 |

The late-chain operators pay for everything they walk past: the mixin-delegation check, negated-operator handling, set operations, junction autothreading, `Whatever` currying, the `Version` branch — about 110 nanoseconds of "is this something special?" before the actual string work.

## 28.3 Cut 1 — cached builtin pointers

Every `RT.callBuiltin("name", ...)` site now routes through a per-name pointer resolved once at program startup:

```
// src/Interpreter.h
const BuiltinFn* builtinPtr(const std::string& name) const;

inline Value rtCallB(Interpreter& I, const BuiltinFn* f, const char* name,
                     ValueList args) {
    if (f) return (*f)(I, args);
    return I.callBuiltin(name, std::move(args));   // by-name fallback
}
```

The generated call `rtCallB(RT, __bfp0, "say", ValueList{...})` **is** the cached call, not a lookup: `rtCallB` is a three-line `inline` shim whose hot path is one indirect call through the already-resolved pointer, and the `"say"` literal is touched only on the fallback branch — nothing is hashed or searched.

The shim also does a required mechanical job. `BuiltinFn` takes `ValueList&`, which a temporary cannot bind to; `rtCallB`'s by-value parameter materialises it into an lvalue.

This cut is **not –O-gated**: it is plumbing rather than speculation, so plain `--exe` gets it too. The fallback keeps semantics byte-identical for names that are not registered builtins.

One portability landmine, recorded because it cost an afternoon: the cached pointer names were originally `__bfN`, so the seventeenth builtin in a program emitted `__bf16` — which is a **reserved built-in type** (bfloat16) on arm64 clang. Hence the `__bfpN` spelling.

## 28.4 Cut 2 — inline string comparisons

`eq ne lt gt le ge` get the `rtEqS` family: two *plain* `Str`s compare byte-wise inline, which is exactly what `applyArith`'s tail does for them. Anything tagged falls back to the full chain.

Value-context forms sit in the `-O` table; **bool-context forms join the always-on condition table**, beside the existing integer `rtLtB` family.

End to end, 2 to 3 million iteration loops:

| Loop | Before | After | |
|---|---|---|---|
| builtin-heavy (`ord(chr(...))`), `--exe` | 407.8 ms | 383.5 ms | 1.06× |
| `$c++ if $a eq $b, --exe` | 712.0 ms | 129.2 ms | **5.5**× |
| `$c++ if $a eq $b, --exe -O` | 621.7 ms | 29.0 ms | **21**× |

The string-compare cut is the headline. An `eq` in a condition used to compile to a boolified `applyArith` call: build a `Bool Value`, walk the whole dispatch chain, then read the truthiness back out. It now compiles to an inline `l.s == r.s`.

## 28.5 Cut 3 — true named builtins, and the analysis that was wrong

An earlier revision of this analysis claimed that a *symbol* call like `b_say(...)` had a measured ceiling of about 1 nanosecond per call, and therefore was not worth building.

**That figure was correct only for renaming the same call shape** — an out-of-line call still taking a `ValueList`. What a real named function unlocks is different in kind:

- **direct `Value` arguments**, so no per-call `ValueList` allocation — which is the actual floor;
- **an inlinable hot path**, because a `std::function` is an opaque wall to the optimizer and an `inline` function is not.

Worse, some builtin lambdas hid extra cost. `abs`'s delegated to `methodCall` — the full method-name ladder on every call, about 370 nanoseconds per iteration in an `abs` loop.

```
// src/Interpreter.h
inline Value rtBAbs(Interpreter& I, const Value& v) {
    if (v.t == VT::Int && !v.big && I.builtinExt_.empty())
        return Value::integer(v.i < 0 ? -v.i : v.i);
    return rtBAbsSlow(I, v);
}
```

`abs`, `chr` and `ord` became real functions. The interpreter's map entries wrap them, **so the interpreter and the generic compiled path get the win too**, and `-O` emits direct calls when a single plain positional argument lines up.

Note the `builtinExt_.empty()` guard: the inline fast path switches itself off the moment a program `augments` a built-in type, so an augmented `.abs` still wins (Chapter 17).

| Loop | cached pointer | named function | |
|---|---|---|---|
| `abs`, `--exe -O` | 1112.9 ms | 198.3 ms | **5.6×** |
| `abs`, plain `--exe` | 1120.5 ms | 363.9 ms | 3.1× |
| `chr+ord`, `--exe -O` | 393.6 ms | 200.8 ms | 2.0× |

The recipe then swept the rest of the viable set — **36 names**: the numeric family, the string family, the twelve trigonometric and hyperbolic functions, and the I/O quartet.

The I/O quartet is worth its own sentence, because it disproved an assumption. A *buffered* one-argument `say` costs about 125 nanoseconds, so the call plumbing was roughly 45% of it: the `say` loop went from 124.9 to 69.6 milliseconds, with byte-identical output. "I/O dominates, so the call cost does not matter" was simply false at this scale.

Each named function is the old registered lambda's one-argument case **verbatim**. Delegators keep their `methodCall`, so `augment`, user objects and junctions are untouched; only `abs` and `sign` have bypassing fast paths, and both are guarded.

Additional measurements: a `sqrt+sin+floor` loop went 731.5 to 363.6 milliseconds under `-O`; `uc+chars` went 672.5 to 574.3 — a smaller win, because the real work in `mapCase` and `graphemeCount` dominates, as it should.

## 28.6 The interpreter got the same treatment

A follow-up the same day: `applyArith` gained a character-dispatched `Str/Str` fast path at the top of its chain, beside the existing integer and number ones, so the *interpreter* also skips the late-chain walk for plain-string comparisons and concatenation.

Interpreted `streq` went from 909.7 to 547.9 milliseconds. The remaining gap to Rakudo there is per-node tree-walk cost, which no operator fast path can remove.

## 28.7 What is deliberately not done

- **The `ValueList` per call** — the dominant cost of the calling convention, about 40 of the 46-nanosecond floor. `-O`'s direct-arity pass removes it for fixed-arity user subs, and cut 3 removes it for the named builtins, but the other ~165 builtins still take `ValueList&` by contract. A wholesale change — small-buffer or span arguments — is a runtime-wide refactor with interpreter implications.
- **`methodCall`'s `if`-ladder.** After the `MName` fix (Chapter 10) name comparison is 8.5% of the profile. A dispatch table would be chasing that 8.5%, and it is not a drop-in: the ladder is not a pure dispatch on the name.
- **The remaining late-chain operators** — `x`, `xx`, `gcd/lcm`, bitwise, `min/max`, `leg/cmp` — still walk the chain. Same recipe as `rtEqS` if any shows up hot; none of the current benchmarks exercise them.

## 28.8 The general lesson

Three cuts, and the third one contradicted the written analysis of the first two. What made the difference was not better reasoning — it was measuring the *right thing*: the earlier figure benchmarked a rename, not the change that a rename enables.

That is the recurring failure mode in performance work, and the defence is the same one used throughout this book: benchmark the full proposal, not the piece of it that is easy to isolate, and keep a control kernel that the change cannot touch.

# Chapter 29

# What a Compiled Binary Keeps

A `--exe` binary carries the whole runtime, and most of the runtime is Unicode tables. `say "Hello"` used to compile to 9.9 MB, of which the program's own code was a rounding error and the character-name table alone was 3.1 MB — data that program could not reach even in principle.

`--slim` is the flag that removes what a program cannot use. The interesting part is not the flag; it is what has to be true of the runtime before a linker can be *asked* to leave something out, and how the compiler decides what is safe to ask for.

## 29.1 The problem is the linker's, not the compiler's

A static archive is pulled in by object file. If any symbol in an object is referenced, the whole object comes along. So a table is droppable only when **nothing in the binary refers to it** — and the runtime referred to everything, because `Unicode.cpp` named every table directly.

That gives the shape of the work, and it came in stages: make the tables reachable only through a seam; split the runtime so each cuttable thing is its own archive; build a stub archive that stands in the gap; and only then write the analysis that decides which gaps to open.

## 29.2 The seam

```
// src/ucd_seam.h
namespace ucd {
struct NameEnt { const char* name; uint32_t cp; };
const NameEnt* namesTable(size_t*);
const int64_t* numvTable(size_t*);
// ...collation, script/block/bidi ranges...
}
```

Four cuttable table groups, each reached **only** through an accessor defined in the same translation unit as its data. That co-location is the whole mechanism: the accessor is the only symbol anyone outside references, so replacing the object replaces the data.

Two details in the header are worth lifting out.

**`Unicode.cpp` no longer declares the tables at all.** A new direct reference is therefore a compile error rather than a quiet hole in the seam. The seam is enforced by the language, not by a convention someone has to remember.

**The never-cut tables keep their direct references on purpose** — general category, binary properties, normalization, case mapping, grapheme break. Those are reached by ordinary string operations, so no program can prove it does not need them, and a seam there would be indirection with no stub ever standing behind it.

There is a performance constraint on the seam, and it is stated where the code is:

> Accessors return pointer and count **once**; callers hoist both into locals before any loop, so inner loops still index raw memory.

Collation touches its table once per collation *element*, so an accessor call per element would have been a measured regression. The gate for that stage was `perf-guard` flat on the collation paths.

## 29.3 Five archives

```
foreach(feat rakupp_ucd_names rakupp_ucd_coll rakupp_ucd_props
             rakupp_parse rakupp_stubs)
  add_library(${feat} STATIC ${${FEAT_VAR}})
endforeach()
```

The runtime became `librakupp_rt.a` plus four feature archives and one stub archive. Three hold Unicode data. The fourth, `rakupp_parse`, holds the **lexer and parser** —

because a compiled program that never calls `EVAL` does not need a Raku compiler inside it.

That last one creates a genuine cycle: `Runtime.cpp` drives the parser, and the parser builds AST nodes whose helpers live in the runtime. Rather than leave it to link order, the cycle is declared:

```
target_link_libraries(rakupp_rt    INTERFACE rakupp_parse)
target_link_libraries(rakupp_parse INTERFACE rakupp_rt)
```

which lets CMake repeat the archives for a single-pass linker instead of producing an unresolved symbol on some platforms and not others.

## 29.4 The stubs

```
// src/stubs/stub_ucd_names.cpp
const NameEnt* namesTable(size_t*) {
    featureMissing("unicode-names",
                   "uniname/uniparse (the Unicode name table)");
}
```

One object per feature, in `librakupp_stubs.a`, linked **in place of** the real archive. The linker pulls exactly the stubs whose archive is absent.

The stubs are deliberately not part of the runtime or of `librakupp`, and the comment says why: an interpreter carrying both the real table and a throwing double would be a one-definition-rule violation waiting for a link-order change.

What they throw is an ordinary typed Raku exception:

```
// src/FeatureGate.cpp
throw FeatureNotBuilt{{Value::typeObj("X::Feature::NotBuilt"),
    std::string(neededFor) + " needs the '" + feature +
    "' feature, and this binary was compiled without it (--slim=-" +
    feature + "). …"}};
```

Catchable with `when X::Feature::NotBuilt`, naming the feature, the operation that wanted it, and the flag that would put it back. **A cut feature is a build decision the program ran into, not an internal error** — so it is reported the way a program can handle rather than as a crash.

`featureMissing` lives in the runtime rather than in a stub, so both sides of the seam can reach it.

## 29.5 The levels

Four, at most one per spec:

| level | what it does |
|---|---|
| `none` | nothing: no dead-strip, symbols kept. For debugging a binary. |
| `safe` | **the default with no flag.** Dead-strip and symbol strip. No feature removed, no analysis run. |
| `auto` | **what bare `--slim` means.** `safe`, plus every feature the scan *proves* unreachable. Sound. |
| `max` | `auto`, ignoring the force-full triggers. Unsound by design. |

`safe` being the default matters more than it sounds: it costs nothing analytically and takes `say "Hello"` from 9.9 MB to 8.1 MB, byte-identical in behaviour. Its only real cost is that a C++-level crash reports addresses rather than names, which is why `+symbols` exists.

A spec naming no level means `auto`, so `--slim=+eval` reads as "automatic pruning, but keep eval".

## 29.6 The scan

```
// src/SlimScan.h
struct SlimScanResult {
    bool used[4];                       // per feature
    std::vector<std::string> triggers;  // why everything was kept
    struct Site { int feat; std::string what; std::string where; int line; };
    std::vector<Site> sites;            // every use, in walk order
};
SlimScanResult slimScan(const Program& prog,
                        const std::vector<BundledModule>& mods);
```

It walks the parsed program **and every module embedded alongside it**, asking per feature: can any site reach this? The governing rule is one sentence in the header:

> The default answer to uncertainty is always "keep".

The detectors are more specific than "does the name appear". `uniprop('Script')` marks `unicode-props`; `uniprop('Name')` marks `unicode-names` instead, because Name and Numeric_Value live in the names tables; a computed property name marks both. A bare `&uniprop` reference marks both, because the argument is unknown. `<:Script<...>>` inside a regex marks props; `\c[...]` marks names, because the engine resolves that name at match time.

The `eval` feature has the widest surface, and most of it is not spelled `EVAL`: `require`, `use-ok`, `eval-dies-ok`, a regex code block, `:my` in a regex, an `s///` replacement that interpolates, `throws-like` with code in a string. Each of those reaches `evalString` at run time, so each keeps the parser.

## 29.7 The triggers

Some constructs mean the program can run code the scan never saw. Under `auto` any of them keeps **everything**:

```
EVAL / EVALFILE / require
a symbolic reference (::($name))
an indirect method call (."$name"())
a metamodel lookup (.^lookup)
a regex interpolating a subregex (<$var> / <{…}>)
a `use`d module that could not be embedded alongside the program
an AST node the scan does not model
```

That last one is the important one, and it is what makes the analysis maintainable: **a node kind the walker does not recognise is itself a trigger.** Adding a construct to the language cannot silently create a hole in the scan; the worst it can do is make `auto` conservative until someone teaches the walker about it.

Literal regex code blocks are **not** triggers. Their source is visible, so the scan parses and walks them like any other code.

And the triggers are reported by name and line, rather than merely suppressing the cut:

```
$ rakupp --exe trig.raku --slim=list
--slim=auto for trig.raku (--exe):
  unicode-names     keep   3.1 MB  kept: a dynamic construct keeps everything
  …
dynamic constructs keeping everything (--slim=max cuts anyway):
  an indirect method call (."$name"()) (in the program, line 2)
would cut 0 KB of 5.0 MB cuttable (archive sizes)
```

## 29.8 What it is worth

```
$ rakupp --exe hello.raku --slim=list
--slim=auto for hello.raku (--exe):
  unicode-names      cut     3.1 MB   proven unused
  unicode-collation  cut     743 KB   proven unused
  unicode-props      cut     141 KB   proven unused
  eval               cut     971 KB   proven unused
would cut 5.0 MB of 5.0 MB cuttable (archive sizes)
```

`say "Hello"`: 9.9 MB unstripped, 8.1 MB at `safe`, **4.6 MB** under bare `--slim`. A program that uses one feature keeps one:

```
$ rakupp --exe uses-names.raku --slim=list
  unicode-names      keep    3.1 MB   used: uniname (line 1)
  unicode-collation  cut     743 KB   proven unused
  unicode-props     keep    141 KB   used: <:Script<…>> in a regex (line 2)
  eval               cut     971 KB   proven unused
```

## 29.9 The directives, and the one that proves it

Four, one per spec:

| directive | |
|---|---|
| `help` | the grammar and the feature table, with the real archive sizes beside *this* rakupp |
| `list` | keep/cut per feature for this program, with the reason and the bytes. Analyses only; does not compile |
| `why:FEAT` | every site — program or module, with the line — that forces FEAT to be kept |
| `verify` | build the slim binary **and** a full reference, run both, and emit the slim one only if stdout, stderr and exit status agree |

`verify` is the interesting one, because it answers the objection the whole feature invites: how do you know the analysis was right? By not trusting it — build both, run both, compare. A nondeterministic program cannot agree with anything, so `verify` refuses that too rather than reporting a false difference.

It composes with an explicit cut, which is exactly when you want the proof: `--slim=safe,-eval,verify` is "I claim this program never compiles code at run time; prove it".

`help` deserves a note of its own. It prints the real archive sizes measured from the rakupp you are running, not numbers baked into a string — so the documentation cannot drift from the build.

## 29.10 Where it declines

Two modes refuse the scan, loudly rather than quietly.

**`--bundle`** embeds source and parses it at run time, so nothing can be proven unused — and `--slim=-eval` is refused there outright, because bundling *is* the eval feature. **`--aot`** keeps every feature until the scan is wired for it. Explicit `±feature` still applies in both.

`--slim` shapes the link, so it applies to the compile modes only. The interpreter never slims.

## 29.11 The manifest

Every compiled binary embeds a one-line build manifest — version, mode, slim level, cut list — which `rakupp --exe-info BIN` prints.

Keeping a string alive in a binary that was built to drop unreferenced data is its own small problem, and the solution is in `FeatureGate.cpp`:

```
static const char* g_exeManifest = nullptr;
void rakuppKeepManifest(const char* m) { g_exeManifest = m; }
```

A pre-main initializer parks the pointer there. The call is what anchors it: **an escaped address into another translation unit is beyond any optimizer's reach.** And because the reader scans bytes rather than symbol tables, the manifest survives symbol stripping — `strings BIN | grep RAKUPP-EXE` finds it too.

## 29.12 The shape of the argument

`--slim` is a static analysis whose failure mode was designed before the analysis was. Every layer has an answer to being wrong:

- an unmodelled construct is a **trigger**, so the scan fails towards keeping;
- a wrong cut throws a **typed, catchable exception** at the point of use, never a crash and never a wrong answer;

- `verify` will **build both and compare** rather than ask to be believed;
- and `max`, the one unsound level, says so in its own help text.

That ordering is the point. An optimisation that removes code from a program is only as good as its worst case, and the worst case here was decided first.

# Chapter 30

# AST Serialization and the Precompiled Parse

Parsing is not free, and a program that pulls in a dozen modules pays for all twelve on every run before it does any work of its own. Every implementation of every language eventually answers that with a precompilation format. This one has both an easier version of the problem and a harder one.

Raku++ executes by walking a tree, so **the tree already is the compiled form**; there is nothing further to compile it into. That makes caching straightforward in one sense — store the tree — and delicate in another, because a stored tree must be indistinguishable from a freshly parsed one.

## 30.1 The format

```
// src/AstSerial.h
inline constexpr uint32_t kAstSerialVersion = 17;
struct AstSerialError { std::string msg; };
std::string serializeAst(const Program& prog);
void deserializeAst(const std::string& blob, Program& out);
```

A self-describing byte string with a header. The version constant is bumped whenever the encoding or the AST changes shape, and an entry carrying a different version is **ignored, never reinterpreted**.

Two properties of the design are load-bearing.

**Both directions are driven by one visitor per node type.** A field cannot be written but not read, which is the failure mode that makes a format like this quietly wrong rather than loudly broken. Adding a field to `SubDecl` and forgetting the reader is not possible; the single visitor handles both.

**The mutable per-node caches are deliberately not stored.**

```
Binary::simpleOp, specialArm, fastShape, litVal
Unary::evalPath
Index::fastShape, litIdx
NumLit::cacheN, cacheD
Block::hoistNeed
StrLit::nfcDone
```

Each has an "undecided" sentinel and is rebuilt lazily on first evaluation (Chapter 19), so a loaded tree behaves identically to a freshly parsed one — it just has not warmed up yet. Storing them would be both larger and more fragile, since `litVal` is a pointer.

Everything else round-trips, **including `line` and `label`**, because diagnostics depend on them.

## 30.2 Two independent switches

```
rakupp --precomp-modules=on    # cache the parse of `use`d modules
rakupp --precomp-files=on      # cache the main program's own parse
rakupp --precomp-info          # what is on, where it lives, what it holds
rakupp --precomp-clean         # empty it (always safe)
```

Both are **off by default** — nothing is written to disk until asked — and they are separate because they are worth very different amounts.

**Modules.** A dependency tree is a lot of source and none of it changes between runs:

| | no cache | cached |
|---|---|---|
| `use XML` (10 files, 1,110 lines) | 16.0 ms | **5.7 ms** |

**Files.** A script's own parse is already sub-millisecond, and the few- millisecond floor of a small program is process startup, not parsing:

| bare file | no cache | cached | saved |
|---|---|---|---|
| 50 lines | 2.8 ms | 2.4 ms | 0.4 ms |
| 1,000 lines | 10.0 ms | 5.1 ms | 4.9 ms |
| 20,000 lines | 158.8 ms | 66.5 ms | **92.3 ms** |

Across the twenty-two fastest example programs, turning `files` on made **no measurable difference at all**. There is also a cost on the run that *writes* an entry — about 22 milliseconds at 20,000 lines — so for a script run once, `files` caching is a small net loss.

If you enable one, enable **modules**. That asymmetry is precisely why these are two switches rather than one.

## 30.3 What invalidates an entry

This is the interesting part, and it is where the naive design would be wrong.

```
// src/Interpreter.h
bool precompLoadProgram(const std::string& srcPath, const std::string& src,
                        const std::vector<std::string>& searchPath,
                        Program& out, std::string& finishOut);
void precompStoreProgram(const std::string& srcPath, const std::string& src,
                         const std::vector<std::string>& searchPath,
                         const Program& prog, const std::string& finish,
                         const std::vector<std::pair<std::string,
                                                     std::string>>& deps);
```

An entry is valid only while **four** things still hold.

**The source has not changed.** Content-validated, not timestamp-validated.

**The rakupp binary is the same one.** A tree from another build may not match this build's node shapes.

**The search path is the same.** This is the one people do not expect, and it is in the signature for a reason: the search path decides which *file* a `use` resolves to for operator scanning, so the same source under a different `-I` can legitimately parse differently.

**The operators of everything it `use`s are unchanged.** This is the deepest one. A module that gains or loses an operator declaration changes how *this* file parses,

without this file changing at all (Chapter 6). So the parser records every source it scanned:

```
// src/Parser.h
std::vector<std::pair<std::string, std::string>> opScanned_;
```

and those pairs go into the entry as dependencies. It is the same list the parser needed anyway; making it a cache key was nearly free.

## 30.4 Where it lives, and what it reports

Settings persist in a small config file under the user's config directory; `RAKUPP_PRECOMP_MODULES` and `RAKUPP_PRECOMP_FILES` override for one invocation, and `RAKUPP_NO_PRECOMP=1` forces both off.

```
// src/Interpreter.h
struct PrecompEntry {
    std::string source; unsigned long long bytes; bool usable; bool orphan;
};
std::vector<PrecompEntry> precompCacheList();
std::pair<size_t, unsigned long long> precompCacheClear();
```

The `orphan` flag distinguishes the one cache state that is *pure garbage* from the ones that are merely misses waiting to happen: the source file is gone, so the entry can never be reused **or** rewritten. Everything else is derived data, which is why clearing the cache is always safe.

## 30.5 What is *not* cached

Only the parse. A module's top-level declarations and its `BEGIN` blocks still execute on every run, which is roughly the 30% of module load time that is not parsing.

Rakudo goes further: its `.precomp` serialises the *objects* that compile-time code produced, so `BEGIN` runs once per compile rather than once per run. Matching that would mean serialising live runtime state — class registries, closures, whatever a `BEGIN` block built — which is a different and much larger project.

Compared with the neighbours:

| | what it stores | where |
|---|---|---|
| Raku++ | the AST | a central content-validated directory |
| Rakudo `.precomp` | bytecode plus compile-time objects | beside the source |
| Python `__pycache__` | bytecode | beside the source |

Storing entries centrally rather than beside the source is what makes caching the **main program** defensible at all. Python never caches `__main__`, and the usual objection is that it would litter the source tree; that objection does not apply here.

## 30.6 The other consumer: embedded modules

The same serialisation carries modules inside a compiled binary (Chapter 25):

```
// src/Interpreter.h
struct BundledModule { std::string name, blob, finish, src; };
void rakuppRegisterModule(const std::string& name, const char* blob,
                          size_t blobLen, const std::string& finish);
```

`collectModuleGraph` resolves the transitive `use` graph at build time and serialises each module, dependencies first. `loadModule` consults the embedded table before the disk.

That the cache format and the embedding format are the *same* format is not a coincidence — it is why a bug in either is found twice as fast.

## 30.7 The AOT emitter, and what it adds

`--aot` is the other consumer of this format, and it is worth being clear that it *is* this format:

```
// src/Ast.h
struct AstEmitError { std::string msg; };
void emitAstProgram(const Program& prog, std::ostream& out,
                    const std::string& fileName, const std::string& finish,
                    const std::vector<BundledModule>& mods);
```

What `emitAstProgram` adds is the wrapper: it calls `serializeAst`, writes the blob out as a `kAst` byte array, and generates a `main` that calls `deserializeAst` at startup. One

encoding, two consumers — the cache reads it from disk and the binary carries it inside itself.

It used to be genuinely different: C++ source with one builder function per node, no reader needed because the compiler was the reader. That is the version this section used to contrast with, and it was replaced because listing each node's fields by hand meant a field nobody listed silently did not survive. Chapter 25 tells that story. The lesson it leaves here is the argument for the visitor: a writer and a reader that are the same source line cannot disagree about what a node contains, and two hand-written lists always eventually can.

Both fail *soft*. An unserialisable module is left out of the binary and loaded from disk; a tree the format cannot carry makes `--aot` fall back to bundling. Neither produces a wrong tree.

That shared property is the design rule worth taking away: **a derived-data mechanism should degrade to recomputation, never to a guess.** Every failure mode in this chapter — a version mismatch, a changed dependency, a missing source, an unhandled node — ends in "parse it again", which is slower and correct.

# Chapter 31

# The JavaScript Back End

Chapter 26 stated the principle the native generator is built on, and everything in it followed from that one sentence: **the generator never reimplements the runtime.** It emits C++ that calls the same functions the interpreter calls, which is why `--exe` and the interpreter produce byte-identical output, and why a feature implemented once works in both.

`--target=js` gives that up. It has to. There is no `librakupp_rt.a` in a browser, and no `Value`, and no `applyArith` — so a JavaScript program cannot call the runtime the other back ends share. Something has to play its part, and that something is a second runtime, written by hand, in JavaScript.

This chapter is about what happens to a code generator when the principle holding it together is unavailable, and what it costs to keep two runtimes honest instead of one.

## 31.1 The interface

One function, and the same refusal type `main.cpp` already catches:

```
// src/codegen/Js.h
std::string transpileToJs(Program& prog, const JsOptions& opt,
                          std::string* dts = nullptr, std::string* map = nullptr);
std::string jsRuntimeSource();     // the runtime, baked into the binary
std::string jsWasmWrapper(const std::string& src, const JsOptions& opt);
```

`transpileToJs` throws `CodegenError` — the same struct `Codegen.h` declares — so the CLI reports a JavaScript refusal in exactly the shape it reports a C++ one. That is a small thing and it is the reason the second back end did not need a second error path through `main.cpp`.

## 31.2 The same program, a third way

Chapter 25 took `demo.raku` through four back ends. Here is the fifth:

```
# demo.raku
sub square($n) { $n * $n }
my $total = 0;
for 1..5 { $total += square($_) }
say $total;
```

```
// Generated by `rakupp --target=js` — rakupp 3.25.0, source f49826b83f947e47
// RAKUPP-EXE-MANIFEST {"rakupp":"3.25.0","mode":"js","source":"f49826b83f947e47"}
import R from "./rakupp-rt.js";
R.main(() => {
    let v___0 = R.Any;
    function u_square(v_n) {
        let v___2_1 = R.Any;
        if (arguments.length !== 1) R.arityError("square", 1, arguments.length);
        if (v_n === R.Mu) R.notAny("$n");
        v_n = R.item(v_n);
        return R.mul(v_n, v_n);
    }
    let v_total = 0;
    {
        let v___2_2;
        L3: for (v___2_2 of R.rangeIter(1, 5, false, false)) {
            {
                (v_total = R.add(v_total, (u_square(v___2_2))));
            }
        }
    }
    R.sink(R.say(R.item(v_total)));
}, { mainExit: true });
//# sourceMappingURL=demo.js.map
```

The shape is deliberate and it is the same bargain the C++ back end makes: the program's subs are hoisted function declarations, its `my` variables are `let`s, its classes are `R.defClass` calls, so a reader recognises their own program. Every *value* opera-

tion is a call into `R`. Control flow is JavaScript's own wherever it is lexically local — that `L3:` is a real labelled loop, and `last` compiles to `break L3` — and only where control has to cross a closure does the emitter fall back to throwing a control object.

Three lines are worth naming because they are what a signature costs: the arity check, the `Mu` check, and the itemisation. They are the price of not reimplementing the dispatcher, and they are also why a transpiled sub recurses about 8,900 deep where a plain JavaScript function manages 10,400 — the guards take stack frames too.

## 31.3 The runtime that had to be written

`src/js-rt/` is thirteen fragments and 5,070 hand-written lines, concatenated in name order:

| | |
|---|---|
| `05-gb-tables.js` | grapheme-break data, so `.chars` counts clusters in a browser |
| `10-core.js` | the value model, `R.add` and the rest of the arithmetic |
| `20-str.js`, `30-list.js`, `40-objects.js` | the three container families |
| `50-builtins.js`, `60-methods.js` | what the emitter is allowed to call |
| `70-host.js`, `80-glue.js` | stdout, exit status, the host boundary |
| `85-js.js` | the `use JS` interop surface |
| `86-async.js`, `87-supply.js` | promises and supplies over the event loop |
| `90-regex.js` | the regex engine, again |

`tools/js/gen-rt-src.raku` bakes that into `src/JsRuntimeSrc.cpp`, so a binary writes the exact runtime it was built with rather than whatever happens to be on disk — the same scheme the grammar shim uses. It goes in as an *array* of raw-string chunks under 15 KB each, because MSVC caps a single string literal at 16 KB and a 200 KB literal does not compile there at all.

That table is the honest cost of the chapter's opening. `90-regex.js` is a second regex engine. `05-gb-tables.js` is a second copy of the grapheme data. Every one of those files is a place where the two implementations can drift, and the reason the next section exists.

## 31.4 Three outputs, and a second tier

```
rakupp --target=js prog.raku -o prog.js          # program + rakupp-rt.js beside it
rakupp --target=js --standalone prog.raku -o p.js # one file, runtime inlined
rakupp --target=js --module lib.raku -o lib.js    # an ES module, plus lib.d.ts
rakupp --target=js --runtime -o rakupp-rt.js      # the runtime alone
```

The sidecar form is 792 bytes of program against a 374 KB runtime; `--standalone` is that same runtime inlined, so the file is 374 KB whatever the program. For a web page that is one cached asset and many small programs, which is why the sidecar is the default.

Anything outside the core is refused with the line and a reason:

```
$ rakupp --target=js prog.raku -o prog.js
note: concurrency/process types (Proc::Async) — P4 of the plan — outside the
    JavaScript core; --fallback=wasm runs it on the WebAssembly engine instead
exit 5, no file written
```

That last clause is the second tier. `--fallback=wasm` emits a 2 KB wrapper that loads the *interpreter* compiled to WebAssembly and runs the embedded source through it — `--exe-info` reads back `"mode":"js-wasm"`. So there are two ways to reach a browser and they are complements rather than rivals: the transpiler is fast and refuses things, the WebAssembly engine accepts everything and carries the whole interpreter. Chapter 31 is about building the second one.

## 31.5 Keeping two runtimes honest

With one runtime, agreement is structural: `--exe` calls `applyArith` and so does the interpreter, so they cannot disagree. Here it has to be *measured*, and `t/js/run.raku` is what measures it.

Every program in `t/regression/` and `examples/` is transpiled with `--standalone`, run under Node, and compared with the same binary interpreting the same file — stdout, stderr and exit status, byte for byte. **The interpreter is the oracle, not a golden file**, which is the important choice: a golden records what the output was, and an oracle records what it should be. The report has three numbers — in core and agreeing, refused with a histogram of reasons, and disagreeing, which must be zero. The refusal histogram is the work queue.

The gate also checks two things that are not about any program: that the generated `src/JsRuntimeSrc.cpp` is current with `src/js-rt/`, and that every builtin the emitter

is willing to call exists in the runtime. Four interop goldens under `t/js/interop/` cover `use JS`, where the interpreter cannot be the oracle because there is no DOM on this side — they run against a stand-in and carry their own expected output.

## 31.6 Where it diverges

Six differences survive that discipline, and they are all the same shape: places where JavaScript's own semantics are visible through the runtime.

| | interpreter | Node |
|---|---|---|
| `printf("%.2f", 0.125)` | `0.12` | `0.13` — JavaScript's tie rounding |
| `(0.1e0).Rat` | `0.1` | `0.100000000000000006` |
| `(2e60).Int` | the exact integer | `2e+60` — past $2^{53}$ it stays a float |
| `"a\r\nb".chars` | 3 | 4 — CRLF on the ASCII fast path |
| `lt`, `leg`, `cmp`, `sort`, hash `.gist` | codepoint order | UTF-16 code-unit order |
| `say` of an unhandled `Failure` | throws, exit 1 | prints `(HANDLED) ...` and continues |

The string ordering one is the least obvious and the most likely to bite: a character above the BMP is a surrogate pair in JavaScript, so ff sorts before 𝕏 here and after it there.

Four claims about the core are also narrower than they look. Multi dispatch ranks by arity, type, literal and `where` — but not by sigil, so an untyped `@` candidate does not beat an untyped `$` one. A scalar's type constraint is not enforced at all: `my Int $x = 'no'` is accepted. Assigning `Nil` leaves `Nil` rather than the declared type's default. And of the match adverbs, `:nth(n)` is off by one and `:x(n)` is ignored.

## 31.7 Speed

Node starts in about 20 ms against the interpreter's 2, so a small program is slower under JavaScript and a program that computes is faster. On the benchmark machine, wall clock including start-up:

| kernel | interpreter | `--target=js` under Node |
|---|---|---|
| fib(29) | 298 ms | 82 ms |
| streq (1M `eq`) | 233 ms | 69 ms |
| loopsum (1M) | 86 ms | 36 ms |

Those are honest numbers and they flatter the wrong thing. What the JIT is beating is the tree-walker, not the native back end — `--exe` on the same kernels is faster than both. The reason to reach for `--target=js` is that the program has to run somewhere there is no binary.

## 31.8 Honest limitations

- **Two runtimes drift, and only a gate says so.** Every fragment in `src/js-rt/` duplicates something the C++ runtime already does. The corpus gate is the only thing standing between that and a silent disagreement, and it can only compare the programs it is given.
- **The refusal list is the plan's work queue, not a boundary.** Concurrency, process types, `nqp::` ops, `EVAL` and NativeCall are refused with a message naming the phase that would add them; `--fallback=wasm` is the answer for a program that needs one today.
- **The divergences above are not bugs the gate can catch.** Each is a case where the honest translation of a Raku rule into JavaScript is the divergence, and closing them means writing more runtime — which is the cost this back end exists to trade away.

# Chapter 32

# Raku in the Browser

The four modes in Chapter 25 all target a machine with a filesystem, threads and a dynamic loader. There is a fifth target that has none of those: the same runtime compiled to **WebAssembly**, which is Raku.js — the interpreter running in a browser tab, with no server.

It is not a fifth back end. It is mode 1, the tree walk, with a different host. What makes it worth a chapter is that the host removes almost everything the interpreter assumes, and the adaptation is **94 lines that live entirely outside `src/` and change nothing in the interpreter**.

## 32.1 The whole boundary

```
// rakujs/rakupp_web.cpp — the three exported functions
EMSCRIPTEN_KEEPALIVE int         rakupp_run(const char* src,
                                            const char* stdin_text);
EMSCRIPTEN_KEEPALIVE const char* rakupp_highlight(const char* src);
EMSCRIPTEN_KEEPALIVE const char* rakupp_version();
```

`rakupp_run` calls `rakupp::rakuppRun` — the same public entry point the native CLI uses — which lexes, parses, builds an `Interpreter`, runs it, and catches `ParseError`, `RakuError` and `std::exception`, reporting them to stderr exactly as the CLI does. The exit code that comes back is the process exit code the CLI would have produced.

The other two exports matter more than they look: `rakupp_highlight` is the tokenizer behind `rakupp --highlight` (Chapter 39), and it is what lets a browser editor

paint Raku with the compiler's own knowledge rather than a JavaScript approximation. That thread is picked up at the end of this chapter.

## 32.2 Input is one `rdbuf` swap

The interpreter reads standard input exclusively through `std::cin`. A browser has no standard input, and Emscripten's TTY device brings its own buffering and EOF state. So feeding a program is a stream-buffer swap and nothing else:

```
// rakujs/rakupp_web.cpp
static std::istringstream web_stdin;
web_stdin.clear();
web_stdin.str(stdin_text ? stdin_text : "");
std::streambuf* old_in = std::cin.rdbuf(web_stdin.rdbuf());

int rc = rakupp::rakuppRun(src ? src : "", {}, "web", "rakupp.wasm", {});

std::cin.rdbuf(old_in);
```

`get`, `lines`, `prompt` and `$*IN.slurp` read that text and then see end-of-file, so **nothing ever blocks** — which matters, because a blocking read in a WebAssembly module with no thread to park is a hung tab.

Two details make it reliable across repeated runs. `rdbuf()` clears `cin`'s eof and fail bits as a side effect, so a program that read to EOF does not poison the next run. And a caller built against the older one-argument signature passes no second argument, which arrives as null and means an empty stdin rather than undefined behaviour.

## 32.3 Output, and the flush that is not optional

Output needs no interpreter change at all: `std::cout` and `std::cerr` are routed by Emscripten to the `print` and `printErr` callbacks the host page installs. What needed care is the *end* of a run.

```
// rakujs/rakupp_web.cpp — after the program returns
std::cout.flush();  std::cerr.flush();
std::fflush(stdout); std::fflush(stderr);
fsync(STDOUT_FILENO);
fsync(STDERR_FILENO);
```

Emscripten's TTY only emits a line to `print` **on a newline**. A program ending in `print` rather than `say` therefore left its final, newline-less line sitting in the device

buffer — and that line then appeared on the *first line of the next run*, attributed to a program that never wrote it.

`fflush` does not clear it. `fsync` does, because that is what triggers the TTY's flush operation. Those two lines are load-bearing, and the bug they fix is the kind that only shows up when someone runs two programs in a row.

## 32.4 The stack is chosen, not inherited

Every native entry point runs the program on a thread with a very large stack (Chapter 13), because the tree walker's recursion depth is the Raku recursion budget. A single-threaded WebAssembly build cannot spawn that thread.

So the web entry point deliberately calls `rakuppRun` rather than `rakuppRunBigStack`, and reserves a large *WebAssembly* stack at link time instead. The interpreter's own recursion guard measures the real stack and throws `X::Recursion` before it overflows, so this is safe rather than merely lucky.

The honest consequence is a much lower ceiling, and the browser sets it rather than the build. A plain recursive sub reaches **58 levels in a Chrome Web Worker and 115 on Chrome's main thread; in Safari, 32 and 419**. The next section is most of the reason in Chrome; the worker section below is the reason in Safari.

## 32.5 Exceptions: `-fexceptions`, not `-fwasm-exceptions`

This is the most interesting engineering decision in the WebAssembly build, and it connects directly to Chapter 15.

The interpreter leans on C++ exceptions for both errors **and control flow**: `last`, `next`, `redo`, `when`, `succeed`, `return` across a boundary. Native WebAssembly exception handling would be faster and would give deeper recursion — but with the toolchain in use its personality routine fails to match the interpreter's by-value control-exception catches. A `catch (LastEx&)` does not catch, so `last`, `next`, `given` and `when` escape to `std::terminate` and the module traps with `RuntimeError: unreachable`. That was verified, not assumed.

So the build ships `-fexceptions`: JavaScript-based exception handling, which handles them correctly. The cost is recursion depth — under `-fexceptions` every throwing call is routed through a JavaScript trampoline and consumes the

**JavaScript engine's** stack, which a page cannot grow. In V8 those trampolines are most of what a Raku level costs.

`-sSTACK_SIZE` does not help, and that was verified too. So was something less obvious: `-Oz` beats `-O2` here on *both* axes — smaller **and** deeper, 206 levels against 174 — because a smaller compiled body means fewer frames per Raku level.

This is also the sharpest illustration of why Chapter 15's cooperative control flow exists. On this host, a C++ throw is not merely slow; it is the thing that bounds how deep a Raku program may recurse. Every `return` and `last` the interpreter resolves with a flag instead of a throw is one that does not touch the JavaScript stack at all.

## 32.6 Off the main thread

`rakupp_run` is **synchronous**: it runs a whole program to completion. On the main thread that freezes the page — no spinner animates, and output appears only at the end, if at all.

So the playground runs it in a dedicated worker:

```
// rakujs/playground/worker.js
importScripts('rakujs.js' + V);          // the MODULARIZE factory

function makeModule() {
  return RakuJS({
    locateFile: p => p + V,
    print:    t => { if (inRun) post('out', { text: t + '\n', cls: ''    }); },
    printErr: t => { if (inRun) post('out', { text: t + '\n', cls: 'err' }); },
  }).then(m => { Module = m; return m; });
}
```

Three things follow from the worker that could not be had without it.

**Output streams.** `print` posts to the main thread as it fires, so a program that redraws animates frame by frame instead of arriving as one block at the end. How that actually reaches the page is the next section.

**A runaway program can be stopped**, by terminating the worker. There is no other way to interrupt a synchronous call.

**A failure is recoverable.** A stack overflow from deep recursion, or an `exit` in user code, leaves the module instance in an unknown state, so the worker throws it away and builds a clean one for the next run. The overflow has a name in every browser

— a `RangeError` in Chrome and Safari, an `InternalError: too much recursion` in Firefox — and the worker tells the page which it was:

```
// rakujs/playground/worker.js
catch (err) {
  Module = null;
  ready = makeModule();
  post('runerror', { message: String(err),
                     deep: /call stack|too much recursion/i.test(String(err)) });
}
```

**A stack overflow gets a second try, on the main thread.** A worker's stack is a fraction of the page's, and in Safari the fraction decides whether a program runs at all. WebKit gives a worker thread a sixteenth of its main thread's stack, and JavaScriptCore's baseline WebAssembly tier spends about a kilobyte on every frame, whatever the function: it reserves a 16-byte spill slot for every register it may allocate. A grammar parse descends a handful of frames for every rule, so the language showcases, which parse their program with a Raku grammar, overflow a Safari worker before they print a line. The page therefore runs an overflowed program again, from the start, on its own main thread, on an engine instance of its own. It is frozen for that run — no spinner, no streaming, no Stop — which is why only an explicit run that has already overflowed the worker takes that path. Only when the main thread overflows too does the page say the recursion is too deep.

The `inRun` flag is a small thing worth noticing: outside a run, `print` is Emscripten's own load-time diagnostics, which belong in the devtools console rather than in the user's output pane.

## 32.7 How a line of output reaches the page

There is a message protocol, a screen *model*, and a coalesced render. All three matter, and none of them is the obvious "append text to a `<div>`".

### 32.7.1 The protocol

The worker posts one of five message types, and the main thread dispatches them to whichever block is currently running:

```
// raku.js
worker.onmessage = function (e) {
  var m = e.data, b = current;
```

```
  switch (m.type) {
    case 'ready':     workerReady = true; …; next();           break;
    case 'out':       if (b) b.feed(m.text, m.cls);            break;
    case 'done':      if (b) b.finish(m.rc, m.ms);
                      current = null; next();                  break;
    case 'runerror':  if (b) { b.error(…); b.finish(1, 0); }
                      current = null;                          break;
    case 'loaderror': …                                        break;
  }
};
```

`current` is the block that owns the run, which is what lets one worker serve every editor on the page: output is routed to a *block*, not broadcast.

### 32.7.2 The screen is a model, not the DOM

`feed` does not touch the document. It appends to an array of `[text, cssClass]` pairs:

```
// raku.js
this._screen = []; var chars = 0, pending = false, CAP = 200000;

function push(text, cls) {
  if (self._clearNext) { self._screen = []; chars = 0;
                         self._clearNext = false; sched(); }
  if (!text || chars > CAP) return;
  chars += text.length; self._screen.push([text, cls || '']); sched();
}
```

Keeping a model rather than mutating the DOM is what makes the next two things possible at all — you cannot *clear a screen* or *drop the oldest output* if the only record of it is already rendered.

`CAP` is a character budget. A program that prints without end fills 200,000 characters and then produces nothing further, rather than growing a DOM node until the tab dies.

### 32.7.3 Rendering is coalesced on a timer

```
// raku.js
function render() {
  pending = false;
```

```
  outEl.innerHTML = self._screen.map(function (p) {
    return '<span class="' + p[1] + '">' + esc(p[0]) + '</span>';
  }).join('');
  outEl.scrollTop = outEl.scrollHeight;
}
function sched() { if (!pending) { pending = true; setTimeout(render, 16); } }
```

Every `push` calls `sched`, and `sched` schedules **at most one** render per 16 milliseconds however many lines arrived in between. A program emitting thousands of lines therefore costs a few dozen repaints, not thousands.

The timer is `setTimeout` rather than `requestAnimationFrame` on purpose: `requestAnimationFrame` pauses in a hidden or background tab, so output would simply stop appearing for anyone who switched away mid-run.

The render is a full rebuild of `innerHTML` from the model, and the scroll is pinned to the bottom afterwards. A full rebuild sounds wasteful and is not, at this scale: the model is capped, and rebuilding is what makes a *clear* free.

### 32.7.4 ANSI, which is how animation works

A program that animates redraws by moving the cursor home. `feed` splits its input on an ANSI escape pattern and treats two of them as "start again":

```
// raku.js
var ANSI = /\x1b\[[0-9;?]*[A-Za-z]/g;

this.feed = function (text, cls) {
  ANSI.lastIndex = 0; var lastI = 0, mm;
  while ((mm = ANSI.exec(text))) {
    push(text.slice(lastI, mm.index), cls);
    var f = mm[0][mm[0].length - 1];
    if (mm[0] === '\x1b[2J' || f === 'H' || f === 'f') clear();
    lastI = ANSI.lastIndex;
  }
  push(text.slice(lastI), cls);
};
```

Erase-display and cursor-home empty the model; every other escape sequence is recognised and discarded, so it neither prints as garbage nor is mistaken for text. That is the whole of the terminal emulation, and it is enough: a Game of Life that clears

and redraws each generation animates in the page, because clearing the model and re-rendering **is** a frame.

### 32.7.5 The deferred clear

The subtlest piece. Pressing Run again does not blank the pane immediately:

```
// raku.js — inside push()
// Deferred clear: keep the previous run's output on screen until the new run
// actually produces something, so re-running never collapses the block.
if (self._clearNext) { self._screen = []; … }
```

The run sets a `_clearNext` flag; the *first output of the new run* is what consumes it. Without this, re-running a slow program blanks the pane and leaves the reader looking at nothing for as long as the program takes to produce its first line — and re-running a program that prints nothing collapses the block to empty, which reads as breakage.

### 32.7.6 Finishing

```
// raku.js
this.finish = function (rc, ms) {
  setRun(false);
  if (self._clearNext) { self._screen = []; chars = 0;
                         self._clearNext = false; }
  if (!self._screen.length) push('(no output)', 'meta');
  push('\n── exit ' + rc + ' · ' + ms + ' ms ──', 'meta');
  stEl.textContent = 'exit ' + rc + ' · ' + ms + ' ms';
};
```

The exit code and elapsed time are pushed as an ordinary screen entry with a `meta` class, so they are styled apart from the program's own output — and the Copy button filters `meta` entries out, so copying an example gives you what the program printed and not the footer the page added.

A run that produced nothing says so, because an empty pane is indistinguishable from a pane that never updated.

## 32.8 Live mode: running as you type

The playground's toolbar carries a **Live** checkbox. With it on, nothing is pressed: the program runs itself a beat after typing stops, so the output pane always shows what the source in front of you does.

The scheduling is a debounce, and the two constants are the whole policy:

```
// play/index.html
const LIVE_DELAY   = 400;   // ms of quiet before a live run starts
const LIVE_TIMEOUT = 2000;  // ms a live run may take before it is killed

function scheduleLive() {
  if (!live) return;
  clearTimeout(liveTimer);
  liveTimer = setTimeout(liveRun, LIVE_DELAY);
}
```

Every repaint of the editor calls `scheduleLive`, and so does the standard-input box — *the program's input is part of the program*, so changing it re-runs too.

`liveRun` then declines in two situations, and both matter:

```
// play/index.html
function liveRun() {
  if (!live || !ready) return;
  if (codeEl.value === lastSrc && stdinEl.value === lastStdin) return;
  if (running) { liveTimer = setTimeout(liveRun, 120); return; }
  startRun(true);
}
```

Nothing changed means nothing runs — moving the caret is not an edit. And if a run is already in flight it is **left alone** and retried shortly, rather than killed: restarting the worker costs a fresh WebAssembly instantiation, and paying that on every keystroke would make typing worse, not better.

### 32.8.1 Why it does not blank while you type

The obvious implementation streams output into the pane as it arrives, which is what pressing Run does. For live mode that is unusable: every keystroke mid-word leaves a half-written program, and a half-written program is usually a **parse error**. The pane would blank and flash a diagnostic between every two characters.

So a live run's output is **double-buffered**. Instead of feeding the screen model directly, it accumulates:

```
// play/index.html
if (buf) bufPush(m.text, m.cls); else feed(m.text, m.cls);
```

and only when the run finishes is the buffer swapped in as one atomic change:

```
// play/index.html
function flushBuf() {
  const b = buf;
  buf = null; bufChars = 0;
  if (!b) return;
  clearScreen();
  for (const [t, c] of b) feed(t, c);
}
```

The pane therefore only ever changes from one *complete* result to the next. That single decision is what makes the mode bearable to type into, and it is the reason the screen is a model rather than the DOM (above) — buffering a list of `[text, class]` pairs and swapping it in is trivial; buffering rendered markup would not be.

### 32.8.2 The watchdog, and when Live gives up

A live run is on a timer. Exceed it and the worker is killed:

```
// play/index.html
function liveTimedOut() {
  if (!running || !curLive) return;
  flushBuf();      // whatever it printed before the kill is still worth seeing
  pushText(`\n── live run stopped after ${LIVE_TIMEOUT / 1000} s ──\n`, "meta");
  killAndRecreate();
  …
  if (++liveStalls >= 2) { setLive(false); … }
}
```

Note that the buffer is flushed *before* the kill is reported: a program that printed something useful and then hung has still told you something, and throwing that away to report a timeout would be a worse trade.

The stall counter is the part worth copying. **Two watchdog kills in a row turn the mode off**, with a line in the output saying why — because each kill costs a fresh WebAssembly instantiation, and re-killing the worker on every keystroke of

a genuinely long-running program helps nobody. A feature that cannot serve you should switch itself off and say so, rather than degrade silently.

The setting persists in local storage, with one exception: a visitor who arrived from a `?live=1` link gets it on for that visit only. They came from a page *about* Live mode, which is not the same as choosing it for every future visit.

### 32.8.3 No incremental compilation, on purpose

There is no REPL here, no partial re-evaluation, no caching of a parse between keystrokes. Every live run lexes, parses and executes the whole program from scratch.

That is not a shortcut around a hard problem — it is the measurement. A whole run of a typical example is a few milliseconds of WebAssembly, comfortably inside the 400 ms debounce, so re-running everything is *also* the fast thing. Incremental machinery would add state to keep correct, in exchange for latency nobody can perceive.

It is the same reasoning as Chapter 19's: count the opportunity before building the clever version.

## 32.9 Embedded editors: one script tag

The playground is one page. The embed is the part that turns any page on any site into a Raku environment:

```
<script src="https://raku.online/raku.js"></script>
<pre data-raku>say "Hello from an embedded editor!";</pre>
```

Every element carrying `data-raku` becomes an editable, runnable editor. That is what makes the specification site, the tour and the course pages show examples that a reader can change and re-run, rather than screenshots of output.

Five decisions hold it up.

**One interpreter per page.** Multiple blocks share a single WebAssembly instance — one download, one instance — so a page with twenty examples does not fetch twenty engines.

**The worker is built from a Blob.** A plain `new Worker('https://...')` is blocked cross-origin; a Blob worker that `importScripts` the cross-origin engine is allowed. That single workaround is why the embed works from someone else's domain at all.

**Runs are queued, not dropped.** Only one program runs at a time, so a request made during another run waits its turn:

```
// raku.js
function enqueue(block) {
  if (queue.indexOf(block) < 0) queue.push(block);
  relabel();
}
```

This used to be a single `queued` slot, and a third request overwrote the second — press Run on several blocks and only the last one ever started. Every path that ends a run now calls `next()`, **including the failures**, because a load error or a killed worker used to clear the current block and leave the rest of the queue stranded for ever.

**Each editor lives in its own Shadow DOM**, so the host page's CSS cannot reach in and the embed's cannot leak out. It looks the same on any site.

**The editor is a transparent textarea over a highlighted `<pre>`.** The two are positioned on top of one another and their scroll offsets are kept in step:

```
// raku.js
hl.innerHTML = html;
hl.scrollTop = ta.scrollTop; hl.scrollLeft = ta.scrollLeft;
```

And the highlighting is the payoff for exporting `rakupp_highlight`. Until the engine has loaded, editors paint with a fast JavaScript approximation; once it is there, **every editor repaints through rakupp itself** — the same tokenizer as `rakupp --highlight`, so an embedded editor colours Raku exactly as the compiler understands it, and keeps doing so as the reader types.

Nothing is sent anywhere. The program runs in the visitor's browser.

## 32.10 What the host takes away

| | |
|---|---|
| **deep recursion** | 58 Raku levels in a Chrome worker, 32 in a Safari one; an overflowed run goes again on the page's main thread (115 in Chrome, 419 in Safari), and past that the page reports a recursion-limit message and recovers |

| | |
|---|---|
| **`start` / `Promise`** | needs real threads; a threaded build needs cross-origin isolation headers, awkward for static hosting (Chapter 38) |
| **sockets** | not available in the browser sandbox |
| **NativeCall** | takes its no-libffi fallback path by construction — there is no shared library to open (Chapter 36) |
| **`--exe` and the code generator** | irrelevant: this ships the interpreter, not the transpiler |
| **`exit`** | aborts the module instance; the worker rebuilds a fresh one |

The recursion limit is the one users actually meet — in Safari even the language showcases meet the worker's — and the message says what it is: a WebAssembly stack limit, not a Raku one. The same program runs natively. Lifting it would mean rewriting the tree walker onto an explicit heap stack — a change to `src/`, and a large one.

## 32.11 Size and memory

A few megabytes of `.wasm`, roughly one to three compressed, **dominated by the Unicode tables** (Chapter 24). It downloads once and is cached, with a hash-based cache tag so a release invalidates it exactly when the engine changes and not otherwise.

Per-instance memory is kept deliberately small — a 16 MiB stack and 32 MiB initial heap, growing on demand — precisely because the worker recreates the instance on every Stop and every crash, and a large fixed footprint would pile up. If instantiation ever fails under memory pressure, the page retries a couple of times before asking for a reload rather than hanging silently.

## 32.12 Why this is worth having

Two reasons beyond the obvious one.

It is a **fourth client of the runtime**, after the interpreter, the native code generator and the extension ABI — and the one with the least in common with the others. Every assumption the runtime makes about its host that turned out to be unnecessary was found by porting it here.

And it is a **test surface**. The playground runs the same example programs the local suite does, so a divergence between the WebAssembly build and the native one is a real bug in something shared. The showcase interpreters run inside it too: a JavaScript interpreter, written in Raku, running on Raku++ compiled to WebAssembly, inside a browser — which is a stack that exercises a great deal at once.

# Part VIII

# Boundaries

# Chapter 33

# Modules

A module is **source text that gets lexed, parsed into its own `Program`, and executed exactly once** — at the point its `use` statement runs, inside the same interpreter, on the same object graph as the importing program.

It gets a fresh `Env` chained to the global one for the duration of the load; when the load finishes, that environment's contents are **copied into the global `Env`**. After that the module has no ongoing identity: its subs are ordinary `VT::Code` values in the global scope, its classes ordinary entries in the one flat class registry. Calling into a module is not a different operation from calling a sub in the same file.

That paragraph is also the source of nearly every divergence at the end of this chapter.

## 33.1 Parse time: only operators are looked at

`use Foo` triggers exactly one compile-time action:

```
// src/Parser.h
void scanModuleOps(const std::string& module);
```

which finds the module's *source file* and **text-scans** it — no lexing, no parsing — for declarations of the five operator categories, registering those names in the parser's tables so the rest of the importing file parses (Chapter 6).

That is the whole compile-time effect, because operators are the only thing about a module that changes how the importing file **parses**:

```
use Vector;          # declares `sub infix:<·>`
say $a · $b;         # a parse error without the scan
```

Two consequences. An operator spelled only in ASCII operator characters is skipped, because `multi infix:<*>(Color, Real)` is almost always extending a built-in and registering it would silently reshape every expression in the importing file. And the scan reads the file a second time — cheap, being a substring search, but it *can* be fooled by an operator name inside a string or a comment.

## 33.2 Run time: `loadModule`

```
loadModule(name)
 ├─ already loaded?                    → return immediately
 ├─ find the source (see below)
 ├─ Lexer → Parser → Program           (its OWN AST)
 ├─ keptPrograms_.push_back(prog)      (the AST must outlive the load)
 ├─ moduleEnv = new Env, parent = global_
 ├─ bind %?RESOURCES / $?DISTRIBUTION into moduleEnv
 ├─ scan the AST for `is export` names
 ├─ cur = moduleEnv;  curPkgEnv_ = moduleEnv
 ├─ hoistSubs(prog->stmts)
 ├─ exec() every top-level statement
 ├─ publish(): copy moduleEnv->vars into global_
 └─ restore cur / curPkgEnv_ / pkgPrefix / finishData_
```

The module's top-level statements run like any other code, once. `BEGIN` blocks in the module run as part of this — there is no separate compile phase for them to run in.

`loadedModules_` is checked on entry and the name inserted immediately, so recursive and repeated `use`s are no-ops. A cyclic `use` therefore will not hang, but the second module sees a *partially loaded* first one.

## 33.3 Where the source is found

`libPaths_` starts as `{"lib", ".", "rakulib"}` and is extended:

| Source | Position |
|---|---|
| `RAKULIB` | appended |
| `ROAST` | appended (adds the test-helper library) |
| `-I dir` | prepended |

| Source | Position |
|---|---|
| `use lib '...'` at run time | prepended |

`RAKULIB` accepts **both `,`** and `:` as separators, with one carve-out: a `:` directly after a lone leading letter is a Windows drive letter, not a separator.

That splitting logic is shared rather than duplicated, and the comment in the header says why:

```
// src/Interpreter.h
// Shared so the PARSER's copy of the search path — which is what finds a
// `use`d module's operator declarations while the importing file is still
// being parsed — cannot drift from the interpreter's. It did: the
// interpreter learned ',' and the parser did not, so `RAKULIB=a,b`
// silently stopped importing operators.
std::vector<std::string> splitSearchPath(const std::string& spec);
```

For each base, both `<base>/` and `<base>/lib/` are tried, with the extensions `.rakumod`, `.pm6`, `.raku` and `.pm`.

If nothing matches, the **installed** repositories are searched. Resolution mirrors a real installation repository: SHA-1 the module name, read the short-name index entry, take its content id, and read the source by that id. So Raku++ loads zef-installed modules directly out of Rakudo's own install tree. That store — its layout, the installer that writes it, and how the engines share it — is Chapter 34.

```
RAKUPP_TRACE=1 rakupp myprogram.raku
```

```
[LibPaths] [lib] [.] [rakulib]
[Load] JSON::Fast <- source lib/JSON/Fast.rakumod
[Load] Test::Util <- installed /Users/you/.raku dist=E3A0…
```

## 33.4 Failure is fatal, deliberately

A module that is missing, fails to parse, or throws while loading **aborts the program**.

It did not always. It used to warn and carry on, and that was a divergence with real consequences: a `use` that silently vanished left the program running against a state nobody designed — the module's `BEGIN` blocks never ran, its exports never materialised. It also flattered every measurement. In the module compatibility

battery, twenty probes "produced output" purely because the load had been skipped, so the comparison against Rakudo was meaningless for them.

The one exemption is `Slang::*`, which are compile-time grammar mutators Raku++ cannot apply at all; failing on them would reject programs it otherwise runs perfectly.

Pragmas with no file behind them — `strict`, `fatal`, `experimental`, `newline`, `precompilation` — are listed **explicitly** rather than being allowed to fall out of a failed lookup. Ignoring an unimplemented pragma must be a deliberate entry, not an accident.

## 33.5 Questions people actually ask

### 33.5.1 Is there an `Env` that holds the module?

**During the load, yes; afterwards, no.** `moduleEnv` exists only for the duration of `loadModule`. Its real job is scoping the module's *own* code: the module's subs are defined there so they see one another, and each `Callable` captures it as its closure. That closure is what keeps the module's lexical world alive after publication — a module sub calling another module sub still resolves it through `moduleEnv`.

What the importer sees is the *copy* in the global environment. There is no stash object, no per-module symbol table to enumerate, no `Foo::EXPORT::DEFAULT`.

### 33.5.2 Is there a separate AST?

**Yes, and it is never walked as a unit again.** Each module gets its own lexer, parser and `Program`. That `Program` is executed once, top to bottom, and then survives only as *storage*: `Callable::params` and `Callable::body` are borrowed raw pointers into it (Chapter 7), which is why it is pushed onto `keptPrograms_` and never released.

Parsing is isolated in **one direction only**. The module is parsed with a fresh `Parser`, so the importing file's operators, lexical regexes and sigilless names do not leak into it. In the other direction the module's operators *do* reach the importer — through the text scan, not the parse.

### 33.5.3 Is calling a module routine different?

**No.** `evalCall` resolves every named call the same way (Chapter 16), and `Env::find` walks the parent chain. A local `my sub` is found nearby; a module sub is found in the

global environment at the end of the chain. The only difference is a slightly longer walk to *find* it; the call itself is byte-for-byte identical.

The same holds for methods: the class registry is one flat map, so a class from a module and a class from the current file are indistinguishable at dispatch time.

### 33.5.4 How does `is export` work?

`is export` is scanned for, but it is **not** what makes a symbol visible. Its one real effect today is the built-in collision carve-out in `publish()`:

> A sub that is **not** `is export` and whose name collides with a built-in stays module-private.

That is what lets a module's `our sub run` coexist with the built-in `run`: inside the module, `run` resolves through `moduleEnv` to the module's own; an importer's bare `run(...)` reaches the built-in.

The `sub EXPORT(*@_)` protocol *is* implemented: if the module defines it, it is called with the `use` statement's arguments and the map it returns is defined in the **using** scope. `use Mod <name:alias>` imports a routine under a second name.

### 33.5.5 `our` and `unit module`?

`tctx_.pkgPrefix` is the mechanism. `unit module Foo;` sets the prefix for the rest of the compilation unit; a braced `module Foo { … }` sets it for the body and restores it after. Then an `our sub bar` is also published as `&Foo::bar`, an `our` variable as `$Foo::x`, a `my` one not at all, and a class or grammar registers under its qualified name.

That last one was a bug until it was not: compound names skipped the prefix, so `grammar Schema::Core` inside `unit module YAMLish` registered unqualified and was unreachable from any importer.

### 33.5.6 When do a module's `END` blocks run?

At **process** end, not at load, and *before* the mainline's own `END` blocks, last-in-first-out, so a module loaded later cleans up earlier. They capture the module scope, so they still see its lexicals.

### 33.5.7 What about --exe?

Modules are **not** compiled into the binary. The generator emits a call to `Interpreter::rtUse`, a thin mirror of the interpreter's `use` handling, calling the same `loadModule`. A compiled binary still interprets its modules — though it can carry their serialised ASTs inside itself (Chapter 25), so it needs neither their sources nor a warm cache to start.

## 33.6 Divergences from Rakudo

These are known and mostly deliberate. They are the reason a module can behave differently under Raku++ than under Rakudo even when nothing is broken.

| # | Raku++ | Rakudo |
|---|---|---|
| 1 | `publish()` copies the module's **whole** environment to global. A `my sub`, a non-exported `our sub`, and its classes are all visible bare to the importer. | Only `is export` symbols are imported. |
| 2 | `need Foo` still makes Foo's symbols visible. | `need` loads without importing. |
| 3 | Types resolve **at run time**. `NoSuchType.new` is a run-time "no such method", after output has appeared. | `Undeclared name` at compile time. |
| 4 | The parse is cached; declarations and `BEGIN` still run every time. | `.precomp` caches the parse *and* the objects compile-time code produced. |
| 5 | A module's operators reach the importer by **text scan**. | Real lexical export of the operator into the importer's grammar. |
| 6 | One flat class registry — no per-compunit type isolation. | Each compunit has its own stash. |

| # | Raku++ | Rakudo |
|---|---|---|
| 7 | `use Foo:ver<...>` is honoured only against the **installed** index (line 1 of a short entry); on the lib path all adverbs are discarded, and `:auth`/`:api` are never consulted. | Full version, auth and API resolution. |

Divergence 1 is the big one and it cuts both ways. It is why some modules work under Raku++ that would otherwise need more machinery, and it is why a module's internal helper can silently shadow something in the importing program.

Demonstrated with a module declaring `my sub private-sub`, `our sub our-sub`, `sub exported-sub is export` and `class Widget`:

| Called from the importer | Raku++ | Rakudo |
|---|---|---|
| `private-sub()` | `"private"` | not visible |
| `our-sub()` | `"our"` | not visible |
| `exported-sub()` | `"exported"` | `"exported"` |
| `Demo::our-sub()` | `"our"` | `"our"` |
| bare `Widget.greet` | `"widget"` | not visible |

## 33.7 Debugging a module problem

1. **`RAKUPP_TRACE=1`** — confirm *which* file was loaded. Half of all module bug reports are the wrong copy being picked up, usually because `.` is on the search path or an installed copy shadowed a checkout.
2. **`RAKULIB` takes `,` or `:`.** If the same value also has to drive Rakudo, write `,`.
3. **Instrument the real module, not a reduction.** Copy its `lib/` to a scratch directory, add `note` calls, run it under both engines. Reductions of grammar and module behaviour have repeatedly *passed* while the real module failed — the reduction drops the one piece of context that mattered.
4. **Load failures are loud now**, so a module that appears to do nothing is loading fine and behaving differently, not being silently skipped.

# Chapter 34

# The Installer and the Store

`rakupp install Foo` downloads a distribution from the Raku ecosystem, runs its tests, and installs it — into **the same on-disk store that Rakudo and zef already share**. That sentence carries the whole design: there is no rakupp package database, no private format, no import step. The store *is* the interface between the engines, and this chapter is about both sides of it — the layout that makes a directory tree function as a protocol, and the installer that writes it.

The order of construction matters here. Raku++ could **read** the store long before it could write it: Chapter 33 ends with `use` resolving zef-installed modules straight out of Rakudo's install tree. Then the zef-compatibility work gave the engine a **writer** — enough of `CompUnit::Repository::Installation` that zef itself, running under rakupp, could install things. `rakupp install` came last, as a thin, careful client of a writer that already existed. Install is a new front door on machinery the engine already had.

## 34.1 The layout

A store is one directory — a *prefix* — with this inside:

```
~/.raku/
├── version              the layout version (2)
├── repo.lock            what writers flock before mutating anything
├── sources/<id>         one file per installed module source
├── dist/<dist-id>       one JSON file per distribution: its full META
├── short/<sha1(name)>/  the index: one entry per (module name, dist)
├── resources/<id>       content of each declared resource file
```

```
├── bin/<id>            content of each bin/ script
└── precomp/            Rakudo's bytecode cache (rakupp ignores it)
```

Every engine's default chain includes at least two prefixes. Rakudo's is `home` (`~/.raku`) then `site/vendor/core` under its own installation; rakupp searches `~/.raku` first and then every Homebrew Rakudo's `site` and `vendor` — so a module zef installed for Rakudo is found by rakupp with no configuration at all. The two installers default to *different ends* of the shared chain: zef writes Rakudo's site store, `rakupp install` writes `~/.raku` (`--to` to aim elsewhere). Both engines read both.

The clever part of the layout is `short/`. A `use JSON::Fast` must find one file among thousands without scanning anything, so the index is **addressed by hash**: the directory name is the SHA-1 of the module name, and each file inside it describes one installed distribution providing that name. The file's *name* is the distribution's id; its *content* is five short lines:

```
$ cat site/short/E0FD4AFC…0FB899A/8270A3B9…41A908A
      # …/short/<sha1("File::Temp")>/<dist-id>
0.0.12                                  ← version
zef:raku-community-modules              ← auth
                                        ← api
DBE25863E06AEC6820058FA548E27780308E6C43   ← the sources/ file to load
1375FC65817F5B56701E20EC2F4101028E8363F6   ← a checksum (see below)
```

Resolution is: hash the name, open the directory, pick an entry whose first line satisfies the `use` version constraint, load `sources/<line 4>`. Three lines of metadata, one pointer, done. The `dist/<dist-id>` JSON is *not* read to resolve a `use` — it exists for everything else: `%?RESOURCES` and `$?DISTRIBUTION` binding, listing, uninstalling.

## 34.2 Two writers, one reader contract

Rakudo's writer and rakupp's writer produce interchangeable stores, but they are not byte-identical, and the differences say what the contract actually is:

| | Rakudo / zef | Raku++ |
|---|---|---|
| `sources/` file name | SHA-1 of *module name* + *dist-id* | SHA-1 of the file's *content* |
| line 5 of a short entry | SHA-1 of the CRLF-normalised source — its precomp layer's validation checksum | the dist-id, repeated |
| `precomp/` | written and maintained | never written, never read |

Neither engine's reader cares about any of these. The `sources/` name is an opaque token — you read whatever line 4 names; nothing recomputes the hash. Line 5 is consulted only by Rakudo's precompilation machinery, which rakupp does not participate in. That is the contract in practice: **filename of the short entry, lines 1–4, and the blob those point at**. Everything else is a writer's private convention riding along in a shared file.

(The formulas above are measured, not quoted from documentation: hash a real store's entries and compare. Line 5 of a 2021-era entry in a long-lived home store matches *no* current formula — the checksum recipe has changed across Rakudo versions, which is itself evidence that nothing load-bearing reads it.)

One Raku++ divergence from Chapter 33 needs an update in this light: `use Foo:ver<1.2+>` adverbs are discarded during lib-path search, but against the *installed* index the `:ver` constraint **is** honoured — it is checked against line 1 of each candidate entry. `:auth` and `:api` are still not consulted.

## 34.3 The writer is the engine's

`CompUnit::Repository::Installation.install` is implemented in C++, in the same method-dispatch code that serves every other built-in class:

```
// src/Builtins.cpp
// $cur.install($dist, :$force) — write the CURI layout under `prefix`
// (sources/<sha>, short/<sha1(name)>/<dist-id>, dist/<dist-id> JSON,
// resources/, bin/). rakupp reads exactly this to resolve `use`.
```

It computes the dist-id as `sha1hex(name + "\0" + ver + "\0" + auth + "\0" + api)`, which is **not** what that line looks like it does. `std::string + const char*` stops at the terminator, so each `"\0"` appends nothing at all and the id is the four fields simply concatenated. That is a bug frozen into a format: every dist-id on every store on disk was computed this way, so adding the separators would rename every record and orphan every installed distribution. The source carries a comment saying exactly that, because it is the kind of line a reader tidies up. It then writes each provided module's source as a content-addressed blob, writes one short entry per provided name, copies `resources/` and `bin/` payloads, and writes the `dist/` record with a `files` map — relative path → blob id — which is what later makes uninstall able to know exactly which blobs are this distribution's. It returns the dist-id, and that return value is load-bearing: the installer records it as provenance.

This writer exists because of zef, not because of `rakupp install`. Running zef under rakupp meant implementing the parts of the `CompUnit` API zef drives, and `.install` is the heart of it. When the time came to build our own installer, the choice was already made: use the engine's writer through the same public spelling zef uses —

```
# tools/install.raku
my $repo = CompUnit::RepositoryRegistry.repository-for-spec("inst#$prefix");
my $dist-id = $repo.install($dist, :force($force));
```

**What went wrong.** The first version constructed the repository by hand:

```
CompUnit::Repository::Installation.new(prefix => $to)
```

which parses, runs, and installs nothing, silently. `.new` does not thread the prefix through to the writer, so every file operation targeted `"/sources"` and failed into the void. The fix is twofold: `repository-for-spec` is the constructor that carries the prefix, and the writer now *refuses loudly* when its prefix is empty rather than failing file-by-file in silence. A writer aimed at a shared store does not get to guess.

## 34.4 The installer is a Raku program the binary carries

`rakupp install` and `rakupp uninstall` are not implemented in the binary — not in C++, anyway. `main.cpp` recognises the command words and rewrites its own argument vector so the command becomes `rakupp install.raku ...`:

```
// src/main.cpp — the same trick `python -m pip` pulls, with a nicer spelling
const char* tool = isDocCmd ? "doc.raku" : "install.raku";
g_embeddedTool = tool;
g_embeddedSrc  = isDocCmd ? rakupp::docToolSource() : rakupp::installerSource();
```

Those two functions return the program's text, and the argument scan recognises the name and takes the source from there instead of opening a file. The text is a byte array in a translation unit that does not exist in the repository: `cmake/EmbedTools.cmake` writes it into the **build** tree at build time, from `tools/install.raku`, `tools/doc.raku` and the two guides `doc` reads, listed as the custom command's `DEPENDS` so editing any of them rebuilds exactly that one file.

Three decisions are packed into that sentence, and each one was arrived at the long way.

**Bytes, not string literals.** MSVC caps a single string literal at 16 KB, and the installer is 85 KB, so a literal would have to be split into chunks — and a chunk boundary is a thing to get wrong. `file(READ ... HEX)` has no cap, needs no escaping, and

has no delimiter the content could accidentally contain. It is what `src/AstEmit.cpp` has always done with serialized ASTs, for exactly this reason.

**CMake, not Raku.** The obvious generator is a Raku program, since everything else in `tools/` is one. But generating the blob would then require a working `rakupp` — which is the thing being built. That forces the generated file to be *committed*, and a committed generated file is a second copy of the installer in git: one diff in the `.raku` and one in the `.cpp` for every edit. CMake needs nothing but CMake, so the output can live in the build tree and the duplication never arises.

**Compiled in, not shipped beside.** Which is the subject of the rest of this chapter.

The guides are the one part that is not a straight copy. `docToolSource()` splices them into `doc.raku`'s `my %DOCS;` declaration as `Q:to/.../` heredocs — `Q` interpolates nothing and unescapes nothing, so Markdown goes in as it is. `doc.raku` reads that hash before it goes looking on disk, so a checkout leaves it empty and reads files, and a shipped binary answers from itself. The one thing that could break it is a line in a guide spelled exactly like the heredoc terminator, which would end the heredoc early and produce a doc tool that fails to *parse* — at run time, in a binary that compiled perfectly. CMake is in no position to check that, so the gate does.

The reasons for Raku-not-C++ are worth spelling out either way, because "write the package tool in C++" is the default instinct and it is wrong here:

- It is dogfood: the project's own tooling running on the interpreter it ships, which is the policy everywhere else in the repo. What an installer actually does — parse index JSON, compare version ranges, join paths, run subprocesses, write a store — is the work Raku is short for and C++ is long for.
- It stays readable as a *program*, which matters for a tool whose failures are all about somebody else's distribution.

What is **not** a reason, though it reads like one: "so that `--exe` binaries do not carry an HTTP client". An `--exe` binary is a compiled user program; it can never be invoked as `rakupp install`, so its payload has nothing to say about how the installer is written. And the size question is settled by where the translation unit is linked, not by its language — the generated blob sits in the CLI-only group beside `Js.cpp`, and a C++ installer in `main.cpp` would have sat there just as well. The engine carries no network code for a different and better reason: fetching is `curl` and unpacking is `tar`, both in a subprocess, so no HTTP client, TLS stack, tar reader or index parser exists in rakupp in *any* language. That is a property of this design, not a consequence of choosing Raku.

### 34.4.1 It used to be a *sidecar*, and that was the bug

The rewrite above once produced a **path**, looked up beside the executable:

```
for (const char* rel : {"/../libexec/rakupp/install.raku", "/../tools/install.raku"}) {
    std::string cand = exeDir + rel;
    if (std::ifstream(cand).good()) { script = cand; break; }
}
```

The argument for shipping the file separately was that the installer changes at ecosystem speed and the engine at engine speed, so decoupling them would let one be fixed without rebuilding the other. In practice they shipped in the same release every time, and the decoupling only ever produced its failure mode: a binary with no installer. `COPY rakupp` into a container, a bare `rakupp.exe` dragged out of the Windows ZIP, a package that installed `bin/` and skipped `libexec/` — each one earned "cannot find install.raku beside this binary", and the fix was always for the *user* to reassemble a pair the project had split.

A cadence that never materialised is not worth a path the user has to get right. Nothing is looked up now, so there is no wrong path to have.

The program is about 1,700 lines and its shape is a pipeline. It has roughly tripled since this chapter's first draft, and the additions are worth naming because none of them is in the pipeline below: `--list` and `--check` and `--gc`, `reinstall` and `test`, installing from a local path, the REA archive fallback when a name is not in the fez index, build hooks, and the `rakulib` shadowing rule that skips a distribution whose name a bundled shim answers.

**Index.** `https://360.zef.pm/index.json` — the fez ecosystem's index, one JSON array of every distribution's META plus an archive path. Cached in `~/.raku/rakupp-install/` for 24 hours; `--refresh` refetches; `RAKUPP_INSTALL_INDEX` substitutes a file or URL, which is how the test suite runs without a network and how a mirror slots in.

**Resolve.** Breadth-first over `depends`, newest satisfying version first, `:from<native>` and `:from<bin>` dependencies reported but not fetched, plan reversed at the end so dependencies install before their dependents:

```
$ rakupp install --dry-run JSON::Class
plan (6 distributions, dependencies first):
  JSON::Fast:ver<0.20.1>:auth<zef:timo>  https://360.zef.pm/J/SO/JSON_FAST/...
  JSON::Name:ver<0.0.7>:auth<zef:jonathanstowe>    ...
  JSON::OptIn:ver<0.0.2>:auth<zef:jonathanstowe>    ...
  JSON::Unmarshal:ver<0.18>:auth<zef:raku-community-modules>    ...
```

```
  JSON::Marshal:ver<0.0.25>:auth<zef:jonathanstowe>    …
  JSON::Class:ver<0.0.21>:auth<zef:jonathanstowe>    …
dry run: nothing written
```

**Fetch and verify.** Transport is `curl` and `tar` — present on every platform this targets, TLS certificate verification on (the installer never passes `-k`). And the archive URLs above contain their own second factor: fez archives are content-addressed, the path stem *is* the SHA-1 of the tarball. A fetched archive is hashed and refused on mismatch, so a compromised mirror or a truncated download cannot become an installed module. When an index entry carries no checksum in its path, the installer says so out loud rather than pretending the gate applied.

**Three ways in, one path out.** Everything above is what a *name* goes through. A directory argument skips resolution entirely — it already is a distribution — and a URL is fetched and unpacked into a directory first. All three then produce the same entry, so Test and Write below have no idea which one they are serving. That is the whole reason `url-dist-entry` ends by calling `local-dist-entry` on what it unpacked rather than building an entry itself: a second construction of the same thing is a second thing to keep correct.

The URL forms are a `.tar.gz` archive and a github.com repo or `/tree/` page, rewritten to the tarball github already serves. `/tree/REF/SUBDIR` reaches a distribution inside a monorepo, which is the shape that prompted it — the URL someone has is the one from their address bar, not one they would have to construct. And the fetch-and-verify paragraph above is exactly where the three stop being equivalent: **a URL cannot be verified.** A fez archive's URL carries the SHA-1 of its own contents; an arbitrary URL names nothing about the bytes it will deliver. So a URL install prints the TLS-only note rather than quietly skipping a gate the reader has just been told about. `uninstall` refuses a URL for a related reason: the store is keyed by name, and learning the name behind a URL would mean fetching it first, which is not a thing an uninstall should do.

Two more kinds of argument arrived later, and both end in the same entry shape. **A directory of module files with no `META6.json`** is not a distribution, but it can stand for one. When it lies under the `lib/` of a checkout — a `META6.json` in some directory above it — the entry is that checkout's, and every module file found in the directory (with `-r`, below it too) that the META's `provides` does not list is added to it. The META is the only place a name, a version and a dependency list are written down. Giving the found files an identity of their own would put a partial second copy of the distribution in the store, and `use` would choose between the two by version. With no `META6.json` above, an identity is made: version 0, named for the

namespace the modules share, its dependencies whatever the files `use`. **A program** contributes no entry at all, only wants: its `use` and `need` lines, read as text, because a program whose dependencies are missing is exactly the program that cannot be compiled to find them.

**Test.** Before a distribution is installed, its own `t/` suite runs under rakupp — dependencies were installed first, so the tests see them. This gate earned its keep on its very first live run: JSON::Unmarshal 0.18's suite failed one test under rakupp, the installer refused the whole chain, and the failure became an engine bug report (a real bug — named-argument unmarshalling — since fixed). `--no-test` is the documented override, and it is a statement about what "installed" means here: not "files copied" but "demonstrated to work under this engine, on this machine".

**Write.** The engine's `.install`, under a real `flock` on the store's `repo.lock` (two new builtins, `rakupp-repo-lock/-unlock`, because the store is shared and a half-written short entry is another engine's crash). Then the returned dist-id is appended to `<prefix>/rakupp-install/owned` — one line per distribution *this tool* installed. That file is the whole provenance system, and uninstall is its consumer.

## 34.5 Update means add

`rakupp install Foo` with a newer version available installs the newer version **beside** the old one. Nothing is removed; the store's resolution — pick among short-entry siblings by version — does the rest, and an installed program pinned to `:ver<1.2.3>` keeps resolving to it. This is not an implementation shortcut, it is what the layout wants: the store is content-addressed and append-friendly, and "replace" would be a delete wearing a trench coat. Reclaiming space is uninstall's job, explicitly.

## 34.6 Uninstall is garbage collection

Deleting from a shared, content-addressed store is the hard half, which is why it landed last and why the checker (below) was written *before* the delete path. `rakupp uninstall Foo` refuses two ways before it touches anything:

- **Not installed by us.** If the dist-id is not in `rakupp-install/owned`, it is zef's (or something else's), and removing another manager's installation silently is a surprise in someone else's tool. `--force` for people who mean it.
- **Still depended on.** Any other installed distribution whose `depends` names a module this one provides blocks the removal, with the dependents named. A

bare name that matches *several* installed distributions is not one of the refusals. Two versions behind one name is the ordinary result of an upgrade — the store keeps versions side by side by design — and `zef uninstall` answers it by matching every installed distribution against the spec and removing each one that matches. rakupp does the same, saying how many it found first. The refusal this replaced asked for a version instead, and the identity it asked to be typed (`Foo:ver<1.0>`) is a redirection to every shell that would have had to pass it through, which left the name unremovable in practice.

Each match still goes through the two gates above on its own, and a match a gate refuses does not stop the others: what was removed is reported line by line, and the exit code reports that something asked for did not happen.

The removal itself is ordered by failure mode, under the same `repo.lock`:

1. **Index entries first** — every `short/<sha1(name)>/<dist-id>` this distribution owns. A crash after this step leaves orphaned blobs, which is wasted disk; the reverse order would leave live index entries pointing at deleted blobs, which is a broken `use`.
2. **Blobs nothing else references.** The `files` maps of every *other* `dist/` record are the mark phase; only unreferenced blobs are swept. Two distributions shipping a byte-identical file share one blob under rakupp's content-addressed writer, and the suite pins that the shared blob survives the first uninstall.
3. **The dist record last** — a crash mid-way leaves a record that still describes what remains to finish.

Note what provenance does *not* do: nothing stops zef from removing a distribution rakupp installed. The store has no owner field, and adding one would break the symmetry that makes the whole arrangement work. `owned` binds only rakupp's own destructive path; politeness is enforced at home.

## 34.7 The checker

`rakupp install --check` walks one store and reports; it fixes nothing:

```
$ rakupp install --check
store: /Users/ash/.raku
store check: 3 distributions, 0 broken, 0 unreferenced blobs
```

It distinguishes two severities. **Broken** — an unreadable `dist/` record, a short entry pointing at a missing dist record, a missing blob behind a live entry, a blob that no longer holds the bytes it was named for, a provided module with no index entry —

each is a `use` that will fail or a record that cannot be trusted, and any of them makes the exit code 1.

That fourth one is why the store being content-addressed is worth the indirection it costs: a blob's file name IS the SHA-1 of its content, so "still the right bytes" is a question with an exact answer and nothing to record alongside. It is asked only of the records this installer wrote — zef and Rakudo name their blobs by something else, so hashing theirs would report every one of them damaged, and the summary says how many were checked for presence only rather than letting `0 broken` imply more than was asked:

```
$ rakupp install --check
store: /Users/ash/.raku
BROKEN: fez (43BE78BF…) lib/Fez/CLI.rakumod: sources/70C8E494… holds different bytes (
1 DA39A3EE…)
  4 distributions not installed by rakupp: blobs checked for presence, not content
store check: 135 distributions, 1 broken, 0 unreferenced blobs
```

A blob truncated to nothing was the case that made presence alone insufficient, and it is worse than an absent one: `stat` is satisfied, so the record still answers "already installed" to every attempt to repair it, and an empty module compiles, so `use` SUCCEEDS and imports nothing. An installed `fez` in that state answered every subcommand with silence and exit 0 — its script is one `use Fez::CLI`, and an empty `Fez::CLI` left the program with no `MAIN` to run. No error anywhere to go on (issue #72). **Unreferenced blobs** are wasted disk, reported and exempt: in a shared store, a blob this engine cannot account for might be another writer's, and a checker that "cleans" what it does not understand is how shared state gets destroyed. There is deliberately no `--fix`.

## 34.8 What went wrong: damage travels between engines

The checker's first run on a real machine found real damage — and the story is a compressed lesson in what sharing a store means.

A July experiment (zef running under rakupp, mid-debugging) had left a `File::Temp` registration in the home store whose source blob no longer existed: a live short entry pointing at a deleted `sources/` file. It sat there for a month without a symptom, because **Rakudo's precomp cache was serving `use File::Temp` without ever touching the store's blob** — the dangling pointer was behind a cache hit. The first mutation of that store in a month (the lock file the new uninstall path created) perturbed the mask, Rakudo revalidated, followed the dangling entry, and began

dying *at compile time* on any `use File::Temp` — while rakupp kept working, because its reader skips an entry whose blob is missing and falls through to the site store, where zef's intact copy lives.

Read that again with the roles labelled: a **rakupp**-era write broke **Rakudo**, a month later, unmasked by a **third** tool's lock file, and the two engines disagreed about whether anything was wrong because their readers degrade differently. A shared store is shared blast radius. That is why every mutation locks, why uninstall orders its deletions by what a crash would leave behind, why the checker exists and reports the store path it checked — and why it was written before the code that deletes.

(The cleanup itself was two file moves — the stale entry and its dist record out of the store — after which both engines loaded File::Temp again. The checker, the refusals and the ordering all predate the first real damage they were designed for; the damage predates them all.)

## 34.9 Interchange, stated precisely

What "rakupp and zef are interchangeable" means, claim by claim, each one gated in CI or verified on a real store:

| Claim | Evidence |
|---|---|
| rakupp reads zef-installed modules | Chapter 33's resolver; the whole module battery runs on zef-installed deps |
| Rakudo reads rakupp-installed modules | a 7-distribution graph (`License::SPDX` ← `JSON::Class` ← ...) installed by `rakupp install` loads under Rakudo from the same store, resources included — `License::SPDX.new.licenses.elems` is 727 under both |
| zef itself runs under rakupp | zef's install path drives the engine's `.install` — the same writer `rakupp install` uses |
| the stores compose | one `~/.raku` holding zef-written and rakupp-written distributions side by side resolves under both engines; the two entry dialects differ only in lines no reader consults |
| mutation is coordinated | both writers `flock` the same `repo.lock` (on Windows rakupp proceeds unlocked, and says so) |

And the honest boundary of the claim: **precompilation does not travel.** Rakudo's `precomp/` is keyed to its compiler build and rakupp neither writes nor invalidates it; rakupp's own AST cache (Chapter 30) lives outside the store entirely, in `~/.cache/rakupp/precomp`, and is off by default. Each engine's cache is private state layered over shared truth — which the File::Temp story shows is exactly where the seams are: the store stayed consistent between engines, and it was a *cache* that made them disagree.

The suite behind all of this is `t/install/run.raku` — 113 checks, fully offline (a fixture index, local archives, a scratch `HOME`), covering the plan, the checksum refusal, the test gate, additive updates, every uninstall refusal, the deletion ordering, shared-blob survival, and `--check` clean before and after every mutation. It runs in CI on every push.

# Chapter 35

# `use nqp`: a Compatibility Subset

Some ecosystem modules — most importantly `JSON::Fast`, which sits under about 170 other distributions — are written in ordinary Raku but reach for **NQP ops** in their hot paths: `nqp::iseq_i`, `nqp::substr`, `nqp::push_s`.

NQP is Rakudo's bootstrap language, and its `nqp::` operations are the low-level primitives the Raku runtime is built on. Rakudo documents them as **internal and unspecified**; every implementation provides whatever subset its ecosystem happens to need.

Raku++ provides that subset. This chapter is about how, and — more interestingly — about why it costs **nothing** for the programs that do not use it.

## 35.1 There is no NQP compiler here

The single most important thing to understand: **`nqp::foo(...)` is already valid Raku syntax.** It is a call to a package-qualified routine named `nqp::foo`. The normal parser reads it with no special help.

There is no NQP grammar, no NQP compiler, and no QAST anywhere in Raku++. What is added is a **semantic layer**: when a program opts in with `use nqp`, the parser recognises those already-parsed calls and gives them meaning, compiling them to dedicated AST nodes instead of leaving them as calls to undefined routines.

```
use nqp;                        # (or `use MONKEY-GUTS`)
my int $x = nqp::add_i(2, 3);  # 5
```

Without `use nqp`, `nqp::add_i(...)` is exactly what it looks like — a call to a routine that does not exist — and running it gives the usual undefined-routine error.

## 35.2 Zero cost when unused

This is a hard design constraint, not an aspiration, and it holds by **construction** rather than by optimisation.

- The parser carries one boolean, `useNqp_`, that starts false and is set only by seeing `use nqp` or `MONKEY-GUTS`. Every recognition site is guarded by it, so in a normal program those checks are a single already-false branch on a token that begins with `nqp::` — which no ordinary identifier does.
- The `NqpOp` node is **only ever constructed** inside that guarded path. A program without `use nqp` builds no such node, so the interpreter's `NqpOp` arm is never reached and the implementation is dead code for that run.
- There are no new fields on any hot struct, no new work in any hot loop, and no change to how normal calls, strings or numbers are handled.

A regression test asserts the invariant directly: that `nqp::add_i` still reports "undefined routine" without `use nqp`.

## 35.3 How the ops compile

Inside a `use nqp` unit, a recognised `nqp::op(...)` becomes one of three things **at parse time**.

**Constants fold to literals.** `nqp::const::CCLASS_NUMERIC` is replaced by the integer `8` in the parser; it never exists as a call. All the character-class flags and normalization modes are handled this way.

**Control forms become native or lazy nodes,** because their arguments must not all evaluate eagerly. `nqp::if` and `nqp::unless` compile to Raku++'s own ternary node, so the untaken branch never runs. `nqp::while`, `nqp::until`, `nqp::repeat_while`, `nqp::repeat_until`, `nqp::stmts` and `nqp::ifnull` become `NqpOp` nodes whose evaluator drives its own argument evaluation. The two `repeat_` forms differ from their plain counterparts in one way that matters: the body runs once before the test is ever taken.

**Leaf ops become eager `NqpOp` nodes** — integer maths, string and list primitives — which evaluate their arguments normally and then do the low-level operation over Raku++'s own `Value`s.

An `nqp::` op **outside** the implemented subset is left as an ordinary call and fails loudly at run time rather than silently misbehaving. The subset grows by measured demand, never by guessing.

## 35.4 In code

The parser branch, condensed:

```
// src/Parser.cpp — the term is already lexed as the qualified name `nqp::add_i`
if (useNqp_ && name.compare(0, 5, "nqp::") == 0) {
    if (name.compare(0, 12, "nqp::const::") == 0) {
        long long cv;
        if (nqpConstValue(name.substr(12), cv))
            return std::make_unique<IntLit>(cv);      // folded here
    }
    if (ExprPtr n = makeNqpOp(name.substr(5), callArgs))
        return n;                                     // an NqpOp, or a Ternary
}
```

The node is deliberately tiny:

```
// src/Ast.h
enum class NqpOpc : uint16_t {
    Stmts, While, Until, IseqI, AddI, Substr, /* … */ };
struct NqpOp : Expr {
    NqpOpc op;
    std::vector<ExprPtr> args;
};
```

and the evaluator handles the lazy forms *before* touching their arguments:

```
// src/BuiltinsNqp.cpp — condensed
Value Interpreter::evalNqpOp(NqpOp* n) {
    auto& a = n->args;
    switch (n->op) {                        // lazy forms drive their own args
        case NqpOpc::While:
            while (boolify(eval(a[0].get())))
                for (size_t i = 1; i < a.size(); i++) eval(a[i].get());
            return Value::nil();
        default: break;
```

```
    }
    ValueList v;                            // everything else: eval once
    for (auto& e : a) v.push_back(eval(e.get()));
    auto I = [&](size_t i){ return v[i].toInt(); };
    switch (n->op) {
        case NqpOpc::AddI:  return Value::integer(I(0) + I(1));   // int64
        case NqpOpc::IseqI: return Value::integer(I(0) == I(1));
        // ... about 40 more leaf ops ...
    }
}
```

## 35.5 The argument buffers

An nqp-heavy program is nqp-heavy in the extreme: a tokenizer written in Raku runs about 1.5 million operations on a 278 KB document. Building a fresh `ValueList` per operation was a malloc and a free per op, for a list of one to four values.

(Those are now exactly the sizes `ValueList` keeps on a free list — see Chapter 12 — so the same op costs a pop and a push instead. The buffers below still pay, because they also save the clear and refill, and because the re-entrancy argument that picks the container has nothing to do with allocation. But this section was written when the allocator was the whole cost, and it no longer is.)

```
// src/Interpreter.h — ExecContext
std::deque<ValueList> nqpArgs;   // one reusable buffer per nesting depth
size_t nqpDepth = 0;
```

The buffers are kept, so their capacity is kept; the contents are still cleared on the way out, so argument lifetimes are exactly what they were.

It is a **deque**, not a vector, and the comment says why: an argument's own evaluation can re-enter `evalNqpOp`, and growing a vector would reallocate under the outer frame's live reference. A deque never moves the elements it already holds.

That is a small illustration of a recurring theme — the reusable-buffer optimisation is easy, and the container choice that makes it *correct* under re-entrancy is the part that takes thought.

## 35.6 Native compilation

The subset is a first-class part of the transpiler, not just the interpreter. A `use nqp` program native-compiles like any other; it does **not** fall back to bundling.

```
$ rakupp --cpp -e 'use nqp; say nqp::add_i(1, 2)'
...
rtCallB(RT, __bfp0, "say", ValueList{
    ([&]()->Value{ ValueList __na = ValueList{Value::integer(1LL),
                                              Value::integer(2LL)};
                   return rtNqpOp(NqpOpc(10), __na); }())});
```

Two emission strategies, mirroring how the ops evaluate:

- **Eager leaf ops** share their implementation with the interpreter through a free function `rtNqpOp(NqpOpc, ValueList&)` in the runtime. `Interpreter::evalNqpOp` delegates to it and the generator emits a call to the *same* function, so there is exactly one copy of the op logic and the compiled binary cannot drift from the interpreted meaning.
- **Lazy control forms** do not route through a runtime call — they emit native C++ directly, a real `while` and a statement sequence, preserving their non-eager evaluation.

The zero-cost guarantee extends here too: a program without `use nqp` emits zero references to `rtNqpOp` or any nqp machinery.

## 35.7 What is covered

Ninety-nine opcodes, reached under 112 spellings, chosen from the actual inventory of the modules in the ecosystem compatibility battery. The tables below are the shape of the coverage rather than the whole list — read `NqpOpc` in `src/Ast.h` for that:

| Group | Ops |
|---|---|
| **Control (lazy)** | `if unless while until stmts ifnull` |
| **Integer** (int64, no bignum promotion) | `iseq_i isne_i islt_i isle_i isge_i isgt_i add_i sub_i mul_i bitand_i` |
| **String** (codepoint-indexed) | `ordat eqat substr chars concat join index chr strfromcodes strtocodes findnotcclass iscclass` |

| Group | Ops |
|---|---|
| **List** | `list` `list_i` `list_s` `elems` `atpos` `atpos_i` `bindpos` `push` `push_i` `push_s` `pop_s` `shift_i` `splice` |
| **Hash** | `hash` `bindkey` |
| **Object / attr** | `create` `istype` `getattr` `bindattr` `p6bindattrinvres` `null` `isnanorinf` |
| **Constants** | `const::CCLASS_*`, `const::NORMALIZE_*` |

Integer ops use plain `int64` semantics, matching NQP's native integers — **no overflow to bignum** — and string ops index by codepoint rather than by grapheme. Both of those are deliberate: an NQP op is supposed to be the low-level primitive, and a module reaching for one is reaching past Raku's semantics on purpose.

## 35.8 Scope and non-goals

- **Not a general NQP runtime.** This is a compatibility shim sized to the ecosystem. Operations nobody's modules use are not implemented until a module needs them.
- **Semantics match Rakudo's observable behaviour, not its internal representation.** `nqp::create(IterationBuffer)` gives back a plain Raku++ buffer, because what the calling module does with it is all that matters.
- **`use nqp` is an opt-in for module code**, not an invitation for application code. There is no reason to reach for `nqp::` operations in a program you are writing, and no support commitment for operations beyond the measured subset.

## 35.9 Why this shape is the right one

The temptation with a compatibility layer is to implement the source language. That would have meant an NQP grammar, an NQP compiler, and a second execution model to keep in step with the first — for a feature whose entire purpose is to let a handful of modules load.

Recognising that `nqp::foo(...)` is *already* valid syntax in the host language turned a compiler project into about a thousand lines of parser branch and evaluator arm. The general principle generalises: **before implementing a foreign language, check whether its surface syntax is already legal in yours.**

# Chapter 36

# NativeCall and the libffi Backend

Sooner or later a program needs something the language does not have: a system call, a compression library, a database client. Raku's answer is to let a program declare the C function's signature in Raku and then call it as if it were an ordinary sub.

```
use NativeCall;
sub strlen(Str --> size_t) is native {*}
sub hypot(num64, num64 --> num64) is native {*}

say strlen("hello");     # 5
say hypot(3e0, 4e0);     # 5
```

`is native` calls a C function directly. This chapter is about how that works inside, which is a more interesting story than the syntax: Raku++ ships as one portable binary with no third-party dependencies, and it would like to keep that property — so it does not link libffi and does not vendor it.

## 36.1 libffi is loaded at run time, and its ABI is restated by hand

```
// src/Ffi.h — the header comment, condensed
// Raku++ ships as ONE portable binary with no third-party dependencies, so
// it does not link libffi and does not vendor it. Instead the library is
// loaded at RUNTIME with dlopen ... and the handful of ABI declarations we
// need are restated here.
```

Two things make hand-declaring another project's ABI safe, and they are worth stating because the technique generalises.

**We never touch `ffi_cif`'s fields.** Its size and layout are target-dependent, so the code hands libffi an over-sized zeroed buffer and only ever passes the pointer back:

```
// src/Ffi.h
struct Cif { alignas(16) unsigned char buf[512] = {}; };
```

512 bytes is far past any target's real size. `ffi_type`'s layout, by contrast, *is* stable public ABI — callers have always had to build struct types by hand — so it is declared normally:

```
// src/Ffi.h
struct Type {
    size_t         size;
    unsigned short alignment;
    unsigned short type;
    Type**         elements;   // null-terminated, for T_STRUCT
};
```

**The calling-convention constant is probed, not tabulated.** `FFI_DEFAULT_ABI` is an enum whose value differs per target *and* per libffi release. On the machine this was developed on, the arm64 library wanted one value while the x86-64 build of the same program wanted another — neither matching what current libffi headers imply.

So the loader tries candidates until one passes a **self-test of real calls** — `strlen`, `abs`, `ldexp`, `ldexpf` — and disables the backend entirely if none does:

```
// src/Ffi.h
struct Lib {
    bool ok = false;
    std::string path, why;
    int abi = 0;                 // FFI_DEFAULT_ABI for this target, probed
    int (*prep)(void*, int, unsigned, Type*, Type**);
    int (*prep_var)(void*, int, unsigned, unsigned, Type*, Type**);
    void (*call)(void*, void (*)(void), void*, void**);
    void* (*closure_alloc)(size_t, void**);
    int (*prep_closure_loc)(void*, void*,
                            void (*)(void*, void*, void**, void*),
                            void*, void*);
    Type *t_void, *t_uint8, /* … */ *t_pointer;
};
const Lib& lib();          // loads and self-tests on first call; never retries
```

The rule that makes this defensible: **a failed probe disables the backend rather than guessing.** Miscalling through a wrong-shaped interface would produce plausible wrong answers, which is the worst possible failure mode for an FFI.

```
rakupp --ffi-info      # libffi: libffi.dylib (abi 1)
```

## 36.2 What happens with no libffi

Not everything keeps working, and being clear-eyed about that is part of the design. The fallback is a **narrower** FFI, not a transparent substitute.

Calls go through a fixed prototype that hands eight integer and eight floating-point arguments to the platform ABI and lets it place them. That is correct for a large majority of real C signatures: pointers, `Str`, integers of every width, `num64`, `CArray`, struct and union handles, `is rw` out-parameters, `nativecast`, `cglobal` and synchronous callbacks all behave exactly as they do with libffi.

Four things it cannot do, and each **throws at the point of the call** rather than computing a wrong answer:

| | |
|---|---|
| a `num32` argument or return | a real C `float` cannot be placed by the fixed prototype |
| a variadic signature | needs `ffi_prep_cif_var` to say where `...` begins |
| more than 8 integer or 8 float arguments | the prototype is that wide and no wider |
| more than 64 distinct callbacks | the trampoline pool holds 64 |

`RAKUPP_FFI=0` forces this path, and the fallback is tested through it: the regression test `t/regression/nativecall-libffi.raku` re-runs the cases only libffi can do in a child process with the backend switched off, and checks each one throws rather than answering. WebAssembly takes the fallback by construction — there is no shared library to open.

## 36.3 How a call is made

1. **Resolve.** The library is `dlopen`ed — the name as written, then the platform-decorated forms — and the symbol `dlsym`ed.
2. **Marshal.** Each argument is converted once, at its *declared* width, into a slot libffi reads directly.

3. **Describe.** The signature becomes a call interface, prepared once per sub — with the variadic form when the signature is variadic.
4. **Call**, then box the return: a pointer becomes a live `Pointer` or `CArray`, a `Str` return is read as a C string, a narrow integer is truncated and sign-extended, and `is rw` out-parameters and mutated buffers are copied back.

Steps 1 and 3 are cached **on the `Callable`**, and the reason is measured:

```
// src/Value.h — Callable, the native fields
bool isNative = false;
std::string nativeLib, nativeSym, nativeLibSub;
const Expr* nativeLibExpr = nullptr;
void* nativeSymCache = nullptr;   // dlopen/dlsym once, not per call:
                                  // 5 dlopen candidates cost a flat ~67 µs
CifSlot nativeCifCache;           // ffi_prep_cif is ~80 ns — 20% of a whole
                                  // crossing — so it must not run per call.
                                  // A struct, not a bare pointer: its copy
                                  // constructor is deliberately empty, so a
                                  // copied Value starts with no cif rather
                                  // than a second owner of the same one.
```

The cost of going through libffi rather than calling blind is about **20 nanoseconds per crossing** — 106 milliseconds against 100 for 300,000 calls of `abs`, where the bare loop is 24. On a crossing that costs about 275 nanoseconds end to end, that is under a tenth, which is why there is one code path rather than a fast one and a general one.

`nativeLibExpr` handles a genuinely awkward case: `is native(EXPR)` where the expression could not be evaluated at declaration time. It is retried once at the first call, and the raw `Expr*` is safe to keep because the AST outlives the interpreter.

## 36.4 Variadics

Variadic C functions are the easiest thing in an FFI to get quietly wrong.

C's `...` is not a formality. On the SysV AMD64 ABI a variadic argument goes in a register but the callee is told how many; on the Apple ARM64 ABI it goes on the **stack**, while a fixed argument in the same position would go in a register. A caller that does not know which arguments are variadic cannot place them, and the failure is silent — you get a number, it is just the wrong one.

Raku++ spells it with a **slurpy marking the position where C's `...` begins**, which is the same spelling Rakudo uses:

```
use NativeCall;
sub snprintf(Buf, size_t, Str, *@args --> int32) is native {*}

my $buf = buf8.allocate(64);
my $n = snprintf($buf, 64, "%s scored %d at %.1f%%", "Ada", 97, 99.5e0);
say $buf.decode('latin-1').substr(0, $n);   # Ada scored 97 at 99.5%
```

Everything beyond the declared parameters is variadic and is typed from the value itself, following C's **default argument promotions**:

| You pass | C receives |
|---|---|
| `Int` | a 64-bit integer |
| `Num` / `Rat` | `double` — never a bare `float` |
| `Str` | `char*` |
| `Buf` / `Pointer` / `CArray` | the pointer |

That promotion table is why `%.1f` works without saying anything: a float argument to a variadic function *is* a `double` in C.

Until this was fixed, Raku++ placed `...` arguments in registers and answered `"0"` for `snprintf($buf, 16, "%d", 42)`. It is a parity fix, not an extension — and it is what makes `curl_easy_setopt`, the `printf` family, `open(2)` with a mode, and `ioctl` reachable at all.

## 36.5 The type map

| Raku | C | Bytes |
|---|---|---|
| `int8` `uint8` `byte` | `int8_t` `uint8_t` | 1 |
| `bool` | `bool` | 1 |
| `int16` `uint16` | `int16_t` `uint16_t` | 2 |
| `int32` `uint32` | `int32_t` `uint32_t` | 4 |
| `int` `int64` `long` `size_t` | 64-bit integer | 8 |
| `num32` | `float` | 4 |
| `num` `num64` | `double` | 8 |
| `Str` | `char*` | pointer |
| `Buf` / `Blob` | the bytes, copied back after the call | pointer |
| `Pointer[T]` `CArray[T]` | the pointer | pointer |

| Raku | C | Bytes |
|---|---|---|
| a `repr('CStruct')` / `CUnion` class | a pointer to it | pointer |
| `&callback (...)` | a C function pointer | pointer |

The table lives in `InterpreterBinding.cpp` and `ffi::scalar(width, sign, isFloat)` maps into it, deliberately keeping the Raku type names in one place.

Structs are laid out with natural alignment; `.new` allocates zeroed native memory, and field reads and writes go to that memory, so a C function can fill one in. `nativesizeof` answers from the same layout code. A member declared with `HAS` is inlined rather than pointed at — it contributes the inner struct's own width and alignment, and its accessor hands back a view onto the outer struct's bytes rather than dereferencing a pointer.

```
// src/Interpreter.h — the pointer helpers
Value ncMakePointer(const std::string& type, void* p);
Value ncMakeLiveCArray(const std::string& type, void* p);
static Value ncReadElem(long long addr, const std::string& ofType, long long i);
static void ncWriteElem(long long addr, const std::string& ofType,
                        long long i, const Value& val);
static long long ncFieldOffset(ClassInfo* ci, const std::string& field,
                               std::string& type);
static long long ncStructSize(ClassInfo* ci);
static long long ncStructAlign(ClassInfo* ci);
```

## 36.6 Callbacks

A `Callable` handed to C becomes a real C function pointer built with an `ffi_closure`. Parameters are typed from the block's own signature:

```
// src/Interpreter.h
long runCallback(int slot, long a0, long a1, long a2,
                 long a3, long a4, long a5);
void runFfiClosure(void* closure, void* ret, void** args);
bool onRakuThread() const { return (bool)tctx_.cur; }
```

The last one is the guard that matters. A callback must fire **during** the native call — `qsort`, `bsearch`, an iteration callback. A callback that C stores and fires later from a thread of its own has no Raku scope to run in, so `onRakuThread()` detects it and the callback is ignored with a warning rather than run against a thread the interpreter never entered.

Note what that predicate actually tests: whether this thread has a current lexical scope. A thread the C library made for itself has none.

## 36.7 Your declaration is a promise nothing can check

A C library exports addresses, not types. Nothing — not Raku++, not Rakudo, not any FFI in any language — can compare a declaration against the real prototype, because that information does not exist at run time. A wrong declaration does not fail; it produces a plausible answer.

```
sub abs(num64 --> num64) is native {*}   # WRONG — C's abs is int abs(int)
say abs(-3.34e0);   # -3.34
say abs(-99.5e0);   # -99.5
```

Every answer is the argument, unchanged, and that is the tell. libffi put the `num64` in the first *floating-point* register, but `int abs(int)` reads the first *integer* register and returns an integer in the integer return register. The floating-point return register is never written, so it still holds what was passed. The call really happened; the wrong question was asked.

Rakudo prints exactly the same three lines. This is not an implementation quirk to work around — it is what an FFI is.

## 36.8 Tracing

```
RAKUPP_FFI_TRACE=1
```

```
[ffi] backend: libffi: libffi.dylib (abi 1)
[ffi] snprintf(0x156f1c9a0, 64, "%s scored %d at %.1f%%", "Ada", 97, 99.5) -
> 22
[ffi] sqlite3:sqlite3_libversion() -> "3.43.2"
```

It shows the resolved C symbol rather than the Raku sub name, and — importantly — **what actually crossed**: the marshalled argument values and the raw return, not the Raku values re-printed. So a marshalling bug shows up in the trace instead of being hidden by it.

An external tracer cannot do this job. `lldb`, `ltrace` and `dtrace` see C symbols, and the interpreter's own C++ calls `strlen` and `malloc` constantly, so a breakpoint on `strlen` cannot distinguish a crossing the program asked for from one the runtime made for itself. Only this layer knows which call came from Raku.

## 36.9 In compiled code

`--exe` emits the same `callNative`, so `is native` behaves identically interpreted and compiled — and a compiled binary looks for libffi at *its* run time, not at build time.

A few shapes make `--exe` fall back to bundling: `is native(&lib-sub)`, a library name computed by an expression, `is rw` out-parameters, and buffer or `CArray` parameters needing copy-back. The compiler says so and produces a bundled binary instead.

## 36.10 Not supported

- **C structs passed or returned by value.** Rakudo cannot express this either — its NativeCall maps a struct to one type code with no by-value variant — so adding it here would mean inventing syntax rather than matching an existing spelling. A return of up to 8 bytes can be declared `int64` and unpacked by hand.
- **Callbacks fired from a thread the C library owns**, as above.

# Chapter 37

# The Extension ABI

Raku++ walks an AST. It does not JIT, so a tight loop written in Raku costs it roughly an order of magnitude more than it costs Rakudo — and some jobs, like tokenizing a 300 KB JSON document, are nothing *but* a tight loop.

An extension module is the escape hatch. A distribution ships C source alongside its Raku, the build step compiles it against a published ABI at install time, and the compiled routines become ordinary Raku subs. Perl calls this XS; Python calls it a C extension. The point is the same, and so is the most important property: **the module versions independently of the compiler.** A faster JSON parser should not require a new release of the language implementation.

## 37.1 Extension or NativeCall?

They solve different problems and the wrong choice is painful.

| | NativeCall | extension module |
|---|---|---|
| calls | an **existing** C library you did not write | C **you** write for this module |
| data | C types: ints, nums, pointers, structs | Raku values: Hash, Array, Int, Rat, Str |
| build | none; the shared object already exists | compiled at install time |
| use when | wrapping libcurl, sqlite3, libgit2 | a hot loop in your own module is the bottleneck |

If you need to return a *Hash of Rats*, NativeCall cannot express it and an extension can. If you need to call `curl_easy_perform`, use NativeCall.

## 37.2 The one rule

**An extension never sees `Value`.** Every Raku value crosses the boundary as an opaque handle.

```
/* include/rakupp/rakupp_ext.h */
typedef struct RkValueOpaque* RkValue;
typedef struct RkCtxOpaque*   RkCtx;
```

This is not fastidiousness. `sizeof(Value)` moved from 392 to 376 to 344 bytes in a single afternoon of ordinary optimisation work, and the representation campaign later halved it twice more — 208, then 128 (Chapter 41). An ABI that exposed the struct would have to freeze the interpreter's internals forever, or silently miscompile every extension built against an older header — the failure mode where a module reads a `Str` out of a field that is now an `Int` and nothing crashes until much later.

Handles cost an indirection. They buy the ability to install a module today and keep it working across compiler releases, which is the entire point.

The corollary: **this is plain C.** No C++ types cross the boundary, no exceptions cross it, and nothing received needs freeing. A binding from Rust or Zig would need nothing beyond the header.

## 37.3 Hello, world

```
#include <rakupp/rakupp_ext.h>

static RkValue answer(RkCtx c) { return rk_int(c, 42); }

static const RkSubDef subs[] = { {"answer", answer}, {0, 0} };
static const RkModule mod = { RAKUPP_EXT_ABI, "Hello::Ext", subs };

RAKUPP_EXT_EXPORT const RkModule* rakupp_ext_init(unsigned host_abi) {
    return host_abi >= RAKUPP_EXT_ABI ? &mod : 0;   // see Versioning, below
}
```

```
use Rakupp::Ext;
rakupp-ext-load('./libhello.dylib');
say answer();      # 42
```

`rakupp-ext-load` installs each of the extension's subs into the **calling scope**, so inside a module they land exactly where an `our sub` would, and that module's `is export` carries them onward.

## 37.4 Both halves of the linking bargain

An extension resolves the `rk_*` symbols from the **host executable** at load time, exactly as a Python C extension resolves `Py_*`. Undefined symbols at *link* time are therefore expected, and must be permitted:

| platform | flag |
|---|---|
| macOS | `-Wl,-undefined,dynamic_lookup` |
| Linux / BSD | none — ELF resolves lazily by default |
| Windows | link against the import library, `lib/rakupp.lib` |

That is the extension's half. The host's half is to actually **publish** those symbols, and for a long time it did not. This was checked rather than assumed, and the answer was worse than expected:

> **Extensions had only ever worked on macOS.** A Mach-O executable exports its globals by default. A plain ELF executable keeps its symbols out of `.dynsym`, so on Linux an extension's first `rk_*` call had always died with an undefined-symbol error. On Windows, the documentation told authors to link against an import library the build never produced.

Nobody had reported it, and the reason is the very pattern this chapter recommends. A well-written module falls back quietly — `JSON::Native`'s whole design is to try the extension and drop back to Raku if it does not load. So on Linux it loaded nothing, ran its fallback, and looked like it was working. Users would have seen the pure-Raku timings and assumed that was the number.

The fix is deliberately narrow: `-Wl,--dynamic-list=…` puts the `rk_*` glob in the executable's dynamic symbol table **and nothing else**, where `-rdynamic` would have exported every C++ symbol in the interpreter as an accidental ABI promise. On

Windows, `RK_API`'s `dllexport` plus `ENABLE_EXPORTS` produce the import library the docs already described.

The lesson generalises past this bug: **a graceful fallback hides the failure it was written to survive.** A module that degrades quietly needs some way to say out loud which path it took, and a build that publishes an ABI needs a test that the symbols are really there rather than a belief that they are.

### 37.4.1 `librakupp` and the export discipline

The same surface is exported by a shared library, built with `-DRAKUPP_BUILD_SHARED=ON`: position-independent, hidden visibility, versioned, installed beside the static archive. It is off by default, because it recompiles every runtime source and the plain CLI neither links nor loads it.

The export list is **the header itself** rather than a second file to keep in sync. Each entry point carries an `RK_API` marker, the library compiles with `-fvisibility=hidden`, and what is marked is exactly what is exported — verified with `nm` rather than assumed: every `rk_*` symbol present, and no `namespace rakupp` symbol leaked.

That is what keeps the boundary honest in the direction that matters most. A C++ symbol accidentally exported from a shared library is an ABI promise nobody decided to make, and the only defence is to keep the visible set small and declare it in one place.

## 37.5 The value vocabulary

```
/* constructing */
RkValue rk_any  (RkCtx c);
RkValue rk_bool (RkCtx c, int truthy);
RkValue rk_int  (RkCtx c, long long v);
RkValue rk_int_s(RkCtx c, const char* decimal);       /* arbitrary precision */
RkValue rk_num  (RkCtx c, double v);
RkValue rk_rat_s(RkCtx c, const char* n, const char* d); /* a Rat, normalised */
RkValue rk_str  (RkCtx c, const char* utf8, size_t len); /* copied, DECODED as UTF-8 */
RkValue rk_blob (RkCtx c, const void* bytes, size_t len); /* raw bytes, ABI 3 */
RkValue rk_array(RkCtx c);   void rk_push(RkCtx c, RkValue a, RkValue v);
RkValue rk_hash (RkCtx c);   void rk_set (RkCtx c, RkValue h,
                                         const char* k, size_t kl, RkValue v);
```

```
void    rk_list (RkCtx c, RkValue array);   /* mark it a List, not an Array */
void    rk_map  (RkCtx c, RkValue hash);    /* mark it a Map,  not a Hash  */
```

`rk_int_s` and `rk_rat_s` take **decimal strings** rather than integers on purpose: Raku's `Int` has no width, and a document may carry a 40-digit number that no C integer type can hold. `rk_rat_s("1", "7")` gives exactly one seventh, not `0.14285714285714285`.

```
/* inspecting */
RkType      rk_type   (RkCtx c, RkValue v);
long long   rk_int_get(RkCtx c, RkValue v);
const char* rk_str_get(RkCtx c, RkValue v, size_t* len);   /* borrowed */
size_t      rk_elems  (RkCtx c, RkValue v);
RkValue     rk_at_pos (RkCtx c, RkValue array, size_t i);
const char* rk_key_at (RkCtx c, RkValue hash, size_t i, size_t* keylen);
RkValue     rk_val_at (RkCtx c, RkValue hash, size_t i);
```

`RkType` is deliberately coarse:

```
typedef enum {
    RK_ANY = 0, RK_BOOL, RK_INT, RK_NUM, RK_RAT, RK_STR,
    RK_ARRAY, RK_HASH, RK_OTHER
} RkType;
```

It describes **what an extension can do with a value**, not where the value sits in Raku's type hierarchy. Anything the ABI has no vocabulary for arrives as `RK_OTHER`, and the advice is to stringify it. A coarse enum is a deliberate choice: a fine one would have to track the language's type system, which is exactly the coupling this design exists to avoid.

Coarse cuts one way only. A `Date` is a hash of year, month and day inside the engine, an enum value is an `Int`, a `Buf` is a `Str` — and none of them arrive as that carrier. They are `RK_OTHER`, and `rk_str_get` gives their `Str` (`"2026-08-10"`, `Green`), because an extension that was told `RK_HASH` would serialise the date's fields, and one told `RK_INT` would write the enum's ordinal. JSON::Native's C writer did exactly the first of those until the mapping was fixed. The one tagged value that is what it looks like is a `Map`, which walks as `RK_HASH`.

```
/* arguments */
size_t  rk_argc (RkCtx c);                    /* positionals only */
RkValue rk_arg  (RkCtx c, size_t i);          /* NULL if absent */
RkValue rk_named(RkCtx c, const char* name);  /* NULL if not passed */
```

`rk_named` returning `NULL` means *not passed*, which is distinct from being passed an undefined value — so `:immutable` and `:!immutable` remain distinguishable.

## 37.6 Calling back into Raku

ABI 2 added the other direction: an extension can invoke Raku.

```
RkValue rk_call      (RkCtx c, const char* name,
                      const RkValue* argv, size_t argc);
RkValue rk_call_value(RkCtx c, RkValue code,
                      const RkValue* argv, size_t argc);
int     rk_can       (RkCtx c, const char* name);
```

`rk_call` resolves a name **the way the extension's own sub would at its call site**: the lexical scope that invoked it, then outward to global. So an extension loaded by a module reaches that module's subs — which is what lets a native fast path delegate the cases it does not handle back to the Raku it replaced. `rk_call_value` is the same for a `Code` value that was handed in, a callback rather than a name. `rk_can` asks first, so an extension can use an optional hook only when the caller supplied one.

The failure model is the interesting part, and it follows from the one rule at the top of this chapter. **A Raku exception thrown inside `rk_call` does not unwind through the extension's frames.** It cannot — the host did not compile them. Instead the call returns `NULL` and leaves a *pending error*:

```
void        rk_die        (RkCtx c, const char* message);
const char* rk_error      (RkCtx c);
void        rk_clear_error(RkCtx c);
```

Return `NULL` from the sub afterwards and the original Raku exception is raised at the call site with its own type intact. Or call `rk_clear_error` and carry on, which is how an extension treats a failed callback as one case among several rather than as a fatality.

That shape — a return value plus a pending error, never a longjmp through foreign frames — is the same discipline `errno` and OpenGL use, and it is forced here by the same thing that forces the handles: neither side compiled the other.

## 37.7 Rooted handles, and the one way to leak

```
RkValue rk_root  (RkCtx c, RkValue v);
void    rk_unroot(RkCtx c, RkValue rooted);
```

`rk_root` lifts a value out of the arena so it can outlive the call. It is **for hosts embedding rakupp, not for extensions**, and it is the only thing in the header you can leak — which is precisely why the arena, where you cannot, is what an extension sees by default.

The guidance in the header is blunt about it: if you think you want a rooted handle, you almost certainly want ordinary C state instead — a compiled pattern, a parsed schema, a cache. Keep those in C.

## 37.8 Lifetime: handles belong to the call

Everything an extension creates is allocated in the context's **arena** and released when the sub returns; the single value it returns is copied out first.

So an extension never frees anything, cannot leak, and — the rule that has to be stated because it is invisible until it bites — **a handle stored in a global and used on the next call is dangling.** There is no reference counting to save you. State that must persist across calls belongs in C, not in handles.

Borrowed pointers from `rk_str_get` and `rk_key_at` are valid until the call returns, which is long enough to copy out of them and no longer. `rk_at_pos` is sharper than that: its handle points into the array's own storage, so a later `rk_push` on the same array can move it *within a single call*. Read it before you grow the array, or copy the value out. (`rk_val_at`'s handle is stable, because the hash keeps node addresses.)

The arena is what makes the boundary safe in both directions: the host cannot be made to leak by a careless extension, and an extension cannot hold a reference into a value model that is free to change underneath it.

## 37.9 Errors

```
void rk_die(RkCtx c, const char* message);
```

Call it and **return `NULL`**. The host raises a Raku exception at the call site, catchable with `try`/`CATCH` like any other. A second `rk_die` keeps the first message, so an error deep in a recursive parse survives the unwind.

Never throw a C++ exception across the boundary. The host did not compile the extension's code and cannot catch it.

## 37.10 Versioning

```
#define RAKUPP_EXT_ABI 3u
typedef const RkModule* (*RkInitFn)(unsigned host_abi);
```

One integer, bumped when the header gains capability an extension cannot detect any other way, or when the meaning or order of anything in it changes. Version 1 was the original surface — construct, inspect, arguments, `rk_die`. Version 2 added re-entering Raku, the pending-error calls, and rooted handles. Version 3 added raw bytes in both directions: `rk_blob` builds a `Buf` from bytes, `rk_blob_get` reads one back, and `rk_is_blob` asks. Before it, a digest or a compressed stream had no way home — see the note under `rk_str` above for why the string constructor is not that way.

The host looks up one symbol, `rakupp_ext_init`, and passes its own version. **Return `NULL` if you cannot serve it** rather than guessing; the host then reports a clean mismatch instead of calling through a wrong-shaped table.

### 37.10.1 The handshake, and why it is a downgrade

Bumping the number would have broken every extension already in the wild, because ABI-1 extensions were all written with the equality test the original documentation showed:

```
return host_abi == RAKUPP_EXT_ABI ? &mod : 0;
```

A host at 2 calling that gets `NULL`. So **the host retries downward**: `init(2)` returning null is followed by `init(1)`, and the old extension answers. Nothing that worked stops working, and it was proven rather than argued — an extension built against the previous day's header was loaded, unchanged, into the new host.

An extension built against the *new* header should use `>=`, which states what it actually needs: a host at least this new. And a module whose `abi_version` is **newer**

than the host's is refused with a sentence saying so, because the host cannot provide what it has not got.

The failure this avoids is undefined symbols. An extension calling `rk_call` on a host that predates it would resolve nothing and abort at the first call under lazy binding; a version check turns a crash into a diagnostic.

All four failure modes report themselves and none corrupt:

```
'…/thing.dylib' was built for a different extension ABI (host is 3)
'…/thing.dylib' reports ABI 99, host is 3 (rebuild it, or upgrade rakupp)
cannot load extension '/nope.dylib'
'/bin/ls' is not a rakupp extension (no rakupp_ext_init)
```

The host side is one function:

```
// src/Interpreter.h
Value extLoadModule(const std::string& path, std::string& errOut,
                    std::vector<std::pair<std::string, Value>>& subsOut);
```

which `dlopen`s the path, checks the ABI, and hands back the subs it declares.

## 37.11 Writing a module that also runs on Rakudo

A module that *only* works on Raku++ is a poor citizen. The pattern that works everywhere is a compiled fast path with a pure-Raku fallback, chosen at load time — and the load-bearing detail is how the loader is reached:

```
my &ext-load = try &::('rakupp-ext-load');
```

**Why not call `rakupp-ext-load(...)` directly?** Because Raku resolves sub names at *compile* time. On Rakudo the name does not exist, so the file fails to compile and the fallback never gets the chance to run.

`&::('...')` is a **symbolic lookup**: the name is a string, resolved at run time. The line is valid Raku either way, so the *same source file* compiles on both engines and simply finds nothing on Rakudo.

```
unit module My::Thing;

my &ext-load = try &::('rakupp-ext-load');
my &fast;

sub load-native(--> Bool) {
    return False unless &ext-load;
```

```
    for libraries() -> $lib {
        next unless $lib.IO.e;
        next unless try ext-load($lib.Str);
        &fast = try &::('my-thing-native');
        return True if &fast;
    }
    False
}
my Bool $native = load-native();

our sub do-the-thing($x) is export {
    $native ?? fast($x) !! pure-raku-version($x)
}
```

`use Rakupp::Ext` is accepted too and is the discoverable spelling for code that is Raku++-only by design — but it is a `use` statement, so writing it makes the file uncompilable on Rakudo.

Note that `&fast` and `&ext-load` keep their sigil at every mention. Writing `fast.defined` would *call* `fast` and ask the result, producing a baffling `No such method 'CALL-ME'`.

## 37.12 Packaging

```
My-Thing/
├── META6.json          "resources": [ "libraries/thing" ]
├── Build.rakumod       compiles src/ at install time
├── src/thing.c
├── lib/My/Thing.rakumod
└── resources/libraries/
```

Raku maps the logical resource name to the platform spelling, so **the build must emit `libthing.dylib`, not `thing.dylib`**, or `%?RESOURCES` will not find it.

`Build.rakumod` should **never abort an install**. If the compiler is missing, or the headers are not there, or the engine is Rakudo, warn and return true: the module then runs on its fallback. A build hook that fails turns an optimisation into a hard dependency.

Find the library through `%?RESOURCES`, falling back to a working-directory path. **Do not derive it from `$?FILE`** — under Raku++ a module's `$?FILE` is the *main program's* path, not the module's, so anything computed from it lands somewhere unrelated. That is a recorded bug, not a design.

## 37.13 The naming convention

A module whose behaviour depends on Raku++ should say so in its name. `JSON::Native` tells a reader at the point of `use` that this is not a portable choice — better than a footnote in a README, and better than discovering it when the deploy target turns out to be Rakudo.

Two rules follow. A `Rakupp::` module is still a normal distribution: installed by zef, carrying a `META6.json`, degrading gracefully elsewhere. And **the compiler must not special-case a module name.** An early draft of `JSON::Native` was built into the interpreter, and `use JSON::Native` intercepted the name before module loading — so the real distribution's tests silently exercised the built-in copy instead of the extension they were written for. If a name can be a module, it must *only* be a module.

## 37.14 Current limits

- **There is a per-value cost.** Each value crosses as an arena-allocated handle and is then copied into its container — roughly one extra `Value` copy per node. Measured on `JSON::Native`: 5.7 ms against 2.7 ms for the same parser compiled into the interpreter. Still 6.6 times faster than Rakudo on that workload, but it means an extension is worth it for **bulk** work, not for a routine called once with two integers.
- **Hash iteration is still index-based, but no longer quadratic.** The host remembers where it was between `rk_key_at` and `rk_val_at` calls, so a sequential walk costs O(1) per key. Jumping between two hashes, or writing to one while walking it, falls back to O(i) positioning — correct either way, just slower.
- **No `--exe` bundling.** A program using a native extension cannot be compiled into a standalone binary; the shared library is loaded at run time and is not part of the module graph the bundler walks.
- **One context per call, single-threaded within it.** Handles are not safe to pass between threads.

## 37.15 The other direction

Extensions are **native code called from Raku**. Embedding is **Raku called from native code**. Same boundary, opposite direction — and the observation that shaped the embedding API is that the harder half was already built:

> The extension ABI has the value vocabulary, the opaque-handle discipline, version negotiation, an error model, and the rule that makes all of it survivable.

So embedding did not get a second value vocabulary. `RkValue` is the value type, `rk_int/rk_str/rk_hash/rk_at_pos` are the accessors, and an earlier proposal for a separate opaque type with its own free function was retracted on exactly that ground.

That is worth stating as a design principle rather than a project note: **when you publish a boundary, publish one.** Two vocabularies for the same values is twice the surface to keep stable, and the whole reason for the handle discipline was to have as little of that surface as possible.

### 37.15.1 What a host actually calls

`librakupp` is the shared library an embedding host links against, and `rk_root` is how it keeps a value past the call that produced it — the two things an extension never needs and an embedder cannot do without. On top of those, `EmbedApi.cpp` publishes a small lifecycle:

```
RkInterp   rk_new (const RkConfig* cfg);   /* NULL = every option off */
void       rk_free(RkInterp rk);
int        rk_eval(RkInterp rk, const char* src, RkValue* out);
int        rk_run (RkInterp rk, const char* src, const char* file,
                   int* exit_code);
const char* rk_last_error(RkInterp rk);
void       rk_set_output(RkInterp rk, RkOutputFn fn, void* userdata);
void       rk_set_input (RkInterp rk, const char* text, size_t len);
int        rk_register(RkInterp rk, const char* name, RkHostFn fn, void* ud);
```

An interpreter is a **session**, not a one-shot: `rk_eval(rk, "my $x = 41")` then `rk_eval(rk, "$x + 1")` answers 42, because the mainline scope persists exactly as it does at the REPL prompt. `rk_run` is the other shape — a whole program, with its `MAIN`, its phasers and its exit code — and it is the one a host reaches for when it is running someone's script rather than evaluating an expression.

### 37.15.2 Three things the CLI does that a library may not

The most instructive part of the API is a struct of things deliberately turned **off**:

```
typedef struct {
    unsigned size;         /* sizeof(RkConfig), so the struct can grow */
    int own_stack;         /* run Raku on a large-stack thread, as the CLI */
```

```
    int handle_sigpipe;    /* ignore SIGPIPE process-wide */
    int own_stdout;        /* flush std::cout/std::cerr after each evaluation */
} RkConfig;
```

Each is something `rakupp` does as a matter of course and a library must not do to its host uninvited. A thread the host did not ask for, a process-wide signal disposition that is the host's business and not ours, a flush of streams the host owns — all three are correct for a program that *is* the interpreter and presumptuous for one that merely contains it. `size` is how a host compiled against an older header keeps working when the struct grows.

The same restraint decides `declCheck` in Chapter 39's gate: an embedding host's interpreter may already hold globals it installed with `rk_register`, so no static pass over the source it hands in could be trusted to judge it.

One interpreter per process is the standing limit, and `--mcp` (Chapter 39) is the largest host of this ABI in the tree: a thousand lines that include `rakupp.h` and never `Interpreter.h`, despite being compiled into the same binary as the interpreter it serves.

## 37.16 A footnote on guessing

The version-2 work was planned as one feature and turned out to be two, and the way that came out is worth recording.

The plan asserted that `JSON::Native`'s `to-json` was still written in Raku because the ABI had no way to call back into Raku, and that `rk_call` would fix it. **The module's own documentation said otherwise**: the real obstacle was that hash access was index-based, `std::advance` from the beginning on every call, so serialising a hash was quadratic in its size.

Both were worth fixing and neither was the other. But the plan had been repeating a guess for as long as it existed, while the answer was written down in the thing it was guessing about.

It is the same failure as the benchmark in Chapter 28 that measured a rename instead of the change a rename enables, and the same remedy: **before designing around a cause, check whether the code already says what the cause is.**

# Chapter 38

# Concurrency

Raku has a full concurrency surface: `start`, `await`, `Promise`, `Channel`, `Supply`, `react`/`whenever`, `Lock`, atomics, and a scheduler. Raku++ implements it on real OS threads, running interpreter compute on all cores by default, with a **global interpreter lock** still there as an opt-in mode.

This chapter is the story of getting there in stages, because the stages are still visible in the code and each solved a specific class of bug.

## 38.1 Stage 1: per-thread execution registers

The state that belongs to one thread of Raku execution — its current lexical scope, the dynamic-variable chain, the recursion depth, the gather and supply collectors — was originally interpreter members. It is now a struct:

```
// src/Interpreter.h
struct ExecContext { /* Chapter 13 */ };
static thread_local ExecContext tctx_;
void saveCtx(ExecContext& c);
void loadCtx(ExecContext& c);
```

Being `static thread_local`, each real worker thread owns its own set, so interpreter execution needs no per-handover register swap. The save and load functions remain because the design allows a parked thread's registers to be stashed and restored; today they merely shuffle within a thread's own copy.

The same treatment was applied, thread by thread, to everything ThreadSanitizer reported:

```
// src/Interpreter.h
static thread_local Value* topicWriteback_;
static thread_local bool noAutothread_;
static thread_local int loopPhaserCtl_;
static thread_local std::vector<RedispatchCtx> redispatchStack_;
static thread_local std::vector<std::shared_ptr<ReactCtx>> reactStack_;
static thread_local bool t_isWorker;
static thread_local Value t_threadSelf;
```

(Two more live outside this header and are worth knowing about: the call-frame pool, and `RVec`'s free list of small `ValueList` blocks — both per-thread caches of released memory rather than per-thread state, so neither needs a lock and neither is visible to another thread.)

The comment on the call registers records the scale: as plain members they were written by every call on every thread, and were TSan's top report — **2,761 lines on a program with no sharing in it at all**.

One member deliberately stayed shared, and the reasoning is a good example of measuring rather than assuming:

```
// src/Interpreter.h — the current source line, for test diagnostics
// Written on EVERY statement, so a thread_local costs a TLV lookup per
// statement on macOS — measured +6% on loopsum. A relaxed atomic member is
// a plain store, defined under concurrency, and TSan-clean; the value being
// process-wide is the same arbitrariness diagnostics always had.
struct RelaxedLine {
    std::atomic<int> v{0};
    void operator=(int x) { v.store(x, std::memory_order_relaxed); }
    operator int() const { return v.load(std::memory_order_relaxed); }
} curLine_;
```

Thread-local storage is not free on every platform, and a diagnostic field does not need to be exactly right.

That lesson was learned again, expensively, on a field that *does* need to be right. Comparing this tree against the shipped v3.24.0 release found the plain assignment loop and the range-sum loop running 1.4 to 2 per cent *slower* than the release, and a bisect narrowed it to one line added to the `ExprStmt` arm of `exec`:

```
tctx_.curStmtExpr = e;   // which expression this statement is evaluating
```

`tctx_` is thread-local, so on macOS that store is a real call to `_tlv_get_addr` — the second-heaviest symbol in an interpreter profile — and placing it at the top of the arm forces a resolution the rest of the arm would otherwise have folded into one. It costs about 34 instructions **per statement executed**: an empty loop body is unchanged, and every non-empty body moves by the same amount, which is how the cost was pinned to the statement rather than to the operation.

The difference from `curLine_` is that this one cannot simply be made approximate. Cooperative `next`/`last`/`redo` read `curStmtExpr` to tell a bare loop control from one nested inside an expression, so the field has to be exact. The fix is to pay for it less often — set it only for statements whose expression *can* contain a bare loop control, which is a parse-time property of the node — or to hoist the thread-local resolution to the top of `exec`, where the function needs it anyway. Neither is done at the time of writing.

The counterweight is worth stating in the same breath, because it stops this reading as "thread-locals are bad". The `ValueList` free list of Chapter 12 is a *new* `thread_local` on the hottest path in the tree — every interpreted call touches it — and it is one of the largest wins in this book. A thread-local access costs what it costs; what matters is how much work rides on the access. One pointer store per statement is a bad trade. Removing a `malloc` per call is a good one.

## 38.2 Stage 2: the symbol-table freeze

The shared symbol tables — the class registry, the global environment, the named regex table, the loaded-module set, each class's method map — are mutated freely while the program is single-threaded. Once concurrency engages they must be treated as immutable, so worker threads can read them without a lock.

```
// src/Interpreter.h
std::atomic<bool> symbolsFrozen_{false};
void noteSymbolMutation(const char* what);
```

The flag flips when concurrency engages, and `noteSymbolMutation` is a tripwire wired into every structural writer. Under `RAKUPP_FREEZE_TRACE` it reports any post-freeze mutation and which thread did it.

That is the interesting part: the tripwire is **empirical evidence** for whether lock-free reads are safe, rather than an argument that they are. Behaviour is otherwise unchanged, so the instrumentation costs nothing in a normal build.

## 38.3 Stage 3: the GIL

```
// src/Interpreter.h
std::mutex gil_;          // held while running Raku
bool gilHeld_ = false;    // engaged once any thread is spawned
void engageGil();         // lazily lock on first async use
```

Only the holder may touch interpreter state. A thread drops it while blocked — in `await`, in a sleep, around a blocking syscall — so another can run.

The lock is **lazy**: a program that never spawns a thread never engages it and pays nothing.

Three operations manage the handover:

```
// src/Interpreter.h
void gilYieldNotify();              // unlock + bump a counter + notify
void yieldToWorker();               // drop the GIL until a worker progresses
bool yieldToWorkerFor(double secs); // ...bounded
void sleepYield(double secs);       // sleep with the GIL released
bool gilPark();                     // release around a blocking syscall
void gilUnpark(bool wasParked);
```

`gilPark` has a strict contract, stated in the header: the parked window must touch no interpreter state — only thread-local buffers and syscalls. That is what lets a child-process wait release the lock so sibling workers can spawn their own children concurrently.

### 38.3.1 Safe points

A background worker doing pure compute has no I/O to yield at, so the interpreter weaves a cheap check into its hot loop:

```
// src/Interpreter.h
inline void safePoint() {
    if (!t_isWorker) return;                // main-thread loops never park
    if (workerAbort_.load(std::memory_order_relaxed)) throw WorkerAbortEx{};
    if (++t_safePtCtr >= 4096) { t_safePtCtr = 0; workerYield(); }
}
```

Two jobs in four lines. It periodically hands the GIL back, so a compute-bound worker cannot starve the main thread. And it unwinds a worker whose result is no longer wanted, at shutdown, by throwing an exception that is deliberately **not** a `RakuError` — so a user's `CATCH` cannot swallow a shutdown.

The main-thread early return means the check is one predicted branch on a thread-local bool for every loop that is not in a worker.

## 38.4 Stage 3a: true parallelism, and then the flip

Worker threads run interpreter compute concurrently instead of serialising on the GIL — safe once the registers are thread-local and the symbol tables freeze. That arrived as an opt-in, and v3.0.0 made it the default:

```
// src/Interpreter.cpp — the constructor decides the mode
const char* g = std::getenv("RAKUPP_GIL");
const char* p = std::getenv("RAKUPP_PARALLEL");
bool gilWanted = (g && *g && std::string(g) != "0") ||
                 (p && std::string(p) == "0");
parallelMode_ = !gilWanted;
```

`RAKUPP_GIL=1` selects the cooperative GIL — the escape hatch, the bisection tool and a CI leg — and `RAKUPP_PARALLEL=0` is honoured as a synonym for symmetry with the old opt-in spelling. `RAKUPP_PARALLEL=1` does nothing: it asks for what is already true.

The member's own initialiser still reads `bool parallelMode_ = false;`, which is a default the constructor overwrites on the next line. It is worth knowing because it is exactly the sort of line a reader — or a chapter — quotes as if it settled the question.

The few genuinely shared internals a parallel worker can still touch — the test counters and TAP output, the worker vectors — are guarded by one mutex. User data mutated without a `Lock` is the user's race, as it is in Rakudo.

For user containers there is a striped lock, and its design is the interesting part:

```
// src/Interpreter.h
struct ParStripe {
    std::unique_lock<std::recursive_mutex> l;
    ParStripe(const Interpreter& I, const void* p) {
        if (I.parallelMode_ &&
            I.liveWorkers_.load(std::memory_order_relaxed) > 0)
            l = std::unique_lock<std::recursive_mutex>(atomicStripe(p));
    }
};
```

Two conditions, not one. Parallel mode **and** live workers — because before the first spawn and after the last join, a single thread cannot race itself. A single-threaded

program therefore pays two predicted branches and nothing else, default mode or not.

That second condition was not an optimisation for its own sake: the unconditional stripe tax was pushing compute-heavy Roast files past the timeout, in files with zero threads in them. The rule it encodes — **the machinery must be free when one thread runs** — is the gate every stage of this work had to pass.

## 38.5 Threads need big stacks

The tree walker recurses deeply, and the default stack for a non-main thread on macOS is 512 KB — a recursive Raku sub inside `start {...}` overflows it within about a hundred frames, producing a bus error and then a process that will not die at exit.

```
// src/Interpreter.h — BigStackThread
const size_t kStack = (size_t)256 << 20;   // 256 MiB, virtual
pthread_attr_setstacksize(&attr, kStack);
if (pthread_create(&h_, &attr, entry, fn) == 0) joinable_ = true;
else { std::unique_ptr<Fn> g(fn); g->f(); }   // creation failed: run inline
```

The reservation is virtual and committed only as used. Windows gets the same through a small shim kept in `Runtime.cpp` so `<windows.h>` stays out of the widely-included header. If thread creation fails, the work runs **inline** rather than being lost.

## 38.6 Worker bookkeeping, and two real crashes

```
// src/Interpreter.h
struct WorkerSlot {
    BigStackThread th; std::shared_ptr<std::atomic<bool>> done;
};
std::vector<WorkerSlot> workers_;
void addWorker(BigStackThread&& th, std::shared_ptr<std::atomic<bool>> fin);
```

`addWorker` is the **only** safe way to register a worker, and the header explains why in terms of a crash report. In parallel mode spawns happen from worker threads too — a `start` block tapping a supply spawns the interval ticker — and the old per-site “reap, then push” pattern mutated the vector from several threads at once.

`vector::erase` corrupted it and the process segfaulted, with the report naming `erase` inside a supply-interval spawn on thread 5 of 292.

The second crash is inside the fix:

```
// Collect finished slots under the lock, JOIN THEM OUTSIDE IT. Joining
// under sharedMut_ deadlocked: the joinee had set its done flag but was
// still unwinding through code that itself wants sharedMut_ (a worker
// spawning a nested worker) — the reaper held the lock waiting for the
// joinee, the joinee waited for the lock.
```

That is the classic shape — hold a lock across a join — and it is worth remembering that the *first* fix for a concurrency bug introduced it.

There is also backpressure, parallel mode only:

```
void throttleSpawn() {
    if (!parallelMode_) return;
    while (liveWorkers_.load(std::memory_order_relaxed) >= 384)
        { /* reap finished workers, wait for the herd to thin */ }
}
```

A tap-and-close loop over interval supplies — four spawners times a thousand activations, each a real thread with a 256 MiB virtual stack — outran teardown and exhausted the address space. Above the cap a spawner reaps and waits.

**Under the GIL this must stay off**, and the reason is a nice illustration of how the two modes differ: a spawner spinning while *holding* the lock would keep the very workers it waits for from ever finishing.

## 38.7 The user-visible layer

| Type | Backing |
|---|---|
| `Promise` | `PromiseState` in `ext`: mutex, condition variable, result or cause, and a list of `.then` continuations fired once on settle |
| `Channel` | shared state plus a stripe from the atomic pool |
| `Supply` | on-demand blocks run through a `SupplyTapCtx`; live suppliers register a tap record |

| Type | Backing |
|---|---|
| `react/whenever` | a `ReactCtx` with an event queue, a live-source count, and its own mutex and condition variable |
| `Lock` | a `std::recursive_mutex`, recursive because `protect` re-enters |
| `Thread` | a `BigStackThread` plus a per-thread identity value |
| `$*SCHEDULER.cue` | a worker with a `CueState` cancellation flag |

Two details in there earn a mention.

**A tap's teardown lives on the context that owns it.** An earlier design kept a close-callback stack as an interpreter member, which was shared across worker threads — two concurrent `react` blocks corrupted it. Making it thread-local fixed the crash and cost 6% on an unrelated benchmark by reshuffling thread-local storage layout on macOS. The right answer was neither: the context object already travels with the block, is already thread-correct, and is free.

**`whenever` activations are deferred.** Rakudo runs the `react` body first and only then activates subscriptions, so a `say` after a `whenever` prints before the first emitted value. The synchronous drains queue into `ReactCtx::deferred`, which the `react` implementation runs after the body.

## 38.8 Signals, and a thing that must be turned off

Signal supplies use the self-pipe trick: a handler writes a byte, a lazily spawned dispatcher thread reads it and runs the `whenever` block under the GIL. When the `react` block ends, externally wired taps must be closed explicitly, or the dispatcher keeps firing the handler after the block is gone — the symptom was a second Ctrl-C re-invoking a stop method on an already-stopped service.

Separately, and simply: **`SIGPIPE` is ignored process-wide.** Without that, a TCP server dies when a client disconnects mid-write. It is set once on the big-stack entry thread.

## 38.9 Testing it

Concurrency bugs do not appear in unit tests. What found the ones in this chapter:

- **ThreadSanitizer** over the stress suite, which drove the thread-local work;
- **AddressSanitizer** for the lifetime bugs;
- **a stress suite designed to exhaust something** — a thousand tap-and-close activations across four spawners is not a realistic program, and that is the point;
- **crash reports read carefully.** "erase inside a supply-interval spawn on thread 5 of 292" named the bug precisely;
- **`sample` on a hung process.** One apparent hang in an async test turned out to be an unrelated pre-existing bug in a supply transform chain, found by sampling the stuck process rather than by reasoning about the test.

## 38.10 Honest limitations

- **A worker thread's own loop is slower than the main thread's.** Measured on the reference machine, one `start` block with nothing to contend with runs at 0.85× — the gap grows with the work, and under `RAKUPP_GIL=1` the same single `start` runs at serial speed, so it is a per-operation cost inside a worker rather than thread setup. It caps every ratio below: a four-way CPU fan-out is 2.9×, eight-way 2.8×, and four threads updating one `atomicint` are 0.79× — a net loss, because the cache line moves between cores faster than any of them makes progress.
- **User data races are the user's.** As in Rakudo, mutating shared data without a `Lock` is undefined; the striped locks protect the interpreter's own structural invariants, not the user's semantics.
- **The symbol freeze is enforced by a tripwire, not by the type system.** A new structural writer that forgets to call `noteSymbolMutation` is invisible.
- **Worker stacks are large.** 256 MiB of virtual address space each is cheap but not free, and it is why the spawn throttle exists.

# Part IX

# Around the Compiler

# Chapter 39

# Tooling Built on the AST

An implementation is judged partly on things that are not the language: whether it can say a program is wrong before running it, colour it, time it, or let someone try one line at a time. Those usually arrive as separate projects that re-parse the language from the outside and drift away from it. Here they are all in the same binary, reading the same tree the interpreter reads.

Once a program is a tree, several useful things become short. This chapter covers the tools that are built on the front end rather than on the runtime, one that is built on the call path, one that is built on the published C ABI and serves the engine to a program that is not a person, and one thing that is not a tool at all: a check the compiler always runs, whose only job is to stop the program.

## 39.1 `--lint`: static analysis that runs nothing

```
// src/Lint.h
struct LintFinding {
    int line = 0;
    char severity = 'W';        // 'E' error, 'W' warning, 'N' note
    std::string rule;           // a stable id, e.g. "unused-variable"
    std::string message;
};
std::vector<LintFinding> lintProgram(Program& prog);
```

It walks an already-parsed tree and reports findings sorted by line and rule. It does not execute the program.

The design constraint is stated in the header and is the whole reason the rule set is small:

> Rules are deliberately conservative — a missed warning is acceptable, a false one is not.

Raku's dynamism is why. Interpolation, dynamic variables, `EVAL`, symbolic references and introspection all mean that a variable which *looks* unused may be reached by name at run time. An over-eager analyser in this language produces warnings people learn to ignore, at which point the tool is worse than nothing.

Every finding carries a **machine-readable rule id** as well as prose, so a project can suppress or gate on a specific rule without matching message text.

No rule here raises the `'E'` severity — the linter only ever advises. It exists because `--lint` also reports the next section's check, and the two must not be printed alike: a warning is something to consider, an error is a file that will not compile. The exit code says the same thing twice over, 1 for warnings and 2 for anything fatal.

## 39.2 The undeclared-variable gate: the same principle, with the program stopped

```
// src/DeclCheck.h
struct UndeclaredVar { std::string name; int line; };

using Path = std::vector<std::string>;   // the module search path

std::vector<UndeclaredVar> findUndeclaredVars(const Program& prog,
                                              const std::string& src,
                                              const Path& searchPath);
```

`--lint` is opt-in and only warns. This one nobody asks for and it refuses to run the program. It is worth reading straight after the linter, because it is the same design constraint with the consequence raised as far as it goes: a false warning is an annoyance, a false *refusal* means a working program will not start.

The problem it solves is one of timing. An undeclared variable was always an error — the interpreter throws `X::Undeclared` from `Interpreter::lvalue` and from `eval`'s `VarExpr` arm — but only when execution *reached* the reference. So

```
my $x = 42;
say $x;
```

```
say $y;
say "done";
```

printed `42` and then died, where a compiler refuses the file. `-c` was worse: it parsed the same program and answered `Syntax OK`. And `--exe`, which never asks the question at all, emitted C++ naming a variable it had never declared and let the C++ compiler report it:

```
1.rakupp.gen.cpp:38:46: error: use of undeclared identifier 'v_sy';
                        did you mean 'v_sx'?
```

which is a diagnostic about generated code the author never wrote.

So the tree is now asked about the whole unit before any of it runs, and the answer is a compile-time report:

```
===SORRY!=== Error while compiling 1.raku
Variable '$y' is not declared
at 1.raku:3
------> say ⏏$y;
```

### 39.2.1 Where it hangs

Not on the interpreter, and deliberately not on every entry point.

```
rakuppRun ──────► rakuppRunOn(…, declCheck = true)  ─► [gate] ─► run
rk_run ─────────► rakuppRunOn(…, declCheck = false) ──────────► run
-c --cpp --bundle --aot --exe ─► [gate] ─► the mode's own pipeline
REPL ──────────────────────────► one line at a time, no gate
```

`rakuppRun` is the CLI's funnel and asks for the gate; `rakuppRunOn` is what an **embedding host** calls through `rk_run`, and it does not. A host's interpreter may already hold globals it installed through the extension ABI, and no static pass over *this* source can see those. The REPL is exempt for a different reason: each line is its own compilation unit, so the late error costs one line, which is what a prompt is for. `--mcp` reaches the interpreter through the same embedding API and is exempt with it.

The two interpreter call sites bracket both branches of the run path — the fresh parse and the precompiled-AST cache hit (Chapter 30) — because a cached tree is the same tree and must be asked the same question.

### 39.2.2 Two layers, and why the second one exists

The first layer is an AST walk with real lexical scoping. It is position-**insensitive** within a scope: a declaration anywhere in the enclosing block counts, even below the use, because the engine is that lenient in places and a static check must never be stricter than the engine it guards. It knows the things a text scan could not:

```
$^bb                     IS the declaration of $bb in its block
our $x                  installs into the package — a sibling scope sees it
loop (my $i = 0; …)      builds no implicit block; $i outlives the loop
has $x                   with no twigil, is read as a bare $x in the class
repeat { … } while $c    evaluates the condition inside the body's scope
my $a = 1 if $c          declares $a even when $c is false
```

The second layer is a scan of the **source text**, and it only ever removes findings: a candidate survives only if the text, too, never spells that name as a declaration — anywhere in the file, in any scope. That layer is there because the parser silently drops things. `repeat until $c -> $x { ... }` has nowhere in `RepeatStmt` to put `$x`; `with $e -> (Int() $v is copy) { ... }` cannot fit a destructuring signature in `GivenStmt::var`; and `stripPseudoPkg` rewrites `$OUR::x` and `$CALLER::y` down to a bare `$x`/`$y` before the AST ever sees them. Each of those makes a correctly declared variable look undeclared, and refusing such a program would be much worse than the late error being replaced.

What the backstop costs is the cross-scope case — a name declared in one routine and used in another is not reported. What it buys is that a finding means *this name is declared nowhere in this file*, which no gap in the parser can falsify. The two layers are not redundant: the AST pass knows about placeholders and signatures, which no text scan could; the text scan knows about binders the AST lost. Neither alone is both safe and useful.

### 39.2.3 Standing down

The same instinct, taken to its conclusion: several constructs end the check for the whole unit rather than risk a guess.

| Construct | Why nothing can be known |
|---|---|
| `EVAL` / `EVALFILE` | compiles new code against this scope at run time |
| `::($name)` | names any symbol at run time — and assigning through one creates it |

| Construct | Why nothing can be known |
|---|---|
| `require` | imports a set of names only the run can determine |
| `use lib $expr` | a computed search path: any module could be behind it |

`no strict` — the pragma that makes an undeclared variable legal — is the one that does *not* end the check, because it is lexical: the flag is saved when the pass opens a scope and restored when it closes one, so `{ no strict; $a = 5 }` leaves the code after the block as strict as it was, and a nested `use strict` turns the check back on. Standing the whole pass down for it was a real divergence: Rakudo refuses the `$b` in `{ no strict; $a = 5 }; $b = 7`.

The same walk has a second reader. `findLaxVars` returns the other face of the answer — not the names the unit fails to declare, but the ones the pragma *auto-vivifies* — and the native backend needs it, because it compiles every variable to a C++ local and a name with no declaration has no local to compile (Chapter 26). The text backstop is what makes that safe: a name reaches the set only if the unit declares it nowhere, so it can never collide with a local codegen does emit. `complete` reports whether the walk finished; a lax unit that stood the pass down is bundled instead of guessed at.

Imports are the one case that is resolved rather than surrendered to, because a module may export a **variable** (`our $setting is export`) and that name is declared nowhere in the importing file. Before reporting anything, the imported module's source is located with `rakuppFindModuleSource` — the loader's own resolver, over the loader's own search path plus any literal `use lib` — and searched for the name. A module that cannot be found, or one carrying a `sub EXPORT` that can export whatever it likes, ends the check.

That lookup reads files, so it is deliberately the **last** thing that happens: it runs only once a candidate already exists, which is to say only on the way to refusing the program. A working run never touches the disk for it.

The match there has to be on the whole name. Matching a prefix let a module's `$setting` clear a bogus `$s` in the program, which quietly switched off detection for most short names in any program that imports anything — a good illustration of how an over-eager *clearing* rule fails silently, in the direction that is hard to notice.

### 39.2.4 The predicate that must not drift

Which names are exempt — twigils, `$_`, `$/`, `$!`, `$0`..., `@_`, `%_`, `$a`/`$b`, every `&`-sigil name, anything package-qualified — is not re-implemented here. `isSpecialVar` in `Interpreter.cpp` stopped being `static` and is declared in `Interpreter.h`, so this pass calls the same function the throw sites call. Two copies of that list would drift, and the failure mode of drift is this check refusing a program the interpreter runs happily. Writing it down once was also how `$¢`, the match cursor, turned out to be missing from it.

### 39.2.5 It reports through `--lint` too

Everything above is about refusing to run a program, which is the opposite of what an analysis tool does — so the check reports through `--lint` as well, as an `error:` line with the rule id `undeclared-variable`, sorted in among the warnings by line.

The alternative was worse than it looks. A tool whose whole purpose is to find problems before running would have answered *no issues found* for a file the compiler refuses outright, which is not merely unhelpful: it is the tool disagreeing with the compiler about whether a program is valid, in the direction that tells you to go ahead. Whatever else a linter does, it must never say less than running the program would.

The rule reaches further than `--lint`, and for a while it did not. `rakupp --lsp` speaks the Language Server Protocol and publishes these findings to an editor — but `src/Lsp.cpp` included `Lint.h` and not `DeclCheck.h`, so for its first release it never reported an undeclared variable: on the file above, `--lint` printed an error and exited 2 while the language server published only the unused-variable warning. An editor showing a clean file that will not run is this same failure one layer further out, and it is worse, because nobody consults a linter they believe has already run.

Both now build the same list — `lintProgram` plus `findUndeclaredVars`, sorted by line and rule — and the server maps `'E'` to LSP severity Error. Two details belong to the server rather than the check. A long-running process may not die of an internal error, so a throw out of the declaration check publishes an informational diagnostic saying the check did not run, rather than either crashing or silently dropping it — a missing error being exactly what this paragraph is about. And `RAKUPP_NO_DECLCHECK=1` switches it off in both tools, so they cannot disagree about being switched off.

### 39.2.6 What it costs

One extra walk of the tree, once, before the program starts: about 0.15 µs per line of source — 0.32 ms for a 2,200-line program. That is 2–3% of a parse-and-exit (-c), 0.2–0.6% of a real run, and immeasurable for anything long-lived. RAKUPP_NO_DECLCHECK=1 switches it off, for the day it is wrong about a program that works.

## 39.3 --highlight: colouring with the compiler's own knowledge

```
// src/Highlight.h
std::string highlight(const std::string& source, const std::string& format);
```

Two renderers: html produces the same CSS token classes Pygments uses, so existing stylesheets apply unchanged; ansi produces terminal escapes.

The class names are borrowed, but the classification is not. A regex-based highlighter has to guess; this one knows. $obj.role is coloured as a method call rather than as the keyword role, because the lexer and parser already established which it is.

That is the general advantage of building a highlighter inside the compiler, and it is why the output is worth the coupling.

## 39.4 --profile: a routine-level wall-time profiler

```
// src/Profiler.h
namespace prof {
    extern bool on;              // read on the hot path
    void setDest(const std::string& dest);
    void enter(const void* key, const char* name, const char* file);
    void leave();
    void report();
}
```

Two hooks — callCallableRaw for routines with a frame boundary, and invokeMethod — feed a per-thread shadow stack. Per routine it aggregates call count, inclusive and exclusive wall time.

Three decisions worth naming.

**Builtins are attributed to their caller**, because the hooks are on *user code* routine entry rather than on the builtin dispatch chain. That is a deliberate simplification: it keeps the hot path to one branch, and a profile that says "this routine spent 40% of its time in built-ins" is usually the answer you wanted anyway.

**Recursion is handled the standard way**: only the outermost active frame of a routine adds to its inclusive time, so a recursive function's inclusive figure does not count the same seconds many times.

**The off-cost is one predicted branch on a plain `bool` per call**, measured at zero against the noise band before it was built. A profiler that cannot be compiled in unconditionally is a profiler people forget to use.

The destination follows Rakudo's convention — the extension selects the format: – writes a table to standard error, a path writes the table there, and a path ending in `.json` writes a machine-readable dump.

One portability note is recorded in the header: everything in it is portable C++17, with no `__builtin_expect` and no attributes, because the prototype's GCC-isms broke the MSVC build once already.

## 39.5 The REPL

```
// src/Repl.h
int rakuppRepl(const std::string& exePath,
               const std::vector<std::string>& libPaths);
bool stdinIsTerminal();
bool replForced();
```

The REPL is entered **only** by a bare `rakupp` attached to a terminal. Anything arriving on a pipe or a redirect is a complete program and keeps running as one; the terminal test in `main` is the whole of that decision.

It lives in the executable rather than the runtime library, so nothing about an interactive session is linked into the binaries the compiling modes produce.

A REPL never calls `run()`. It keeps **one** `Interpreter` alive and feeds it `evalString` per line, so the mainline scope *is* the session. Two functions cover what `run()` would otherwise have done at either end:

```
// src/Interpreter.h
void replStart(std::vector<std::string> args);  // define @*ARGS, arm `state`
```

```
void replFinish();                              // run END phasers, once
std::vector<std::string> replNames() const;     // for tab completion
```

What a line evaluated to is echoed back, dimmed so it reads as the REPL talking rather than as program output — with one exception, taken from Rakudo: a statement that already wrote to stdout is not echoed at all. `say 42` prints `42` and stops there; `for 1..10 { say $_ }` prints the ten numbers and nothing under them. The rule is about intent rather than about the value — you asked for output, so the return value was a by-product. It counts *bytes*, so `print ""` wrote nothing and its `True` is still shown, and it counts *stdout*, so a `note` suppresses nothing.

The count is taken at `std::cout`'s streambuf rather than inside `ioEmit`, because `say` is not the only thing that reaches the terminal — `$*OUT.print`, the Test builtins and MAIN's usage text write to the stream directly. Taken at the stream it is exactly "did a byte reach the terminal", and it cannot drift as output sites are added. The one gap is a subprocess, which inherits the descriptor and writes past the buffer; Rakudo has the same gap for the same reason, its own count living in the `$*OUT` handle.

The value being echoed is the statement's, which is why a loop *statement* is `Nil` and not an undefined `Any` — the same answer `sub f { while 0 {} }` gives. Values survive only where the loop was written as an expression (`do for`, `(for …)`), which the AST marks with `asExpr`.

The nicest detail is how an incomplete line is recognised. A parse that dies on end-of-input is a request for more input, not a syntax error:

```
// src/Parser.h — ParseError
bool atEof = false;
```

```
// src/Interpreter.h
Value evalString(const std::string& src, bool mainlinePH = false,
                 bool* incompleteOut = nullptr);
```

The flag is set by the lexer's runaway-construct diagnostics and by the parser when it fails on the end token, so typing `sub f {` at the prompt asks for a continuation line while `sub f )` reports an error. One boolean, threaded from the lexer to the prompt.

## 39.6 --mcp: the same session, driven by an agent

```
// src/McpServer.h
namespace rakupp::mcp {
struct Options {
    int timeoutSecs = 120;              // 0 disables the watchdog
    std::vector<std::string> preload;   // -M modules, as `use <module>;`
};
int runServer(const Options& opt);      // serves until stdin closes
}
```

The REPL's shape — one interpreter, fed a line at a time, the mainline scope as the session — turns out to be what an AI agent wants too. `rakupp --mcp` serves exactly that over the Model Context Protocol: JSON-RPC 2.0, one message per line, on stdio. A client launches the process, discovers its tools and calls them mid-conversation.

Two tools, and the interesting thing is that neither is new engine work:

- **`raku`** evaluates source in one persistent session. `my $x = 41` in one call and `$x + 1` in the next is `42`, for the same reason it is in the REPL: it *is* that evaluation path. The reply is what the code printed, or, when it printed nothing, the `.gist` of its last statement after `=>`, which is again the REPL's convention.
- **`raku-parse`** compiles a grammar from source text and parses with it, answering a JSON tree — or, on a non-match, a diagnosis naming the line, column and deepest rule reached, which is what a client needs in order to fix its grammar and try again.

### 39.6.1 It is a host, not a private door

The whole server reaches the engine through the public C ABI of Chapter 37 — `rk_new`, `rk_eval`, `rk_set_output`, and for grammars the same `rk_grammar_shim()` source every language binding loads. Nothing in `McpServer.cpp` includes `Interpreter.h`.

That is the load-bearing decision, and it is worth stating as a rule: **a new front door must be a client of the published boundary, not a second one.** An agent therefore gets byte-for-byte what a Python binding or plain `rakupp -e` would get, and every ABI fix reaches all of them at once. The alternative — a server reaching into the interpreter directly because it lives in the same binary — would have been shorter to write and would have produced a third set of semantics to keep in step.

It is also why `--mcp` is exempt from the declaration gate earlier in this chapter: every evaluation arrives through `rk_eval`, like any other embedding host's, and the embedding path does not ask for the gate.

### 39.6.2 The watchdog, and a bug worth keeping

`rk_eval` cannot be interrupted mid-evaluation — an `rk_interrupt` is future ABI work — so a call that never returns would wedge the client forever. A watchdog thread, armed around every engine call with that request's id, answers the request after `--timeout` seconds (120 by default, `0` to disable) and then ends the process. The client starts a fresh server on its next call and the error text says the session was lost. That is a poor outcome; a wedged agent is worse.

Two details of it are the kind this book exists to record.

The timeout reply is **assembled by hand**, not built through the server's JSON type, because the main thread may be wedged deep inside Raku and the dog must not allocate its way through shared machinery to say so.

And the thread is **joined, not detached**:

> destroying a condition variable with a parked waiter is not the no-op macOS makes it — glibc's `pthread_cond_destroy` WAITS for the waiter to leave

A detached dog parked on its condition variable meant the server's return blocked forever on Linux and nowhere else, which is how it reached CI as six-hour job timeouts rather than as a test failure. The destructor now sets `quit_`, notifies, and joins. The timeout path never runs it at all: `_Exit` ends the process with the thread still parked, deliberately.

### 39.6.3 Two smaller consequences of embedding

`exit` in evaluated code does not end the server, because an embedded evaluation refuses to end its host process; it comes back as an error naming the code and the session continues. And the interpreter's standard input is pinned to EOF with `rk_set_input`, so `get/lines/$*IN` read nothing instead of eating the protocol's own bytes off fd 0 — the server reads that descriptor itself.

The JSON layer is about 300 lines of hand-written C++ rather than a dependency, for the reason the rest of the engine has none. It lives in `src/JsonLite.h`, shared with the Jupyter kernel below — two protocols, one little value type, still no library.

`tools/mcp-smoke.raku` drives a real server over stdio exactly as a client does and pins the handshake, both tools, session persistence, the surviving `die`, the tree and the diagnosis, and the watchdog's answer-then-exit contract.

### 39.6.4 What it does not do

The `raku` tool runs arbitrary Raku with the privileges of the process. There is no sandbox, and registering the server grants an agent the trust that handing it a shell does. Saying so is the whole of the mitigation here; a sandboxed variant is separate work, and the WebAssembly build of Chapter 32 is the natural cage for it.

## 39.7 `--jupyter`: the same session, in a notebook

```
// src/JupyterKernel.h
namespace rakupp::jupyter {
struct Options {
    std::string connectionFile;         // Jupyter's {connection_file}
    std::vector<std::string> preload;   // -M modules, as `use <module>;`
};
int runKernel(const Options& opt);
int installKernelspec(const InstallOptions& opt);   // --jupyter-install
}
```

A notebook is a REPL with an editable scrollback, so this is the third face of the same thing: `rakupp --jupyter FILE` serves the session to a Jupyter frontend, and the rule from `--mcp` holds unchanged — nothing in `JupyterKernel.cpp` includes `Interpreter.h`. A cell arrives as `rk_eval`, its output leaves through `rk_set_output`, and `jupyter-display` — the one routine the kernel adds to the language — is an `rk_register` host function that publishes `display_data`. A notebook cell, an agent's tool call and a `.raku` file therefore agree by construction.

### 39.7.1 The transport is the interesting part

Jupyter's protocol rides on ZeroMQ, and this project links no third-party libraries. So the kernel speaks ZMTP itself, over plain TCP: the 64-byte greeting, the NULL-mechanism `READY` handshake, `MORE`-chained frames — and only the three socket behaviours a kernel needs. `ROUTER` for shell and control, where the reply goes back down the connection it arrived on; `PUB` for iopub, fanning out to subscribers by topic prefix; `REP` for the heartbeat, which is an echo. Everything else a real ZeroMQ does — reconnection, high-water marks, the other twelve socket types — is absent deliberately: the peer is one frontend that connects once.

Two decisions are worth recording, both of the "advertise less, accept more" kind. The kernel announces version **3.0** rather than 3.1, which keeps the peer on the

older subscription form and off ZMTP heartbeats — less protocol on a link that carries one client — while the code still honours the 3.1 `SUBSCRIBE` command and answers `PING` with `PONG`, so a libzmq that changes its mind does not break it. And every message is signed with HMAC-SHA256 over its four JSON parts, using the SHA-256 in `src/Digest.h` (about 120 lines of FIPS 180-4, shared with the digest builtins; the kernel keeps a five-line wrapper) for the same reason as the JSON: one that does not verify is *dropped*, not answered.

### 39.7.2 The bug the capture mechanism sets

`rk_set_output` swaps `std::cout` and `std::cerr`'s stream buffers process-wide so a cell's printing can be captured. The kernel's own diagnostics therefore must not use `std::cerr`: a log line written that way arrives in the notebook wearing the user's output as a disguise. They go to the C `stderr` instead, which the swap does not touch and which Jupyter puts in its log. It cost one silent debugging session to notice, which is why it is written here.

### 39.7.3 What it does not do, out loud

The engine has no interrupt point inside an evaluation, so `interrupt_request` answers, says so on iopub, and *exits* — the frontend restarts the kernel and the session is gone. `get` reads EOF, because a frontend may have no way to answer a prompt and a kernel blocked on one is a hung notebook. `is_complete_request` answers `unknown` rather than guessing at what the parser would say. All three are the same choice: a visible gap beats a convincing lie.

`tools/jupyter-smoke.raku` is a **Jupyter client written in Raku** — it opens the five sockets, does the ZMTP handshake, and carries its own HMAC-SHA256 pinned to the RFC 4231 vectors, so the gate passes only if two independent implementations of both halves agree.

## 39.8 Pod and --doc

```
// src/Pod.h
ValueList parsePod(const std::string& src);
```

Pod blocks are parsed into the `$=pod` DOM — a list of `Pod::Block` values, each a `Hash` tagged `"Pod"` with a class, a name, a level, a configuration and contents. Delimited

(`=begin/=end`), paragraph (`=for`) and abbreviated (`=head1 …`) forms, nested blocks, and whitespace-collapsed paragraphs.

Note the representation: a Pod block is not a new `VT` or a C++ class. It is a tagged `Hash`, which is the same technique `Set`, `Proxy` and `DateTime` use (Chapter 8). The Raku program manipulating it sees an ordinary object.

Declarator documentation — `#|` above a declaration, `#=` beside or below it — is collected by the lexer keyed by line and attached by the parser, answering `.WHY` on routines, classes and attributes.

`--doc` runs `DOC` phasers and prints the rendered content, and `=finish` data travels from the lexer to `$=finish`.

## 39.9 `--dump-ast`, and the tool built on it

```
rakupp --dump-ast prog.raku
RAKUPP_DUMPTOKENS=1 rakupp prog.raku
```

`AstDump.cpp` prints the tree as an indented outline. It is the first thing to reach for when a parse produces something unexpected.

It is also an **interface**, not just a debugging aid. `tools/ast-opportunity.raku` reads its output and counts syntactic patterns across a corpus — which is the tool that measured, in two minutes, that constant folding had almost nothing to fold in real Raku and that operand shapes were 25 per thousand nodes (Chapter 19). A textual tree dump turned out to be a perfectly good static analysis substrate.

## 39.10 The tools that are not in the binary

A standing rule of the project: **build the ecosystem tooling in Raku, run it with rakupp.** Not because it is elegant, but because it is the largest available body of real-program testing.

| Tool | What it does |
|---|---|
| `tools/run-roast.raku` | the Roast harness; `--suite=rakudo` or `--suite=mutsu` runs that project's own `t/` instead |

| Tool | What it does |
|---|---|
| `tools/run-bench.raku` | the benchmark harness, four engines; `--rusage` adds CPU and peak memory; `--suite=mutsu` times mutsu's benchmark files instead |
| `tools/perf-guard.raku` | the release performance gate |
| `tools/run-optbench.raku` | compiles each optimiser showcase twice, checks byte-identical output, then times |
| `tools/gen-unicode.raku` | generates Unicode tables |
| `tools/ast-opportunity.raku` | counts AST patterns |
| `tools/doc-examples-diff.raku` | runs every documented example on both engines |
| `tools/recheck-divergences.raku` | re-tests the recorded divergence list |

Each of these is a substantial Raku program that runs on every release. When one of them breaks, it has usually found a bug in the compiler rather than in itself — which is exactly the point of the rule.

The showcase programs go further. Interpreters for JavaScript, Perl 5, Python 3, Lisp and Forth, each written in Raku, each producing byte-identical output to the real implementation on its examples. Writing one is the single most productive bug-finding activity in the project's history: each of them, on first completion, yielded a list of genuine defects — grammar mis-parses, a lost `return`, dynamic-variable restore, slip flattening, precedence traps.

An interpreter for another language exercises grammars, recursion, closures, string handling and dispatch simultaneously and at a scale no unit test reaches. It is a test suite that writes itself, and it has an oracle: the language it implements already has one.

# Chapter 40

# Measuring, and Proving

Almost every decision in this book was made against a number. This closing chapter is about how those numbers are produced, why several of them were wrong the first time, and what a release has to pass.

## 40.1 The benchmark policy

Stated once, applied everywhere:

- **interleave the variants.** Run A, B, A, B — not all of A then all of B — so machine drift and thermal state hit both equally.
- **discard a warm-up**, then report the median or the minimum of several repetitions. Which one depends on the noise shape; the important thing is to say which.
- **keep a control kernel** that the change under test cannot possibly affect.
- **state the conditions**: machine, architecture, compiler, date.
- **prefer a load-independent metric when the machine is not yours to quiet.** `/usr/bin/time -l` reports **instructions retired** and **cycles elapsed** on macOS 12 and later. Instructions retired does not move when a browser or a build is running alongside, which is what makes a developer box measurable at all — and it is what `perf-guard --check` cannot offer, since it refuses to report under load rather than reporting something misleading.

Read the two counters together, because the pair says more than either:

| reading | meaning |
|---|---|
| instructions down | work was removed; the size of the drop is the size of the win |
| instructions flat, cycles down | the same work done more efficiently — better locality, a bulk `memcpy` instead of a loop |
| instructions up, wall clock down | suspect code layout, and say so rather than quoting the wall clock |

One addition to the control rule, learned the hard way: the control should be a **control binary**, not only a control kernel. Build it from the same tree with just the line under test reverted. A rename, a new header, an inlining decision that shifts a 30,000-line translation unit — any of these moves code layout by itself, and comparing against the last release charges that to the change.

The control is the part people skip and the part that carries the argument. Chapter 19's table has a row for a pure method-dispatch loop, containing nothing node specialisation can touch:

| kernel | base | specialised | delta |
|---|---:|---:|---:|
| vars | 840.8 ms | 686.5 | −18.3% |
| fib | 478.6 | 422.4 | −11.7% |
| **ctl — method dispatch only** | 327.9 | 327.0 | **−0.3%** |

Without that last row the others are only evidence that the machine was quieter the second time.

## 40.2 The performance gate

```
build/rakupp tools/perf-guard.raku --check
```

`perf-guard` compares the current binary against a recorded baseline and **fails** on a regression. The release checklist gates on it.

That it exists at all is a lesson learned twice: a release has shipped with a performance regression because someone eyeballed the numbers and thought they looked fine. Eyeballing does not work — the noise band is a few per cent, the regressions

that matter are often a few per cent, and human judgement about which is which is unreliable at exactly that scale.

The rule that follows: **a performance change is an interleaved A/B against `perf-guard`, not an opinion.**

`perf-guard` has one honest limitation: it refuses to report on a machine under load, by its own detector, which on a developer's daily box means most of the time. That refusal is correct — a gate that reports noise is worse than one that reports nothing — but it leaves a gap, and the instruction-count A/B above is what fills it. The gate still owns the release; the counters own the day-to-day.

## 40.3 Three kernels, three resolutions

The number that makes the previous section usable is not a ratio at all. It is the **spread of a kernel measured against itself**: the same unchanged binary, run eight times.

| kernel | spread over eight runs |
|---|---|
| `fib` | 6.4093–6.4119 G instructions, ±0.02% |
| grammar JSON parse | 1.8493–1.8839 G, ±1.9% |
| `streq` | 4.90–5.00 G, ±2% |

Two orders of magnitude between the tightest and the loosest, on the same machine, in the same minute. The mechanism is the parallel runtime: its worker threads do variable amounts of work, so the variance lands on kernels that allocate and thread and not on tight arithmetic loops. A 1% ratio measured on `fib` is a finding. A 1% ratio measured on the grammar parse is nothing at all.

**Measure a kernel's spread on one binary before trusting any ratio from it.** The next section is what happens when you do not.

## 40.4 The correctness gates

| Gate | What it is |
|---|---|
| **Roast** | the Raku specification suite, run by a Raku harness |

| Gate | What it is |
|---|---|
| **the local suite** | examples and showcases, byte-compared against golden output |
| **regression tests** | one file per fixed bug |
| **stress tests** | concurrency and memory, also under TSan and ASan |
| **compiler agreement** | every deterministic example must produce identical output interpreted, `--exe`, and `--exe -O` |
| **the libffi fallback** | `t/regression/nativecall-libffi.raku`, which re-runs the libffi-only cases in a child with `RAKUPP_FFI=0` and checks each throws |

Compiler agreement is the one that catches the most. Three execution paths must produce **byte-identical** output for the same program; a divergence is a bug in one of them, and it is found automatically rather than by someone noticing.

Roast is reported two ways, and the difference matters. **Files fully passing** is an all-or-nothing bar; **assertions passing** gives partial credit. Roughly ninety per cent of declared assertions pass. Both numbers are published, because either alone is misleading — a file can fail on one obscure assertion out of two hundred, and a file can "pass" by skipping everything.

## 40.5 Oracles

A test needs something to be right *against*. Four are used, in decreasing order of authority.

**Roast.** The specification. If Roast asserts it, it is the answer.

**Rakudo.** For anything Roast does not cover, the reference implementation is run side by side. The documentation harness does this in bulk: every documented example, on both engines, three-way classified.

**The previous rakupp binary.** For a change that is supposed to be *semantically invisible* — an optimisation — the baseline binary is the oracle, not another implementation. That habit came directly from a bug: the first `evalIndex` fast path wrapped negative subscripts from the end, and `my $i = -1; @a[$i]` must throw. Rakudo could not have caught it, because Rakudo rejects that spelling at compile time. Diffing against the previous binary did.

**The language being implemented.** The showcase interpreters compare their output against `node`, `perl`, `python3` and the real Lisp — which is an oracle for thousands of lines of Raku that nobody wrote assertions for.

## 40.6 Error messages are not behaviour

A rule with a sharp edge: **do not copy Rakudo's message prose unless Roast or the documentation asserts it.**

Behaviour must match. Message wording need not, and chasing it produces churn that no test protects. Where a diagnostic *is* asserted, it is asserted by exception class and attributes rather than by text — which is why typed exceptions are built with attributes rather than formatted strings (Chapter 15).

## 40.7 What has and has not paid

The pattern across every optimisation in this book is consistent enough to state as a finding.

**What paid** — all of it removing work or allocation:

| Change | Effect |
|---|---|
| copy-on-write strings | removed a quadratic; 142 ms to 24 ms on a copy benchmark |
| the property cache on the string body | removed the *other* quadratic |
| interned tag fields | removed a constructor, destructor and copy from every `Value` |
| the packed-prefix name comparison | 60% of a profile to 8.5% |
| `strtod` instead of a caught `stod` | removed a C++ throw from the hottest path |
| node specialisation | removed a `Value` copy and a literal rebuild per evaluation |
| direct-arity calls | removed the per-call `ValueList` — in compiled code |
| the small-block free list | removed the same allocation in *interpreted* code: 32.35 ns to 9.48 |
| one-pass relocating list growth | removed a whole second walk over the buffer |
| native integer lanes | removed the per-operation `Value` |
| the key-once sort | removed an asymptotic factor |

**What did not pay:**

- **constant folding** — measured first, with a corpus of 51,353 nodes; 0.07% of nodes, none of it in a loop. Not built.
- **link-time optimisation and `-mcpu=native`** — measured, no effect.
- **`-Os`** — smaller by nothing that matters, slower by 20 to 50% of the lane speed-up.
- **a dispatch table for the method ladder** — would chase 8.5% of a profile, and is not a drop-in because the ladder is guarded and order-sensitive.
- **a flat threaded execution loop** — perl's `run.c` in one line, and the most frequently proposed change to any tree-walker. Measured twice, a month apart: the opcode `switch` is worth 0.32 ns against a node visit costing 46 to 85. Under one per cent. Chapter 42.

The generalisation: **on this codebase, removing an allocation has always paid and removing a branch almost never has.** That is a property of a tree-walker over a fat value type, and it is worth re-deriving before assuming it holds somewhere else. The two cleanest witnesses sit at opposite ends of the two lists above and were measured within days of each other: the free list is pure allocation removal and paid; the threaded loop is pure branch removal and did not.

## 40.8 Four ways the measurement itself was wrong

Worth more than the successes.

**Benchmarking a rename instead of the change it enables.** An early analysis concluded that turning a builtin into a named C++ function was worth about 1 nanosecond per call. That was true for an out-of-line function still taking a `ValueList` — and completely missed what a real named function unlocks: direct `Value` arguments, no allocation, and an inlinable body. The corrected measurement was 5.6 times on an `abs` loop (Chapter 28).

**Restructuring the shared path while adding a fast one.** The first node specialisation made the *control* 5.7% slower, because binding an operand through an optional cost the general path its copy elision. The rule extracted: add an early exit, never restructure the code underneath it.

**Assuming where the cost was.** "I/O dominates, so call plumbing does not matter" was false: a buffered one-argument `say` costs about 125 nanoseconds, of which the plumbing was 45%. And the method dispatch ladder was exonerated twice by a

correct observation — that a method 177 comparisons in cost the same as one 812 in — before a profiler showed the cost was `strlen` on a literal, not the ladder's length.

**Two agreeing single runs, agreeing on a wrong number.** A string-representation change was published as an 8.9% win on grammar JSON parsing. It is 0.1%. The measurement was one run per binary; it was taken twice, on different corpora, and both times said 8 to 9 per cent, which felt like confirmation. It was not. The grammar parse spreads ±1.9% on an unchanged binary, and the two readings had agreed on a value about 12% above the kernel's own floor — an agreement between two draws from the same wide distribution, which is exactly what independent confirmation is supposed to rule out and does not, when the draws share a bias. Best of five put the real figure at 0.1%, and the change's honest case turned out to rest on a different kernel entirely.

The rule that came out of it is the previous section: the spread is a property of the kernel, it varies by two orders of magnitude across the suite, and it has to be measured before any ratio from that kernel is quoted. Repeating a measurement is not the same as repeating it enough.

## 40.9 Where the remaining time is

For an interpreted method-heavy loop, after everything above:

| | share |
|---|---|
| heap allocate and free | 31% |
| `Value` copy and destroy | 11% |
| method-name comparison | 8.5% |
| the dispatch function's own body | 6.1% |

Forty-two per cent in allocation and value churn. The shape of the fix looked known when this was written: pass the invocant and argument list by reference rather than by value, and shrink `Value` — both to be approached carefully rather than quickly, because the first trades away an accidental safety property and the second is a representation change that the extension ABI was specifically designed to survive (Chapter 37).

Half of that came true and half did not, which is the interesting part. `Value` was shrunk twice, 344 to 208 to 128, and the ABI did survive it exactly as designed.

The argument list was never passed by reference — the 178-call-site aliasing audit is still undone — and the cost was taken anyway, from a third direction nobody had written down: making the allocation itself nearly free (Chapter 12), which needed no audit at all.

That is the more useful lesson to carry out of this chapter. A cost can be correctly identified, and the *shape of the fix* can still be wrong. "Pass it by reference" and "make the allocation cheap" address the same 31% and have nothing else in common: one is a 178-site audit of what may alias, the other is sixty lines in one header. Naming a cost is worth more than naming its cure.

## 40.10 Honesty as a practice

Every chapter in this book has a "honest limitations" section, and that is a deliberate convention rather than a stylistic tic.

The list for the project as a whole: about ninety per cent of Roast; depth-first method resolution rather than C3; no ambiguity error in multiple dispatch; role composition that is last-writer-wins; modules that publish their whole environment; a flat class registry; no macros, and slangs only at fixed seams; laziness capped at a million elements; a `gather` block that can run more than once; a reference cycle that leaks.

Every one of those is a real difference from the reference implementation, and every one is written down where someone hitting it would look. That is the difference between a document and a brochure, and it is the reason the divergence lists in `docs/dev/findings/` are maintained as running logs rather than occasionally cleaned up.

A last observation, which is the closest thing this book has to a thesis. The mechanisms in it are mostly ordinary — a Pratt parser, a tagged struct, a backtracking matcher, a transpiler. What made them work was not cleverness. It was measuring before building, keeping a control, writing down the reason next to the code, and being specific in public about what does not work yet.

# Part X

# The Speed Campaign

# Chapter 41

# Constant Factors

On 2026-08-21 the fastest way to run the `hashfill` kernel built here was to compile it to a native binary — and that binary, at 113 ms of wall clock, still lost to the `perl` binary's 82 ms. Our own interpreter was 3.3× behind `perl` on the same program. Two days later the interpreter led Rakudo on all ten benchmark kernels, the native binary led `perl` 2.8× on the kernel that started it, and `sizeof(Value)` had fallen from 344 bytes to 128. This chapter is the story of those two days: four batches of work, each one an application of the same few decisions, and each one measured under Chapter 40's rules before it was believed. A fifth batch arrived ten days later, from the same probe and a different direction, and it is here too — partly for what it bought, and partly because it produced the campaign's cleanest example of a measurement that was wrong in a way no test could catch.

Nothing in the campaign is an algorithm. Every batch attacks a *constant factor* — the cost per variable read, per hash probe, per value copied, per assignment executed — and the campaign's tools are the ones this book has already described: the census before the change, the interleaved A/B with a control kernel, the falsifier written into the plan, and Roast as the veto.

## 41.1 The trigger, and the method

The trigger was external: a remark that a Raku program ran "twice slower than Perl 5". The report named no program, so one was built to probe it — `hashfill`, a twin pair of files, Raku and Perl, identical line for line. The Raku side (`tools/bench/hashfill.raku`):

```
my %h;
for 1 .. 200_000 -> $i {
    %h{"key$i"} = $i * 2;
}
my $sum = 0;
for %h.values -> $v { $sum += $v }
my $s = '';
for 1 .. 50_000 { $s ~= 'x' }
say $sum, ' ', $s.chars;
```

And the Perl twin, the same work with byte-identical output:

```
my %h;
for my $i (1 .. 200_000) {
    $h{"key$i"} = $i * 2;
}
my $sum = 0;
$sum += $_ for values %h;
my $s = '';
$s .= 'x' for 1 .. 50_000;
print "$sum ", length($s), "\n";
```

It is a kernel made of exactly what a scripting language is asked to do all day — interpolated hash keys, a values sweep, a built-up string — and `perl` won it in every mode we had.

The response was not to guess. It was to read the Perl 5 sources — five files: `sv.h`, `hv.h`, `pad.h`, `run.c`, `pp_hot.c` — and write down what thirty years of interpreter maintenance had settled on, as a ranked findings document — `docs/dev/findings/engines/PERL5-TECHNIQUES.md` in the repository. Chapter 42 tells how that one document grew into a nine-engine survey. This chapter follows the items that got applied, in the order they landed.

The loop each batch ran:

1. **Measure first.** A profile or a census names the cost; the plan document records a prediction and a falsifier — what result would prove the idea wrong.
2. **Change one layer.**
3. **Interleave the A/B** on the same machine, same day, with control kernels that the change cannot touch.
4. **Gate on Roast.** A per-file diff against the pre-change run; jitter is distinguished from regression by re-running the flapping file, not by optimism.

5. **Re-measure the public numbers** and update BENCHMARKS.md the same day.

## 41.2 The recurring kernels

Two harnesses supply the names this chapter keeps using. `perf-guard` (Chapter 40's regression gate) times interpreter-only one-liners; the `tools/bench` set times full programs in every mode, against Rakudo and — for `hashfill` — against `perl`. Four of the guard kernels are short enough to show whole:

```
# fib
sub fib($n) { $n < 2 ?? $n !! fib($n-1) + fib($n-2) }; say fib(29);
# asg
my $x = 0; for ^2_000_000 { $x = $x + 1 }; say $x;
# loopsum
my $t = 0; for 1 .. 1_000_000 { $t += $_ }; say $t;
# hash
my %c; for 1 .. 100_000 { %c{$_ % 1_000}++ }; say %c.elems;
```

The rest, in a line each:

| kernel | what the program does |
|---|---|
| `strscan` | per-character `.substr` over a 200 KB string, 200k calls — catches any op that re-scans or copies the invocant per call |
| `strpass` | a 200 KB string passed to a sub 200k times — catches lost `CowStr` sharing (a memcpy per call when broken) |
| `subcall` | a typed `is rw` signature called 200k times — catches per-call binder work that belongs on the AST |
| `rats` | 200k short-lived `Rats` created, summed and read once each — added by this chapter's own footnote, below |
| `strcat` | build a string by repeated `~=` |
| `streq` | a million `eq` comparisons in a hot condition |
| `regex` | one simple pattern matched many times |
| `sortnums` | sort 50,000 pseudo-random integers |
| `arrayops` | a `grep`/`map`/`sum` pipeline over a range |
| `bigint` | factorial(5000) as a running product — machine words overflow almost immediately |

The guard kernels were chosen adversarially — each stresses the path a batch is most likely to slow down — and `strscan/strpass/subcall` exist precisely because an earlier release changed the string representation and the then-current guard, all Int-and-Array work, could not see it. `rats` was added for the same reason one batch later, and by the same argument: batch two below moves the `Rat` pair out of the inline value, and every kernel listed above is Int, Array or string work.

## 41.3 Batch one: the hash payload

The hash payload behind every `%h` was `std::map<std::string, Value>` — a red-black tree. Every lookup walked O(log n) nodes doing a full string comparison at each, every node carried a then-344-byte `Value`, and the `hashfill` profile showed the cost plainly as `memcmp` samples under `rtIndexRef`.

Perl has not paid that price since the nineties: a key's hash is computed once and stored beside it, lookups compare the stored hash before they touch a single byte, and the table is an open-addressed array. `ValueHash` (src/ValueHash.h) is that design under two contracts the interpreter already relied on:

- **Reference stability.** Autovivification hands out `Value&` into the payload and keeps it across further inserts. Entries therefore live in a deque — `push_back` never moves elements — and deletion only marks. A churning hash carries tombstones; that is the price of stable references, and Perl pays the same one with its lazy deletes.
- **Iteration shape.** The interpreter iterates `pair<const string, Value>` in insertion order, skipping the dead.

The subtlety was not the table; it was the *ordering audit*. `std::map` had been silently sorting, and rendering sites got sorted `.keys` for free. The audit came to exactly three test files, and two of them were gifts: a pair of assertions that had only ever passed by coincidence, and one real latent bug — `Mix.total` summed fractional weights in a `double`, so its rounding depended on iteration order. It now sums exactly through the numeric tower, which is also Rakudo's answer. A data-structure swap found an arithmetic bug; that is what the ordering audit was for.

| kernel | `std::map` | `ValueHash` | |
|---|---|---|---|
| hashfill, interp (user) | 0.26 s | 0.21 s | −19% |
| hashfill, `--exe -O3` (wall) | 0.103 s | 0.078 s | −24% |
| 500k lookups, 100k-key hash | 0.41 s | 0.33 s | −20% |

| kernel | `std::map` | `ValueHash` | |
|---|---|---|---|
| bench `hash` kernel | 0.06 s | 0.04 s | −30% |

The compiled binary crossed `perl` on `hashfill` with this batch — 78 ms against `perl`'s 82.

## 41.4 Batch two: the census and the cold block

Chapter 8 ends by admitting that `Value`'s size "is the thing to attack first if the design is ever revisited". The revisit began with a census, not a redesign: a build flag (`RAKUPP_PTR_CENSUS`) that made every `Value` destructor record which of its eleven pointer fields were actually set. The corpus was chosen to be broad on purpose — the local test suite plus a feature-targeted Roast slice, some 30 million destructions — and pointedly *not* the benchmark kernels, which would have been a blinkered witness (`hashfill` barely creates the fields in question). The verdict:

| live pointer fields | destructions |
|---|---|
| none | 25,583,692 |
| `arr` | 4,268,636 |
| `big` | 539,483 |
| `ratN ratD` | 466,950 |
| `obj` | 181,374 |
| `code` | 67,662 |

25.6 million of 30 million values carried none of the eleven pointers — and the half-million BigInts and Rat pairs are as much a part of the design as the zeros, because they are the values the change could have hurt.

The top row licensed the cold block: `im`, the `BigInt` slots, the Rat pair, the shape vector, the range fields, `ofType` and friends — about 148 inline bytes — moved behind one lazily allocated, copy-on-write `shared_ptr<ValueExt>`. Reads go through an accessor that never allocates (`xr()`, returning a shared empty block at namespace scope — a function-local static's thread-safe guard, run 256 times per regex byteset build, had already cost one 28% regression and taught that lesson); writes clone a shared block first. `sizeof(Value)`: 344 → 208.

The second half applied the same census to the five payload pointers — `arr`, `hash`, `code`, `pairVal`, `obj`. They were mutually exclusive on every live value except the

Match family, which legitimately carries positionals, nameds and `.made` at once. So the five collapsed into **one** slot plus a kind byte, and Match got a combined body behind the same slot. The kind byte is the design point worth keeping: a handful of sites convert a value in place — tag first, payload after — and a self-describing slot keeps that legal without proving a global invariant over the whole interpreter. `sizeof(Value)`: 208 → 128.

One census blind spot surfaced, and it is the honest footnote to the method: `is default` containers carried their element default in `pairVal` *alongside* a payload — a co-occurrence the census never sampled, because the programs it ran never did it. The census bounds what typical programs do, not what the language allows; the field moved to the cold block and the lesson moved here.

| effect (batches together) | |
|---|---|
| sortnums | −26% then −17% |
| arrayops | −20% then −14% |
| hashfill (interp) | −15% then −13% |
| hashfill (`--exe`) | — then −20% |
| JSON::Fast interpreted parse | −8% then −13% |
| hashfill peak RSS | −39% then −32% |

A plain Int copy, eleven pointer checks and a string construction in August's first week, is now two pointer checks and an inline `CowStr`.

Who pays in the new layout, and how that was checked. A value that *uses* a cold field pays one small allocation when the block is first written and one pointer hop per read — while every **copy** of it drops from a 344-byte memcpy to a 128-byte one with fewer refcount touches. Copying is what the profile said dominates, so even the penalised types should net flat or ahead — and the suite contains the adversarial case to test that: `bigint`, whose every hot value keeps its payload behind the block, improved with the rest (the −3…−13% band) rather than regressing, and the grammar parse — Match values genuinely carry payloads — measured flat. The wins were also broadest *away* from `hashfill` (sortnums −26% against hashfill's −15%), which is the distribution you expect when a change targets the representation rather than a benchmark. The residual exposure is narrow and nameable: a program that mass-creates short-lived Rats or Ranges and reads each once pays the allocation without harvesting the copy savings. No kernel isolated that shape — which was an argument for adding one to the guard, not for the old layout, and the argument was taken. `rats` joined `perf-guard` and `tools/bench` on 2026-08-22: 200k

`Rats` created, added into an accumulator and read once each. It is not a synthetic shape — every decimal literal in Raku is a `Rat`, so any money or unit-conversion loop is this program — and the same loop with an Int multiplier instead of `0.01` runs in 60 ms against its ~370, so what it times is Rat handling rather than loop overhead. This is the same move the string kernels made one batch earlier: when a representation change creates a class of value the guard cannot see, the answer is a kernel, permanently, not an argument.

## 41.5 Batch three: pads, and the lever the falsifier exposed

Until this batch every variable lived in an `unordered_map<string, Value>` on an `Env`, and every read hashed the name at each level of the parent chain. Perl resolves a `my` to an integer offset at compile time; a read is one indexed load. The port is not literal, because `Env` does three jobs at once — lexical scope, closure environment (a block captures the `shared_ptr<Env>` chain), and the dynamic-lookup spine for `CALLER::` and `$*dyn` — and any pad design has to keep all three working through the same object.

The design that landed: a `PadLayout` per pad *owner*, keyed by **body** rather than by `Callable` (`.assuming` wrappers share the AST body, and slot annotations live on shared AST nodes, so slot assignment must be a function of the body alone); a fixed-size `pad` vector on owning frames; and a liveness bitmask, because a slot exists from frame entry but must answer lookups only after its `my` executed — declaration is an event in the scope's timeline, and the mask is that event made cheap.

The bug of the batch deserves its page. The first design tracked the current pad frame in a register, maintained RAII-style by `run()` and the call path. It survived every targeted edge test — shadowing, closures, EVAL, `temp`, recursion — and then the Forth showcase said "stack underflow". The minimal reproduction:

```
sub f($n is rw, $d) { $d > 0 ?? f($n, $d - 1) !! ($n = 42) }
```

The `is rw` write-through evaluates the caller's argument expression after re-pointing the current scope to the caller — but not the tracked pad register. In a recursive sub, the caller's `$n` is the *same AST node* as the callee's; its annotation validated against the innermost frame's layout, and the write landed on the wrong activation. The interpreter has about 109 places that temporarily re-point the current scope; every one was this bug waiting. The fix was to **stop tracking and start deriving**: the pad frame is the nearest layout-carrying ancestor of the current scope, computed at use. A register is a claim about global state that every call site must maintain;

a derivation is a per-use proof from state the site already holds. Both RAII guards were deleted, and all 109 sites became correct at once.

Then the plan's own falsifier fired. The prediction was at least 10% on the assignment kernels from pads alone; the first A/B measured `asg` *flat*. A `sample` of the loop answered better than theory: the remaining samples were not in name lookup at all — they were in the per-iteration topic insert and the iteration-scope machinery. Pads had already removed the lookups; the kernel's cost was the scope itself, two million times. That is Perl's other `foreach` lesson: the loop variable aliases one pad cell for the whole loop. The equivalent here: when a static scan proves the body cannot define anything into the iteration scope, the loop keeps one scope and overwrites the topic in place — and capture safety stays dynamic, because a closure taking the scope bumps its use count and the next iteration forks it.

| kernel | pre | after | |
|---|---|---|---|
| perf-guard asg | 545.1 ms | 406.7 ms | −25% |
| perf-guard loopsum | 226.5 ms | 149.5 ms | −34% |
| perf-guard hash | 34.0 ms | 26.0 ms | −24% |
| perf-guard fib | 575.1 ms | 546.0 ms | −5% |
| bench hashfill (interp) | 184.0 ms | 155.2 ms | −16% |
| bench strcat | 17.2 ms | 13.1 ms | −24% |

The pads infrastructure was not wasted by the falsifier — the flat-loop lever lands *on top of it*, and the next batch builds on the same frame.

## 41.6 Batch four: the price of an assignment

With lookups and scopes cheap, the while-shaped kernels named the next cost: the store. Six one-line loop bodies priced an interpreter iteration's anatomy, 2 million iterations each, best of three:

| loop body | ns/iter | the increment buys |
|---|---|---|
| `{ }` | 65 | the loop floor: body dispatch, safe point, topic |
| `{ $x }` | 102 | +37 — one pad read, with the sink copy |

| loop body | ns/iter | the increment buys |
|---|---|---|
| `{ $x = 1 }` | 173 | +108 — the assignment ceremony for a constant store |
| `{ $x = $x + 1 }` | 201 | +28 — the specialised add itself |
| `{ $x += 1 }` | 127 | the compound path — proof a leaner lane exists |
| `{ $p = $p + 4; $n = $n + 1 }` (while) | 389 | condition eval plus two full assigns |

The arithmetic costs 28 ns; the act of *storing* its result costs 108. The assignment ceremony cost four times the arithmetic inside it.

Perl's answer is TARG: every op owns a preallocated result slot in the pad. Translated literally that is wrong for a value-returning tree-walk — but the diagnosis translates perfectly, and it became a decided-once **simple-assign lane**: a store into a slot that was declared untyped and unconstrained skips the full container ceremony. The eligibility rule carries a lesson of its own: a typed `my Int $x` leaves *no mark on its slot's Value* — the constraint lives in the scope's default table — so the lane's licence had to be a per-slot bit computed from the declaration at layout-build time, not anything inspectable at the store site.

The follow-up slices each carry a moral in miniature:

- **`op=` joined the lane.** The general compound path allocated a substring of the operator name and ran a cascade of string compares — on every `+=`. The lane classifies the operator once and stores an op class on the node: the cost of *re-deriving per evaluation*, removed by deciding once.
- **Native ints stopped bailing** on a rule read out of the engine itself: only uppercase type names register an assignment check, so a lowercase declared type is lane-eligible *by the engine's own definition* — parity by construction beats parity by testing.
- **The binder stopped string-matching types.** Its fast-accept caches what is a property of the name (its accept class) and re-checks per call only what can change underneath (whether a user subset now shadows the name — two hash counts). Fast-accept only, never fast-reject, so every error message stays byte-identical with the full matcher's.

- **Conditions answer as bool** — with the first iteration deliberately taking the slow path, so Chapter 19's shape verdict is decided by the code that owns it and the fast path only ever reads it. Layers stay in their lanes.

Combined: subcall −19.5%, strpass −19.5%, strscan −12%, loopsum −8%, the `my int` while-shape −45% in isolation; a closing slice gave parameters the same slot annotations variables have, worth another −4% on fib and subcall.

## 41.7 Batch five: the list container itself

Ten days later, and from a different direction. The size campaign's probe had priced a `Value` copy from every angle; on the way it measured something that was not about `sizeof` at all. Building a million-element list spent more time in `std::vector`'s reallocation than in the pushes — because a reallocation with a non-trivial element type is *two* passes over the buffer, one to move-construct into the new storage and one to walk the old storage destroying each source.

`ValueList` stopped being a `std::vector`. `RVec<Value>` (src/ValueVec.h) implements the `std::vector` subset the tree uses, with `std::vector`'s semantics, and grows in one pass — and, where the standard library tolerates a bitwise move, that pass is a `memcpy`, because a `Value` is trivially relocatable. Chapter 12 has the container.

Then the argument list. A `ValueList` is not only every Array's payload; it is built, filled, passed and destroyed on every interpreted call, and for the one- or two-element shape the heap block was almost the whole cost. Small blocks now come off a thread-local free list, kept per exact capacity for capacities one to four: allocation is a pop, release is a push, 32.35 ns down to 9.48. That is Perl's item 8 — `sv.c`'s arenas and free lists — applied to the one allocation two separate investigations had already named.

Measured against the batch's own starting binary, instructions and cycles, minimum of five runs: arrayops 1.303/1.201, listbuild 1.259/1.223, sortnums 1.188/1.201, multimeth 1.139/1.136, a two-argument call loop 1.130/1.088, textsplit 1.116/1.111, fib 1.086/1.043, hashfill 1.054/1.051. Nothing below parity.

### 41.7.1 The probe that was wrong all day

Batch two had its census blind spot and batch three had its falsifier firing. This batch's self-correction is the sharpest of the three, because nothing failed.

The `memcpy` path is guarded by `bitwiseRelocOk()`, which decides whether the standard library's `std::string` survives a bitwise move. Its first version asked whether a short string's `data()` pointer lay inside the string object. It does — on every implementation, because that is precisely what the small-buffer optimisation *is*. So the probe answered "not relocatable" everywhere, libc++ included; the `memcpy` path never ran; and every number measured that day came from the fallback loop.

No test could have caught it. The fallback is correct — it is what `std::vector` does — so the only symptom was a number smaller than it should have been, in a project where nobody yet knew what it should have been. It was found by asking the binary a question nobody had asked: a five-line program printing which path was live.

The right question is whether the object stores a *pointer* to those characters. libstdc++ aims `_M_p` at its own `_M_local_buf`; libc++ and MSVC compute the address from `this`. The probe now relocates a string and checks where the copy's characters come from.

Two things came out of it. The first is a number: turning `memcpy` on added `listbuild` 6.1% of cycles, `arrayops` 4.2%, `sortnums` 1.1%, and nothing elsewhere — so **most of this batch was never the `memcpy`**, it was the second walk. The second is a habit, now a file: tools/reloc-probe.cpp prints which path is live, because a correctness-preserving fallback can hide a dead fast path indefinitely, and this codebase now has two of them.

### 41.7.2 And a size that cost memory

The free list has an obvious form: round every small request up to one four-element block, one size class, one list. Simplest code, and marginally the faster of the two on call-shaped work.

It also made a program holding a million one-element arrays go from 292–296 MB to 560–663, and pay 22% of its cycles in the extra memory traffic. A `ValueList` is an argument list *and* an Array payload, and the two uses want opposite things from a block-sizing policy. Per-capacity lists give both what they want.

Which is batch two's lesson in a new place: a census of what typical programs do bounds neither what the language allows nor what a different user of the same structure needs.

## 41.8 Where it landed

The re-measured standing on 2026-08-22, same machine as every earlier revision of BENCHMARKS.md. The interpreter against Rakudo, all ten kernels:

| kernel | interp | Rakudo | faster |
|---|---|---|---|
| strcat | 9.7 ms | 181.1 ms | 18.7× |
| hash | 18.2 ms | 223.0 ms | 12.3× |
| bigint | 31.5 ms | 251.8 ms | 8.0× |
| sortnums | 34.1 ms | 249.4 ms | 7.3× |
| regex | 39.7 ms | 278.9 ms | 7.0× |
| arrayops | 64.9 ms | 280.6 ms | 4.3× |
| hashfill | 112.9 ms | 455.4 ms | 4.0× |
| loopsum | 107.6 ms | 259.7 ms | 2.4× |
| fib | 394.1 ms | 453.6 ms | 1.2× |
| streq | 248.4 ms | 284.3 ms | 1.1× |

`streq` was the last holdout and crossed with the assignment lane; `fib` had crossed the same day. Both had been Rakudo's for the whole life of the benchmarks file, both are tiny-body kernels where a JIT should be at its best, and both margins are thin — the file reads them as "level, our side", which is the honest reading. Compiling widens everything again: `--exe` puts `fib` 5.3× ahead and `streq` 13.9×.

Against `perl`, on the kernel that started the campaign:

| engine | hashfill |
|---|---|
| Raku++ `--exe` | 37.4 ms |
| Perl 5 | 104.0 ms |
| Raku++ interp | 112.9 ms |
| Rakudo | 455.4 ms |

The interpreter, 3.3× behind `perl` two days earlier, is within 10% of it. (The absolute numbers differ from the batch-one day's — 78 against 82 — because this harness times all four engines in one run, startup included, best of six; the batch-one figures were warm wall-clock. Ratios, not milliseconds, are the comparable thing across the chapter.)

And the costs are stated with the wins: startup grew 0.3–0.6 ms — laying out a pad is a fixed per-process cost — and long-lived churning hashes carry tombstones. Both were prices worth paying; neither is hidden.

And the leftover list is part of the result. The 128-byte `Value` still carries its `CowStr` inline; the head/body endgame is deliberately coupled to the container refactor, one piece of which batch five has now landed.

The loop floor and the per-node `eval` return protocol are also untouched, and they used to be described here as the opening argument for a threaded execution loop — a design document away, not a weekend. That document was written, in the form of a measurement, and the answer is no: the opcode `switch` is worth 0.32 ns against a node visit costing 46 to 85 nanoseconds. Under one per cent. What *did* change is the objection that had made a partial lowering impossible — the escape-analysis tax on parking an intermediate in addressable storage, +11.2 ns per un-lowered node when it was first measured and −0.02 ns now, because it was a property of a 376-byte `Value` and this campaign shrank it. The structural barrier fell; the motive never arrived. Chapter 42 carries the numbers.

The campaign stopped where the next step stopped being a constant factor and started being an architecture.

## 41.9 The three decisions

Every batch above is one of three decisions, made at a new layer:

1. **Pay per compile, not per use.** Slot numbers instead of name hashes; op classes instead of string cascades; accept classes instead of type matching; stored key hashes instead of re-hashed probes.
2. **Stop carrying per value what only some values need.** The cold block; the payload slot; tags as interned handles; the topic written in place.
3. **Keep the block instead of giving it back.** The frame pool, then the `ValueList` free list — the same decision applied to memory rather than to compilation, and the one Perl's arenas are the canonical instance of.

Perl's speed is mostly these three decisions applied everywhere for thirty years. The campaign compressed the applying; the deciding had been done for us, and reading it cost two days. What else the other engines had already decided — and which of it this codebase had independently decided the same way — is the next chapter.

# Chapter 42

# Reading Other Engines

The last chapter followed one findings document — the Perl 5 study — through four applied batches. This one is about the shelf that document started. Over 2026-08-21 and -22 the project read nine implementations of dynamic languages at the design level and wrote one findings doc per engine, each pairing the engine's primary sources with the measured state of our own structures. The full ledger lives in the repository under `docs/dev/findings/engines/` and stays maintained there; this chapter records the method, the shape of what came back, and the honest accounting of which ideas were imports and which were convergences.

## 42.1 The method

A study here is not a survey paragraph. The rules that made the shelf worth keeping:

- **Primary sources, fetched and dated.** Nikita Popov's zval and hashtable write-ups for PHP; the PEPs and release notes for CPython; the object-shapes proposal and the basic-block-versioning paper for Ruby; MoarVM's own repo docs; the Lua 5.0 implementation paper read in full; Filip Pizlo's speculation post for JavaScriptCore; the V8 team's parser posts; Rakudo's optimizer source and dispatcher inventory. Where a source was unreachable and memory had to serve — some of the MoarVM material — the doc says so rather than laundering recollection as citation.
- **Ground every comparison in a measured local number.** The day the PHP study was written, a scratch program printed `sizeof` for `Value`, `CowStr`, `Value`–

`Hash` and its entries, and `ObjectData` on this machine — so "their bucket is 32 bytes, our entry is 168" is arithmetic, not impression.

- **Rank by transfer, and say what does not transfer.** Every doc carries an honest frame — how much of the engine's speed is a smaller semantic surface rather than technique — and a section of things deliberately not taken, with reasons. LuaJIT's ceiling is not our ceiling; PHP's request-scoped allocator does not fit a long-running process; Python's everything-boxed layout is the base cost our `Value` avoided.
- **Sort every finding into one of three buckets**: already implemented here independently; imported and applied, with measurements; or a design input parked in a plan document. The buckets are the point — a pile of interesting facts is not a findings doc.

## 42.2 Nine engines, one line of inheritance each

**Perl 5** contributed the applied playbook of Chapter 41 — pads, result slots, the stored-hash table, arenas and free lists, the head/body diagnosis — and the framing sentence the whole shelf keeps confirming: pay per compile, not per use; don't carry per value what only some values need. Its arena discipline was the last item on the list to be applied and produced the largest single-shape win in the tree: the argument list a call allocates, 32.35 nanoseconds down to 9.48.

**PHP 7** is the same family's playbook executed as one deliberate rewrite — roughly 2× on real applications with no JIT — and the best-documented proof that the order is layout first, dispatch second, compilation machinery a distant third. Its new levers for us: per-callsite dispatch caches, compile-time attribute slots, and `zend_reference` — the production precedent for "a container cell should exist only where binding demands one", which is the shape of our container refactor.

**CPython** supplied the control layer: adaptive specialisation with counters and cheap de-optimisation (PEP 659), the type version tag as the invalidation half of any cache design, the twelve-year migration of attribute storage toward slots, and lazy frames that survive introspection as demanding as `CALLER::`. Its free-threading design (PEP 703) reads as corroboration from the other direction: the harden-the-runtime road our parallel plan had already chosen is the road it takes, and its worked-out mechanisms — biased refcounts, immortal objects, per-container critical sections — now feed that plan.

**Ruby** is the closest language sibling, and its two contributions are a key and a warning: object shapes show the attribute-cache key can work *across* classes (which roles make more relevant here than it is in Ruby), and the years its method cache spent behind one global serial — any definition anywhere flushing everything — are the cautionary tale our `ClassInfo` serial is designed against. YJIT's lazy basic-block versioning also gave a name to a habit this codebase already had: specialise on first observation, not on a counter.

**MoarVM** — the reference VM for this same language — is less a bag of techniques than a tax map: the guards its specialiser inserts are a ranked list of what Raku semantics cost, and containers sit at the top, which is independent confirmation of our refactor's aim. Its one directly actionable export is the interned callsite: the argument shape of a call is static, so the binder should compute a binding plan once per callsite-and-callee, not rediscover named arguments by scanning every argument list on every call.

**Lua** contributed the sharpest single design in the shelf: upvalues — per-variable capture cells, pointing into the frame while it lives and closing over the value when it dies — which is the closure half of the container refactor, reached by a third independent road. Its register VM paper is filed as the tightest starting spec for a threaded execution loop — which is now filed rather than queued, for the reason two sections below.

**JavaScriptCore** published the constants everyone else implies: a wrong speculation costs three to four orders of magnitude more than a right one saves, so speculate only when the probability of success is indistinguishable from one — the quantified form of a rule our decided-once flags and MoarVM's statistics both already practise. Its watchpoints completed the invalidation menu (check per use, subscribe and jettison, or let the key miss), and Bun — a runtime shell around JSC, not a faster engine — is market evidence for this project's founding bet: startup plus a native runtime surface plus drop-in compatibility wins users before peak throughput does.

**V8** earned a scoped doc for one idea: don't compile what you don't run. Preparse everything, parse a body on first call, and save the skipped body's summary so nothing is ever parsed twice — with a documented trap (superlinear reparsing) and a sequencing insight we would not have found alone: lazy bodies are trivial under whole-frame capture and hard after per-variable capture, so those two designs must be written together. At a 2–3 ms startup none of it is today's bottleneck; the doc is explicitly a when-the-time-comes study, and the time is module-scale programs.

**Rakudo**, ninth and last, required a decision recorded in its opening: until 2026-08-22 this project deliberately never read Rakudo's code. Reading it at the design level produced the only catalogue of its kind — eighteen *Raku-legal* static optimisations, each one a shortcut the language's own designers pre-litigated — and a dispatcher inventory that prices the semantic sites (in the reference implementation, even assignment and boolification are guarded dispatches). The stance that emerged is stated in the study and in the project's founding documents alike: read designs, never port code; Roast remains the only definition of correct.

## 42.3 Already ours, before any study

The shelf could leave the impression that every fast mechanism here traces to someone else. The record says otherwise, and the series index states it plainly: several of the findings were implemented in this codebase before the corresponding engine was read. Copy-on-write strings with cached scan state predate the Perl 5 study that expected to teach them (Chapter 9's `CowStr`, out of the string-scanning work); in-place ~= append predates the two engines whose rope designs it answers; the decide-once-at-first- execution habit predates the literature that names it; conditions-as-bool landed from our own profiles days before the PHP and Lua studies found the same fusion called "smart branch" in both; the lazily materialised rare-case block (`EnvExtras`) predates the engines that institutionalise the pattern; and the Array/Hash split means three engines' packed-array machinery solves a problem this design never had.

The distinction matters beyond credit. An idea reached independently here and by an engine under different constraints is *stronger* evidence than an idea copied — most of the convergence list below is agreement this project is one of the witnesses to, not a syllabus it received.

## 42.4 What the engines agree on

Nine engines, read against each other, agree more than they differ. The short form of the convergences the index records:

- The value head is small; the body is behind a pointer. Four independent data points bracket the endgame at 8–24 bytes; with containers and the numeric tower, ours lands at 16–24.

- Arguments belong in the callee's frame, not in an argument object — Lua register windows, PHP's SEND, MoarVM's callsite buffers.
- Strings intern once and carry their hash — five engines.
- Specialisation runs under one of two policies — counted warmup or first observation — and always counts its failures and retires its losers.
- Invalidation has exactly three modes, and mutation frequency picks one.
- Ropes are a last resort; both engines that rope grew flattening heuristics when reads suffered.
- Resumable control flow forbids a C-stack-recursive runloop — two VMs, independently, which is why first-class `gather` resumption waits for the threaded loop rather than arriving before it. That is now the *only* surviving argument for the loop here; the speed one was measured away.
- Layout before dispatch before compilation machinery. The JITs bolted on late moved real workloads least; the interpreters' layout years moved them most.

## 42.5 The one item the shelf recommends and this engine does not take

Perl's `run.c` is the most quotable thing on the shelf:

```
while ((PL_op = op = op->op_ppaddr(aTHX))) ;
```

The whole interpreter, flattened: each op does its work and returns `op_next`, a pointer the compiler filled in once. No recursion, no dispatch on node kind, one indirect call per op. It is item 3 of the Perl study and the change most often proposed for any tree-walker, including by people who have read this book's earlier chapters and drawn the obvious conclusion.

It has been measured here twice, a month apart, and the answer both times was no. The opcode `switch` alone costs **0.32 ns**. A node visit costs **46 to 85 ns** — `fib` 46, `asg` 61, `call` 65, `loopsum` 72, `method` 85, counted with a `-DRAKUPP_NODE_COUNT` build and divided into wall clock. Flat dispatch is between four and seven tenths of one per cent of a node visit. The interpreter is not slow because it walks a tree; it is slow because of what it does at each node, which is what every batch of Chapter 41 attacked instead.

The second measurement did change one thing, and it is worth recording because it inverts an argument this book used to make. A partial lowering — an IR that handles what it can and calls back into the tree-walker for the rest — used to be impossible, because the callback was expensive: parking an interpreter intermediate in address-

able storage rather than a non-escaping local cost **+11.2 ns per un-lowered node**, which put break-even at around 42% of all executed nodes lowered. There is no incremental path through a number like that.

That tax is now **−0.02 ns**. It was never really about registers; it was a property of a 376-byte `Value` carrying five `std::string`s and eleven `shared_ptr`s, and destroying and re-constructing one in memory the optimiser cannot reason about. At 128 bytes it has vanished. The break-even fraction went from 42% to zero.

So the structural objection is gone and the motive never arrived. If an IR is ever built here it will be for the thing that would actually pay — unboxed typed registers, a slot known to hold a `long long` for the extent of a loop, with a `Value` built only where it escapes — or for resumable control flow, which is the bullet above. "Flat instructions are faster than a tree" is not a reason, and two measurements a month apart say so.

## 42.6 Where it stops, for now

The campaign and the reading programme end at the same line: everything cheap and local is banked, and what remains is architecture — the head/body endgame coupled to the container refactor, the callsite cache programme, lazy bodies, and a threaded loop that is now possible and still unmotivated. Each has its design inputs parked in a plan document with the relevant studies cited, so the work can start from evidence instead of from recollection.

The next campaign is not speed at all — it is modules. The shelf will still be there when the profiles point back.

# Chapter 43

# The Carry Chain

Chapter 42 closed with a prediction: the shelf will still be there when the profiles point back. They pointed back ten days later, and this chapter is what happened — the campaign's method run once more, end to end, against a different competitor and a different kind of cost. Chapter 41's batches were about copying and lookup; this episode's first half is about *latency*, a cost no instruction count names. Its second half is the copies again, because it is always the copies. And its centrepiece decision is one Chapter 42 prepared: the competitor's fastest design, measured, priced, and declined.

## 43.1 The trigger

On 2026-08-31 mutsu — a Raku implementation in Rust, with a bytecode VM and a Cranelift JIT — was measured against Raku++ across all fifteen benchmark kernels. The interpreter won eleven, drew two and lost two. `bigint` was the clear loss, and it came with the one detail that made it actionable: `bigint` was the only kernel mutsu led *even against* `--exe`. Compiling the program bought nothing, so the time was not in the loop around the multiply — dispatch, scopes, assignment, everything Chapter 41 worked on — it was inside the runtime's own multiply routine, which the interpreter and the native binary share.

Chapter 41's trigger was a number without a place. This one named the row and not the reason, which is nearly as blank: *slower at big integers* covers the representation, the algorithm, the loop, and everything either side of it.

The kernel is a running product:

```
my $f = 1;
$f *= $_ for 1 .. 5000;
say $f.chars;
```

`5000!` is 16,326 decimal digits. Chapter 11 described the representation that has to carry it: a magnitude of **limbs** base $10^9$ — nine decimal digits each, about 1,814 of them by the end — and multiplying by a small number walks the limbs, multiplying each and carrying, exactly like long multiplication on paper.

There was even a prediction on file. The FAQ had named `bigint` as the row a dependency-taking implementation should win before anyone measured it: mutsu links `num-bigint`, a mature Rust library, and Raku++ hand-rolls its bignums because it takes no dependencies. The prediction was right about the row and wrong about the reason — the library's advantage is real (its base, below), but the measured gap was mostly ours to close without touching the representation at all.

## 43.2 First pass: one limb at a time

The general multiply had one schoolbook loop for every shape of operand. For the running-product shape — thousands of limbs times one small limb — that loop routed every limb's carry through the result array, a store the next iteration had to load back, and needed a second inner pass to place the carry. A dedicated one-limb path keeps the carry in a register, and splits the limb product *before* folding the carry in, so the split — a multiply-high and a shift, five-odd cycles — runs ahead of the carry chain instead of on it.

That took `bigint` from 45.2 ms to 11.4 compiled and 60.4 to 12.6 interpreted: level with mutsu. Level is not ahead, and the second pass is where the episode earned its chapter.

## 43.3 Second pass: the loop could not get out of its own way

Each limb's carry feeds the next one. However wide the processor is, a single-chain loop retires one limb per round trip through that dependency — and measured, the loop cost about five cycles per limb for work whose instruction count says one and a half. Nothing was being computed slowly; almost nothing was being computed at all. The chip was waiting for its own previous answer.

The fix is to stop having one chain. The magnitude is cut into **eight contiguous segments**, each multiplied with its own carry starting from zero — a lie, but a cheap one. The chains are independent, so they issue in parallel and the loop becomes throughput-bound. Afterwards the lie is paid for: segment *j*'s carry-out belongs at the first limb of segment *j+1*, which was computed as if nothing came in from below. Paying it back is one add with a ripple that stops at the first limb that does not overflow — it cascades only when the digits happen to be all nines — and the payments commute, so their order does not matter. Seven small additions at the end buy eight times as much work in flight.

Then something counterintuitive fell out: **the clever code became the slow code.** The split-first step above exists for exactly one purpose — keeping the division off the carry chain — and pays twelve machine instructions per limb to do it. With eight chains there is no waiting left to protect, so the arrangement is pure overhead, and the naive form, which folds the carry in first and pays one division, is six instructions — load, `umaddl`, `umulh`, shift, `msub`, store:

```
static inline void mulStep(uint32_t* d, const uint32_t* s, std::size_t i,
                           uint32_t m, uint32_t& carry) {
    uint64_t p = (uint64_t)s[i] * m + carry;
    uint64_t q = p / BigInt::BASE;            // a multiply-high and a shift
    d[i] = (uint32_t)(p - q * BigInt::BASE);  // one fused multiply-subtract
    carry = (uint32_t)q;
}
```

All three candidate step forms were run in one harness on the factorial kernel at eight chains:

| step form | 5000-step run |
|---|---|
| fold the carry in first (**shipped**) | **2.46 ms** |
| reciprocal-of-multiplier | 2.80 ms |
| split the product first (the first-pass form) | 3.48 ms |

The reciprocal form — two multiply-highs in place of the divide — spends the same three multiply-class operations, which is the real floor here, plus four more scalar ones. And eight is measured, not chosen: four and six chains land within 3%, two is 1.5× worse, sixteen gives the fold-back more segments than the loop saves. Below 32 limbs a single chain wins outright, and that is what runs.

An optimisation that was correct becomes overhead when the constraint it served disappears — Chapter 41's batches never hit that shape, because removing a cost there never changed what the remaining code was *for*. Removing the latency did.

## 43.4 And then the copies, again

Every one of the 5,000 steps was doing this around its one multiply:

1. copy the entire running total into the multiply routine — `toBig()` returns by value;
2. build a whole new magnitude for the answer;
3. copy that answer back into the variable, through `make_shared<BigInt>(const BigInt&)`.

Three full passes over thousands of limbs to move data around, wrapped around one pass of actually multiplying — and no line of source asks for any of them. Two go away with a reference (`toBigRef`) and a move overload (`Value::bigint(BigInt&&)`). The third needs the destination and the left operand named *once* instead of twice, which is what `applyArithInto` is: the compound-assign form of `applyArith`, emitted by the code generator in place of `lhs = applyArith(op, lhs, rhs)` and called by the interpreter's `op=` paths. When the box provably owns its magnitude alone, the multiply writes over it and allocates nothing.

*Provably* is two ownership counts, not one, and each catches a shape the other misses. Chapter 8's cold block is copy-on-write, so a plain copy of a `Value` shares the block and leaves the *magnitude's* use count at one while two Values plainly reach it. And a `Range` built over a big endpoint splices the same magnitude into a *different* cold block, so the block can be unshared while the magnitude is not. Both counts must be one. The rest of the guard is the fields a fresh `Value::bigint(...)` would have reset and an in-place write would instead preserve — an enum identity, a native width, a readonly binding, an itemized tag. And it refuses outright while worker threads are live: growing a magnitude *reallocates*, so a racing reader that has already loaded the limb pointer would read freed memory rather than a stale-but-valid limb.

Counted with a `malloc` shim over the whole benchmark, against a control that runs the same loop without the bignum:

| | allocations | bytes copied |
|---|---|---|
| before | 32,163 | 34,019,701 |
| after | **2,297** | **131,852** |

Seventy-four allocations for 4,966 bignum steps — one vector growing geometrically — and 33.9 MB of copying gone.

## 43.5 The road not taken

The obvious fix was on the table the whole time: do what the competitor does. `num-bigint` stores raw binary limbs, base $2^{64}$; measured against base $10^9$ that is about 3.4× on this multiply and about 10× on the general big-by-big product.

It was measured and declined, and the reason is Chapter 11's, now with prices attached. Base $10^9$ is what makes printing a big integer **linear**; every power-of-two base makes it quadratic. A 16,326-digit number converts to decimal in 0.087 ms as stored, against 0.73 ms from base $2^{64}$ — and the 0.73 is *with* a divide-and-conquer conversion written specially for the comparison. At 261,211 digits it is 1.3 ms against 129. This is a language, so `say $n` has to stay fast for real programs, not just for a benchmark that prints once. A base change would also silently move the boundary in `Value.cpp`'s `dblExact`, whose one-limb test currently means "below $10^9$, hence exact in a double" and would come to mean "below $2^{64}$" — a change to Rat→Num rounding that no test named. Base $10^{18}$ was measured too and is the worst of the three: the same total rewrite for 1.16×, because dividing a 128-bit product by $10^{18}$ exactly costs three multiplies where base $2^{64}$ costs none.

This is Chapter 42's discipline pointed at a tenth engine: read the design, ground the comparison in measured local numbers, and say what does not transfer and why. mutsu's base is the right choice for a library and was priced as the wrong one here — so the speedup had to come from inside the existing representation, and it did.

## 43.6 Where it landed

Measured through `tools/run-bench.raku` against a purpose-built binary of the commit before, one interleaved sitting at load average 2.3 — on the M1 development machine, so the milliseconds are not comparable with Chapter 41's tables, only the ratios are:

| lane | before | after | mutsu |
|---|---|---|---|
| interp | 13.0 ms | **7.4 ms** | 11.2 ms |
| `--exe` | 11.1 ms | **6.2 ms** | 11.2 ms |

Five control kernels — `loopsum`, `fib`, `hash`, `streq`, `rats`, both lanes each — all landed within ±2%, which is that box's run-to-run spread. Roast gated the change as Chapter 40 requires; the five files that differed from the baseline run behaved identically on a binary built from the prior commit, so all five were the machine's evening load, not the change.

And the episode left coverage behind, because there was none. `t/regression/bigint-multiply-kernel.raku` sweeps magnitude lengths 28–257 limbs across the eight-segment boundaries, cross-checks the one-limb kernel against the general schoolbook one, and probes the shared-magnitude hazard six ways. `tools/optbench/bigmul.raku` exists because nothing else in that directory leaves `int64` — so the compound-assign lane the code generator emits for `*=` had no four-lane agreement check at all until this work added one. The honest leftover is stated with them: the performance gate still has no bignum kernel, so it can gate this work's correctness but not its speed.

## 43.7 What the two episodes share

The full pairing lives in the repository's findings shelf, in `docs/dev/findings/HASHFILL-AND-BIGINT.md`; the short form belongs here, because it is the campaign's closing evidence.

Both reports named a number, not a cause, and both times the useful first move was to build or run the smallest program that could carry the claim. Both gaps were dominated by cost that appears nowhere in the source as work — most of `hashfill`'s in copying, `bigint`'s split between copying and a dependency chain the instruction count cannot see. Both fixes are constant factors: `hashfill` is still a hash fill and a sweep, `bigint` is still schoolbook long multiplication. Both times the competitor's own answer was measured and turned out to be the wrong answer here — perl's hash design was worth adopting but was one fix of four; mutsu's limb base would have bought the multiply by selling the printing. And both kernels stayed: the point of each episode was that nothing in the harness could see the workload until someone outside pointed at it, and the permanent fix for that is a kernel, not a recollection.

What remains open is the same list Chapter 41 ended on, one item longer. The eight-chain technique is applied in exactly one place — a magnitude times a single limb; addition and subtraction have the same strictly serial chain and would take the same treatment, and nothing measured has needed it yet. The general big-by-big product is untouched, and there the interesting prize is a better algorithm, not more chains. And `bigint` is now dominated by process startup — at ~3.9 ms of a 7.4 ms run it is the largest single line item left, and it has nothing to do with bignums at all.

# Chapter 44

# Compiling While It Runs

The campaign's two measured episodes attacked *constant factors*: what it costs to read a variable, to copy a value, to carry one limb of a multiply. Neither changed what the machine does with a program. A tree walk stayed a tree walk, and the native back end stayed a translation made before the program ran.

This chapter is the other kind of change. For one shape of code — a hot arithmetic loop — the interpreter stops walking the tree partway through and runs machine code instead, and it picks the loop because of what the program was *observed* to do rather than because anyone asked. Two back ends do it, landed a day apart, behind one harness. `--jit` hands the loop to the C++ compiler on the machine; `--cnp` stitches it out of machine-code snippets carried inside rakupp itself.

**Both are off by default, and both are work in progress.** They are one feature with two back ends, kept apart so each can be measured against the interpreter on its own, and the expectation is that `--cnp` becomes the default and `--jit` is removed. Treat the two spellings as scaffolding rather than as interface; this chapter describes mechanisms, and `docs/guide/JIT.md` is the flag reference that moves with them.

## 44.1 What tiering up is

Five steps, of which only the last is unusual.

1. **Counting.** Every `while`, `until`, C-style `loop` and `for` over a Range of integers counts its iterations. Nothing else does.

2. **Checking.** A loop that goes hot is checked against a whitelist. One that fails is marked and never examined again.
3. **Emitting.** The loop becomes a kernel — a small compiled function with the variables it did not declare bound into the live frame.
4. **Compiling.** By whichever back end is on.
5. **Entering.** At the loop's next iteration boundary, control moves into the kernel and the rest of the loop runs natively.

Step five is the one that usually needs machinery. Entering compiled code partway through a running loop is *on-stack replacement*, and in a bytecode VM it means reconstructing a native frame from an interpreter frame: which register held what, where the stack pointer should be, which values were spilled. Here it needs nothing at all. A `while` or `loop` keeps its entire state in named variables, which live in the interpreter's environment either way, so entering at an iteration boundary *is* entering at the top. Nothing has to be guessed, replayed or reconstructed.

That is not a trick; it is a consequence of the tree walk (Chapter 13) keeping no hidden execution state between statements. The design decision that makes the interpreter simple is the one that makes this cheap.

## 44.2 The harness, and the whitelist that makes it sound

A candidate loop must be unlabelled and not in expression position, and its body may contain only:

| | |
|---|---|
| literals | integer, number, Rat, string, Bool |
| variables | plain `$` scalars with no twigil: not the topic, not a dynamic, not an attribute |
| declarations | `my $x`, with no type, trait, coercion or default |
| operators | arithmetic, comparison, string and logical, `++` and `--`, the ternary, assignment and its compound forms |
| statements | `if` with `elsif` and `else`, bare blocks, nested loops, unlabelled `last` and `next` |

Anything else refuses the loop: a call of any kind, a method, an index, a regex, an inner `for`, `return`, `die`, a phaser, `state`, a closure.

The list looks like a list of things not yet implemented. It is really **one invariant, spelled out**: *nothing in the loop may call anything*. Everything that follows in this chapter — binding variables by pointer, holding them in machine registers for the whole loop, never checking a statement boundary — is sound because no code inside a kernel can observe the loop's variables except through the kernel itself. A widening that admits a call has to answer that question again, and the answer is not "probably fine".

### 44.2.1 The second gate, at the moment of entry

The whitelist is about the loop's *text*. The second gate is about the containers it will touch, and it runs every time the kernel is entered.

A slot the kernel would **write** is refused if its container has behaviour that a direct assignment would skip: a native width (`my int8` wraps at eight bits), a readonly binding, a coercion, a default, an `is rw` write-through, `is dynamic` (Chapter 12). A kernel assigns the container, so it would honour none of them. A slot the kernel only **reads** is exempt, which is what lets a loop inside a routine use its own parameters.

The wrapped answer is the proof the guard fired:

```
my int8 $n = 0;
my $i = 0;
while $i < 300 { $n = $n + 1; $i = $i + 1 }
say "$n $i";        # 44 300 — the int8 wrapped, so no kernel ran
```

A counted `for` has a rule of its own. Its loop variable is a read-only binding, so a body that writes it refuses the loop — and that refusal is load-bearing rather than tidy. The interpreter rebinds the variable from the counter on every iteration, so a write to it is forgotten at the next one, while a kernel holds one slot for the whole loop and would carry the write into the counter and change the trip count.

## 44.3 The first back end: hand the loop to the machine's compiler

`--jit` emits the loop as a small C++ function — the same one the code generator would print for it with `-O` (Chapters 26 and 27), with the undeclared variables bound as references into the live frame — runs a C++ compiler on it in a background thread, and `dlopen`s the result. The interpreter keeps iterating meanwhile and never waits.

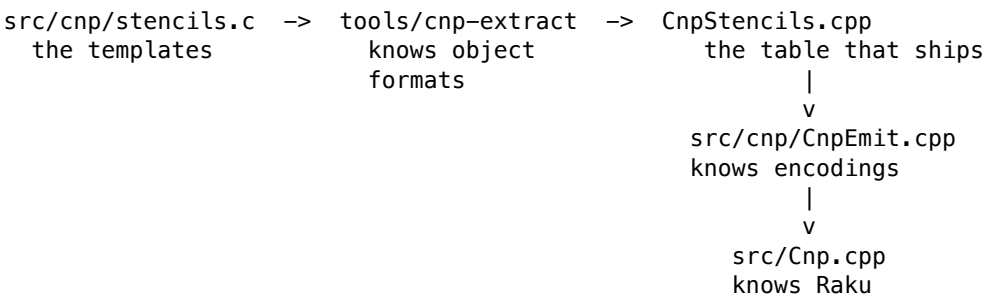
What makes it a JIT rather than a build step is that it happens during the run, on code chosen by how the program behaves, and takes over mid-loop. What makes it an unusual one is the back end: there is **no instruction encoder and no executable-memory buffer**, because the compiler on the machine is the assembler. The kernel is linked with its runtime references undefined and bound to the running rakupp when it loads, so a tiered loop and an interpreted one cannot disagree about semantics — they run the same machine code for everything but the loop's own control flow.

The price is in the numbers. A kernel takes about 0.83 s to compile, or 0.55 s with a precompiled header, and is about 50 KB on disk. That is why the threshold is 1000 iterations and why there is a cache, keyed on the kernel's source, this build of rakupp and the compiler that built it: a program that finishes in under a second gets nothing from its own first run, and the cache is what makes the *second* one fast.

## 44.4 The second back end: copy and patch

`--cnp` removes the compiler and the cache. A **stencil** is a snippet of machine code with holes in it, compiled when rakupp itself was built and carried in the binary as bytes. To compile a loop, the emitter copies the snippets it needs into a page of memory, one after another, and fills the holes with this run's values: which register, which constant, where to go next.

Three programs, meeting at one header:




`src/cnp/CnpTable.h` is the whole of what passes between them: an abstract patch kind and which hole it wants. Nobody has to know both halves. The extractor runs on the machine that *builds* rakupp, which is why there is no cross-object-format case to handle — it only ever reads the host's own.

This build carries **53 stencils, 5,324 bytes of template code**, sitting in read-only data and costing nothing in a run that never turns the flag on.

### 44.4.1 A stencil, and the holes in it

The holes are relocations, and that is the trick the whole back end turns on. `stencils.c` declares undefined symbols — `_JIT_OP0` to `_JIT_OP3`, `_JIT_CONT`, `_JIT_TARGET`, `_JIT_SLOW` — and reads their *addresses*. The C compiler, with no idea what they are, emits its ordinary address-of-a-global sequence and one relocation per reference. The extractor lifts those relocations out of the object file; the patcher rewrites the instructions.

Here is the stencil for a loop condition — "branch out when `$i < k` is false" — as the compiler left it, with the holes marked:

```
_rk_st_jnlti:
   adrp x8, 0          ; hole: OP0, the register number
   ldr  x8, [x8]       ;   an ADRP/LDR pair through the GOT
   ldrb w9, [x2, x8]   ; the register's tag
   cmp  w9, #0x1
   b.eq <the Num lane>
   cbnz w9, <cold>     ; neither Int nor Num
   ldr  x8, [x1, x8, lsl #3]
   adrp x9, 0          ; hole: OP1, the loop bound
   ldr  x9, [x9]
   cmp  x8, x9
   b.ge <target>       ; hole: _JIT_TARGET
   b    <next>         ; hole: _JIT_CONT
```

The two `ADRP/LDR` pairs are where most of the win is. The compiler had to assume an arbitrary symbol, so it reached it through the global offset table — two instructions and a **load**. By patch time the value is known, and when it fits in 32 bits the same two instruction slots hold `MOVZ` and `MOVK` instead: no table slot, and no memory access at all on the hot path. Register numbers and loop bounds almost always qualify; a `double`'s bit pattern does not, and falls back to a table slot in the same mapping.

The rewrite is refused unless the two instructions are adjacent and all three register fields agree. Anything looser and the scratch register might be one another instruction in the stencil is still reading, which would be a wrong answer rather than a slow one.

Every stencil that can fall back to the interpreter's own operator dispatch is written in **two pieces**. The fast one handles the machine-number lanes and calls nothing, so the compiler gives it no prologue and no epilogue; everything it cannot do it tail-calls to a cold block the emitter lays out past the end of the loop. Written as

one function with the call inline, the first draft paid six stack-memory operations per operation for a frame only the cold half needed.

### 44.4.2 The layout, and why nothing is ever out of range

One mapping holds the lot:

```
[ stencil code, one block per op ] [ veneers ] [ GOT slots ]
```

so a branch between stencils is always inside the range a single instruction can express, and an address-of-page instruction always reaches the table area, which is kilobytes away rather than wherever the heap landed relative to the executable. The **veneers** handle the other direction: a stencil calling a runtime helper is calling into the host executable, which may be anywhere, so it branches to a three-word thunk in the buffer that loads the real address and jumps. Cold, and free on a path that never calls a helper.

The buffer is mapped read/write, patched, then flipped to read/execute with `mprotect`, per region and never both at once. On Apple silicon the documented alternative is `MAP_JIT` with `pthread_jit_write_protect_np`, and it is deliberately the fallback: that call toggles write protection for every such mapping in the process and for the calling thread only, so patching one kernel while another thread runs a second one is a race by construction. rakupp runs work on several threads (Chapter 38). `mprotect` has no such coupling.

### 44.4.3 What it costs to make one

Measured rather than asserted: a program of 4,000 distinct one-iteration loops, lowered and patched at a threshold of zero, costs 63 ms more than running it plainly — about **15 µs per kernel**, and that figure includes entering each one. Against `--jit`'s 0.55–0.83 s that is four to five orders of magnitude, which is the whole reason this back end needs no cache: there is nothing worth storing. It also lets the threshold drop from 1000 iterations to 100, and it makes the **first** run of a program the fast one.

## 44.5 The register file, and the one IR in this engine

Chapter 3 classifies the back end by what is missing: no bytecode, no three-address code, no SSA, no control-flow graph — the AST is the sole representation the whole

way through. This back end is the exception, and it is worth stating exactly what kind.

A kernel does not run on `Value` objects. It runs on a flat register file: `r[k]` holds an int64, the bit pattern of a double, or 0/1 for a Bool, and `t[k]` says which. Anything else — a bignum, a `Str`, a `Rat`, an object, an enum value — lives in a parallel array of real `Values` with the tag `RK_T_BOX`. Slots are unboxed into registers at kernel entry and written back at exit, which is what makes the arithmetic one instruction instead of a call.

To emit that, the lowering walks the loop subtree once and produces a flat list of ops — a stencil index, up to four operands, and three continuations: the next op, a branch target, and the cold block a guard bails to. The `while` loop

```
my $s = 0; my $i = 0;
while $i < 200000 { $s = $s + $i; $i = $i + 1 }
```

becomes sixteen of them: seven on the hot path — the fused compare-and-branch, an add, a move, an add-immediate, a move, the back edge, the return — and nine more in cold blocks, one per guard that can fail, each ending where the fast op would have gone. Then the patcher copies 1,224 bytes of stencils into a page and fills in the holes.

That list is an intermediate representation by any honest definition. What it is not is a middle end: **nothing runs over it.** No folding, no scheduling, no register allocation past a bump counter and a high-water mark for temporaries, no serialisation, and nothing outside the function that builds it ever sees it. It exists for about 15 µs and is then thrown away. Chapter 3's claim survives as written for the language pipeline, and this is the footnote to it.

Three kinds of value are deliberately kept out of registers, and the first two are there because the alternative is a wrong answer nothing would have reported:

- a **bignum** `Int` cannot be an int64 at all (Chapter 11);
- an **enum value** is an `Int` with a name, and the name is its identity — `Less` unboxed to its ordinal prints as `-1` the moment it is copied into another container;
- everything else — `Str`, `Rat`, `Complex`, objects — stays a `Value`.

## 44.6 The four costs, and what each came to

The plan that proposed this back end named four things it would cost before any of it was written. Recording what they actually came to is more useful than the estimate was.

**Two instruction sets.** Real, and half paid. The patcher implements arm64 and x86-64; arm64 is the one that has been run. The x86-64 encodings were written from the manual, and when CI ran them the result was worse than untested: a kernel *is* entered and returns the **wrong answer**, while the statistics line cheerfully reports success. That is the one outcome worse than not compiling at all, because nothing about it looks like a failure from outside. `--cnp` now refuses x86-64 at startup, names the plan in the reason, and runs the loop interpreted.

**Two object formats.** Smaller than it looked, for the reason given earlier: the extractor runs on the build machine, so there is no cross-format case. Mach-O and ELF are both implemented.

**W^X and the executable-memory rules.** Smaller still, and the default route needs no entitlement and no special mapping — `mprotect` per region, as above.

**`die` is a C++ throw, and a code buffer has no unwind tables.** This was the hard one, and it is answered by construction rather than by registering unwind information. The rule is one sentence: **no helper a stencil calls may throw.** Every helper wraps its body in a catch-all, puts the exception in the frame as an `exception_ptr` and returns a status; the stencil returns an error code. Because every stencil tail-calls the next one, the whole kernel is a **single machine frame**, so that return goes straight back to the trampoline — an ordinary C++ frame, with ordinary unwind tables — which rethrows.

That is a constraint on the entire stencil-callable interface, and it is affordable only because the whitelist is small: six helpers cover everything 53 stencils can reach. The cost is real and it is the thing to weigh before every widening, because the sixth helper was free and the sixtieth would not be.

It has a visible consequence. The registers are hoisted, so a kernel that dies half way has to write them back before the throw propagates:

```
my $i = 0; my $n = 0; my $d = 5;
{
    while $i < 100 { $n = $n + 10 div $d; $d = $d - 1; $i = $i + 1 }
    CATCH { default { say "caught " ~ .^name ~ ": i=$i n=$n d=$d" } }
}
```

The division by zero happens at iteration five, inside a cold block, inside machine code with no unwind tables — and the `CATCH` must still read `i=5 n=22 d=0`, byte-identical to the interpreted run. It does.

## 44.7 Hoisting, and the bug that priced it

Holding the loop's variables in registers is where the speed comes from, and it is unobservable for exactly one reason: on this thread, the whitelist admits no calls. On *another* thread it is not unobservable at all, and no amount of reasoning about the whitelist would have found it, because the whitelist is about what is inside the loop. The corpus gate found it as a hang:

```
my $stop = False;
my $noise = start { my $n = 0; until $stop { $n = $n + 1 } };
$stop = True;                  # the kernel never sees this
```

The kernel read `$stop` once, and the write it was waiting for landed in the container it had stopped looking at. Nothing about that program is exotic — a flag set by another thread is how you stop a worker.

So a `--cnp` kernel is **not entered at all while another Raku thread is live**, which is what the concurrency runtime's live-worker and cued-load counters already say (Chapter 38). That loop stays interpreted and behaves as it always did. The test comes before the slot binding, because the interpreter asks again on every iteration and a refusal has to cost one relaxed load; the site is not retired, so a program that joins its workers gets its kernel back.

There is a third case that none of the harness's guards can see: two names bound to a **single container** would be two registers here and one cell there, and the write-back would drop whichever went last. That is checked on the spot, with a pairwise scan over the handful of slots.

`--jit` has none of these questions, because it does not hoist: its kernel binds a reference and re-reads the container on every use. This is the one place where the faster back end is the more restricted one, and closing it — by reloading read-only slots per iteration, if the measured cost of one inline load is worth the generality — is the open item.

## 44.8 Measured

Minimum of seven, arm64 Release build, on a machine carrying a load average of 5–8, so these are factors rather than percentages and none of them belongs in a benchmark table until it is re-taken under Chapter 40's quiet-machine protocol. `--jit` is shown with a warm cache, which is its second and every later run; `--cnp` has no such distinction.

| | interp | `--cnp` | `--jit`, warm | `--exe -O` |
|---|---|---|---|---|
| 5M-iteration integer `while` | 959 ms | **45 ms** | 42 ms | 27 ms |
| the Mandelbrot example | 121 ms | **78 ms** | 95 ms | 71 ms |
| a kernel entered 200,000 times | 784 ms | **140 ms** | 116 ms | 34 ms |

Each row says something different. The first is the headline: 21× over the interpreter, landing just above `--jit`'s warm cache. The second — the Mandelbrot program in the examples directory — reads as only 1.6× and needs reading: `--exe -O` compiles the *whole* program and still takes 71 ms, so most of that 121 ms is startup and output rather than the loop. What the loop itself costs falls from roughly 50 ms to roughly 7 ms.

**The third row was written to find out whether it would go the other way.** An ineligible outer loop around a ten-iteration inner one enters a kernel 200,000 times, and every entry binds slots, allocates a register file and boxes the results back. If entry were expensive the row would read *slower* than interpreting. It reads 5.6× faster. `--jit` is ahead of it here, and that gap *is* the entry cost: its kernel binds a reference and allocates nothing, which is the other side of the trade the register file makes.

That is what a tiered loop is worth. How often a program *has* one is a different measurement, and a more sobering one. Over the 40 programs in `examples/` and `tools/bench/` that run without an argument: 20 have no countable loop at all, 33 loop sites are examined, 7 pass the whitelist, and 6 programs enter a kernel. Counting loop keywords across the 904 Raku files in the repository says why — `for` out-

numbers `while` by six to one, and `.map` and `.grep` together outnumber it by two and a half to one, and none of those is counted unless it is a `for` over a Range of integers.

**The most common loop shape is now reachable; the most common loop sources are not.**

## 44.9 How it is proved

Three gates, and the second exists because the first can be satisfied by doing nothing.

**The differential gate.** Every program in the tier-up cases, the regression corpus and `examples/` is run twice by the same binary — once plainly, once at a threshold of zero, so every eligible loop lowers on its first iteration — and stdout, stderr and exit status are compared byte for byte. The interpreter is the oracle, as it is for the JavaScript back end (Chapter 31). Twenty-five cases are written for this specifically: the fifteen `--jit` brought and ten of `--cnp`'s own, each pinning one thing a register file could get plausibly wrong — a Bool that must print as `True` and not `1`, an enum that must keep its name, a `Rat` that must stay a `Rat`, NaN at the edge of a comparison, a string appended sixty times, a kernel that dies half way.

**The coverage check.** A case that agrees only because nothing compiled is not evidence. So every case that is not explicitly marked as testing a refusal must be shown to have entered a kernel.

Together they read, on the build this chapter describes: **725 programs agreeing, 0 differing**, 6 skipped as non-deterministic, and 23 of the 23 cases that are not marked as refusals shown to have entered one.

**Roast at a threshold of zero**, which turns the specification suite into this back end's differential test as well: every eligible loop in it lowers on entry. The two lanes are run interleaved, so that both meet the same machine.

The Roast rounds taught a lesson that is not about this back end at all. A single non-interleaved pair had read 23 assertions fewer under `--cnp`, and every one of those 23 came from two files hitting the harness's ten-second timeout in that one run. It looked like a finding. It was not: this suite times eleven or twelve files out in the *plain* lane every round, so a timeout carries no information about which lane it happened in. **A suite with a double-digit timeout baseline cannot be read one run at a time**, in either direction — and the first reading of it here was published as a characterisation on the strength of one sample, before the replication existed to support it.

## 44.10 What went wrong on the way

Four. Three were caught by a gate; the fourth by reading the machine code the first draft produced.

**The outermost loop's init was emitted, and re-ran.** A kernel is entered at an iteration boundary, which for `loop (my $i = 0; ...)` is *after* the interpreter has run the init. The first draft walked it anyway and re-ran `$i = 0` on entry, restarting the loop. The gate caught it as exactly one extra iteration, which is what a re-zeroed counter looks like from outside.

**The topic form answered 211 where the interpreter answered 210.** `for 1 .. N { ... $_ ... }` tiers up under `--cnp` and is refused under `--jit`, and the reason is in the C++ back end: the code generator does not resolve the topic to a lexical, so the slot bound under that name was written by the synthetic increment and read by nothing, while the body read the interpreter's live topic. Wrong, not failed — the outcome the whitelist exists to make impossible, caught by the differential gate and by nothing else.

**An append loop was right and 570× slower.** `$s ~= ...` in a kernel went through the general operator path, which copies the accumulator out of its box, builds a whole new string beside it and copies that back — three passes per append, so $n$ appends move $O(n^2)$ bytes. 400,000 appends took 34.4 s against the interpreter's 0.06 s, and the answer was correct the whole time, which is why no test caught it. The fix was to call the same in-place append the interpreter and `--exe` already share.

**The fast path paid for the slow path's frame**, which is the measurement that produced the two-piece stencil described above.

## 44.11 What it does not claim

- **It is not more general than `--jit`.** The 53 stencils are the ceiling; where `--jit` can in principle compile whatever `--exe -O` emits, this refuses and leaves the loop interpreted, which costs nothing.
- **It is not a speculating JIT.** There is no type feedback and no deopt path. The register tags are checked per operation, which is what lets a guard miss fall back *within* the kernel instead of leaving it.
- **It has run on one instruction set.**
- **It does not tier up threaded code.**

- **A kernel costs a page.** Each mapping is at least one page, so forty kernels is about 640 KB of address space. Pooling them into an arena is obvious and not done.
- **Neither back end helps anything that is not a hot arithmetic loop.** Method dispatch, object construction, hash and array work, regexes, and anything that calls a routine are untouched — and so is the loop around them.

## 44.12 Where it goes

The end state is no flag at all: hot loops tier up because that is what the interpreter does. Three things stand between here and there. The x86-64 patcher has to be shown correct on Linux too — on macOS it now agrees with the interpreter, after a fix to how it fills the GOT — because making a back end the default on a platform it has not run on would be making it the default on trust. The threaded gap has to be closed or consciously accepted — as a flag it is a documented limitation, as a default it is a silent cliff in threaded programs. And then `--jit` can go, which deletes the C++-emission lane, the precompiled header, the on-disk cache and two flags, and nothing else, because the harness was shared from the start.

What this chapter adds to the campaign is a different kind of entry in the ledger. Chapters 41 and 43 made the same work cost less. This one skips the work: for the narrow shape it reaches, the dispatch, the environment lookup and the `Value` copies are not made cheaper, they are **not performed at all**. The narrowness is the honest part of the result, and widening it is the same question every time — what can a kernel be allowed to do and still be unable to observe the difference?

# Appendices

# Appendix A

# The Tag Sets

The three closed enumerations everything else switches on.

## A.1 `VT` — runtime value tags

`src/Value.h`. Eighteen tags. Several Raku types are an existing tag plus a marker rather than a tag of their own; those are listed in the second table.

| Tag | Live fields | Notes |
|---|---|---|
| `Nil` | — | the absent value |
| `Any` | — | the default undefined value |
| `Bool` | `b` | |
| `Int` | `i`, or `big` when it overflows | `enumName`/`enumType` make it an enum value |
| `Num` | `n` | |
| `Str` | `s` | |
| `Array` | `arr`, `isList`, `itemized`, `shape`, `ext` | List, Seq, Junction and lazy sequences all live here |
| `Hash` | `hash`, `hashKind`, `objKeyed` | Set, Bag, Mix, Proxy, Failure, Date and more |
| `Code` | `code` | sub, block, method, builtin, WhateverCode |
| `Range` | `rFrom`, `rTo`, `rExFrom`, `rExTo`, `rNum`, `ext` | `ext` holds the original endpoint objects |
| `Pair` | `s` or `pairKey`, `pairVal`, `namedArg` | |

| Tag | Live fields | Notes |
|---|---|---|
| Type | s | a type object; s is the type name |
| Whatever | — | * |
| Object | obj | a user class instance |
| Rat | ratN, ratD, fatRat | normalised, denominator positive |
| Regex | s (pattern), hashKind (flags) | |
| Match | s, rFrom, rTo, arr, hash | five fields live at once |
| Complex | n (real), im (imaginary) | |

### A.1.1 Types with no tag of their own

| Raku type | Representation |
|---|---|
| List, Seq | Array with isList |
| an itemized array | Array with itemized |
| Junction | Array with enumName in any/all/one/none |
| a lazy or infinite sequence | Array with a LazySeqState in ext |
| Set, Bag, Mix, and their mutable forms | Hash with hashKind |
| Proxy | Hash with hashKind == "Proxy", holding FETCH/STORE |
| Failure | Hash with hashKind == "Failure" |
| Date, DateTime | Hash with hashKind |
| Buf | Str with hashKind == "Buf" and an identity token in ext |
| Blob | Str with hashKind, compared by value |
| Instant, Duration | Num with hashKind and an identity token |
| IntStr, RatStr, NumStr, ComplexStr | the numeric tag plus s and hashKind |
| an enum value | Int with enumName and enumType |
| FatRat | Rat with fatRat |
| a native integer container | any numeric tag with natBits and natSigned |
| Promise, Channel, Supply, Tap, Lock | a tag plus state in ext |

## A.2 NK — AST node kinds

src/Ast.h. Forty-nine kinds.

**Expressions.** IntLit, NumLit, StrLit, InterpStr, BoolLit, VarExpr, ListExpr, Assign, Binary, Unary, Call, MethodCall, Index, Ternary, Range, Pair, Block–

`Expr`, `ArrayLit`, `HashLit`, `NameTerm`, `RegexLit`, `SubstLit`, `ChainExpr`, `SymbolicRef`, `AllomorphLit`, `NqpOp`, `Whatever`, `SelfTerm`.

**Statements.** `ExprStmt`, `VarDecl`, `SubDecl`, `IfStmt`, `WhileStmt`, `ForStmt`, `LoopStmt`, `Block`, `ReturnStmt`, `LastStmt`, `NextStmt`, `RedoStmt`, `UseStmt`, `EmptyStmt`, `GivenStmt`, `WhenStmt`, `RepeatStmt`, `ClassDecl`, `EnumDecl`, `NamedRegexDecl`, `SubsetDecl`.

A great deal of Raku's surface syntax lowers into a few of these. Every operator — built-in, meta, hyper, set, and every user-declared one — is a `Binary`, `Unary` or `Call`. `class`, `role`, `grammar`, `module` and `package` are all `ClassDecl`. A phaser is a `Block` with a name.

## A.3 `K` — regex node kinds

`src/Regex.h`. Twenty kinds.

| Kind | Construct |
|---|---|
| `Lit` | a literal string |
| `Any` | `.` |
| `Class` | `<[…]>`, `\d`, `<:Nd>`, a named class |
| `Seq` | concatenation |
| `Alt` | `\|` (ranked) and `\|\|` (first match) |
| `Conj` | `&` |
| `Rep` | `*`, `+`, `?`, `** N..M`, with `%` separators |
| `Group` | `[ ]`, `( )`, `$<x>=[…]` |
| `AnchorStart`, `AnchorEnd` | `^`, `$`, `^^`, `$$` |
| `WBLeft`, `WBRight` | `<<`, `>>` word boundaries |
| `Nop` | an empty branch |
| `Subrule` | `<name>`, `<.name>`, `<alias=rule>`, `<r($x)>` |
| `Look` | `<?…>`, `<!…>`, `<?after …>` |
| `Code` | `{…}`, `<?{…}>`, `<!{…}>`, `:my` |
| `VarMatch` | a `$var` atom evaluated at match time; backreferences |
| `CapStart`, `CapEnd` | `<(` and `)>` |
| `CondRef` | `(?(N)yes\|no)` under `:P5` — branch on group N's state |

## A.4 `NqpOpc` — the `use nqp` subset

`src/Ast.h`. About fifty operation codes in seven groups: lazy control forms, 64-bit integer operations, codepoint-indexed string operations, list and hash primitives,

object and attribute helpers, buffer read and write, and folded constants. The full list is in Chapter 35.

## A.5 Exception and control structs

`src/Interpreter.h`.

| Struct | Meaning |
|---|---|
| `RakuError` | every user-visible Raku exception: payload plus message |
| `ReturnEx` | `return` across a callable boundary |
| `LastEx`, `NextEx`, `RedoEx` | labelled or cross-frame loop control |
| `BreakGivenEx` | `when` / `succeed` across a boundary |
| `ProceedEx` | `proceed` |
| `LeaveEx` | `leave` |
| `ResumeEx` | `.resume` inside a `CATCH` |
| `DoneEx` | `done` in a supply, whenever or react body |
| `StopGatherEx` | a lazy `gather` has produced enough |
| `ExitEx` | `exit` |
| `WorkerAbortEx` | unwind a background worker at shutdown — deliberately **not** a `RakuError` |
| `ParseError` | a compile-time diagnostic, optionally typed |
| `CodegenError` | `--exe` cannot transpile this; bundle instead |
| `AstEmitError` | `--aot` cannot rebuild this; bundle instead |
| `AstSerialError` | a cache entry is unreadable; parse instead |
| `StepLimitExceeded` | a regex exceeded its backtracking budget |
| `ObsoleteEscape` | a retired Perl 5 metacharacter in a pattern |

# Appendix B

# Flags and Environment Variables

Everything that changes what the compiler does from the outside. This is a reference for the internals; the user-facing command line is documented in `docs/guide/CLI.md`.

## B.1 Command-line flags that select a mode

| Flag | Effect |
|---|---|
| *(none)* | lex, parse, interpret |
| `-e 'code'` | interpret the argument |
| `--bundle` | embed the source bytes in a standalone binary |
| `--aot` | parse at build time, emit C++ that rebuilds the AST |
| `--exe` | transpile to C++ and compile natively |
| `--cpp`, `--emit-cpp` | print the generated C++ instead of compiling it |
| `-O` | run the codegen optimizer. Any suffix (`-O2`, `-Ofast`, `-Os`, ...) turns it on too and is forwarded to the C++ compiler; there is one flag here, not five |

## B.2 Inspection and tooling flags

| Flag | Effect |
|---|---|
| `--ast`, `--dump-ast` | print the parsed tree as an indented outline |
| `--ast-roundtrip` | serialise and deserialise the tree, then run it — the cache format's own test |
| `-c`, `--target=parse` | compile-check only: parse and check every variable is declared |
| `--lint` | static analysis; does not run the program |
| `--highlight` | syntax-highlight the source |
| `--html`, `--ansi`, `--terminal` | pick the highlighter's renderer |
| `--doc` | run `DOC` phasers and print rendered Pod |
| `--profile[=dest]` | the routine-level wall-time profiler |
| `--mcp` | serve the interpreter over the Model Context Protocol on stdio |
| `--timeout=SECS` | `--mcp` only: the watchdog's limit; `0` disables it |
| `--jupyter FILE` | run as a Jupyter kernel against Jupyter's connection file |
| `--jupyter-install` | write the kernelspec that lets Jupyter launch this binary |
| `--ffi-info` | which FFI backend is live, or why none is |
| `--fmt` | format Raku source to stdout (`--target=raku`); `--check` lists the files that would change, `--diff` shows what |
| `--rakuast` | print the RakuAST *view* of the program beside the Raku each node renders back to (`--target=rakuast`); `=tree`, `=attrs`, `=compunit` vary it |
| `--jit-info` | what is in the tier-up kernel cache (Chapter 44) |
| `--jit-clean` | empty it |
| `--precomp-info` | what the parse cache holds |
| `--precomp-clean` | empty it |
| `--precomp-modules=on\|off`, `--precomp-files=on\|off` | the two cache switches |
| `--version`, `-v` | the release, build and platform, on one line |
| `-V` | the full report — the split is Rakudo's, but the report is a readable block, not Rakudo's `key=value` config dump |
| `--info`, `--version-full` | ours; Rakudo spells this `-V` and nothing else |
| `--help`, `-h`, `--quiet`, `-q` | as expected |
| `-x` | skip everything before the `#!` line |
| `--json` | machine-readable `-c` and `--lint` findings |
| `--ll-exception` | every frame of an uncaught error |

| Flag | Effect |
|---|---|
| `--exe-info FILE` | the build manifest embedded in a compiled binary |
| `--stagestats` | phase timings and module loads on stderr |
| `--trace` | print each statement as it runs |
| `--repl-after` | run the program, then open a session on its state |
| `--completions=bash\|zsh\|fish` | print a shell completion script |
| `--lsp` | run the Language Server |

## B.3 Flags that shape a build

| Flag | Effect |
|---|---|
| `-o FILE` | output file (compile modes, `--target=js`) |
| `-I DIR` | add a module search directory |
| `--slim[=MODE]` | cut unused runtime subsystems from the binary; MODE is `safe`, `auto`, `max`, `none`, `help`, `list` or `verify` |
| `--standalone` | a module that cannot be embedded is a build error |
| `--static` | link the C++ runtime into the compiled binary (Linux, MinGW) |
| `--target=js` | transpile to JavaScript (Chapter 31) |
| `--verify` | emit JavaScript only if it agrees with the interpreter |
| `--module` | JavaScript: export the subs, classes and `MAIN` instead of running |
| `--runtime` | write just the JavaScript runtime |
| `--fallback=wasm` | accept a program outside the JavaScript core, on the WebAssembly engine |

## B.4 Flags that change how a run behaves

| Flag | Effect |
|---|---|
| `--jit[=SPEC]` | **work in progress**: compile hot loops with the machine's C++ compiler while the program runs, off by default. SPEC words: `off`, `on`, `sync`, `verbose`, `stats`, `nocache`, `pch`, `threshold=N` (Chapter 44) |
| `--cnp[=SPEC]` | **work in progress**: the same by copy-and-patch — no compiler and no cache. SPEC words: `off`, `on`, `verbose`, `stats`, `threshold=N` (Chapter 44) |
| `--seed` | pin the random generator |
| `--stack-size` | the stack of the program thread, and so the recursion ceiling |
| `--env-file FILE` | load `KEY=VALUE` lines into the environment |
| `--color=auto\|always\|never` | ANSI colour on stderr and in the REPL |
| `--watch` | re-run the program whenever it or a library file changes |
| `-l` | accepted; lines already arrive chomped |
| `-0` | NUL-separated records |
| `--name`, `--prefix DIR` | kernel name and location for `--jupyter-install` |

Flags are position-independent, and the perl-compatible one-liner family (`-n`, `-p`, `-a`, `-F`, `-i`, `-0777`, `-M`) is documented in the CLI guide.

This list is checked against the binary rather than maintained by hand: `tools/check-book-appendix.raku` reads `kFlagDocs` out of `src/main.cpp` — the same table `--help` prints from — and fails if a flag is in one and not the other. It was written because this appendix was missing twenty-seven of the sixty-six flags the binary accepts, including a whole run mode.

## B.5 Environment variables

### B.5.1 Search and loading

| Variable | Effect |
|---|---|
| `RAKULIB` | extra module search paths; **both** `,` and `:` separate |

| Variable | Effect |
|---|---|
| `ROAST` | adds the specification suite's test-helper library |
| `RAKUPP_HOME` | where the binary considers itself installed |
| `RAKUPP_CONFIG` | override the settings file location |
| `RAKUPP_VM_NAME` | what `$*VM.name` answers instead of `cpp`, for modules that branch on it |
| `XDG_CONFIG_HOME`, `XDG_CACHE_HOME` | the settings and cache directories |

### B.5.2 The parse cache

| Variable | Effect |
|---|---|
| `RAKUPP_PRECOMP_MODULES` | override the module-cache switch for one run |
| `RAKUPP_PRECOMP_FILES` | override the file-cache switch |
| `RAKUPP_NO_PRECOMP=1` | force both off |
| `RAKUPP_PRECOMP_DIR` | where cache entries live |

### B.5.3 The tier-up compilers

| Variable | Effect |
|---|---|
| `RAKUPP_JIT_THRESHOLD` | iterations before a loop counts as hot, for whichever back end is on |
| `RAKUPP_JIT_VERBOSE` | narrate every tier-up decision to stderr |
| `RAKUPP_JIT_DIR` | where `--jit` keeps compiled kernels |
| `RAKUPP_CNP` | a bundled binary only, which has no option surface of its own: `0` or `off` turns the baked-in back end off for one run, `1` on, anything else is read as a `--cnp` spec |
| `RAKUPP_CNP_X86=1` | run the x86-64 patcher on Linux, where it is refused at startup until it has been verified on ELF (it runs on macOS x86-64 without this) |

### B.5.4 The foreign-function interface

| Variable | Effect |
|---|---|
| `RAKUPP_FFI=0` | force the no-libffi fallback path |
| `RAKUPP_FFI=/path` | use *only* that library; if it fails to load, report and fall back rather than silently substituting the system copy |
| `RAKUPP_FFI_TRACE=1` | log every crossing, with marshalled arguments and raw returns |

### B.5.5 Memory

| Variable | Effect |
|---|---|
| `RAKUPP_CYCLE=0` | no cycle collection: reference counting alone (Chapter 12) |
| `RAKUPP_CYCLE=dry` | find cyclic garbage and count it, free nothing |
| `RAKUPP_CYCLE_STATS=1` | at exit, report collections, candidates, walks and what was freed |
| `RAKUPP_CYCLE_POISON=1` | give a collected object a class whose every method dies, instead of clearing it |
| `RAKUPP_CYCLE_TRIGGER=N` | collect whenever N candidates wait; `1` collects at nearly every scope exit |

### B.5.6 Concurrency

| Variable | Effect |
|---|---|
| `RAKUPP_PARALLEL=1` | true parallelism: workers run interpreter compute concurrently instead of serialising on the GIL |
| `RAKUPP_GIL` | control the lock explicitly |
| `RAKUPP_FREEZE_TRACE=1` | report any symbol-table mutation after concurrency engaged, and which thread did it |

### B.5.7 The regex engine

| Variable | Effect |
|---|---|
| `RAKUPP_LTM=1` | use the NFA ranker for longest-token matching where it is gap-free |
| `RAKUPP_LTM_DEBUG=1` | rank both ways and print every disagreement — works without `RAKUPP_LTM` |
| `RAKUPP_LTM_RANKDUMP=1` | print each decided alternation's ranking |

### B.5.8 Diagnostics

| Variable | Effect |
|---|---|
| `RAKUPP_TRACE=1` | the module search path and every resolution |
| `RAKUPP_DUMPTOKENS=1` | the token stream |
| `RAKUPP_NO_DECLCHECK=1` | skip the undeclared-variable gate (Chapter 39) |
| `RAKUPP_ACTTRACE=1` | grammar action firing |
| `RAKUPP_TAP_TRACE=1` | the test harness's own emission |
| `RAKUPP_KEEPGEN=1` | keep the generated C++ from a compiling mode |
| `RAKUPP_NO_AOT=1` | `--exe`: keep the embedded modules' routines interpreted (`RAKUPP_AOT_VERBOSE=1` says why one stays interpreted) |
| `RAKUPP_DEBUG_MAKE`, `RAKUPP_DEBUG_REPLAY` | grammar `make` and replay diagnostics |

### B.5.9 The interactive session

| Variable | Effect |
|---|---|
| `RAKUPP_REPL=1` | force a session even when standard input is not a terminal |
| `RAKUPP_HISTORY` | the history file |

### B.5.10 Test-harness variables

`RAKU_TEST_DIE_ON_FAIL` stops a suite after a real failure; `RAKU_EXCEPTIONS_HANDLER=JSON` serialises uncaught exceptions as JSON. Both follow Rakudo's spelling because test harnesses set them.

## B.6 Build-time switches

| Define | Effect |
|---|---|
| `RAKUPP_PTR_CENSUS` | count which combinations of `Value`'s eleven pointers are actually live together — the empirical input to shrinking the struct |
| `_GLIBCXX_USE_CXX11_ABI` | relevant only because it is the ABI break copy-on-write strings caused (Chapter 9) |

The pointer census is a good example of the project's habit: before collapsing `Value`'s pointers into tag-dispatched slots, a special build **counts** which sets are live simultaneously, rather than reasoning about what the type tags suggest.

## B.7 A note on flags that change behaviour

Two of the variables above change *results* rather than speed: `RAKUPP_LTM` and `RAKUPP_PARALLEL`. Both are off by default, and the policy for both is the same: the old path stays available for at least one release after the default changes, so a regression can be bisected to the switch rather than to the release.

`RAKUPP_VM_NAME` is a third, and a different kind. It changes one answer — `$*VM.name`, and the `$*RAKU.VMnames` list it has to stay a member of — so that a module which branches on the name and dies on the `else` takes its `moar` branch instead. Nothing sets it on your behalf, and no measurement this project publishes runs with it set: the default is what this engine is, and asserting a dialect is the caller's statement, not the engine's. `$*VM.config` is untouched either way, because the toolchain values in it were already real.

Everything else here either reports something or selects a code path that is required to produce identical output.

# Appendix C

# Source Map

The vocabulary the book uses — the general compiler terms and this project's own — is in Appendix D.

## C.1 Where to look for what

| If you are changing | Start in | Also read |
|---|---|---|
| tokenization, quoting, heredocs | `Lexer.cpp`, `Token.h` | Chapter 4 |
| statement or expression syntax | `Parser.cpp`, `Ast.h` | Chapters 5 and 7 |
| a user-declared operator | `Parser.cpp` `parseSub`, the `userInfix_` family | Chapter 6 |
| what a value *is* | `Value.h` | Chapter 8 |
| string performance | `Value.h` `CowStr`, `BuiltinsShared.h` | Chapters 9 and 24 |
| the number tower | `BigInt.cpp`, `IntOps.h`, `Value::rat` | Chapter 11 |
| scoping, assignment, binding | `InterpreterCore.cpp` `lvalue`, `evalAssign` | Chapter 12 |
| how lists and argument lists are stored | `ValueVec.h` `RVec`, `Value.h`'s `ValueList` | Chapter 12 |
| calls and signatures | `InterpreterCore.cpp` `callCallableRaw`, `bindParams` | Chapter 14 |

| If you are changing | Start in | Also read |
|---|---|---|
| `return`, `next`, `last`, `when` | the cooperative registers in `ExecContext` | Chapter 15 |
| a built-in routine | `BuiltinsRegister*.cpp` `registerBuiltins` | Chapter 16 |
| a built-in **method** | the six `methodCall` segments, **in order** | Chapters 2 and 16 |
| classes, roles, mixins | `InterpreterModules.cpp` `ClassDecl` handling, `Value.h` | Chapter 17 |
| laziness, `gather` | `LazySeqState`, `seqOp`, the gather stack | Chapter 18 |
| interpreter speed | `evalBinary`, `evalIndex`, the decided-once fields | Chapter 19 |
| regex syntax | `Regex.cpp` `parseAtom` | Chapter 20 |
| regex matching | `Regex.cpp` `matchNode` | Chapter 21 |
| grammars | `GrammarMatcher`, `Interpreter::grammarParse` | Chapter 22 |
| alternation ranking | `LtmNfa.cpp` | Chapter 23 |
| Unicode | `Unicode.cpp`, `tools/ucd/`, the generators | Chapter 24 |
| the CLI and compile drivers | `main.cpp` | Chapter 25 |
| the native compiler | `Codegen.cpp`, the `rt*` helpers in `Interpreter.h` | Chapters 26 to 28 |
| what a binary keeps | `SlimScan.cpp`, `ucd_seam.h`, `src/stubs/` | Chapter 29 |
| tier-up: counting, eligibility, entry | `Jit.cpp` — the candidate walk, `Site`, `runIfReady` | Chapter 44 |
| copy-and-patch: stencils and the patcher | `src/cnp/`, `Cnp.cpp`, `tools/cnp-extract.cpp` | Chapter 44 |
| the parse cache | `AstSerial.cpp` | Chapter 30 |
| the JavaScript back end | `src/codegen/Js.cpp`, `JsRuntimeSrc.cpp`, `src/js-rt/` | Chapter 31 |
| the browser build | `rakujs/rakupp_web.cpp`, `rakujs/build.sh`, `raku.js` | Chapter 32 |
| module loading | `InterpreterModules.cpp` `loadModule`, `Parser.cpp` `scanModuleOps` | Chapter 33 |
| the installer and the store | `tools/install.raku`, `Builtins.cpp` `.install` | Chapter 34 |
| `nqp::` ops | `Parser::makeNqpOp`, `Interpreter::evalNqpOp` | Chapter 35 |

| If you are changing | Start in | Also read |
|---|---|---|
| NativeCall | `Ffi.cpp`, `Interpreter::callNative` | Chapter 36 |
| the extension ABI | `rakupp_ext.h`, `ExtApi.cpp` | Chapter 37 |
| embedding — Raku inside a host | `rakupp.h`, `EmbedApi.cpp` | Chapter 37 |
| threads, the GIL, supplies | `Interpreter.h`'s concurrency section | Chapter 38 |
| lint, highlight, profile, REPL | `Lint.cpp`, `Highlight.cpp`, `Profiler.cpp`, `Repl.cpp` | Chapter 39 |
| the undeclared-variable gate | `DeclCheck.cpp`, `isSpecialVar` in `Interpreter.cpp` | Chapter 39 |
| the MCP server | `McpServer.cpp` | Chapter 39 |

## C.2 Rules that are easy to break by accident

Collected here because each has cost real debugging time.

**The method-dispatch chain is order-sensitive.** The four segments are ordered slices of one function; later arms deliberately catch what earlier ones decline. Moving an arm for readability is a behaviour change.

**A decided-once field may hold a fact about the syntax, never a value that can change.** The literal cache is the one exception, and a literal is a constant by definition.

**Never store an `FnRef`.** It borrows the caller's lambda; storing it dangles.

**A pointer is only a valid map key when its target's lifetime is at least the map's.** AST nodes are never freed, so a flip-flop state map keyed on one is fine. Regex nodes are freed and their addresses recycled, which is why a cache keyed on one produced automata built for other patterns.

**Intern a closed vocabulary, never an open one.** The intern table is append-only, so a field that can hold arbitrary runtime data would leak an entry per distinct value.

**Do not add a non-`const` `operator[]`, `begin()` or `data()` to `CowStr`.** That is exactly the interface that made copy-on-write non-conforming for `std::string`.

**Nothing in a `Value` may point at itself.** `ValueList` relocates its buffer with a `memcpy`, which is sound only while every member survives having its bytes moved without

the source being destroyed. A field with an interior pointer, a self-registering handle or an intrusive list node breaks it — and breaks it silently, by producing wrong data rather than by failing to compile. The near-miss is already in the struct: libstdc++'s short `std::string` points at its own inline buffer, which is why the path is chosen by a run-time probe. Run `tools/reloc-probe.cpp` after touching the struct.

**Add an early exit, never restructure the general path underneath it.** The first node specialisation cost the control 5.7% by doing the latter.

**No helper a stencil calls may throw.** A copy-and-patch code buffer carries no unwind tables, so an exception crossing one reaches `std::terminate`. Every `rk_cnp_*` helper wraps its body in a catch-all, stashes the exception in the frame and returns a status instead. Widening what a kernel may call means answering this for the new helper first (Chapter 44).

**A memo is only sound if the thing memoised is a function of the key.** A grammar rule that reads a dynamic variable is not a function of (rule, position), which is what the `dynDep` flag records.

**`gilPark`'s window must touch no interpreter state.** Only thread-local buffers and syscalls.

**Never join a thread while holding the lock the joinee might want.**

**A derived-data mechanism must degrade to recomputation, never to a guess.** Every failure mode in the caching and emitting paths ends in "parse it again".

# Appendix D

# Glossary

Two kinds of word are collected here. Most are the ordinary vocabulary of compilers — *lexer*, *AST*, *packrat*, *SSA* — which would mean the same in a book about any other language implementation. The rest are this project's own, or Raku's: *fat struct*, *decided-once field*, *twigil*, *slip*. Both are listed together and alphabetically, because when you meet an unfamiliar word in a chapter you do not yet know which kind it is.

Each entry says what the word means in general, then, where it applies, what it means *here* — the two are not always the same, and the difference is usually the interesting part. A chapter reference points at the long version.

## If you have never built one before

Eight terms carry most of the rest, and they are easiest read in the order a program moves through them.

A compiler is handed **source** — text. The **lexer** reads it character by character and produces **tokens**: the words of the language, so that the next stage can think about `my`, `$x`, `=` and `42` rather than about letters. The **parser** reads that flat run of tokens and works out what groups with what, producing an **abstract syntax tree** — one node per construct, nested the way the program means rather than the way it was typed.

What happens to the tree is where implementations differ. Most compilers lower it into an **intermediate representation**, optimise that, and emit machine code or

**bytecode** for a virtual machine to run. Raku++ does none of that in three of its four modes: an **interpreter** walks the tree directly, evaluating each node by visiting its children, which is called a **tree-walking interpreter**. The fourth mode walks the same tree and writes C++ instead.

Everything else in this glossary hangs off those eight.

## A

**ABI** (application binary interface) — the machine-level contract two separately compiled pieces of code agree on: how arguments are passed in registers and on the stack, how a struct is laid out, what a symbol is called. An API says what you write; an ABI says what the bytes must be. Raku++ defines one of its own for extension modules (Chapter 37) and restates libffi's by hand in order to call C (Chapter 36).

**A/B interleaving** — the benchmark discipline of alternating the two versions under test within a single run, rather than measuring one and then the other, so that any drift in the machine hits both equally (Chapter 40).

**Abstract syntax tree (AST)** — the tree a parser builds: one node per construct, with the punctuation that guided the parse discarded. `2 + 3 * 4` becomes an addition whose right child is a multiplication, so the precedence now lives in the shape rather than in the text. In Raku++ the AST is the *only* representation of a program — there is no bytecode or IR beneath it — so it is both what the interpreter walks and what the C++ emitter reads (Chapter 7).

**Adaptive grammar** — a grammar that the program being parsed can change while it is being parsed. Declaring `sub infix:<⋄>` adds an operator, and tokens after that declaration parse differently from tokens before it. No fixed parse table can be built for such a language, which is why every language with user-declared operators — Raku, Haskell, Prolog, Agda — has a top-down front end (Chapters 3 and 6).

**Adverb** — Raku's named modifier, written `:name` or `:name(value)`: `:i` on a regex, `:kv` on a hash lookup. To the parser it is a colon-pair in a particular position; to the callee it is a named argument.

**Allomorph** — a value that is simultaneously a number and its own string, such as `IntStr`. Represented as a numeric tag with the original text kept alongside it (Chapter 8).

**AOT (ahead-of-time)** — compiling before the program runs, as opposed to a JIT, which compiles while it runs. The `--aot` mode does the *parse* ahead of time and

emits C++ that rebuilds the AST at startup; execution is still a tree walk (Chapter 25).

**Arena** — a region of memory whose contents are all released together when the region dies, so nothing inside needs its own free. Everything an extension creates is allocated in the arena belonging to the call, which is what makes the extension ABI difficult to leak through (Chapter 37).

**Arity** — how many arguments a routine takes. A *direct-arity call* in the C++ emitter is one whose count is known at emit time, so the generic argument-list machinery can be skipped (Chapter 27).

**Associativity** — which way a run of equal-precedence operators groups. `a - b - c` is left-associative and means `(a - b) - c`; `a ** b ** c` is right-associative. Raku lets a declared operator state its own, and the precedence-climbing loop reads it out of the operator table (Chapters 5 and 6).

**Autothreading** — running an operation once per value inside a junction and recombining the results, so that `1|2 == 2` is true (Chapter 18).

## B

**Back end** — everything after the tree: the half of a compiler that turns an analysed program into something that runs. Three of Raku++'s four modes have almost no back end, because the tree *is* the program; only `--exe` has one, and what it emits is C++ (Chapters 3 and 26).

**Backtracking** — trying an alternative, and on failure rewinding to where you started and trying the next. The regex matcher does it constantly and by design (Chapter 21); the parser does it at exactly four places in seven and a half thousand lines, each a short speculative probe that restores its position on failure (Chapter 5). Same word, unrelated machinery.

**Binding** (`:=`) — making a name refer to the same *container* another name refers to, instead of copying a value into a container of its own. Assignment writes through a container; binding replaces it (Chapter 12).

**Binding power** — the number a Pratt parser attaches to an operator to decide whether it claims the expression on its left or yields it. Sixteen named ones cover Raku's two dozen precedence levels here (Chapter 5).

**Bottom-up parsing** — building the tree from the leaves toward the root, recognising a construct only once its final token has been read. LR and its relatives are bottom-up. Raku++ is not (Chapter 3).

**Bundle** — the `--bundle` mode: the source bytes are embedded in a standalone binary that lexes, parses and interprets them at startup (Chapter 25).

**Bytecode** — a compact instruction set invented for one language and executed by a loop that switches on an opcode. CPython, the JVM and MoarVM all have one. Raku++ deliberately has none; Chapter 13 says what the tree walk costs instead, and Chapter 42 reports the measurement behind the decision.

**Byteset** — a 256-bit bitmap cached on a regex character-class node, which answers "does this byte match?" without re-deriving the class (Chapter 20).

## C

**C3 linearisation** — the standard algorithm for ordering a class's ancestors when it has more than one parent, and the order Raku specifies. Raku++ walks parents depth-first instead, which differs in the diamond cases (Chapter 17).

**Callable** — anything invokable: a sub, a method, a block, a closure. Here it is one value shape carrying a cached view of its own signature (Chapter 14).

**Calling convention** — the protocol for making a call: where the arguments go, who cleans up, what a return value looks like. Raku++'s is a `ValueList` in and a `Value` out, and its cost is the largest single line item in a compiled call (Chapter 28).

**Capture** — two senses, kept apart by context. In a regex, a parenthesised group whose matched text is kept (Chapter 21). In Raku, the entire argument list of a call reified as an object (Chapter 14).

**Closure** — a function together with the environment it was written in, so it keeps working after that scope has been left. `my $n = 0; my &c = { $n++ }` yields one. Chapter 12 is the environment it closes over; Chapter 26 is what a closure has to become in emitted C++.

**Constant folding** — computing at compile time whatever the constants already determine, so `2 + 3` is emitted as `5`. Measured here and found to be worth almost nothing on real Raku, which is why the node work went elsewhere (Chapters 7 and 19).

**Container** — in Raku, the box a variable names, as distinct from the value inside it. `$x = 5` writes into `$x`'s Scalar container; `$x := $y` makes the name refer to a different container altogether. Most of what assignment means in Raku is a fact about containers (Chapter 12).

**Context-sensitivity** — when the same characters mean different things depending on what came before them. A bare `/` opens a regex where a term is expected and divides where an operator is expected. Raku++ settles this inside the lexer, from the lexer's own one-token history (Chapters 3 and 4).

**Control kernel** — in a benchmark, a workload that the change under test cannot possibly affect, run alongside the one it should. If the control moves too, the machine moved and not the code (Chapter 40).

**Cooperative control flow** — implementing `return`, `next`, `last` and `when` with a flag and a frame counter rather than a C++ exception, in the common case where no callable boundary was crossed (Chapter 15).

**Copy-and-patch** — building machine code at run time by copying pre-compiled snippets and filling in the blanks, rather than by running a compiler or an instruction encoder. The `--cnp` back end (Chapter 44). See *stencil*.

**Copy-on-write (COW)** — letting several values share one buffer and duplicating it only when one of them is written to. `CowStr` does this for strings; Chapter 9 also records why the C++ standard library gave the technique up for `std::string`.

**Coroutine** — a routine that can suspend in the middle, hand a value out, and later resume where it stopped. `gather/take` is the Raku construct that wants one; Chapter 18 shows how it is built without one, and where the substitute diverges from the real thing.

## D

**Dead code** — code that can never run. In this book it usually means a branch compiled into the binary that a particular program can never reach, which is what `--slim` exists to cut (Chapters 7 and 29).

**Decided-once field** — a mutable field on an AST node holding a fact about the *syntax*, computed on first evaluation and never recomputed. It may never hold a value that can change (Chapter 19).

**Declarative prefix** — the leading part of a regex alternative made only of literals, character classes and quantifiers, up to the first construct that needs code to run. It is what longest-token matching ranks by (Chapter 23).

**Desugaring** — rewriting a convenient surface form into the smaller construct it stands for, so that the rest of the compiler only ever sees the smaller one. Raku++ does very little of it: the tree keeps the shape the programmer wrote, because four run modes read the same tree.

**DFA (deterministic finite automaton)** — a matcher with a single current state and no choices to make, so it reads each input character exactly once and never backtracks. Fast, but incapable of expressing captures, embedded code or recursion — which is why Raku's regexes cannot be compiled into one (Chapter 3).

**Dispatch** — deciding which piece of code a call actually runs. Raku layers several kinds: name lookup, multiple dispatch on the argument types, method dispatch through the class hierarchy, and re-dispatch with `callsame` (Chapter 16).

**Dragon Book** — *Compilers: Principles, Techniques, and Tools*, the standard text. "A compiler in the Dragon-Book sense", in Chapter 3, means one with the classical pipeline of IR, optimisation passes and code generation — which three of the four modes here are not.

**Dynamic scope** — a name resolved by walking the *call* stack rather than the enclosing text, so what it means depends on who called you. Raku spells it with the $*$ twigil: `$*OUT`. Contrast lexical scope (Chapter 12).

## E

**Eigenstate** — one of the values inside a junction (Chapter 18).

**Emscripten** — the toolchain that compiles C and C++ to WebAssembly. Its `-fexceptions` mode routes C++ throws through JavaScript, which is what bounds recursion depth in the browser build (Chapter 32).

**Environment** — the run-time structure holding one scope's variables, with a pointer to its parent; resolving a name means walking that chain. Raku++ has no separate symbol table — the environment is it (Chapter 12).

**Epsilon transition** — an edge in an automaton that consumes no input. The longest-token automaton treats certain regex constructs as one, on the grounds that the real matcher will enforce them later (Chapter 23).

## F

**Fat struct** — the `Value` design: a single struct carrying a type tag and a field for every kind of payload, several of which may be live at once, rather than a class hierarchy or a variant. Chapter 8 is the entire argument.

**FFI (foreign function interface)** — the machinery for calling a function written in another language, here C. Raku spells it `is native`; Raku++ implements it by loading libffi at run time rather than linking against it (Chapter 36).

**Flip-flop** — Raku's `ff/fff` operator, which is off until its left side is true and then stays on until its right side is. It needs one bit of state per occurrence in the source, which is why there is a map keyed on the AST node (Chapter 18).

**Free list** — a chain of released blocks of one size, kept for reuse instead of being handed back to the allocator. There is one per capacity for small argument lists, and the frame pool is the same idea for scopes (Chapter 12).

**Front end** — everything before the tree: lexing, parsing, and whatever analysis happens on the way through. Raku++'s is a hand-written recursive-descent parser with a Pratt expression core (Chapter 3).

## G

**Garbage collection** — reclaiming memory automatically by tracing what is still reachable from the roots. Raku++ has none: lifetime is `shared_ptr` reference counting, which is simpler, is deterministic, and leaks reference cycles (Chapter 1).

**Gate** — a check that a release must pass, run automatically rather than eyeballed. The book uses the word for the correctness gates (Roast, the example suite, compiler agreement between the run modes) and for the performance gate, which fails a build on a regression against a recorded baseline (Chapters 1 and 40).

**GIL (global interpreter lock)** — a single lock that only one thread may hold while touching interpreter state. Raku++'s is engaged lazily, on first concurrent use, and can be switched off (Chapter 38).

**GLR, Earley** — parsing algorithms that pursue every possible reading at once and produce a parse forest, leaving ambiguity to be resolved afterwards. Raku++ produces one tree and settles ambiguity where it meets it (Chapter 3).

**Grammar** — in theory, the rules defining which strings a language admits. In Raku, also a language feature: a class whose methods are regexes, usable as a parser (Chapter 22). The book uses both senses; context separates them.

**Grapheme** — what a reader would call a character, which may be several Unicode code points (`e` followed by a combining acute). Raku indexes strings by grapheme; Raku++ stores them as UTF-8 bytes, and Chapter 24 is largely about the gap between the two.

**Greedy, possessive** — a greedy quantifier takes as much as it can and hands characters back under backtracking; a possessive one takes as much as it can and hands nothing back. Raku's `token` and `rule` make their quantifiers possessive — see *ratchet* (Chapter 20).

## H

**Hand-written** — of a lexer or parser: written directly as code, rather than generated from a grammar file by a tool. The alternative is a **parser generator** — yacc, bison, ANTLR, CPython's pegen — which reads a grammar and emits state tables. Raku++'s lexer and parser are hand-written C++: there is no grammar file, no generated table and no generator in the build. This is not a shortcut small projects take because they must; Clang and GCC are hand-written too, for the two standard reasons — a generated parser is hard to give good error messages, and awkward to make context-sensitive. The price is that every rule is code somebody maintains. The return, here, is a parser that can change its own operator table in the middle of a parse and point an error at an exact token (Chapter 3).

**Handle** — an opaque `RkValue` in the extension ABI: a token the extension passes back to the runtime, so that it never sees a `Value` (Chapter 37).

**Heredoc** — a quoting form whose terminator is a word the programmer chooses and whose body runs to the line bearing it. It is a lexer problem, because the extent of the body cannot be known from the opening delimiter alone (Chapter 4).

**Hoisting** — making a declaration take effect earlier than its position in the text. Named subs are hoisted here, which is why a sub needs no forward declaration — but the hoist is done by the interpreter rather than the parser, which is exactly why a user-declared *operator* must still appear before its first use (Chapters 3 and 6).

**Hyper operator** — Raku's » and « metaoperators, which distribute an ordinary operator over the elements of a list: `@a »+« @b` (Chapter 18).

## I

**Inlining** — replacing a call with the body of what it called. The emitter inlines a few `int64` arithmetic shapes itself and leaves general inlining to the host C++ compiler, which is much of the point of emitting C++ at all (Chapters 26 and 27).

**Interning** — keeping one canonical copy of each distinct value from a *closed* vocabulary and passing a small identifier around instead. Method names and similar fixed vocabularies are interned here. An open vocabulary must never be, because the table is append-only and would grow an entry per distinct runtime value (Chapter 10).

**Intermediate representation (IR)** — a form between the tree and the machine, invented so that optimisation passes have something regular to rewrite: three-address code, SSA, a control-flow graph. Raku++ has none of them in the path from your source to its execution, which Chapter 3 states as a classification and Chapter 42 revisits as a trade. The one exception is the opt-in `--cnp` tier-up back end, which lowers a single hot loop to a flat op list and discards it immediately; no pass ever rewrites it (Chapter 44).

**Interpreter** — a program that executes another program directly, rather than translating it into something else first. See *tree-walking interpreter*.

**Invocant** — the object a method is called on: the thing to the left of the dot. Much of method dispatch is guarded on its type (Chapter 16).

## J

**JIT (just-in-time compilation)** — compiling parts of a program into machine code while it is already running, guided by what it is observed to do. MoarVM has one. Raku++ has two, both opt-in: `--jit` hands a hot loop to the C++ compiler on the machine, and `--cnp` stitches it from machine-code snippets carried in the binary. With neither flag nothing is compiled during the run, and `--exe` compiles ahead of time instead (Chapters 3 and 44).

**Junction** — a Raku value holding several values at once, `1|2|3`, which operations distribute over. See *autothreading* and *eigenstate* (Chapter 18).

## L

**Lazy evaluation** — producing elements only when they are asked for, which is what makes an infinite sequence a usable value. Chapter 18 covers `Seq`, `gather`, and the lazy tail.

**Lexer** (also **scanner**, **tokenizer**) — the pass that turns a stream of characters into a stream of tokens, so the parser can work in words rather than letters. Raku++'s runs to completion and returns a flat vector before a parser exists at all (Chapter 4).

**Lexer hack** — the arrangement in C compilers where the parser feeds its symbol table back to the lexer, so the lexer can tell a type name from an ordinary identifier. It is named for how unpleasant it is. Raku++ has no such loop: the lexer has finished before the parser starts (Chapter 3).

**Lexical scope** — a name resolved by the enclosing text, fixed when the code is written rather than by who calls whom. `my` variables are lexical. Contrast dynamic scope (Chapter 12).

**LL, LR** — the two classical parser families. LL is top-down and builds a leftmost derivation; LR is bottom-up and table-driven. LL(1) means one token of lookahead suffices. Raku++ is in the LL family but is neither LL(1) nor LL(k) for any fixed k — and, because its grammar changes mid-parse, it is outside the classification altogether (Chapter 3).

**Longest-token matching (LTM)** — Raku's rule for choosing among a `proto` regex's alternatives: the one whose declarative prefix reaches furthest wins, not the one written first. Chapter 23 is the two rankers that implement it.

**Lookahead** — how many tokens beyond the current one a parser must see to decide what to do. Several Raku constructs need more than one, which is what places the parser outside LL(1) (Chapter 3).

**Lookaround** — a regex assertion that tests what lies ahead or behind without consuming it (Chapter 21).

**Lowering** — translating a construct into a simpler, more machine-like form, usually in a pass of its own. There is no lowering step between the tree and any of the four run modes here, which is why a language feature is implemented once rather than four times (Chapters 3 and 25).

**Lvalue** — an expression denoting a place that can be written to, as opposed to a value. `@a[0]` is one, and the interpreter has a separate evaluation mode for producing them (Chapter 12).

## M

**Maximal munch** — the scanning rule that the longest spelling wins, so that `<=` lexes as one token rather than as `<` then `=`. Raku++ gets the same effect from an operator table ordered longest-first by hand (Chapter 4).

**Memoisation** — caching a result against the arguments that produced it, so a repeat is a lookup. Sound only when the result really is a function of the key: a grammar rule that reads a dynamic variable is not a function of (rule, position), which is what the `dynDep` flag records (Chapter 22).

**Metaoperator** — in Raku, an operator that takes another operator as its argument: `Z+`, `X~`, `R-`, `»+«`, `[+]`. The lexer knows the shapes; the tree keeps them as ordinary nodes (Chapters 4 and 7).

**Mixin** — attaching a role to one object at run time, so that object alone gains the methods: `$x does Serialisable` (Chapter 17).

**Model gap** — a regex construct the longest-token automaton builder could not model, as distinct from one that genuinely ends the declarative prefix. A gap forces a fallback to the other ranker (Chapter 23).

**Multiple dispatch** — choosing among several routines of the same name by the types of *all* the arguments, not merely the invocant. Raku spells it `multi`. Raku++ scores the candidates and breaks ties by declaration order, where Raku specifies an ambiguity error (Chapter 16).

## N

**NFA (nondeterministic finite automaton)** — an automaton permitted to be in several states at once, which is what lets a single linear scan answer "how far could each of these alternatives reach?". Built here by Thompson construction, and used for exactly one job: ranking longest-token alternatives. It never matches on the engine's behalf (Chapters 3 and 23).

**NFG (normalization form grapheme)** — Raku's string model: text is normalised on the way in, and indices count graphemes rather than bytes or code points (Chapter 24).

**Node specialisation** — caching on an AST node the decision its syntactic shape implies, so the second evaluation skips the dispatch the first one did. A fast path on a tree walk, not a compilation step (Chapter 19).

**NQP (Not Quite Perl)** — the small Raku subset Rakudo is written in. Raku++ implements a subset of its `nqp::` ops, so that code written against them runs (Chapter 35).

## O

**On-stack replacement (OSR)** — moving a running loop from the interpreter into compiled code partway through, without restarting it. Usually the hard part of a tier-up compiler; here it costs nothing, because a loop's whole state is in its variables (Chapter 44).

**Opcode dispatch loop** — the `switch` at the heart of a bytecode VM. Measured here at 0.32 ns, against a tree-node visit costing 46 to 85 ns; that ratio is the number behind the decision not to build one (Chapters 40 and 42).

**Operator table** — the parser's map from an operator's spelling to its precedence, its associativity and the node it builds. Raku++'s is mutated during the parse and rolls back at scope exit (Chapters 5 and 6).

**Oracle** — a trusted implementation whose output a test is checked against. Rakudo is this book's principal oracle, and a file verified against it records which version it was verified with (Chapter 40).

## P

**Packrat memo** — the cached match of a grammar rule at a position. Legitimate for a ratcheting rule, because such a rule does not backtrack (Chapter 22). A **packrat parser** is one that memoises every rule at every position this way; the main parser here memoises nothing, which is one of the reasons it is not a PEG parser.

**Pad** — the storage for one scope's variables, laid out once per scope rather than looked up by name on every access. The term is Perl 5's, and so is much of the design (Chapter 41).

**Parse forest** — the set of all valid trees for an ambiguous input, which is what GLR and Earley parsers produce. Raku++ produces one tree (Chapter 3).

**Parser** — the pass that turns a stream of tokens into a tree, deciding what groups with what (Chapter 5).

**Parser generator** — a tool that reads a grammar file and emits a parser. See *hand-written* for why there is none in this build.

**PEG (parsing expression grammar)** — a grammar formalism with ordered choice, in which the first alternative that matches wins and ambiguity cannot arise by construction; usually paired with packrat memoisation. Raku++ has neither the ordered-choice formalism nor the memo (Chapter 3).

**Peephole optimisation** — a local rewrite that inspects a small window and improves it, with no knowledge of the program as a whole. Everything the `--exe` optimizer does is peephole-class by any standard classification (Chapter 27).

**Pratt parser** — an expression parser that resolves precedence with one loop and a table of binding powers, instead of one mutually recursive function per precedence level. Also called precedence climbing, or top-down operator precedence after Pratt's 1973 paper. It is what lets `parseExpr` fit on a page despite two dozen precedence levels (Chapters 3 and 5).

**Precedence** — which operator binds tighter, so that `2 + 3 * 4` is 14 and not 20. See *Pratt parser* and *binding power*.

**Proto** — in Raku, the declaration that names a family: `proto sub` for a multi family, `proto regex` or `proto token` for an alternation resolved by longest-token matching (Chapters 16 and 23).

**Proxy** — a Raku container whose reads and writes run code you supply, so an ordinary-looking variable can compute its value on access (Chapters 8 and 12).

**Publish** — the step at the end of module loading that copies a module's environment into the global one. Raku specifies exporting selected symbols; this publishes all of them, and Chapter 33 says so.

## R

**Ratchet** — the property of `token` and `rule` that their quantifiers are possessive and their matches commit, so that no backtracking crosses them. It is what makes memoising a rule sound (Chapters 20 and 22).

**Recursive descent** — parsing by writing one function per construct, each calling the functions for its parts, so that the C++ call stack mirrors the tree being built. The oldest and by far the most common way to write a parser by hand (Chapters 3 and 5).

**Reference counting** — freeing an object when the number of references to it falls to zero. Raku++ uses `shared_ptr`: deterministic, cheap to reason about, and unable to collect reference cycles (Chapter 1).

**Register allocation** — deciding which values live in machine registers. A back-end pass Raku++ does not have, because it emits C++ and lets the host compiler do it (Chapters 3 and 26).

**Regression** — a change that makes something that used to work stop working, or something that used to be fast stop being fast. The performance gate exists to fail the build on the second kind (Chapter 40).

**REPL** — read–eval–print loop. Raku++'s shares its session machinery with the MCP server and the Jupyter kernel (Chapter 39).

**Roast** — the official Raku test suite, and the specification in practice. Results against it are reported two ways, because either number alone misleads (Preface, Chapter 40).

**Role** — a bundle of methods composed into a class at compile time, or into a single object at run time as a mixin. Raku's answer to multiple inheritance (Chapter 17).

## S

**Seam** — an accessor that is the only route to a piece of data, so that the object defining it can be swapped for a stub at link time. It is what makes a Unicode table cuttable (Chapter 29).

**Self-hosting** — implementing a language in itself. Rakudo is self-hosted through NQP, and therefore needs a bootstrap; Raku++ is written in C++ and does not (Chapter 3).

**Sigil** — the character that opens a Raku variable name and declares what shape it holds: `$` item, `@` positional, `%` associative, `&` callable. Mostly a parse-time fact here (Chapters 4 and 12).

**Sigspace** — the `:s` adverb and the `rule` declarator, under which whitespace written in a pattern means "match optional whitespace here". Implemented by wrapping atoms with a `<ws>` subrule at compile time (Chapter 20).

**Single-pass** — reading the source once and doing the work on the way through, rather than in separate analysis passes over the tree. The parse here is single-pass, with three qualifications — a lexer pre-scan, two seams that re-lex the unread tail, and a slang's code running mid-parse — and it remains the reason there is no whole-program analysis to optimise from (Chapter 3).

**Sink context** — a statement whose value is discarded, signalled down the evaluation so that an assignment need not materialise its result (Chapter 13).

**Slang** — in Raku, a swapped-in sublanguage that changes how later source is parsed. It requires user code to run during the parse, which here happens at a fixed set of seams: the module runs in an interpreter of its own, and its tokens are called where the lexer would have started the productions it overrode. A slang that reaches past those seams is refused by name (Chapters 3 and 6).

**Slip** — `|@a`: a list that splices into the surrounding list or argument list instead of nesting inside it (Chapter 12).

**Slurpy** — a parameter that takes everything left over: `*@rest`, `*%opts` (Chapter 14).

**Source-to-source compiler** (also **transpiler**) — a compiler whose output is another language's source rather than machine code. `--exe` emits C++ and hands it to the host compiler (Chapters 25 and 26).

**Specificity** — the score a multi-dispatch candidate is given, which decides which candidate wins (Chapter 16).

**SSA (static single assignment)** — an IR discipline in which every variable is assigned exactly once, which makes many optimisations easy to state. LLVM's IR is in SSA form. Raku++ has no IR in the path from source to execution and no SSA anywhere; the one op list it does build, in the `--cnp` back end, is not in SSA form (Chapters 3 and 44).

**Stash** — Raku's package symbol table, reachable through `.WHO`. There is no per-module stash object here (Chapters 17 and 33).

**Static analysis** — inspecting a program without running it. `--lint` and the undeclared-variable gate are the two instances in this book (Chapter 39).

**Stencil** — a snippet of machine code with holes in it, compiled when the compiler itself was built and carried in the binary. The holes are relocations, filled in at run time with a register number, a constant or the address of the next snippet (Chapter 44).

**Superinstruction** — a fused node kind standing for a common pattern of several. The approach node specialisation deliberately did not take (Chapter 19).

**Symbol table** — a compiler's map from a name to what it denotes. Raku++ has no separate one: the run-time environment chain is the only such map (Chapter 12).

**Syntax-directed** — of a translation: driven by the shape of the construct being recognised, with the action attached to the rule that recognised it, rather than performed by a later pass over a finished representation.

## T

**Tagged union** — one struct able to hold any of several payload types, with a tag field saying which is live. `Value` is a loose one — loose because several payloads may be live at the same time (Chapter 8).

**Thompson construction** — the standard method for turning a regular expression into an NFA: one small automaton per operator, glued together with epsilon transitions. Used here for the longest-token ranker and nothing else (Chapters 3 and 23).

**Three-address code** — an IR of instructions with at most three operands, `t1 = a + b`. Named in this book only among the things that do not exist here (Chapter 3).

**Tier-up** — compiling a piece of a program that has been observed to be hot, while it runs, and entering the compiled version mid-flight. Raku++ has two tier-up back ends, both opt-in (Chapter 44). See *JIT* and *on-stack replacement*.

**Token** — one lexical unit: a number, an identifier, an operator, a string literal. The lexer's output and the parser's input. Raku's `token` declarator is an unrelated thing — a non-backtracking regex (Chapters 4 and 20).

**Top-down parsing** — building the tree from the root downwards, deciding what you are about to read before you have read it. Recursive descent is the hand-written form, and it is what an adaptive grammar requires (Chapter 3).

**Trait** — in Raku, a declarative modifier applied at declaration: `is rw`, `is native`, `is tighter`. Several of them are instructions to the parser rather than to the interpreter (Chapters 6 and 12).

**Transparent** (of a regex construct, in ranking) — treated as an epsilon transition by the longest-token automaton, because the real matcher will enforce it (Chapter 23).

**Tree-walking interpreter** — one that executes by recursively visiting the nodes of an AST, with no instruction stream in between. The simplest design that can run a real language, and what Raku++ is in three of its four modes (Chapters 3 and 13).

**Trigger** (in `--slim`) — a construct implying the program can run code the scan never saw, so nothing may be cut. An AST node the scanner does not model is one (Chapter 29).

**Trivially relocatable** — of a type: its bytes may be moved to a new address without the source being destroyed, and the result is as if it had been move-constructed and the original then destroyed. `Value` is; libstdc++'s short `std::string` is not. It is what lets the list container grow with a `memcpy` (Chapters 8 and 12).

**Twigil** — the second character of a Raku variable name, after the sigil: `*` dynamic, `!` and `.` attribute, `^` placeholder, `?` compile-time (Chapter 4).

## U

**UCD (Unicode Character Database)** — the data files the Unicode Consortium publishes. Raku++ pins a version and generates its tables from them at build time (Chapter 24).

**UTF-8** — the variable-width encoding strings are stored in. A grapheme index is not a byte offset, and Chapter 24 is largely about the distance between the two.

## V

**Virtual machine (VM)** — the program that executes a bytecode. MoarVM is Rakudo's. Raku++ has none (Chapter 3).

## W

**WebAssembly (Wasm)** — the portable binary instruction format browsers execute. The whole runtime is compiled to it with Emscripten (Chapter 32).

**Whatever** — Raku's `*` in term position, which turns the expression around it into a closure: `*.is-prime` is a one-argument block (Chapter 18).

**Wrapper stack** — the `&routine.wrap({...})` layers on a Callable, run outermost first, each able to `callsame` into the next (Chapter 16).